Money Matters

Straight to the money.
I'm going to give it all I got.
I'm going straight to the top.

A Tharun Industries Production



Tharun Nadeswaran
Student Code -
Centre Number -

Contents

Analysis **5**
- Background and Problem Identification 5
- Initial Survey Results 6
- Survey from the Foresters Friendly Society 9
- Interview with Luke Giles 11
- Current Methods And Problems 13
- Primary Objectives 20
  - Objective 1 - News Tab 20
  - Objective 2 - Economic Figures & Diagrams 21
  - Objective 3 - Economy Rater 22
  - Objective 4 - Investment Simulator 23
  - Objective 5 - Budget Tracker 24
  - Objective 6 - Costs of Production 25
  - Objective 7 - Login System 26
  - Objective 8 - Friendly to use UI Design 27
  - Objective 9 - User Dashboard 28
- End-Users 29
- User Needs 30
- Limitations of the System 31
- Proposed System 32
  - Mathematical Formulae 32
    - Cobb-Douglas Production Function 32
    - Exponential Modelling 32
    - Optimising Cost Of Production 33
    - Outlier Detection 34
  - Flowchart 35
  - Entity Relationship Diagram 36
  - Hand Drawn UI 37
  - Hierarchy Chart 40

Documented Design **41**
- Functionality of Proposed System 41
- Refined Flowcharts and Data Flow Diagrams of the Proposed System 45
- OOP Class Diagrams 55
- Database Design & Normalisation 65
- SQL Queries 70
  - DDL Scripts 70
  - DML Scripts 73

DQL Scripts....77
Security and Integrity of Data....84
Algorithm Design....85
Vernam Cipher....85
Recursive Merge Sort Algorithm....88
Recursive Binary Search Algorithm....90
Risky Purchases....92
Combinatorial Subset Search....95
API Endpoints & Sample Data....97
BBC News....97
The Telegraph....99
The Economist....101
The Independent....102
CNBC....104
Frankfurter API....106
WorldBank API....109
Yfinance Python Library....114
User Interface Designs....116
File Structure....132
The Prototype....141
Technical Solution....149
Contents....149
MoneyMatters Package....170
Algorithms Package....174
Controllers Package....192
EconomicFigures Package....239
Logic Package....263
PythonFlaskAPI Package....290
Records Package....316
SQLQueries Package....325
Users Package....380
JSON Files....390
Frontend Code....401
Testing....537
System Tests....537
Unit Tests....546
Evaluation....574
Objective Analysis....574
User Feedback....576
Possible Extensions....578

Appendix..... 579
Appendix A - Initial Survey Results..... 579
Appendix B - The Prototype Code..... 585
References..... 625
Figures..... 625
Guides..... 627

Analysis

Background and Problem Identification

In today's society, consumers and producers struggle to understand how the shifts within the economy can either generate wealth or lead to financial ruin.

Currently, there is no good way to understand the economy as people find it challenging to interpret macroeconomic data, such as changes in real GDP, and how that would influence their decisions. The current method to research news about the economy is neither effective nor efficient as most individuals get their news from social media or google searches which may provide broad results, but these sources may be biased or unverified. On the other hand, some individuals may collect their information using government reports, which does provide accurate data, yet this can be too technical and overwhelming for the average person to understand.

Furthermore, due to the consequences of the COVID-19 pandemic, global economies and markets have changed, which makes it harder for users to determine what macroeconomic indicator is relevant.

Given these challenges, it makes it difficult for consumers and producers to analyse economic trends before implementing them into the real world.

Initial Survey Results

In order to understand what tools my end users would need to become more financially literate, I sent a 19 questions survey on Microsoft Forms that is primarily aimed at 6th Formers at William Perkin CofE High School, aged between 16 to 18 (at the time of recording the responses). 21 6th Formers have responded to my survey.

I chose my sample to be 6th Formers as they come from a range of socio-economic, and cultural backgrounds. Their views provide a broad range of opinions on current and prior economic matters. Also, since 6th Formers are representative of the younger population that will soon be making decisions and contributing to the economy, their views may address concerns about their lack of economic understanding.

There are 4 main sections of my survey:

- Measuring the financial literacy among this sample
- How users can access information about the economy
- How consumers spend their money, and whether their decisions are influenced by the economy.
- Features users want to help them understand the economy

By understanding how users perceive the economy, it provides me with a list of issues that can be a problem to users. These sections are highly important to solve this problem, helping me see the challenges people deal with, allowing me to create a system that would benefit users.

The full list of survey questions and responses can be found in Appendix A. (Page 576) For the open-ended questions, the most common responses have been recorded in Appendix A.

Measuring the financial literacy among this sample

I began my survey by asking the respondents how well they know basic economic terms and the current state of the UK economy, shown in Questions 3 - 6. These results highlight that there is a huge gap of financial knowledge as 81% of participants claim they have “a basic understanding” of economic terms, like inflation and GDP, they are unable to use their understanding to make optimised decisions regarding their finances. “Seems complex from someone outside financial spaces” and “Just don’t know enough and it’s boring” were some of the responses.

Additionally, the average rating of financial literacy is 6, among that sample, which is further reinforced from respondents' uncertainty to describe the current state of the economy, where 38% of responses being “I don't know”.

Whilst some individuals do demonstrate a general understanding of the economy "Inflation is quite bad currently, cost of living crisis, housing crisis" and "From what I know, it's been losing its value over the years meaning the economy is failing (i think)", ultimately the majority of respondents are either unable to be informed due to the complex nature of becoming financially literate, or the lack of effort, as some deem it to be unnecessary.

How users can access information about the economy

Questions 7 - 12, assesses how intune the respondents are with the economic news and figures. Even though 52% of people view their news from popular news outlets like BBC, 67% of respondents gain their news from social media or podcasts, which can spread false information to the public, compared to trustworthy news outlets.

Furthermore, 43% of people read about the economy weekly or more frequently, yet 57% of people view news about the economy quite infrequently. Users had some benefits, and drawbacks when using those methods to analyse the economy. For example, some advantages were "Helps with budgeting, understanding what exactly you are spending on and why.", "You can make informed decisions" and "I am more knowledgeable of the current state of the economy and I can make more financial decisions". However some disadvantages were "A lot of right wing content, it would be amazing if you could sort by different types of viewpoints on websites" and "Misinformation and not knowing where to search."

Then, I asked the respondents their own opinion on whether society seems to over exaggerate the current economic climate with some arguing that news outlet do not exacerbate current affairs "No because it's a serious thing with trump throwing tariffs around chaotically", while others argue that news outlets amplifies the situation, creating unnecessary chaos, "Yes, because in the case of most innovations in the past more jobs have been created than those lost".

Lastly, I asked respondents for their opinion of the Government attempting to manage inflation and prices. Shockingly, 17 respondents said they "don't know", while 4 of the respondents had some idea of the Government interventions that are set in place, such as "They have just increased interest rates and increased prices which is not good, especially for loans for borrowing." and "Borrowing more and spending less on public services will solve these issues in the long-term."

Therefore, this section of questions highlight that users are unable to properly understand the news fed to them, due to the fears of complex/misinformation, it is boring and the lack of care of researching Government interventions.

How consumers spend their money, and whether their decisions are influenced by the economy.

Questions 13 - 15 shows how willing the respondents were to manage their money. 52% of people answered that they do acknowledge inflation impacts personal spending, where only 10% do not pay attention to the consequences of inflation.

All the respondents agree that they would love to have a system that tracks their budget. Common responses were “Yes, I spend too much on things I don't need and then realise too late when I have barely any money left”, “Yes, it would allow me to create a steady savings amount each month.”, “Yes, easier to track how much you have left and to reach a certain goal if you want to buy something” and “I find it beneficial because I would like to save enough for future spending like university costs”.

However, I do have to mention that the respondents do not necessarily make the best financial decisions based on their finances. For example, only 38% of respondents always look for the best deals when purchasing their goods, while 43% sometimes and 19% rarely make decisions that would best optimise their satisfaction. This highlights that people are unable to make confident financial decisions, without thinking about the consequences.

Features users want to help them understand the economy

Finally, the last set of questions (16 - 19) conveys the features users would love to assist them to become more financially literate and confident when making financial decisions.

From reviewing those questions the most liked feature is a New Tab and Budget Tracker, while the most disliked features (with 3 votes each) is the Investment Simulator, as people believe that “it still doesn’t fundamentally teach you anything, it just allows you to try your luck if you don’t have the knowledge” or “I don’t dislike the aim of the feature, but it must be clear to help users to minimise the risks of their investments.” and the Rating the Economy as “subjective opinions may differ based on social class and other factors”.

It is clear that while some users may disagree with some of the features, due to moral concerns, the majority of the respondents either do not mind or would appreciate the feature.

From analysing all the results I have collected from my survey, there is a clear relationship between young peoples’ lack of financial literacy and their limited understanding of the current economy. This means that consumers are unable to make optimal choices leading to resources being wasted and minimising consumer satisfaction.

Survey from the *Foresters Friendly Society*

After reviewing the results of my survey, I conducted further research to determine if similar studies on financial illiteracy had been carried out. The *Foresters Friendly Society* conducted a study aimed at assessing levels of financial illiteracy.



Figure 1: Foresters Friendly Society Stats(*Foresters Friendly Society*, 2023)

This secondary data that the "*Foresters Friendly Society*" have collected highlights that a large proportion of the UK Population suffer from financial illiteracy. One example of this is shown in [Figure 1]. Whilst 65% of the participants believed that they had a good understanding of personal finance, only 35% answered at least 1 of the 7 of the Financial Literacy Quiz correctly.

They were closely matched when it came to their understanding of how inflation works – with 48% of Millennials and 46% of Gen Z-ers answering correctly.

- Of those that were unable to answer the question correctly, one in three Gen Z-ers admitted they didn't know the answer over taking a guess. Double the amount of Millennials answered this question incorrectly over saying they didn't know.

Extract 1a: Foresters Friendly Society Stats(*Foresters Friendly Society*, 2023)

[Extract 1a] highlights the misunderstandings of how the economy functions. As "*one in three Gen Z-ers admitted they didn't know the answer over taking a guess*" and "*Double the amount of Millennials answered this question incorrectly over saying they didn't know.*" suggests that the lack of financial literacy is not only an issue for the younger

generations, but originates from older generations due to lack of education and the spread of misinformation.

"These results really surprised us. We'd have expected the introduction of financial education to the National Curriculum to have been more positive in its impact on young people's understanding of money. Financial skills are absolutely invaluable throughout all stages of adulthood – putting us in the most informed position to make big life decisions. Our survey results have highlighted just how much work there still is to be done around equipping not just Gen Z-ers but Millennials too with the right financial skills."
Extract 1b: Sally Waterfield, Head of Marketing (*Foresters Friendly Society*, 2023)

Additionally, [Extract 1b] shows that even though there are current systems to assist humans to make better financial decisions, the results of the survey contradicted this. They had expected that since financial education is now part of the school curriculum, young people would have a stronger understanding of money and the economy. Waterfield highlights that financial skills are "*absolutely invaluable throughout all stages of adulthood* ", helping people with budgeting, saving, and investing. However, this survey showed that both Gen Z and Millennials do not have sufficient information and important money skills to confidently spend their income, to make optimal decisions. Therefore, more work is needed to ensure that young adults are able to become more financially literate.

These statistics demonstrate that there is a lot of inaccurate information about the economy, which would eventually lead for the public to gain a sense of false hope. This can create economic instability, as economic stakeholders will make decisions that harm their wealth.

Interview with Luke Giles

After collecting the results from the survey I interviewed my economics teacher, Mr. Giles, to gain deeper insight into how individuals with a strong understanding of the global economy perceive how news is being spread, and respond to shifts in macroeconomic indicators.

1) What are the current methods for ordinary individuals to look at the economic climate?

   Mr. Giles: BBC News, Social Media

2) What are the benefits and disadvantages of using these methods?

   Mr. Giles: A disadvantage is that there is a political bias. However the benefit is that you can be critical when analysing the different sources of information in order to form your own understanding.

3) How reliable and/or biased is the news and social media when describing changes to the economy?

   Mr. Giles: Social Media (X, Elon Musk) tend to be more Right Wing, while BBC News, ITV News, are more Centrist.

4) What methods of accessing the economic news do you find the most useful?

   Mr. Giles: Financial Times, This Week Magazine, News Outlets (BBC News), Social Media

5) Compared to 20 years ago, how easy is it to become financially literate?

   Mr. Giles: It is much easier nowadays, much more readily available information on the Internet.

6) How important do you think it is for people to understand the economy in today's world?

   Mr. Giles: Very important, understanding where one fits into the current economic system.

7) Do you think it is difficult for ordinary consumers to make financial decisions that optimise their utility and satisfaction?

Mr. Giles: Of course

Based on the survey I conducted, the Foresters Friendly Society's survey, and Mr. Giles' interview, there is a clear trend: young people nowadays are less in tune to the changes within our economy. As a result, this lack of knowledge, such as inflation rates or GDP growth, contributes to a larger issue. Many young people are financially illiterate, so they may struggle to make informed decisions about managing their finances, that would maximise their satisfaction or understanding how the economy impacts their livelihoods.

Current Methods And Problems

From the feedback of the results of the survey and Mr. Giles' interview, I observed the most common ways consumers access economic news and recognise the struggles ordinary individuals face in order to understand the current economy.

The most common problem I identified is the sense of bias from popular news outlets. *The Guardian* leans towards the left holding a liberal bias, while *The Telegraph* has a tendency to favour the right-wing ideals, with a Conservative bias. Although *BBC News* claims to be Centrist, they have been accused to be biased towards both right and left wing. As the public have no clear indicator of the bias within the news they consume, this can drastically impact how people interact with the economy, which in return could lead to economic instability from asymmetric information being spread.

Two of the most notorious processes are the following:

1. The Telegraph - The Complex Nature
2. Financial Times - The Paywall

The Telegraph - The Complex Nature

1) When users visit *The Telegraph* to learn about the economy, they may be greeted with a few challenges. In [Figure 2a], being entered to the website with a large advertisement can be distracting and may discourage users from continuing their search for information.

2) Additionally, the wide range of news subcategories can be overwhelming for users. To access economic news, users are required to navigate to the "Money" tab. However, this may cause confusion, as individuals may mistakenly click on the "Business" tab, not realising the difference in news between the two sections.

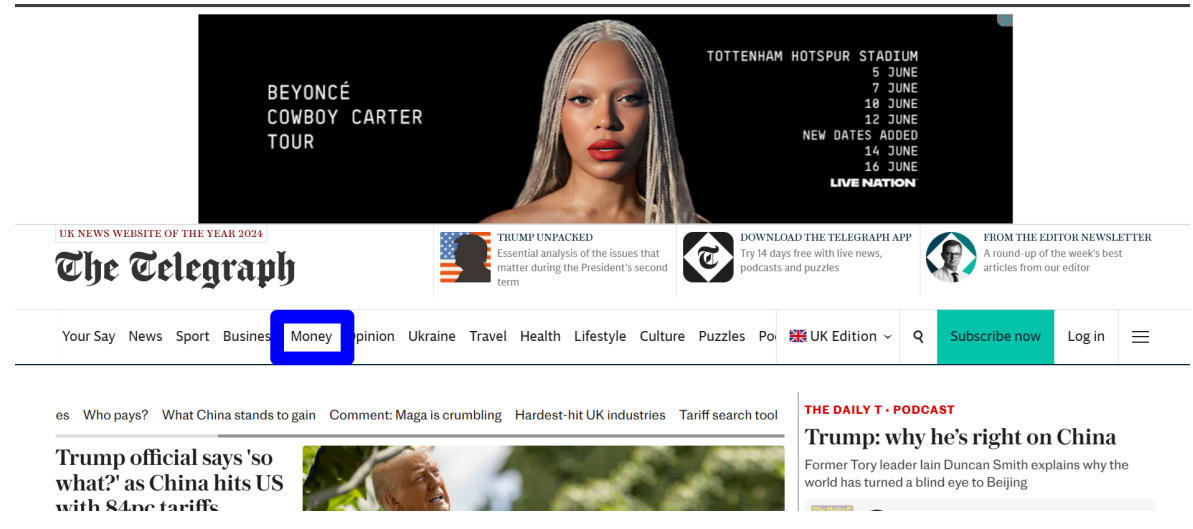


Figure 2a: The Telegraph Webpage(The Telegraph, 2025)

3) Once users navigate to the Economic news section, they have a wide range of sources to read and learn from, shown in [Figure 2b]. While this is beneficial for the ordinary consumer, individuals may feel overwhelmed as some articles may exacerbate issues about the current state of the economy. Given that The Telegraph is known for leaning towards right-wing ideals, it is most likely to promote capitalism and have scepticism toward government regulation. Some users may not realise this, and may give into any exaggeration from The Telegraph, adding towards asymmetric information.

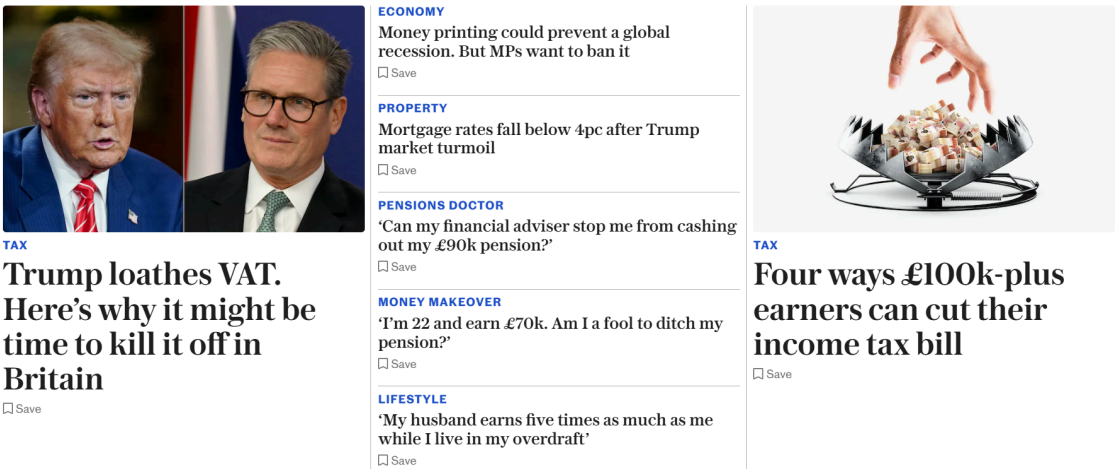


Figure 2b: The Telegraph Webpage(The Telegraph, 2025)

4) In [Figure 2c], users are forced to make an account for The Telegraph. This can further disincentive users to learn more about the economic state as it can be time-consuming to create an account, so users would believe that the hassle isn't worth it to quickly check information. While it is possible to read news from The Telegraph, most consumers would easily give up.

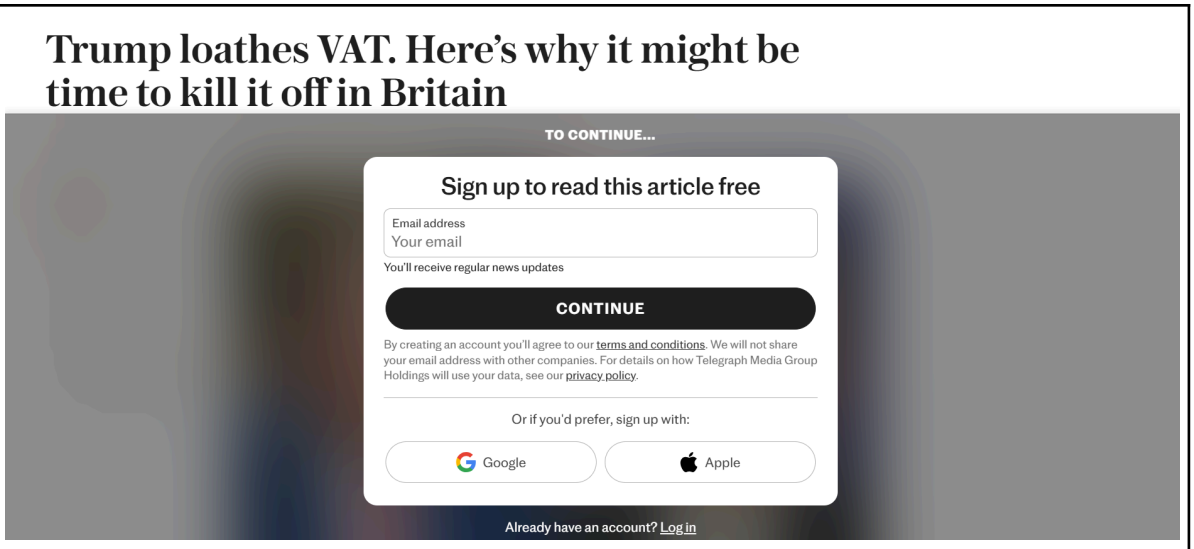


Figure 2c: The Telegraph Webpage(The Telegraph, 2025)

5) Some users may encounter a pop-up, shown in [Figure 2d], that encourages them to pay £25 for a subscription to The Telegraph, along with an offer of "*a free tote bag*." (also shown in [Figure 2e]). From the ads, forced sign-ups and pop-ups, this could create a barrier to accessing essential economic news. This will give users the impression that staying financially informed is only possible through costly subscriptions, which may hinder them from being financially literate.



Figure 2d: The Telegraph Webpage(The Telegraph, 2025)

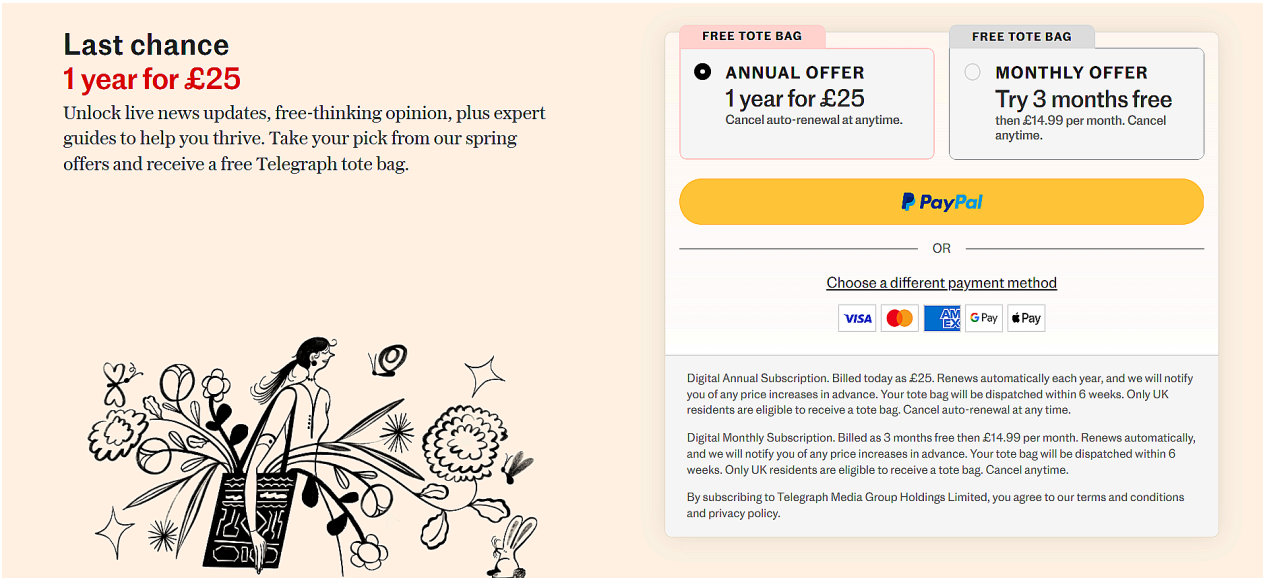
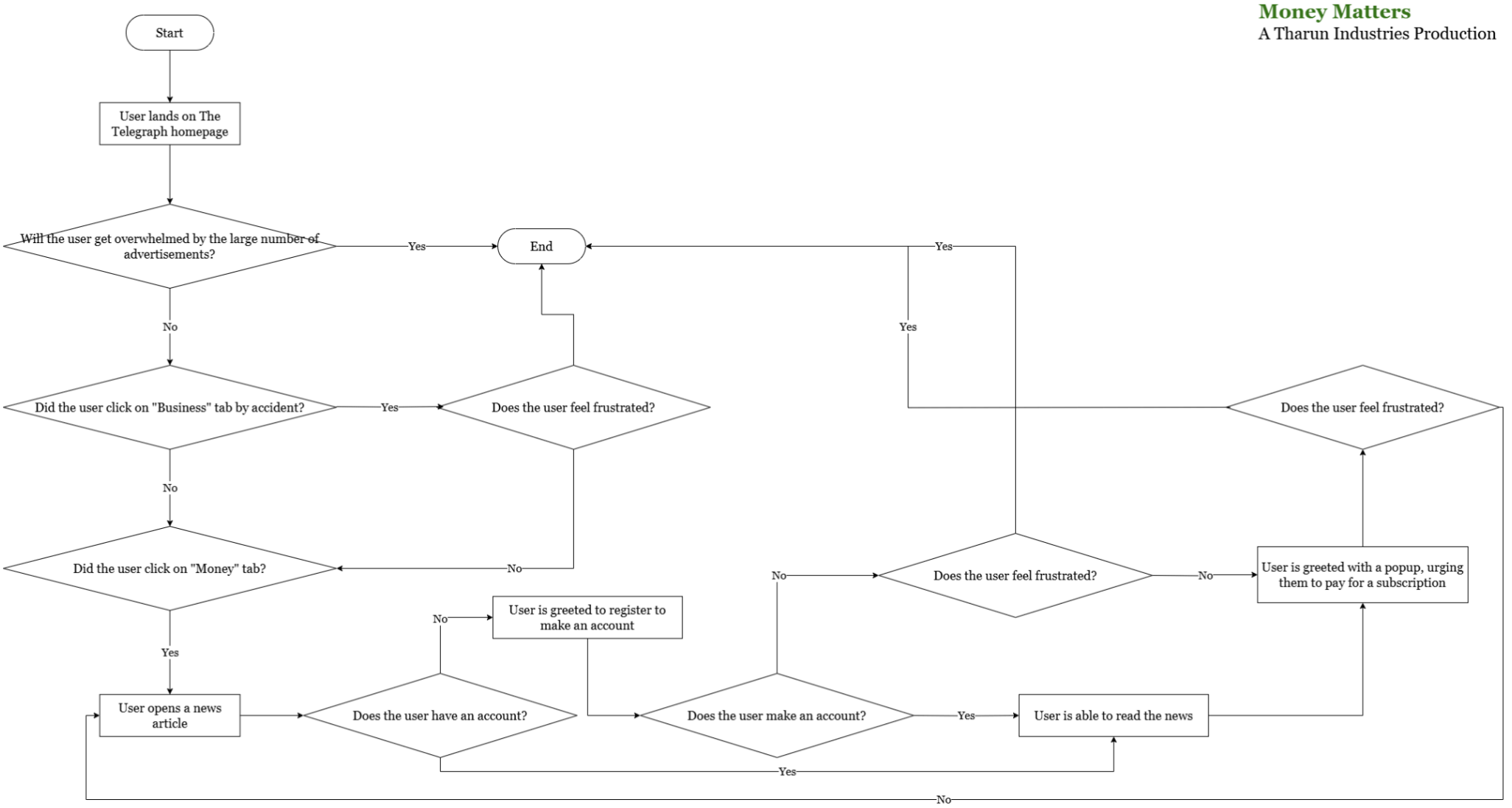


Figure 2e: The Telegraph Webpage(The Telegraph, 2025)

Flowchart of The Telegraph



The flowchart above shows how people use The Telegraph to find news about the economy. This highlights the parts of the system where users are most likely to give up, often because of how complex it is to navigate the site. It is evident that without creating an account, users can't really do much on the site. This often leads to frustration and causes them to leave, missing out on important updates about the economy.

Financial Times - The Paywall

1) When consumers visit the Financial Times website, they are met with a paywall that restricts access to detailed economic news [Figure 3]. While users can view article headlines, these offer little value to the public, as they lack the essential details needed to fully understand and assess changes in economic policies.

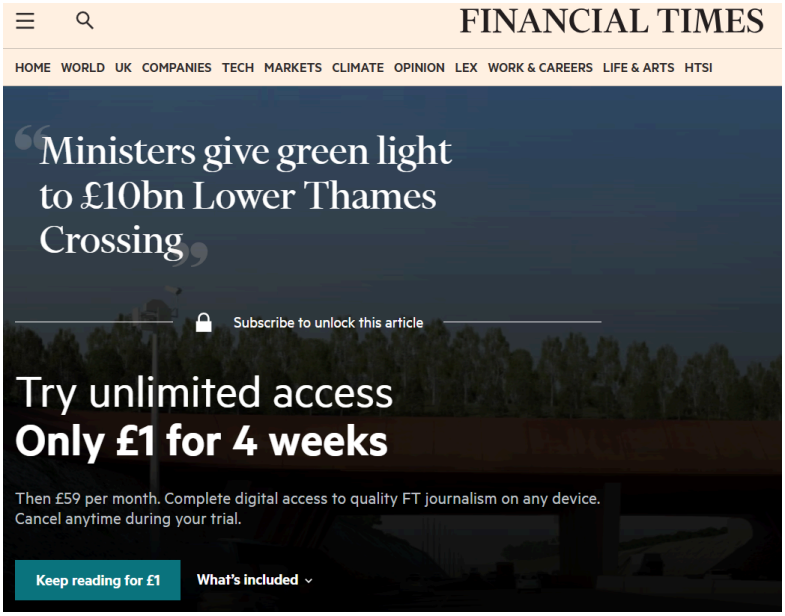


Figure 3: FINANCIAL TIMES Webpage(FINANCIAL TIMES, 2025)

2) Additionally, headlines may mislead the public which can exacerbate the economy, by inducing unnecessary panic or confidence to the consumer.

3) For example, the headline "*Ministers give green light to £10bn Lower Thames Crossing*" [Figure 3], does not provide the user enough context. Headlines lack the basic information, such as, how the lowering of the Thames Crossing can impact the public, or any challenges the Government may face during its construction.

Primary Objectives

After reviewing the current systems used to access economic news and analysing feedback from the surveys and my interview I conducted with Mr. Giles, I developed a set of objectives, providing me clear goals with deadlines, allowing me to track progress and make any adjustments when necessary.

As I will be building the NEA project using Spring Boot, I will be using a **Complex Business Model**, throughout the whole project, as I will be parsing data from multiple sources such as JSONs, Databases, XML Feeds and APIs.

Objective 1 - News Tab	
Specific	Collects news from multiple reputable news sites and uses artificial intelligence to summarise the content for users. ● Users are able to view: ○ Headlines ○ Description of the article ○ Published Date ○ News Source ○ Bias ○ Link to access full article ○ Image (If Applicable) ● As a stretch goal, I would like to implement an A.I. should be able to summarise the article and explain key economic changes and predict their impact on everyday life ● Users can to sort the news by date or by headline and can search news by headline
Technical Requirements	● Binary Search or similarly efficient searching algorithm ● Merge Sort or similarly efficient sorting algorithm ● Research how to create a web scraper that can be used for different news outlets ● List Operations ● File(s) organised for direct access ● Queue Operations (MultiThreading)
Achievable	● Research how to extract data from RSS .XML feeds ● There are many local AI models I can use to summarise news ● There are many videos explaining the process of multithreading.
Time-Bound	2 weeks

Objective 2 - Economic Figures & Diagrams	
Specific	Create a feature that retrieves real-time economic data from multiple APIs and generates diagrams (such as line charts or bar charts). ● A user could view the following statistics and figures, such as: ○ Real GDP ○ GDP per Capita ○ Inflation Rates (CPI) ○ Unemployment Rates ○ Global Exchange Rates ○ Interest Rates ● A user can view diagrams showing the changes in economic data. ● Users can use a currency converter to see how much their currency is in terms of another
Technical Requirements	● Calling parameterised Web service APIs and parsing JSON/XML to service a complex client-server model ● Complex mathematical models ● Files(s) organised for direct access
Achievable	● Research how to use World Bank API to extract macroeconomic indicators ● Research how to use FrankFurter API to extract data about exchange rate ● There are many guides on how to use Plotly (Javascript library) to generate graphs.
Time-Bound	2 weeks

Objective 3 - Economy Rater	
Specific	Develop a feature that allows users to rate their satisfaction with the economy based on macroeconomic indicators (e.g., unemployment rate, inflation, GDP) to provide feedback for others. ● Consumers have the ability to rate the economy by answering a series of questions based on these factors: ○ Value of assets owned ○ Inflation Rates ○ Unemployment ○ Salary ○ Interest Rates ● Other consumers are able to view other consumers' feedback, and based on this information, they are able to have a common understanding of the UK economy, which could show degrees of income inequality.
Technical Requirements	● Server-side scripting using request and response objects and server-side extensions for a complex client-server model ● Complex data model in database (eg several interlinked tables) ● Aggregate Functions ● Files(s) organised for direct access
Achievable	● Store user question, answers and the weight for each answer in a JSON file ● There are many guides on how I am able to extract data from JSON files
Time-Bound	1.5 weeks

Objective 4 - Investment Simulator	
Specific	Using an API, to retrieve live data of the stock market. Develop a feature where households and firms can invest a fake currency in real-life stocks. • Consumers and Producers are able to view diagrams showing changes in the stock price, with details describing the past trend of stock price, and the number of people investing in the stock. • These economic agents are able to invest into stocks that are based on real world data. • They can earn profit by selling their stocks, when the price of the stock is greater than the initial investment • They can also lose money if the price of the stocks falls below the initial investment. • Economic agents can analyse a stock's trend through its 10, 50, and 100-day Simple Moving Averages. • If economic agents lose all of their money, they can earn £1000.
Technical Requirements	• Calling parameterised Web service APIs and parsing JSON/XML to service a complex client-server model • Aggregate SQL functions • Complex data model in database (eg several interlinked tables) • Cross-table parameterised SQL • List Operations • Queue Operations (MultiThreading) • Advanced Matrix operations • Linear Regression
Achievable	• Research how to use Yfinance (Python library) in order to extract real-world stock data. • There are many videos on how to use Python Flask API so that I can parse JSON payloads from Python to Java and vice versa. • There are videos on how I am able to use pandas and numpy in Python to extract relevant data, and to implement Linear Regression
Time-Bound	3 weeks

Objective 5 - Budget Tracker	
Specific	Implement a budget tracker that enables consumers to input their daily/weekly and monthly budget and add their spendings. ● Consumers can add the following information: ○ How frequent they receive their budget ○ The size of the budget ○ The price of goods users purchase ○ The quantity of goods purchased ● After each day, the user is informed how sustainably they spend their budget ● Consumers are able to view graphs, to visualise a breakdown of their spending ● By using mathematical calculations, the consumer is given a graph to show their spending throughout the day, and based on current spending, it would predict future budgets. A graph should be made to visualise future budgets. ● Users can sort their spending based on date or cost and can search items they have spent. ● Users are able to view any risky purchases they have made, if the price of the good they purchase, doesn't typically follow the trend of the previous goods they bought
Technical Requirements	● Complex mathematical models ● Cross-table parameterised SQL ● Aggregate SQL functions ● Complex user-defined algorithms (eg optimisation, minimisation, scheduling, pattern matching) or equivalent difficulty ● Binary Search or similarly efficient searching algorithm ● Merge Sort or similarly efficient sorting algorithm ● Queue Operations (MultiThreading)
Achievable	● Research how to perform complex SQL Commands ● Research how to perform calculation with vectors and matrices ● There are many videos explaining the process of multithreading.
Time-Bound	1 week

Objective 6 - Costs of Production	
Specific	Add a feature that enables firms to create, read, update, and delete data related to their production costs. ● Producers can add information of factor inputs: ○ Type of factor input (Capital, Labour, Land) ○ A Job description ○ Name ○ Marginal Cost ○ Marginal Revenue ○ Marginal Profit ● Producers are able to view graphs, one graph showing how a change in costs leads to a change in output and another graph showing how a change in output leads to a change in revenue. ● By using mathematical calculations, the producer is able to gain advice on optimal production levels. ● Producers can use an undo button to undo actions, such as accidentally firing a worker, or updating workers data incorrectly.
Technical Requirements	● Complex mathematical models ● Cross-table parameterised SQL ● Aggregate SQL functions ● Complex user-defined algorithms (eg optimisation, minimisation, scheduling, pattern matching) or equivalent difficulty ● Stack Operations
Achievable	● Research how to perform a Combinatorial Subset Search ● There are many videos showing features of a stack
Time-Bound	2.5 weeks

Objective 7 - Login System	
Specific	Develop a secure login system that allows users to either sign in with existing credentials or register a new account using their personal information. ● For registering a new account, the user must enter their full name, date of birth, email, phone number and whether they are a consumer or producer ○ For each economic agent, email addresses should be unique. ○ Passwords must follow the following conditions: ■ More than 8 Characters ■ At least 1 Uppercase Character ■ At least 1 Lowercase Character ■ At least 1 Special Character ■ At least 1 Digit ○ Passwords must be stored in an encrypted form, using a Vernam Cipher. ○ User credentials should be stored in a SQL database, in the appropriate table depending on their role.
Technical Requirements	● Complex user-defined use of an OOP model, eg classes, inheritance, composition, polymorphism, interfaces ● Dynamic generation of objects based on complex user-defined use of OOP model ● Complex data model in database (eg several interlinked tables) ● Hash Tables
Achievable	● Research how to store the password in ciphertext in a database and store the vernam key in a separate JSON file
Time-Bound	2.5 weeks

Objective 8 - Friendly to use UI Design	
Specific	Create a website with a user-friendly UI/UX design that allows users to easily navigate and use the features from the previous objectives. ● Clarity ○ There should be a simple, clean layout with easy-to-understand language for any user. ○ Use of navigation menu ● Responsiveness ○ The website should load elements very quickly with smooth interactions. ● Flow ○ Important elements should stand out, logical flow, such as titles, headings, and sections being clearly differentiated, or having clear error messages ○ Use of buttons, text fields, tables, graphs. ● Simplicity ○ The design should have minimal clutter. ● Error Handling ○ This is able to prevent permanent mistakes, such as an undo feature, or showing error messages, when the user inputs invalid input.
Technical Requirements	● Research HTML, CSS and Javascript using W3Schools ● Server-side scripting using request and response objects and server-side extensions for a complex client-server model ● Complex client-server model
Achievable	● There are many videos on how to use HTML, CSS and Javascript.
Time-Bound	5 weeks

Objective 9 - User Dashboard	
Specific	• Users are able to learn and search common economic terms they may be unfamiliar with through a glossary, which will be stored as a collection of objects in a JSON file • Use Python to use Ollama AI to answer any user questions about the economy, where the JSON responses will be sent through a Python Flask API.
Technical Requirements	• File(s) organised for direct access • Complex data model in database (eg several interlinked tables) • Complex user-defined use of an OOP model, eg classes, inheritance, composition, polymorphism, interfaces • Records
Achievable	• Copy and paste words and definitions from the ONS and store them in a JSON file. • There are many local AI models I can use to summarise news
Time-Bound	1.5 weeks

End-Users

There are a handful of users that will use the system. These would range from consumers and producers. I decided to split the “public” into these specific categories, as different economic agents have different incentives to fulfill.

The target audience are as follows:

Users	Description
Young Adults* (Consumers)	Consumers are able to navigate current and predicted economic statistics and understand the changes in economic news, without any difficulty, improving quality of life. Consumers should also be advised how to spend their income with the aid of complex algorithms that analyse the changes in the economy. Furthermore, they can use the investment simulator to learn how the market works and practice without risk.
Firms (Producers)	Producers are also able to navigate current and predicted economic statistics and understand the changes in economic news, without any difficulty. They can track their costs of production and revenues with complex algorithms deciding how to minimise their costs of production. They can be informed how to set prices of goods and services to maximise profit and utility for the consumer.

***** I will assume that the primary consumer will be young adults, as they are more lenient to use technology, compared to the older generations.

However, for simplicity, the end users for consumers would be 6th Formers at William Perkin CofE High School as they are in need to be more financially literate, especially as they are nearly going to be adults. The end user for producers would be smaller firms, aiming to minimise their costs and maximise output and profit.

User Needs

The user role matrix below outlines the program requirements for each type of user.

	Consumer	**Producer**
View News Tab	X	X
View current and Economic Diagrams and Statistics	X	X
Use Consumer Rating System	X	
View Consumer Rating System	X	
Investment Simulator	X	X
Budget Tracker	X	
Viewing costs of production		X

Limitations of the System

As I don' t have all the resources available to make this project perfect, I have made some acceptable limitations. While these limitations do restrict my ability to accurately predict and simulate the economy, they will not impair the overall functionality to the system. The limitations are as follows:

- Complexity of Financial Models
    - As I don't have a university level of economic theory and mathematical ability, developing highly advanced financial models may be challenging for someone without a solid foundation in finance and algorithms.

- Realistic Simulation of Financial Markets
    - It may be difficult to simulate the unpredictable nature of markets, due the complexity of human psychology and external factors, so I won't be implementing a system that can predict any changes in economic macroindicators or stock data.

- Limited Hardware Components
    - As my laptop has 8GB of RAM, 4GB of GPU and Intel Core i5-1035G1 @ 1.00 GHz, some processes of my system take longer to load in, which makes the user wait for long periods of time to get responses. For example, retrieving stock data or A.I. responses using Flask API to send payloads from Python to Java may take a while. I will also be limited in the choice of A.I. models I can use, as I can only use smaller and weaker models, such as 1B or 3B parameter models, instead of 7B or 20B parameter models, so the A.I. may not generate accurate answers.

Proposed System

In order to create a proper system to fulfill the users' requirements, I have created a few diagrams such as flowcharts, to visualise and simplify processes and ensure each procedure is able to communicate with each other without conflicts and ERDs to model databases by clarifying how the tables are linked together. I have also created a few prototypes, to show a proof of concept.

Mathematical Formulae

As I do not have a university level of understanding with economics, I researched formulae that can perform the following tasks:

- Advising users for sustainable budgeting
- Advising firms for minimising costs of production, while receiving maximum profit
- Deciding whether user's purchases are deemed to be risky

Cobb-Douglas Production Function

- The Cobb-Douglas Production Function models the relationship between output and factor inputs. This will be useful to show how firms contribute to economic growth.

$$Y = A \times K^{\alpha} \times L^{\beta}$$

- Y = total output (e.g. GDP)
- A = total factor productivity (TFP), a constant
- K = input of capital
- L = input of labour
- α, β = output elasticities of capital and labour, respectively (i.e. how much output changes when input changes). These will be random.

Exponential Modelling

For the budget tracker, I aim to model a realistic projection of how consumers' budgets will grow over time, rather than assuming it stays fixed. By understanding that a consumer's budget can grow exponentially, which is based on the current inflation rate, the model produces a clear 12-month forecast that can support better planning for spending, saving, and setting financial goals.

To calculate this, I have used vectors and matrices to make it efficient to calculate multiple income components across many months at once.

Let r_m be monthly growth rate:

$$r_m = (1 + r)^{\frac{1}{12}} - 1$$

Let m be the months as a column vector:

$$m = \begin{bmatrix} 0 \\ 1 \\ 2 \\ \vdots \\ 11 \end{bmatrix}$$

Let G be the growth factor:

$$G = \left(1 + r_m\right)^m = \begin{bmatrix} \left(1 + r_m\right)^0 \\ \left(1 + r_m\right)^1 \\ \left(1 + r_m\right)^2 \\ \vdots \\ \left(1 + r_m\right)^{11} \end{bmatrix}$$

Let F be the future budgets
Let I be the budget components (Essentials, Leisure and Savings):

$$F = G \times I$$

<u>Optimising Cost Of Production</u>

I aim to create a system that finds the best combination of factor inputs that maximises profit under a cost constraint. To simulate this I have researched powerset generation, which finds all possible subsets, and combinatorics. It first generates all non-empty combinations of the factor inputs, where each factor input can either be included or excluded from a particular subset. The code will iterate through all possible subsets, select the subsets that satisfy the maximum cost constraint, and choose the subset that maximises total profit.

Outlier Detection

By applying my knowledge in A-Level Maths Statistics, I aim to analyse consumer purchase data and detect outliers. It first groups purchases by item, then calculates the mean and standard deviation of purchase totals for each item. The mean represents the central value of prices, while the standard deviation shows how much prices fluctuate around the mean (volatility). Using these values, the code computes the z-score for each purchase, which measures how many standard deviations a purchase deviates from the mean. Purchases with a z-score exceeding a threshold, where the absolute value of z > 1.5, are flagged as outliers. By detecting outliers, consumers are able to realise unusual spending patterns in the data.

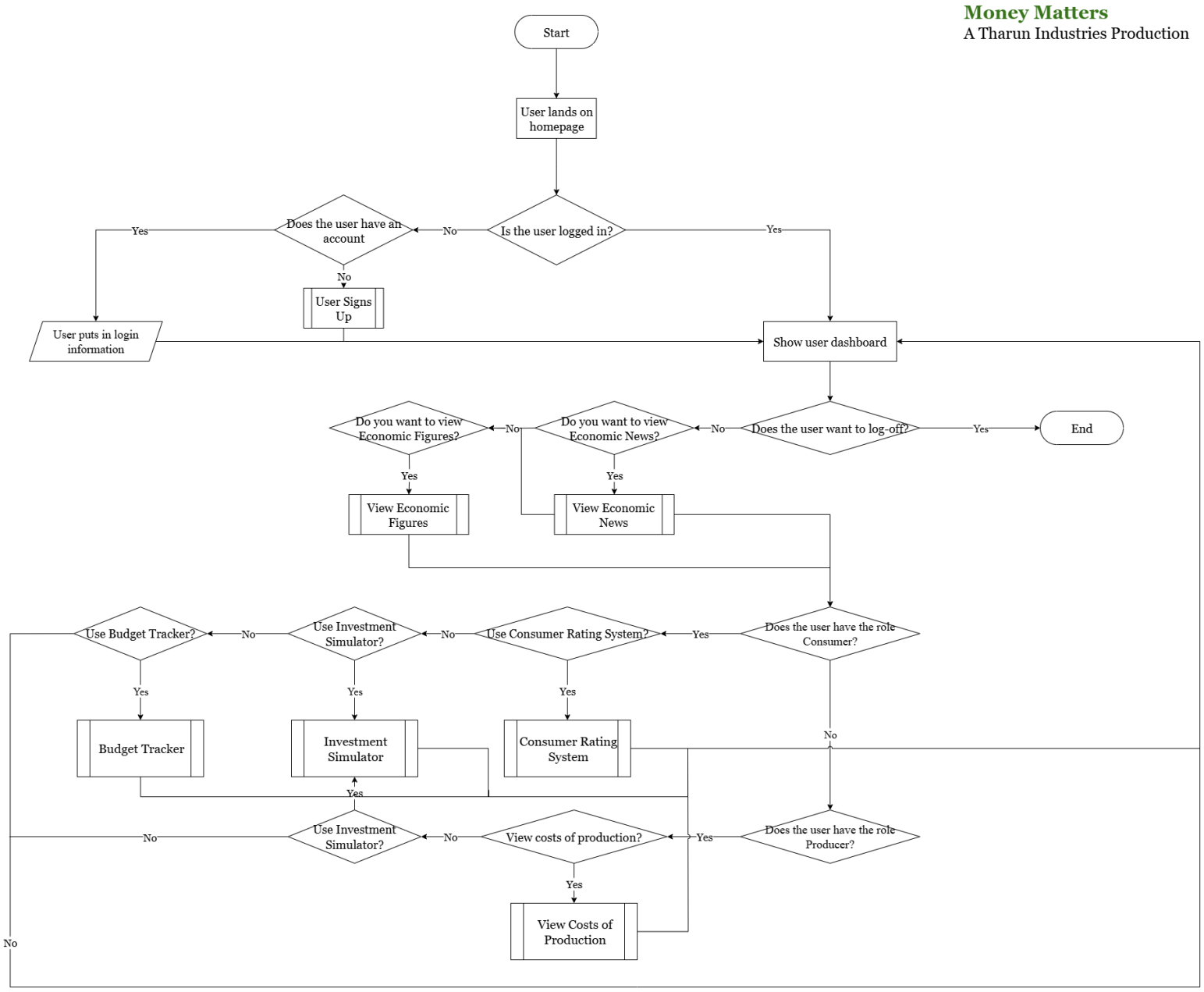
$$mean = \frac{\sum_{r=1}^{n} x_r}{n}$$

$$variance = \frac{\sum_{r=i}^{n} (x_r - mean)^2}{n}$$

$$std\ dev = \sqrt{variance}$$

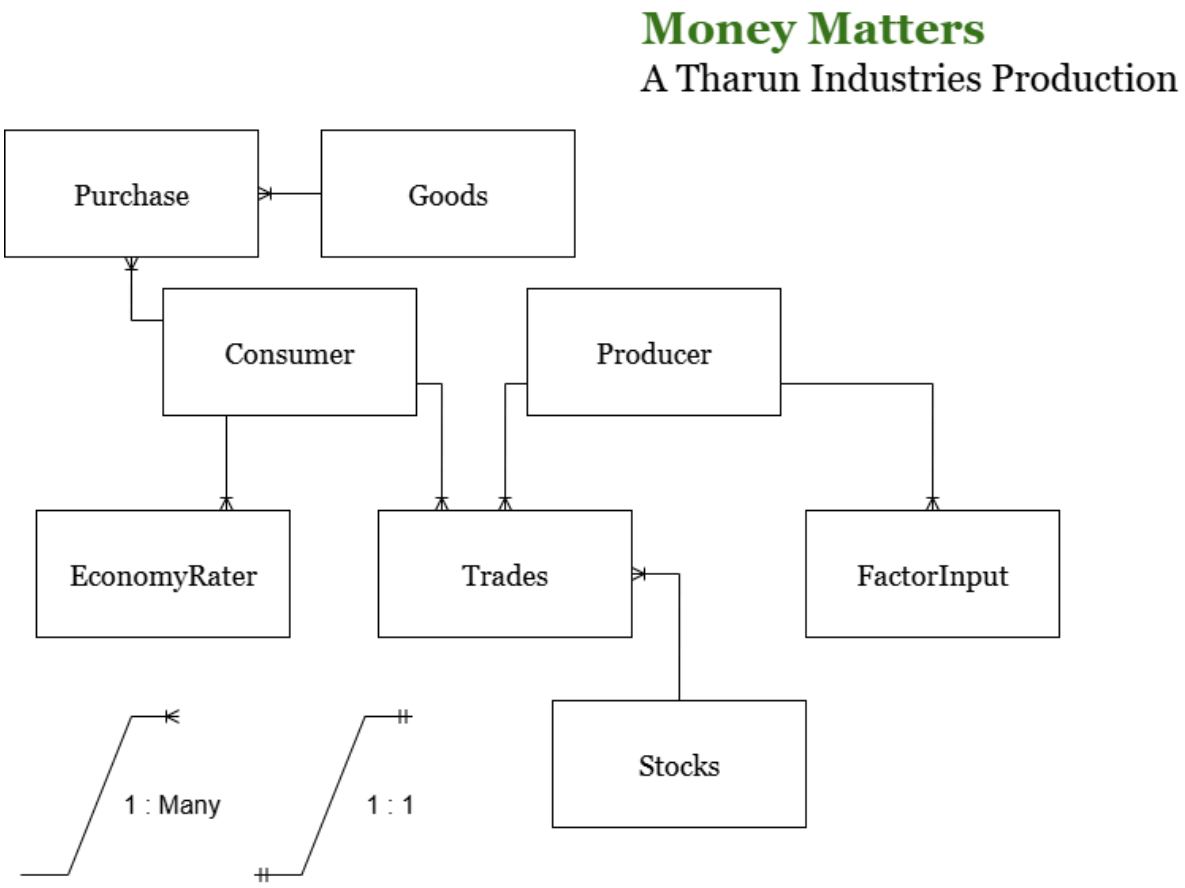
$$z = \frac{x_r - mean}{std\ dev}$$

Flowchart



This flowchart shows the proposed system. It shows the possible features users are able to use, which are split up into consumer and producer features. Each subroutine (rectangle with 2 vertical lines) are further explored in the **Refined Flowcharts and Data Flow Diagrams of the Proposed System**.

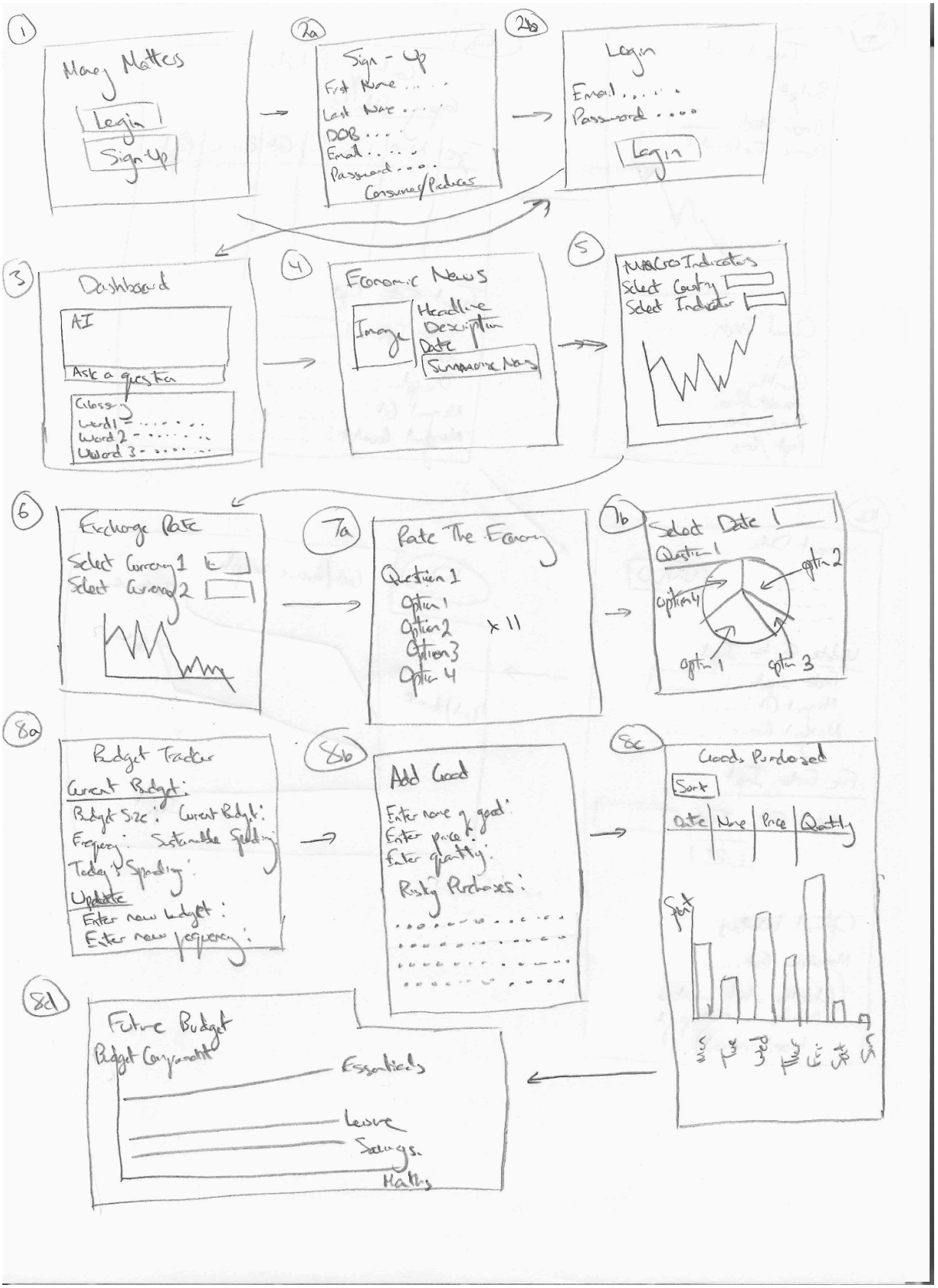
Entity Relationship Diagram

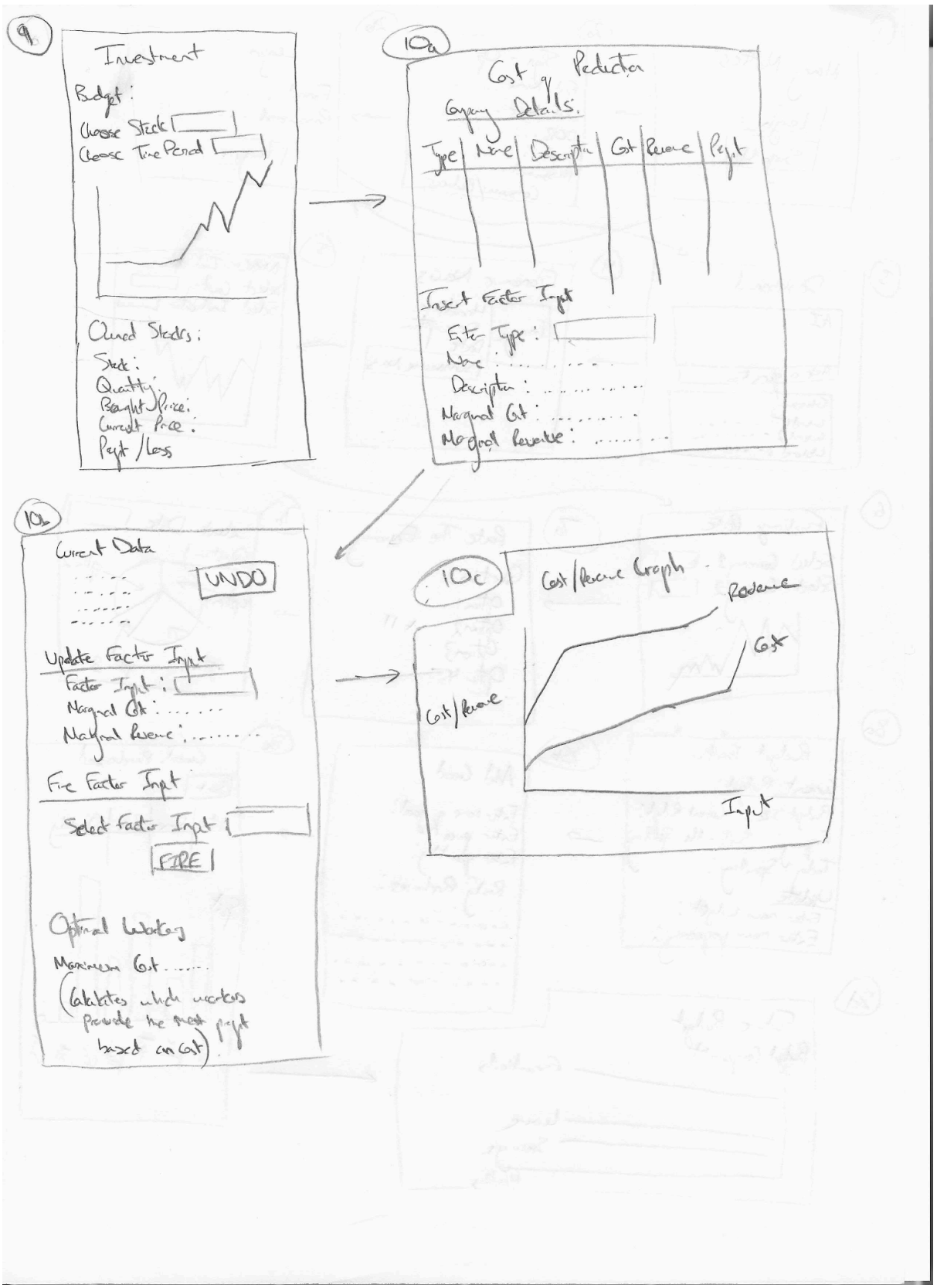


The diagram above shows an Entity Relationship Diagram (ERD) of the Proposed System showing the one to one and one to many relationships of how each table is linked to one another. This ERD Diagram will be further developed **Database Design & Normalisation**.

Hand Drawn UI

My first UI designs were drawn on paper, to visualise how each objective clearly links to each other.




9
Investment
Budget:
Choose Stock
Choose Time Period
Owned Stocks:
Stock:
Quantity:
Bought Price:
Current Price:
Prof / Loss
10a
Cost & Reduction
Company Details:
Type
Name
Description
Cost
Revenue
Profit
Insert Factor Input
Enter Type:
Name:
Description:
Marginal Cost:
Marginal Revenue:
10b
Current Data
UNDO
Update Factor Input
Factor Input:
Marginal Cost:
Marginal Revenue:
Fire Factor Input
Select Factor Input
FIRE
Optimal Workers
Maximum Cost:
(Calculates which workers provide the most profit based on cost)
10c
Cost/Revenue Graph
Revenue
Cost
Cost/Revenue
Input

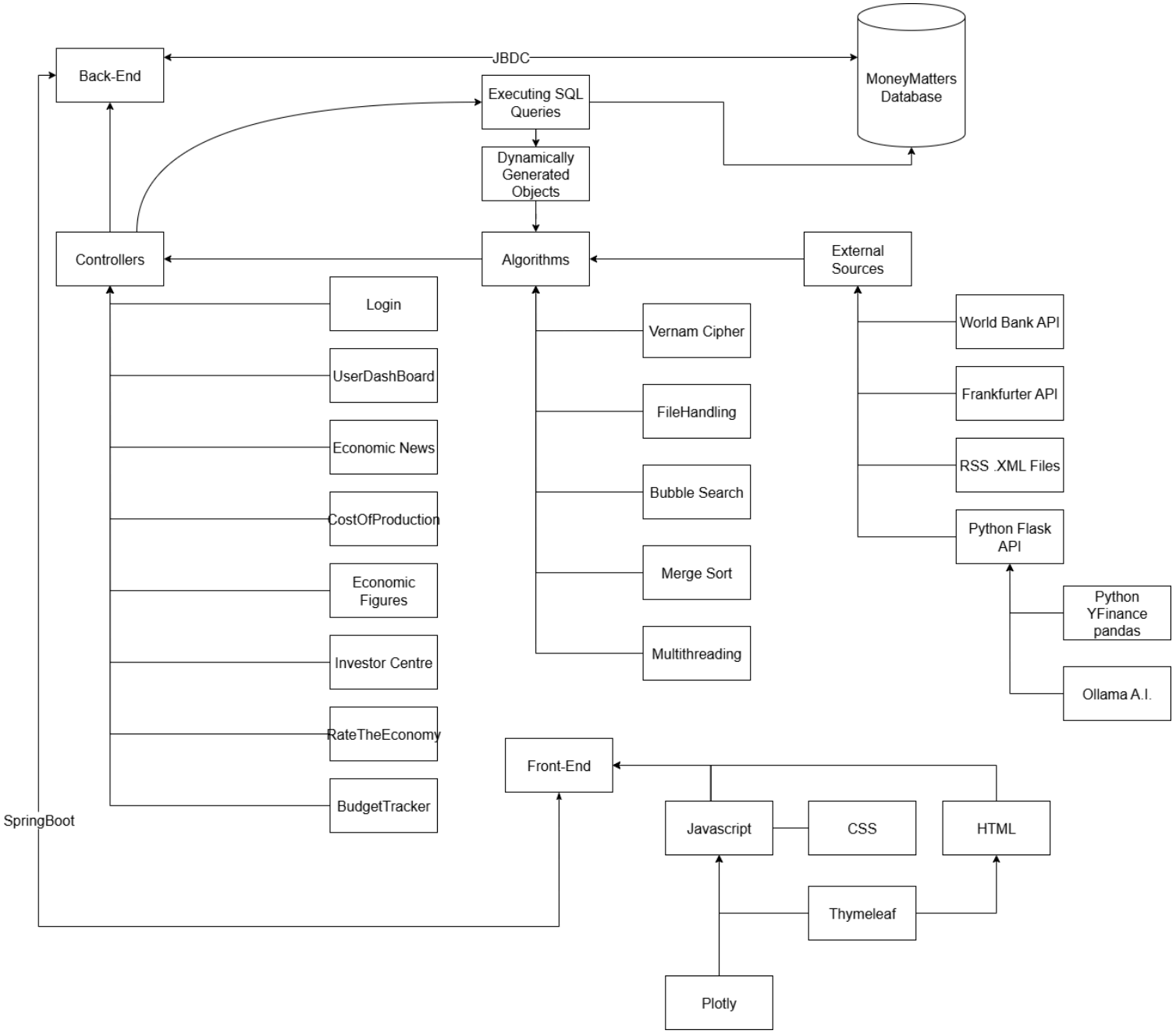
After discussing with my end user (Mr Giles), this is the following feedback.

“I find the early UI design very promising. The flow from login to dashboard and through the various sections, like economic news, macro indicators, and exchange rates, is very clear and accessible for new users. I like how the interactive features such as economy ratings and charts make it easier for students to engage with economic data.

However, the design could benefit from clearer labeling and simpler visual hierarchies in some areas. For Instance, the ‘Rate the Economy’ and ‘Add Good’ sections feel quite text-heavy and might overwhelm new users. The graphs are a strong feature, but it would be helpful to ensure that the data sources and units are visible.

I also like the tools, such as, the budget tracker and investment section as they make the experience more practical and relatable for students learning how to manage their finances. Overall, the UI designs effectively support new users to have a better understanding of the economy, but could be improved by making the presentation of information clearer. By simplifying how data and tools are being displayed, people can interpret graphs, understand the relationships between indicators, and draw meaningful conclusions about the economy.”

Hierarchy Chart



The Hierarchy Chart shows how different elements of the proposed system will interact. The backend systems will handle all the logic, controllers, repositories, file handling and more. The front end systems will be tasked to present details to users, and handle input validation as well.

Documented Design

Functionality of Proposed System

I am going to use Java, Python, HTML, CSS, Javascript to create my project, where data will be stored in either a text file or an SQL Database. I will also use a server so that clients are able to access news and other features without any difficulty.

The new system is a website that will allow the public to access news about the economy they are currently unsure about, so that they can properly understand economic changes that impact the world. It will allow all users to read up to date economic news, without any difficulty, look up recent economic statistics and figures. Additionally, those with the consumer role are able to rate the current state of the economy, which can indicate to others the potential hardships they may face, learn how to invest their income in order to maximise capital gain and are able to effectively manage their spending using the budget tracker. Producers are also able to use the investment simulator and can view their costs of production, giving advice on how to minimise costs and improve the production process.

- Login System (Objective 7)
    - Existing users can log into their accounts.
    - First time users will need to register for an account, by providing their information.
    - Passwords are stored encrypted, through the Vernam cipher, where the password as cipher text and Vernam key gets changed every time the user logs in. The vernam key will be stored separately in a JSON file rather in the database.

- User Dashboard (Objective 9)
    - Access their profile, and can edit their information, and change their preferences, which also means updating relevant information in the relevant SQL table.
    - Users can change preferences like their favorite news sources, or ways to visualise data (tables or graphs).
    - Familiarise themselves with common economic terminology that they may not properly understand using a glossary.
    - Use Python to integrate A.I. to explain economic terms and answer user questions.

- News Tab (Objective 1)
    - Implementing a web scraper that collects news from multiple reputable sources and using A.I. to provide a summary of the whole article to the user. The metadata of the articles will be stored in a JSON file.
    - The A.I. should also explain key economic changes and predict their impact on everyday life.
    - Users can sort articles by publish date and by headline names
    - Users can search for specific economic articles.
    - Enable multithreading, through using threads and queues, so multiple Python Flask API calls can run concurrently, meaning that the system is able to summarise multiple news articles at the same time.

- Economic Statistics and Figures (Objective 2)
    - The system should retrieve real-time economic data from multiple APIs and present the changes of current economic statistics, using visual diagrams such as line graphs, using the Plotly library.
    - Each statistic includes a brief description explaining what it represents.
    - Using mathematical algorithms, the user should be able to view predictive trends, which forecasts changes to the economy.
    - Users can search and filter economic statistics

- Economy Rater (Objective 3)
    - Users can rate their satisfaction with the economy by answering questions related to macroeconomic indicators.
    - Using a mathematical algorithm, consumer responses are converted into a number which is representative of their opinion on the economy.
    - The average of the responses can be viewed by all consumers, which gives them a rough estimate of how they think of the economy.
    - Consumers can view other consumers' feedback, while being anonymous, to have a general sense of how the public views the UK economy, which are displayed using pie charts.

- Investment Simulator (Objective 4)
    - Using Python's yfinance library and pandas, which manipulates and fetches relevant real-time stock data, so consumers and producers can invest.
    - Use Linear Regression to calculate generic trends of the stocks short term and long terms simple moving averages
    - Use Python Flask API to send JSON payloads from the information about stocks in Python to Java.
    - Enable multithreading, through using threads and queues, so multiple Python Flask API calls can run concurrently, reducing the overall time to fetch stock data.

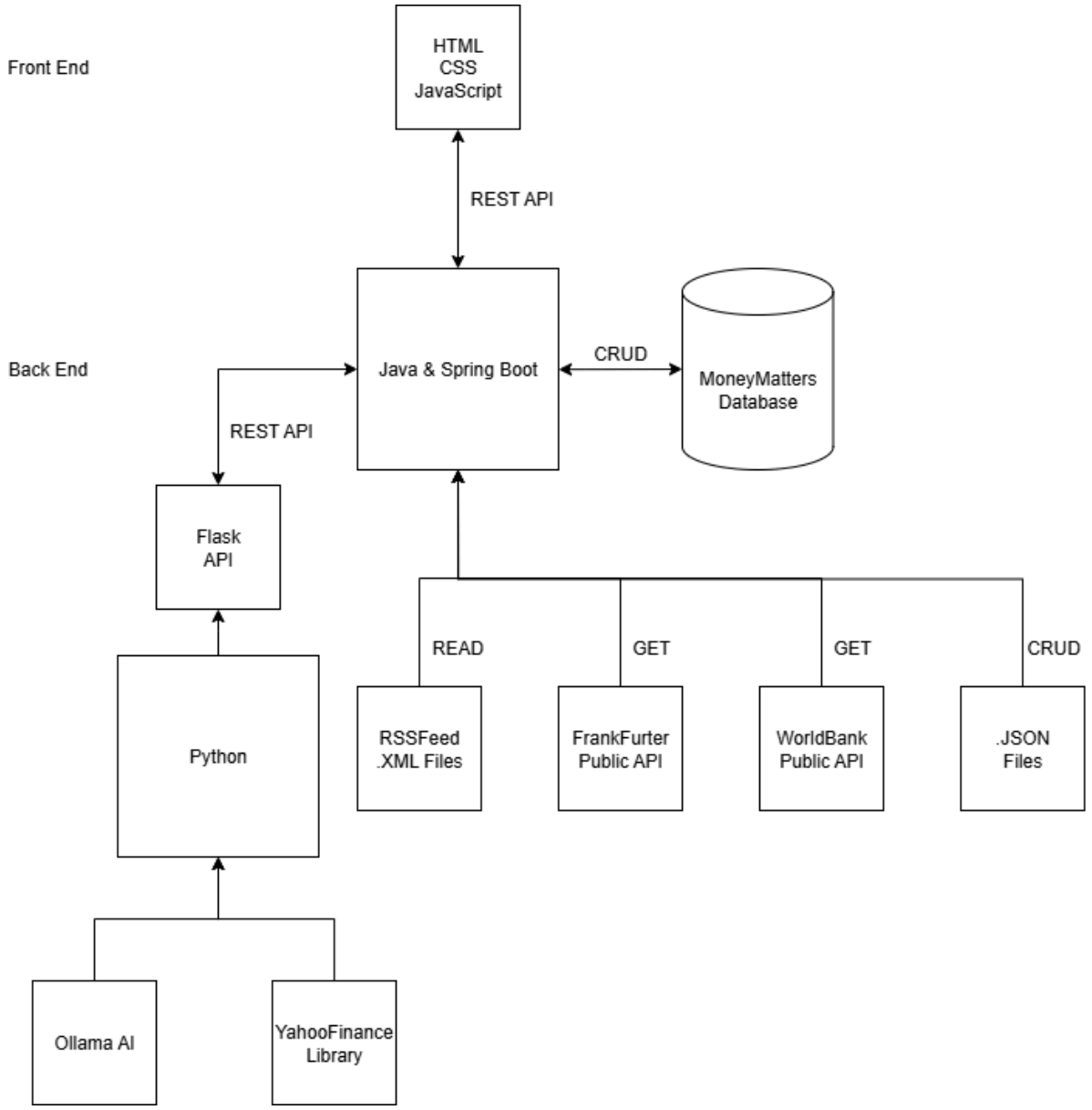
    - Through the Plotly library, users can view interactive charts showing stock price trends and information about how many people are investing in each stock.
    - Users can earn profit by selling when prices rise above their ask price or make a loss on their investments when they sell below the ask price, leading to relevant updates to the Investment Table.

- Budget Tracker (Objective 5)
    - Using exponential modelling and matrix calculations, model consumers' budgets as growing with inflation rather than remaining fixed, producing a realistic 12-month forecast. This is then projected to a graph.
    - If the user renews their budget monthly, users can receive advice on how they can sustainably spend based, helping them reach their financial goals.
    - Analyse consumer purchase data using z-scores to detect outliers in purchases, by grouping purchases by item. Then calculate the mean and standard deviation, and identifying cases where $|z| > 1.5$—so consumers can spot unusual spending patterns.
    - Using sorting algorithms, users can view their spending by date, cost and quantity bought.

- Viewing Costs of Production (Objective 6)
    - Producers can create, read, update, and delete data about their factor inputs, which leads to relevant changes to the FactorInput Table.
    - Implement a stack that allows producers to undo the last few changes they made when modifying their costs of production.
    - Producers can view two key graphs: one illustrating how changes in output affect costs, and another showing how output changes impact revenue.
    - Using non-linear transformations, producers should be able to view their impact on the GDP, through the Cobb-Douglas production function, with random α values between 0.3–0.4 and β values 0.6-0.7 to maintain constant returns to scale.
    - By generating non-empty combinations of factor inputs using powerset generation, I aim to calculate the best factor inputs that bring the most profit, under a cost constraint, solving a linear programming problem.
    - Producers can sort their costs of production based on date or cost and can search individual factor inputs.

- Technical Requirements:
    - OOP model, using classes, inheritance, composition, polymorphism and interfaces

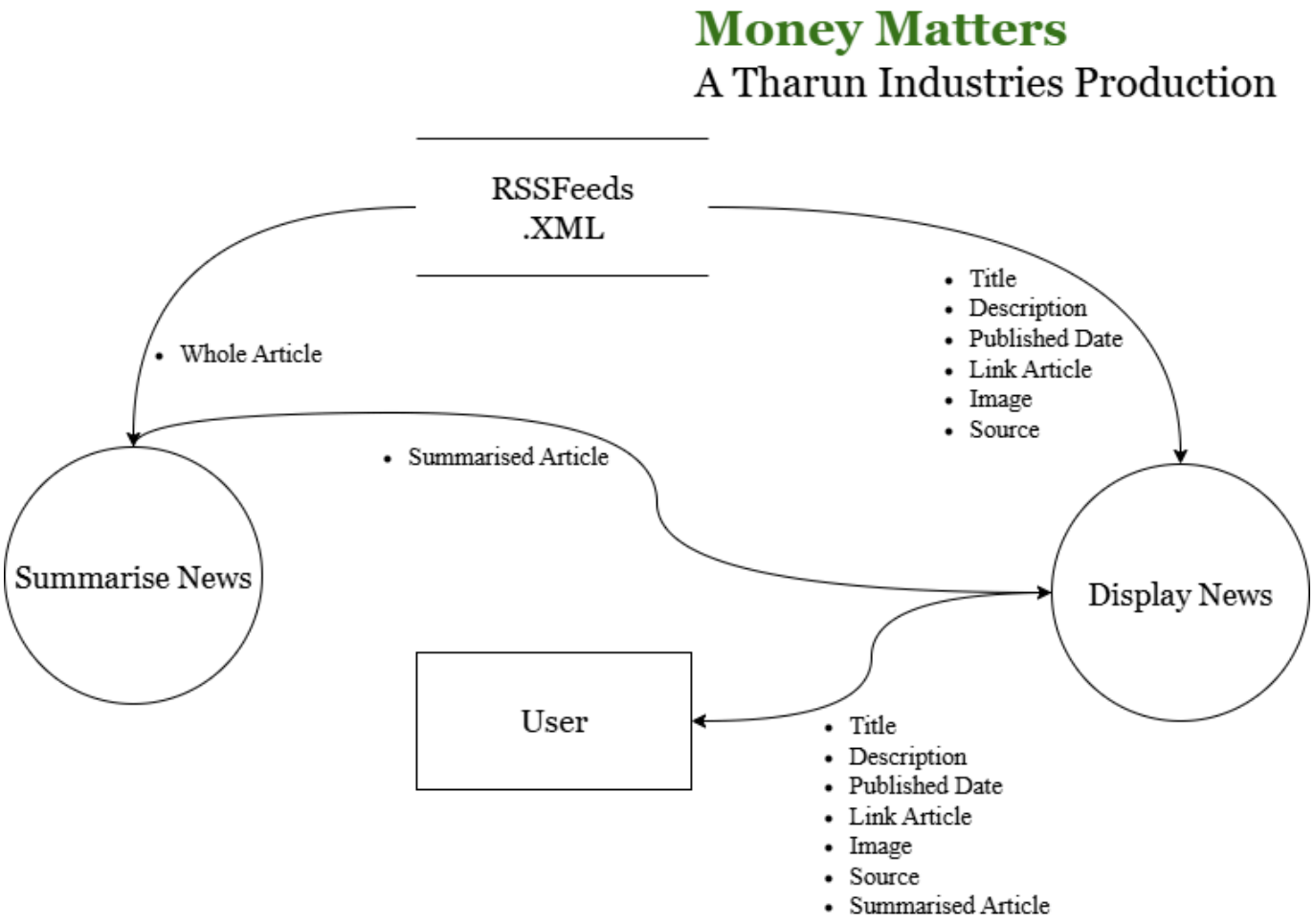
- Encryption, such as the use of the Vernam Cipher, to ensure that user data is stored confidentially.
- In Python, use the Ollama library to implement an A.I. tool that answers any economic question users may have.
- Set up a Python Flask API to output data as JSON payloads, which can then be processed in Java.
- Create an efficient sorting and searching algorithms, to aid users to retrieve relevant data
    - Recursive Merge Sort Algorithm
    - Recursive Binary Search Algorithm



Refined Flowcharts and Data Flow Diagrams of the Proposed System

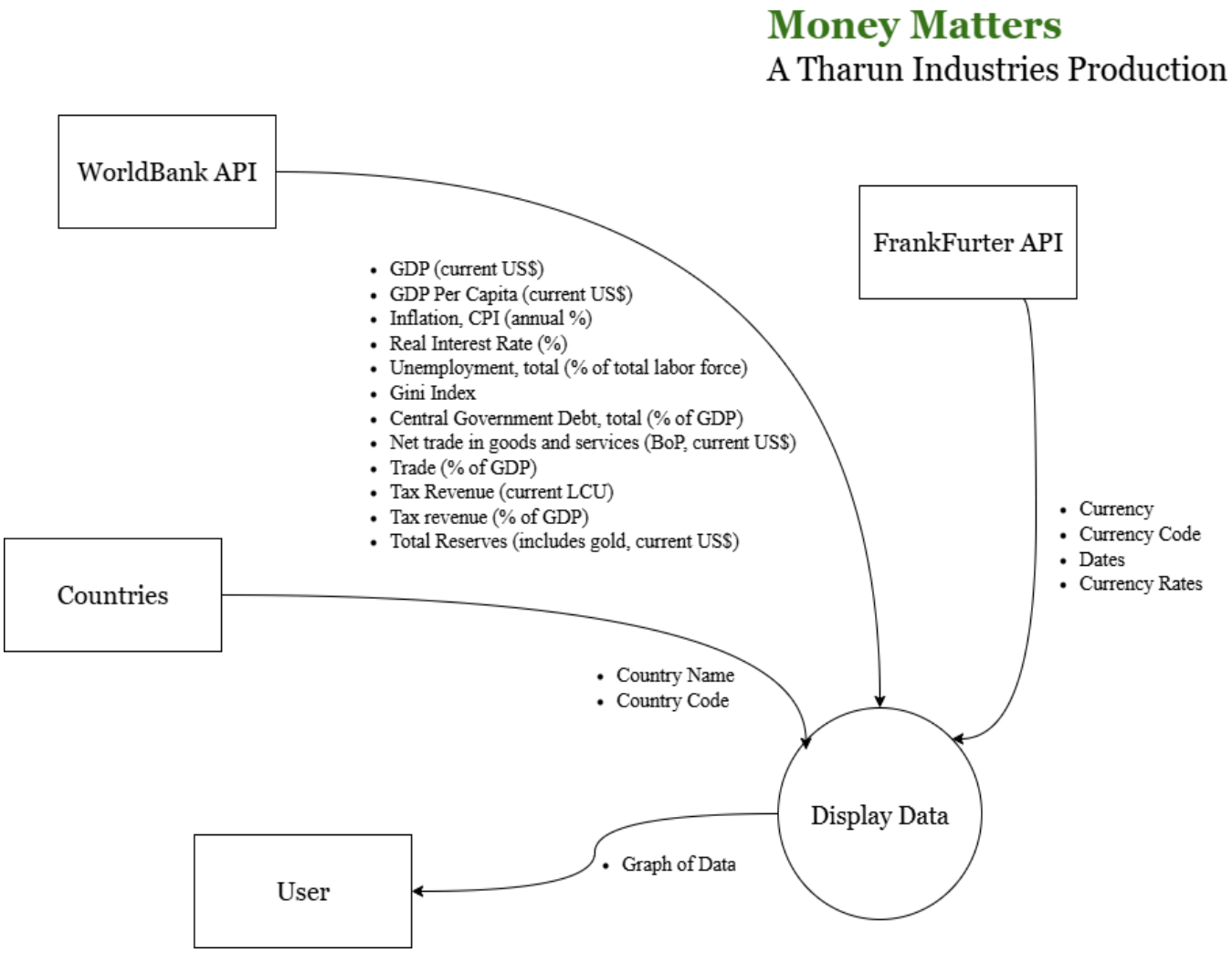
In order to illustrate how I would successfully implement my objectives, I have either created more detailed flow charts or data flow diagrams to highlight how each of my objectives work. These diagrams have been created through abstraction to remove the complexities of certain one-off issues and are built off from my original diagram I created in the Analysis Section.

Objective 1



By extracting key details, such as the title, description, and image, from RSS XML feeds of various news sources, I can present essential economic news in a clear format. Each article is then processed using Ollama AI to generate a summary, allowing users to quickly grasp the main points and stay informed. The summary will be processed via Python, and using a FlaskAPI I will send the data from Python to my main Java project.

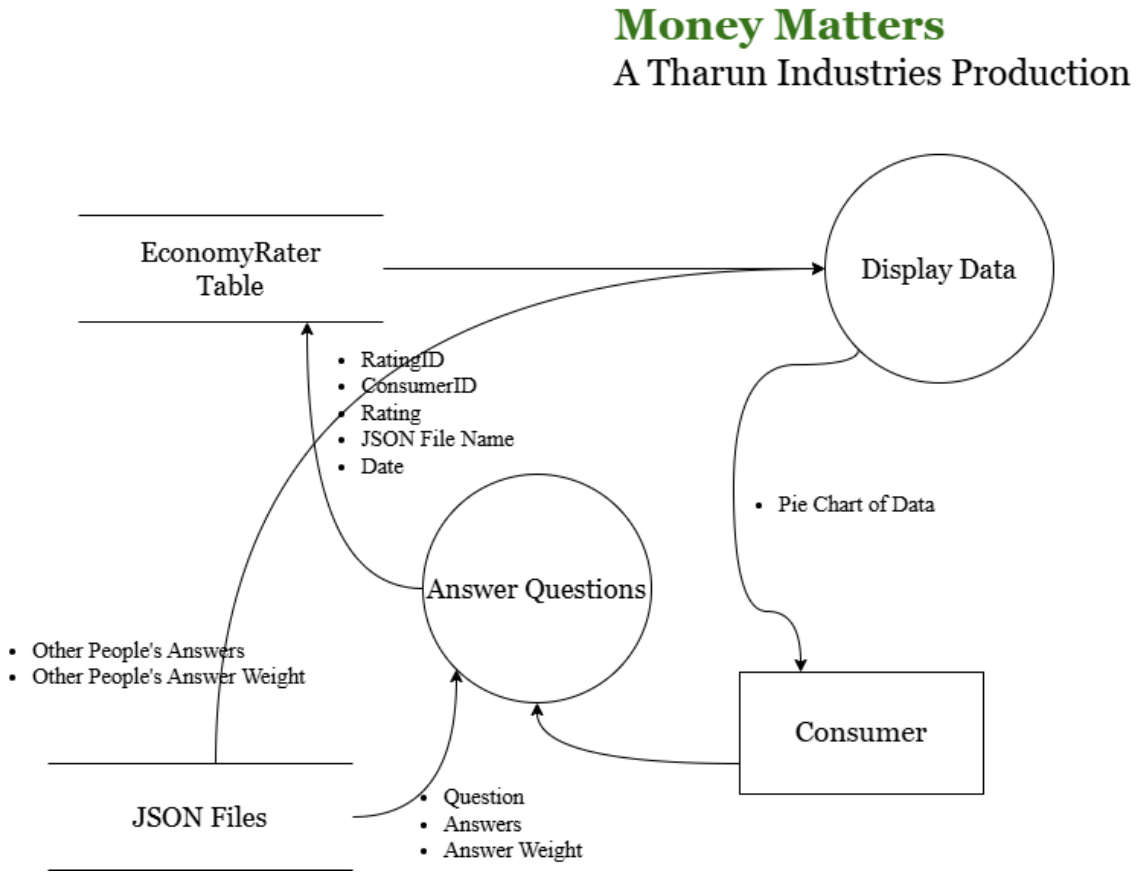
Objective 2



By using country and indicator codes from the World Bank API through JSON payloads, which provides a value based on the indicator, users can view graphs that display trends for various macroeconomic indicators.

Also, by using currency codes and dates from the Frankfurter API through JSON payloads, which provides a currency value, users can view graphs to compare multiple currencies over time. There is a currency converter that allows users to calculate the value of one currency relative to another.

Objective 3



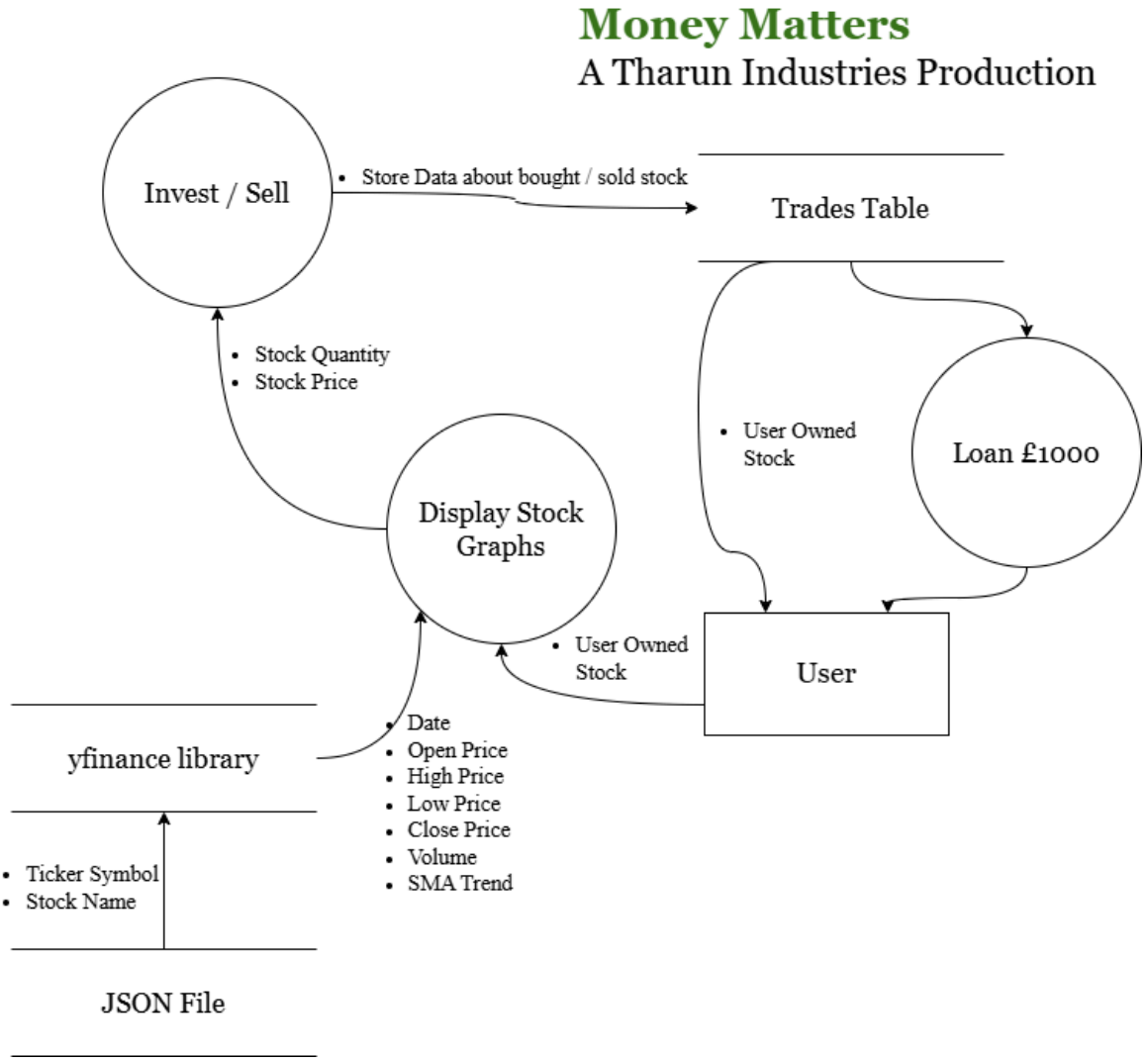
I will create a JSON file containing a list of objects, where each object includes a question along with a dictionary of possible answers and their corresponding weights (1 to 5). A weight of 1 indicates that the economy is failing, while a weight of 5 indicates that the economy is thriving.

These questions and multiple-choice answers will be presented to the user, where the user selects one option per question. After all questions are answered, the selected questions, answers, and weights are stored in a JSON file. This file is saved in a folder named after the date the user answers the questions, and the JSON file is named "userOpinion" + consumerID.

The consumer's overall rating (calculated as the sum of their selected weights as a percentage of the total possible weights) along with the submission date and the path to their JSON file, will be recorded in the EconomyRater Table.

By accessing the JSON files of users for a specific date, the system can generate pie charts to see society's collective opinion of the economy, displaying a percentage breakdown of each response option.

Objective 4

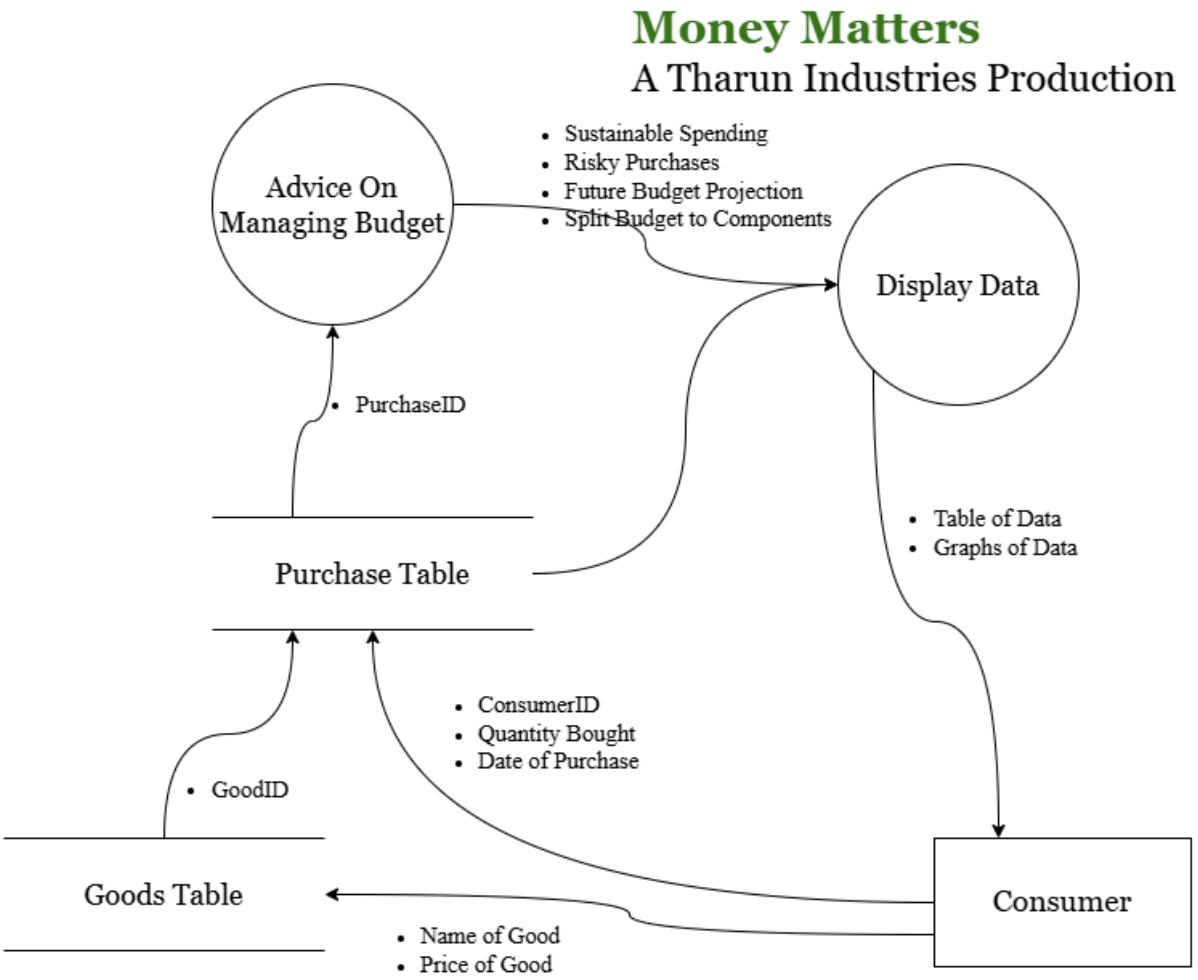


Using Python's yfinance library, I will extract key stock data such as open and close prices, and volume. I will use Linear Regression to identify trends and determine the gradients of both short-term and long-term Simple Moving Averages, where the SMA represents the average stock price over a specific number of days, providing insight into stock prices over time.

A JSON file will store 30 stock names along with their corresponding ticker symbols. The Flask API will be used to transmit this data from Python to the main Java project. Users will be able to search for specific stocks listed in the JSON and view their performance through graphs.

Also, users can buy or sell stocks based on the current ask price. Each transaction updates the Trade Table, recording whether a user bought or sold a stock, at what price, and enabling them to track profits or losses. If a user becomes bankrupt and holds no remaining stocks, they will automatically be loaned with £1000 to continue trading.

Objective 5



First-time users will need to input and update their budget information, including their total budget amount and the frequency of budget renewal. Each user's budget is divided into three categories: Essentials (45%), Leisure (30%), and Savings (25%).

If the budget renewal frequency is set to "Monthly," users can view their sustainable daily spending limit, calculated by dividing the total essential expenses evenly across 30 days.

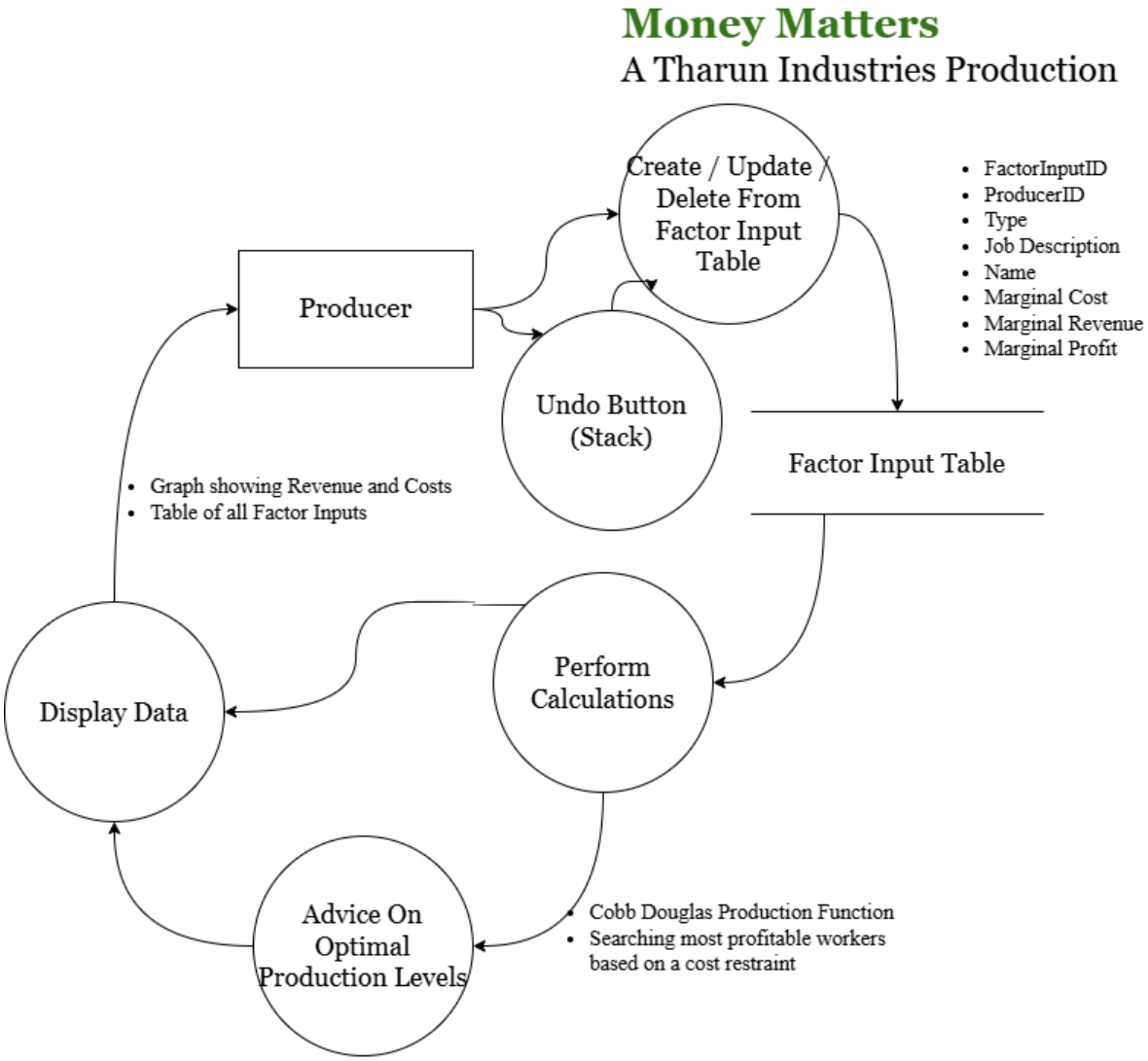
Users can purchase goods, with each item's name and price stored in the Goods Table. The Purchase Table records additional details, including the date of purchase, quantity purchased, and goodID.

The system displays multiple tables and graphs:

- A graph showing the future budget projections based on the current inflation rate
- A sortable table displaying all purchased goods. (Table can be sorted by the price and the quantity bought)
- A bar chart showing daily spending throughout the week.

Additionally, users can identify any risky purchases, where risky goods are priced at 1.5 times greater than the standard deviation of that item's typical price range.

Objective 6

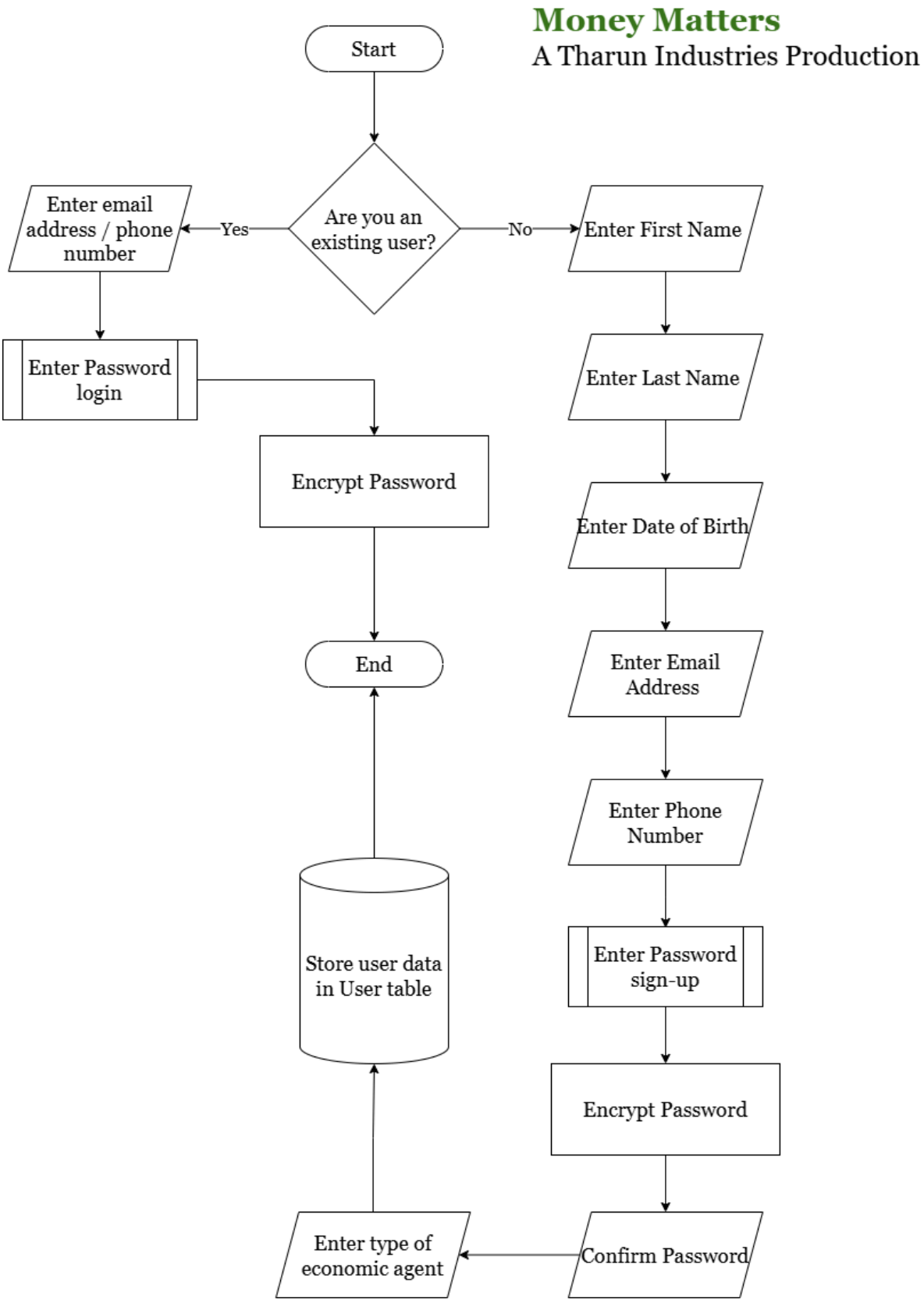


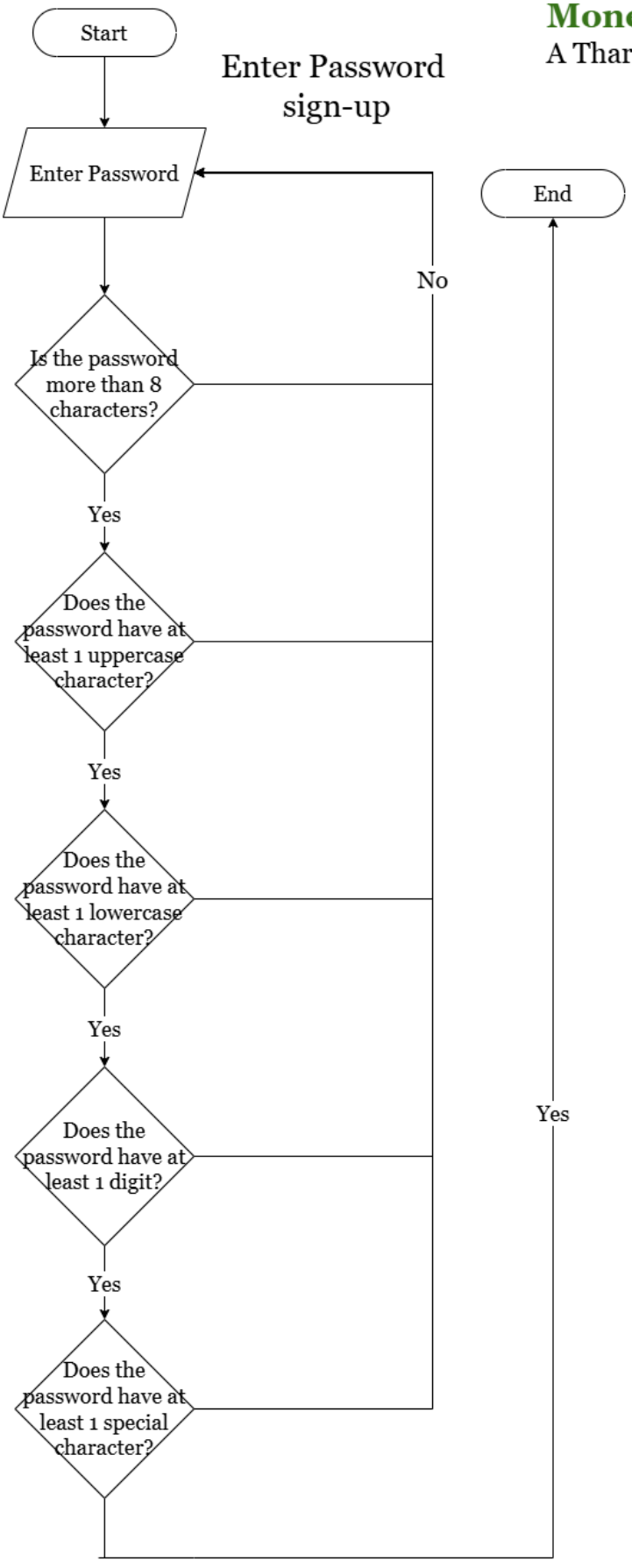
Producers can hire, update, and fire factor inputs, with all changes reflected in the database. After entering their factor inputs, producers can perform calculations, such as the Cobb-Douglas Production Function, to determine the additional GDP generated by a firm, and identify the most profitable workers within their cost constraints.

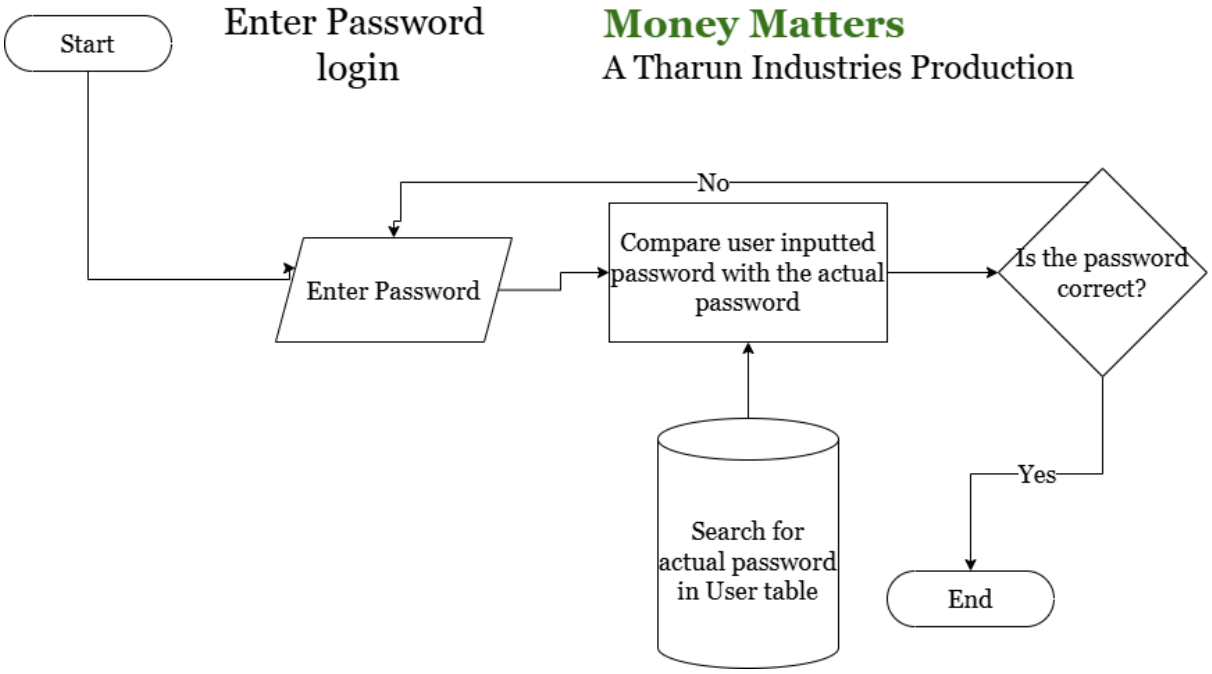
Producers also have access to an undo feature, allowing them to revert any changes made to the database. However, if the stock is empty, the undo functionality is disabled.

Additionally, producers can view a graph showing the relationship between total costs and revenue, making it easy to assess profits and losses. They can also view a table listing all factor inputs, with detailed information about each input displayed. This is achieved by extracting data from the database and parsing all factor inputs for a given producer to the frontend.

Objective 7


Money Matters
A Tharun Industries Production
Start
Are you an existing user?
Yes
No
Enter email address / phone number
Enter Password login
Encrypt Password
End
Store user data in User table
Enter First Name
Enter Last Name
Enter Date of Birth
Enter Email Address
Enter Phone Number
Enter Password sign-up
Encrypt Password
Confirm Password
Enter type of economic agent


Start
Enter Password sign-up
Enter Password
End
No
Is the password more than 8 characters?
Yes
Does the password have at least 1 uppercase character?
Yes
Does the password have at least 1 lowercase character?
Yes
Does the password have at least 1 digit?
Yes
Does the password have at least 1 special character?
Yes

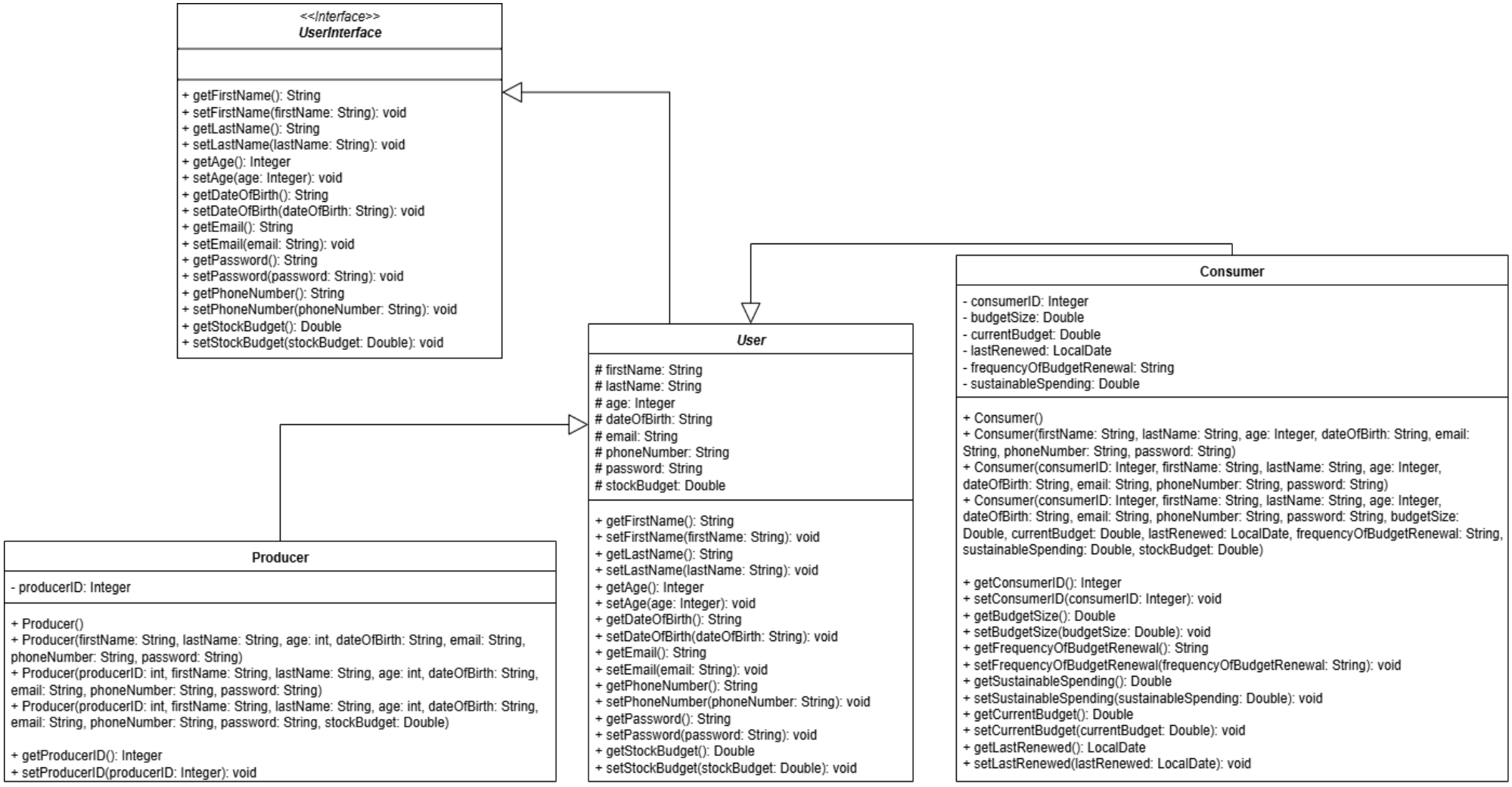


When a user creates an account for the first time, they can sign up by entering their personal information, which will be stored in the database. If the user's password is strong enough, it will be encrypted using the Vernam Cipher. A random Vernam key will be generated and stored in a separate JSON file (kept outside the database for security), while the resulting ciphertext will be saved in the database. If the password is deemed too weak, the user will be prompted to create a stronger one.

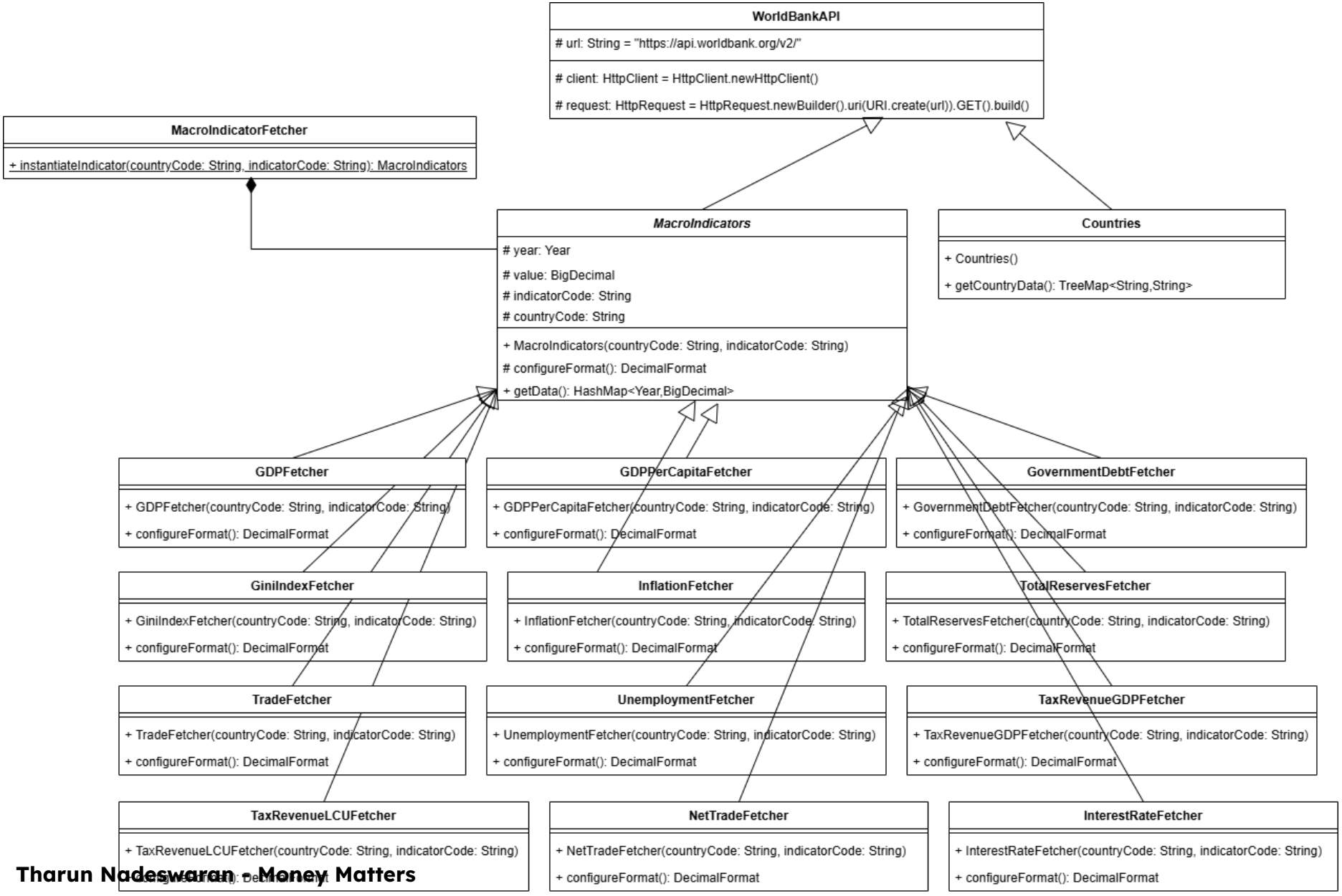
When an existing user attempts to log in, the system retrieves the password they entered along with the corresponding Vernam key from the JSON file. The Vernam Cipher is then applied on the ASCII version of the user inputted password to generate ciphertext based on the entered password. If this ciphertext matches the one stored in the database, authentication is successful. To maintain security, a new Vernam key and ciphertext are then generated, where the key is stored in the JSON file and the ciphertext is updated in the database.

OOP Class Diagrams

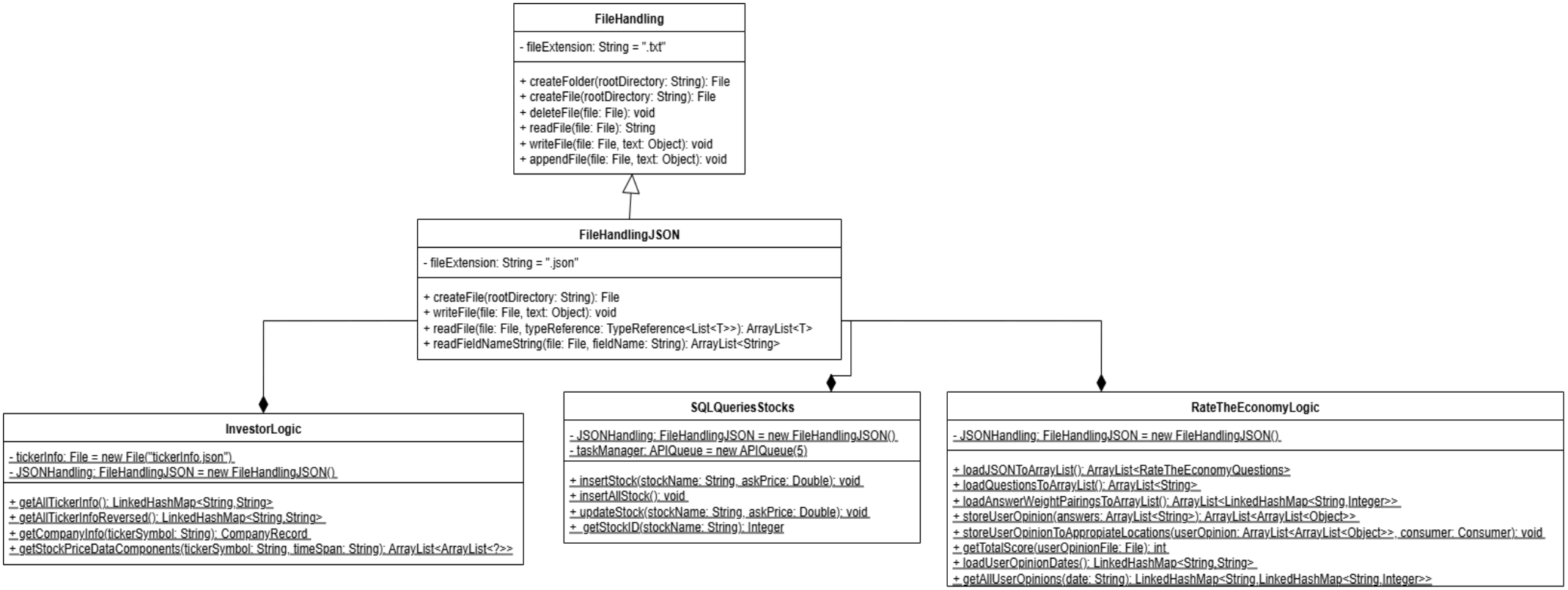
The following OOP diagrams show how I have used OOP concepts. These diagrams will show my use of instantiating objects at run time, inheritance, where subclasses are able to access relevant attributes and methods from parent classes or interfaces, polymorphism, which is shown in the relationship between the FileHandling class and the FileHandlingJSON class. I have used composition and aggregation, where it is shown where I handle .TXT and .JSON files, and my use of multithreading to fetch API calls from Java to Python quickly to retrieve stock data.


<<Interface>>
UserInterface
+ getFirstName(): String
+ setFirstName(firstName: String): void
+ getLastName(): String
+ setLastName(lastName: String): void
+ getAge(): Integer
+ setAge(age: Integer): void
+ getDateOfBirth(): String
+ setDateOfBirth(dateOfBirth: String): void
+ getEmail(): String
+ setEmail(email: String): void
+ getPassword(): String
+ setPassword(password: String): void
+ getPhoneNumber(): String
+ setPhoneNumber(phoneNumber: String): void
+ getStockBudget(): Double
+ setStockBudget(stockBudget: Double): void
User
firstName: String
lastName: String
age: Integer
dateOfBirth: String
email: String
phoneNumber: String
password: String
stockBudget: Double
+ getFirstName(): String
+ setFirstName(firstName: String): void
+ getLastName(): String
+ setLastName(lastName: String): void
+ getAge(): Integer
+ setAge(age: Integer): void
+ getDateOfBirth(): String
+ setDateOfBirth(dateOfBirth: String): void
+ getEmail(): String
+ setEmail(email: String): void
+ getPhoneNumber(): String
+ setPhoneNumber(phoneNumber: String): void
+ getPassword(): String
+ setPassword(password: String): void
+ getStockBudget(): Double
+ setStockBudget(stockBudget: Double): void
Consumer
- consumerID: Integer
- budgetSize: Double
- currentBudget: Double
- lastRenewed: LocalDate
- frequencyOfBudgetRenewal: String
- sustainableSpending: Double
+ Consumer()
+ Consumer(firstName: String, lastName: String, age: Integer, dateOfBirth: String, email: String, phoneNumber: String, password: String)
+ Consumer(consumerID: Integer, firstName: String, lastName: String, age: Integer, dateOfBirth: String, email: String, phoneNumber: String, password: String)
+ Consumer(consumerID: Integer, firstName: String, lastName: String, age: Integer, dateOfBirth: String, email: String, phoneNumber: String, password: String, budgetSize: Double, currentBudget: Double, lastRenewed: LocalDate, frequencyOfBudgetRenewal: String, sustainableSpending: Double, stockBudget: Double)
+ getConsumerID(): Integer
+ setConsumerID(consumerID: Integer): void
+ getBudgetSize(): Double
+ setBudgetSize(budgetSize: Double): void
+ getFrequencyOfBudgetRenewal(): String
+ setFrequencyOfBudgetRenewal(frequencyOfBudgetRenewal: String): void
+ getSustainableSpending(): Double
+ setSustainableSpending(sustainableSpending: Double): void
+ getCurrentBudget(): Double
+ setCurrentBudget(currentBudget: Double): void
+ getLastRenewed(): LocalDate
+ setLastRenewed(lastRenewed: LocalDate): void
Producer
- producerID: Integer
+ Producer()
+ Producer(firstName: String, lastName: String, age: int, dateOfBirth: String, email: String, phoneNumber: String, password: String)
+ Producer(producerID: int, firstName: String, lastName: String, age: int, dateOfBirth: String, email: String, phoneNumber: String, password: String)
+ Producer(producerID: int, firstName: String, lastName: String, age: int, dateOfBirth: String, email: String, phoneNumber: String, password: String, stockBudget: Double)
+ getProducerID(): Integer
+ setProducerID(producerID: Integer): void

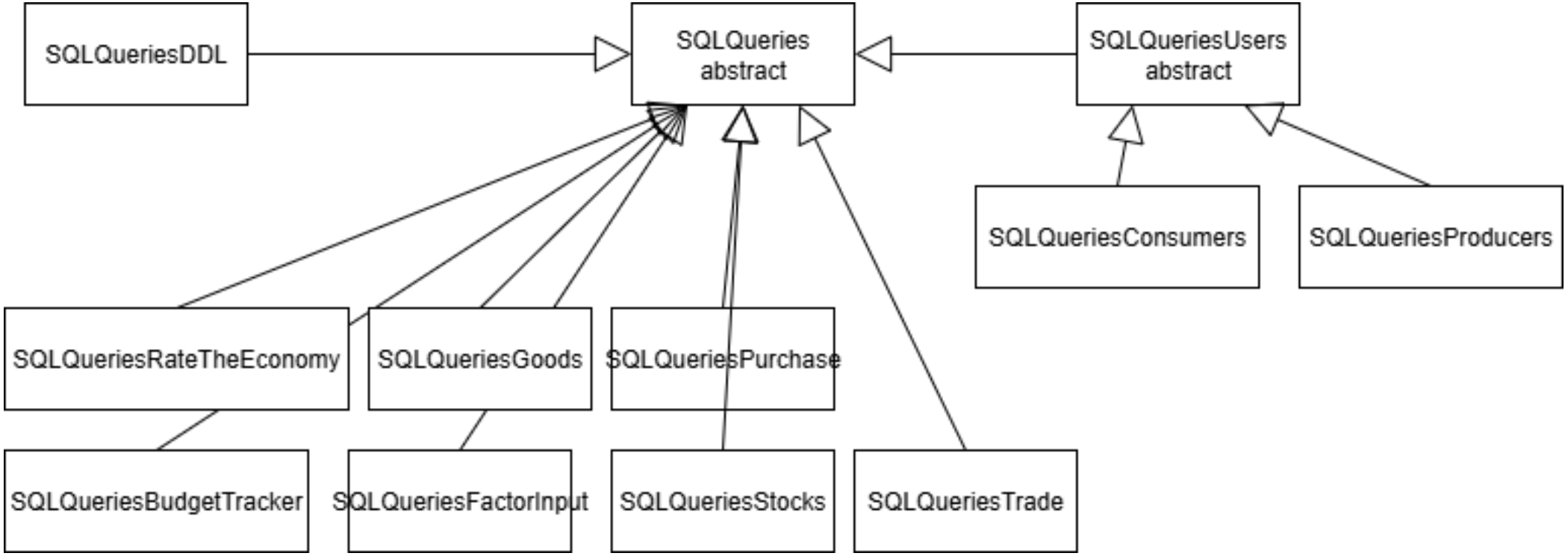
I decided to use an interface, an abstract class, and then two subclasses as it will help me to keep my code organised. The interface lets me set out the basic rules that every type of user must follow, while the abstract class holds shared attributes and methods that all users have in common. This means I don't need to repeat the same code for each user type. My two subclasses then focus on their own specific purposes: the Consumer manages budgets and the Producer handles stock. By structuring my program this way, I've made it easier for myself to maintain, and build on the project further.


WorldBankAPI
url: String = "https://api.worldbank.org/v2/"
client: HttpClient = HttpClient.newHttpClient()
request: HttpRequest = HttpRequest.newBuilder().uri(URI.create(url)).GET().build()
MacroIndicatorFetcher
+ instantiateIndicator(countryCode: String, indicatorCode: String): MacroIndicators
MacroIndicators
year: Year
value: BigDecimal
indicatorCode: String
countryCode: String
+ MacroIndicators(countryCode: String, indicatorCode: String)
configureFormat(): DecimalFormat
+ getData(): HashMap<Year,BigDecimal>
Countries
+ Countries()
+ getCountryData(): TreeMap<String,String>
GDPFetcher
+ GDPFetcher(countryCode: String, indicatorCode: String)
+ configureFormat(): DecimalFormat
GDPPerCapitaFetcher
+ GDPPerCapitaFetcher(countryCode: String, indicatorCode: String)
+ configureFormat(): DecimalFormat
GovernmentDebtFetcher
+ GovernmentDebtFetcher(countryCode: String, indicatorCode: String)
+ configureFormat(): DecimalFormat
GiniIndexFetcher
+ GiniIndexFetcher(countryCode: String, indicatorCode: String)
+ configureFormat(): DecimalFormat
InflationFetcher
+ InflationFetcher(countryCode: String, indicatorCode: String)
+ configureFormat(): DecimalFormat
TotalReservesFetcher
+ TotalReservesFetcher(countryCode: String, indicatorCode: String)
+ configureFormat(): DecimalFormat
TradeFetcher
+ TradeFetcher(countryCode: String, indicatorCode: String)
+ configureFormat(): DecimalFormat
UnemploymentFetcher
+ UnemploymentFetcher(countryCode: String, indicatorCode: String)
+ configureFormat(): DecimalFormat
TaxRevenueGDPFetcher
+ TaxRevenueGDPFetcher(countryCode: String, indicatorCode: String)
+ configureFormat(): DecimalFormat
TaxRevenueLCUFetcher
+ TaxRevenueLCUFetcher(countryCode: String, indicatorCode: String)
NetTradeFetcher
+ NetTradeFetcher(countryCode: String, indicatorCode: String)
+ configureFormat(): DecimalFormat
InterestRateFetcher
+ InterestRateFetcher(countryCode: String, indicatorCode: String)
+ configureFormat(): DecimalFormat

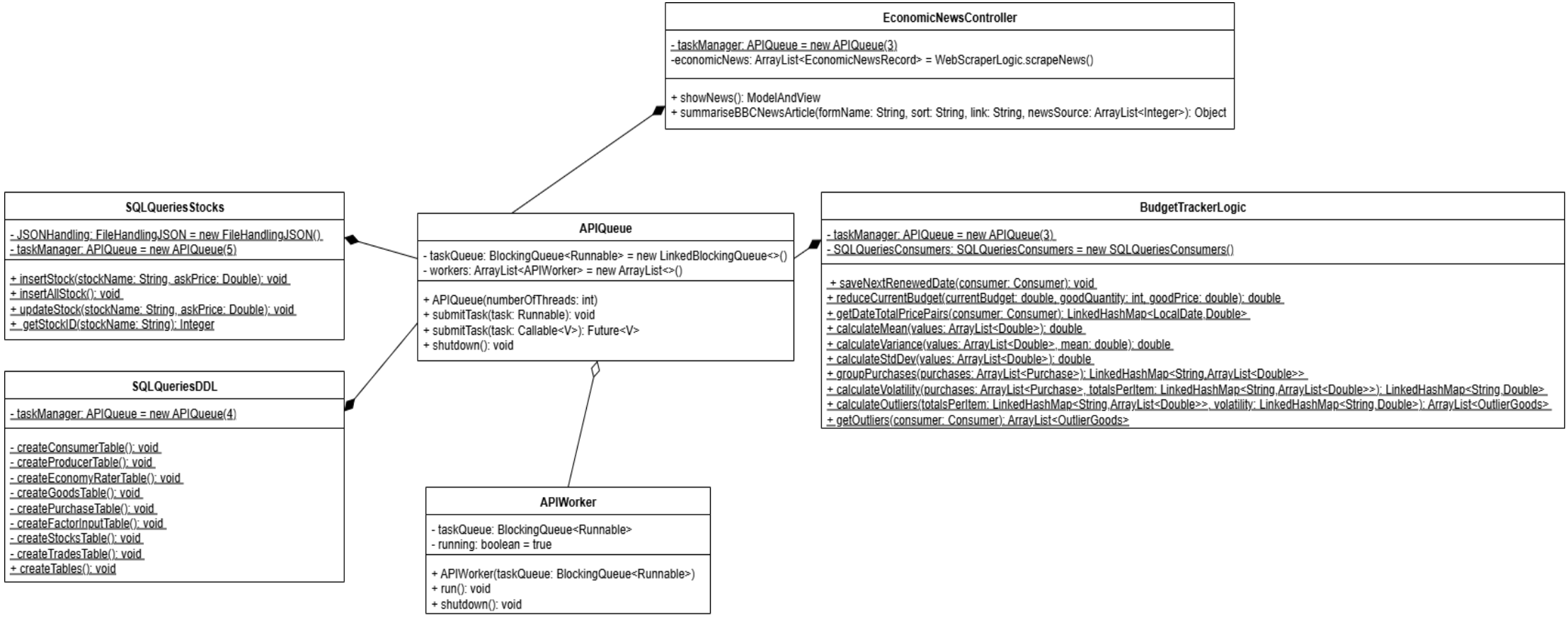
I structured the program using abstract classes and specific fetcher classes because it helps me keep everything easy to manage. I will use the WorldBankAPI abstract class to handle the shared setup for connecting to the API, so I won't have to repeat the same code in every subclass. The MacroIndicators abstract class holds common attributes and methods for all economic indicators, while each fetcher class, like GDPFetcher or InflationFetcher, focuses on the formatting of its own data. This way, I can add new indicators or update existing ones without affecting the rest of the program.


FileHandling
- fileExtension: String = ".txt"
+ createFolder(rootDirectory: String): File
+ createFile(rootDirectory: String): File
+ deleteFile(file: File): void
+ readFile(file: File): String
+ writeFile(file: File, text: Object): void
+ appendFile(file: File, text: Object): void
FileHandlingJSON
- fileExtension: String = ".json"
+ createFile(rootDirectory: String): File
+ writeFile(file: File, text: Object): void
+ readFile(file: File, typeReference: TypeReference<List<T>>): ArrayList<T>
+ readFieldNameString(file: File, fieldName: String): ArrayList<String>
InvestorLogic
- tickerInfo: File = new File("tickerInfo.json")
- JSONHandling: FileHandlingJSON = new FileHandlingJSON()
+ getAllTickerInfo(): LinkedHashMap<String,String>
+ getAllTickerInfoReversed(): LinkedHashMap<String,String>
+ getCompanyInfo(tickerSymbol: String): CompanyRecord
+ getStockPriceDataComponents(tickerSymbol: String, timeSpan: String): ArrayList<ArrayList<?>>
SQLQueriesStocks
- JSONHandling: FileHandlingJSON = new FileHandlingJSON()
- taskManager: APIQueue = new APIQueue(5)
+ insertStock(stockName: String, askPrice: Double): void
+ insertAllStock(): void
+ updateStock(stockName: String, askPrice: Double): void
+ getStockID(stockName: String): Integer
RateTheEconomyLogic
- JSONHandling: FileHandlingJSON = new FileHandlingJSON()
+ loadJSONToArrayList(): ArrayList<RateTheEconomyQuestions>
+ loadQuestionsToArrayList(): ArrayList<String>
+ loadAnswerWeightPairingsToArrayList(): ArrayList<LinkedHashMap<String,Integer>>
+ storeUserOpinion(answers: ArrayList<String>): ArrayList<ArrayList<Object>>
+ storeUserOpinionToAppropiateLocations(userOpinion: ArrayList<ArrayList<Object>>, consumer: Consumer): void
+ getTotalScore(userOpinionFile: File): int
+ loadUserOpinionDates(): LinkedHashMap<String,String>
+ getAllUserOpinions(date: String): LinkedHashMap<String,LinkedHashMap<String,Integer>>

I created a base FileHandling class for general file operations and extended it with FileHandlingJSON to handle JSON-specific tasks, which will demonstrate inheritance and code reuse. I will also use polymorphism by overriding methods like createFile and writeFile, allowing me to treat all file operations the same for multiple file types. I will apply composition by including objects like FileHandlingJSON inside logic classes such as InvestorLogic and RateTheEconomyLogic, meaning these classes will be built from smaller objects that they fully control. Similarly, I will use aggregation when logic classes interact with external APIs or database repository classes.


SQLQueriesDDL
SQLQueries
abstract
SQLQueriesUsers
abstract
SQLQueriesConsumers
SQLQueriesProducers
SQLQueriesRateTheEconomy
SQLQueriesGoods
SQLQueriesPurchase
SQLQueriesBudgetTracker
SQLQueriesFactorInput
SQLQueriesStocks
SQLQueriesTrade

I will create an abstract SQLQueries class to keep all database connection details and common methods in one place, and then extend it with classes for specific tables like SQLQueriesConsumers, SQLQueriesProducers, and SQLQueriesStocks. I will also use polymorphism by overriding methods in these subclasses, so I can handle all database queries in a similar way. Composition will be applied by including objects, such as FileHandlingJSON or APIQueue, inside query classes like SQLQueriesStocks, to allow me to easily use multithreading and read data from JSON.


EconomicNewsController
- taskManager: APIQueue = new APIQueue(3)
-economicNews: ArrayList<EconomicNewsRecord> = WebScraperLogic.scrapeNews()
+ showNews(): ModelAndView
+ summariseBBCNewsArticle(formName: String, sort: String, link: String, newsSource: ArrayList<Integer>): Object
SQLQueriesStocks
- JSONHandling: FileHandlingJSON = new FileHandlingJSON()
- taskManager: APIQueue = new APIQueue(5)
+ insertStock(stockName: String, askPrice: Double): void
+ insertAllStock(): void
+ updateStock(stockName: String, askPrice: Double): void
+ getStockID(stockName: String): Integer
APIQueue
- taskQueue: BlockingQueue<Runnable> = new LinkedBlockingQueue<>()
- workers: ArrayList<APIWorker> = new ArrayList<>()
+ APIQueue(numberOfThreads: int)
+ submitTask(task: Runnable): void
+ submitTask(task: Callable<V>): Future<V>
+ shutdown(): void
BudgetTrackerLogic
- taskManager: APIQueue = new APIQueue(3)
- SQLQueriesConsumers: SQLQueriesConsumers = new SQLQueriesConsumers()
+ saveNextRenewedDate(consumer: Consumer): void
+ reduceCurrentBudget(currentBudget: double, goodQuantity: int, goodPrice: double): double
+ getDateTotalPricePairs(consumer: Consumer): LinkedHashMap<LocalDate,Double>
+ calculateMean(values: ArrayList<Double>): double
+ calculateVariance(values: ArrayList<Double>, mean: double): double
+ calculateStdDev(values: ArrayList<Double>): double
+ groupPurchases(purchases: ArrayList<Purchase>): LinkedHashMap<String,ArrayList<Double>>
+ calculateVolatility(purchases: ArrayList<Purchase>, totalsPerItem: LinkedHashMap<String,ArrayList<Double>>): LinkedHashMap<String,Double>
+ calculateOutliers(totalsPerItem: LinkedHashMap<String,ArrayList<Double>>, volatility: LinkedHashMap<String,Double>): ArrayList<OutlierGoods>
+ getOutliers(consumer: Consumer): ArrayList<OutlierGoods>
SQLQueriesDDL
- taskManager: APIQueue = new APIQueue(4)
- createConsumerTable(): void
- createProducerTable(): void
- createEconomyRaterTable(): void
- createGoodsTable(): void
- createPurchaseTable(): void
- createFactorInputTable(): void
- createStocksTable(): void
- createTradesTable(): void
+ createTables(): void
APIWorker
- taskQueue: BlockingQueue<Runnable>
- running: boolean = true
+ APIWorker(taskQueue: BlockingQueue<Runnable>)
+ run(): void
+ shutdown(): void

Database Design & Normalisation

For my system, I will be using a database from Microsoft SQL. In order to make effective use of my database, I will normalise each table to 3NF, to improve data consistency and reduce data redundancy. See the image below to see all the fully normalised tables, with data types for each attribute, and the relationship between the tables.

Attributes in **bold** signify a primary (or composite) key.
Attributes in *italics* signify a foreign key.

Consumer Table (1NF)

ConsumerID	FirstName	LastName	Age	DateOfBirth

Email	PhoneNumber	Password	BudgetSize	CurrentBudget

FrequencyOfBudgetRenewal	SustainableSpending	LastRenewed	StockBudget

ConsumerOpinion	ConsumerOpinionFile	ConsumerOpinionDate

Whilst this table is in 1NF, there are many partial key dependencies and non-key dependencies. For example, the attributes FirstName, LastName, Age, DateOfBirth, Email, PhoneNumber, Password, BudgetSize, FrequencyOfBudgetRenewal, and SustainableSpending only rely on the primary key **ConsumerID**, hence they belong to the Consumer table. Furthermore, BudgetSize, CurrentBudget, FrequencyOfBudgetRenewal, SustainableSpending, LastRenewed, StockBudget also rely on **ConsumerID** as consumers will only have 1 set of attributes at all times so there won't be data redundancy.

However, the attributes ConsumerOpinion, ConsumerOpinionFile, ConsumerOpinionDate could cause data redundancy if they are in the Consumer Table, as Consumers may have reviewed the economy multiple times, which unnecessarily repeats personal information.

Therefore I have created a separate table EconomyRater, with the following attributes: **RatingID**, *ConsumerID*, ConsumerOpinion, ConsumerOpinionFile and ConsumerOpinionDate and removed ConsumerOpinion, ConsumerOpinionFile, ConsumerOpinionDate from the Consumer Table. This means that both the Consumer Table and EconomyRater Table are in 3NF.

Consumer Table (3NF)

ConsumerID	FirstName	LastName	Age	DateOfBirth

Email	PhoneNumber	Password	BudgetSize	CurrentBudget

FrequencyOfBudgetRenewal	SustainableSpending	LastRenewed	StockBudget

EconomyRater Table (3NF)

RatingID	*ConsumerID*	ConsumerOpinion	ConsumerOpinionFile

ConsumerOpinionFileDate

BudgetTracker Table (1NF)

GoodID	*ConsumerID*	GoodName	GoodPrice	PurchaseDate

QuantityBought

This table is in 1NF as the table is vulnerable to data redundancy. This is because multiple people can buy the same good, which could mean that basic information about the good, such as GoodName, and GoodPrice may be unnecessarily repeated.

Therefore, to ensure the table is in 3NF, I would create 2 tables, one table that contains the information about the Goods and the other table is about Purchases. The Goods Table will have the attributes: **GoodID**, GoodName and GoodPrice. The Purchase Table will have the attributes: **PurchaseID**, *GoodID*, *ConsumerID*, PurchaseDate and QuantityBought.

Goods Table (3NF)

GoodID	GoodName	GoodPrice

Purchase Table (3NF)

PurchaseID	*GoodID*	*ConsumerID*	PurchaseDate	QuantityBought

Producer Table (1NF)

ProducerID	**InvestmentID**	FirstName	LastName	Age

DateOfBirth	Email	PhoneNumber	Password	InitialInvestment

CurrentPrice

This table is in 1NF, because the attributes InitialInvestment and CurrentPrice only rely partially on the composite key (it only relies with **InvestmentID**), while the other attributes rely on **ProducerID**.

To remove any partial key dependencies, I created a Trades Table and a Stocks Table and removed the InitialInvestment and CurrentPrice attributes. The Stock Table will have the **StockID**, StockName, AskPrice and LastUpdated attributes.

The Trades Table will have **TradeID**, *ConsumerID*, *ProducerID*, *StockID*, TradeDate, TradeType, AskPrice (the price when the user purchases the stock) and Quantity attributes. By having both *ConsumerID* and *ProducerID* as attributes that can be set to NULL means that both Consumers and Producers can make investments.

Producer Table (3NF)

ProducerID	FirstName	LastName	Age	DateOfBirth

Email	PhoneNumber	Password	StockBudget

Stocks Table (3NF)

StockID	StockName	AskPrice	LastUpdated

Trades Table (3NF)

TradeID	*ConsumerID*	*ProducerID*	*StockID*	TradeDate

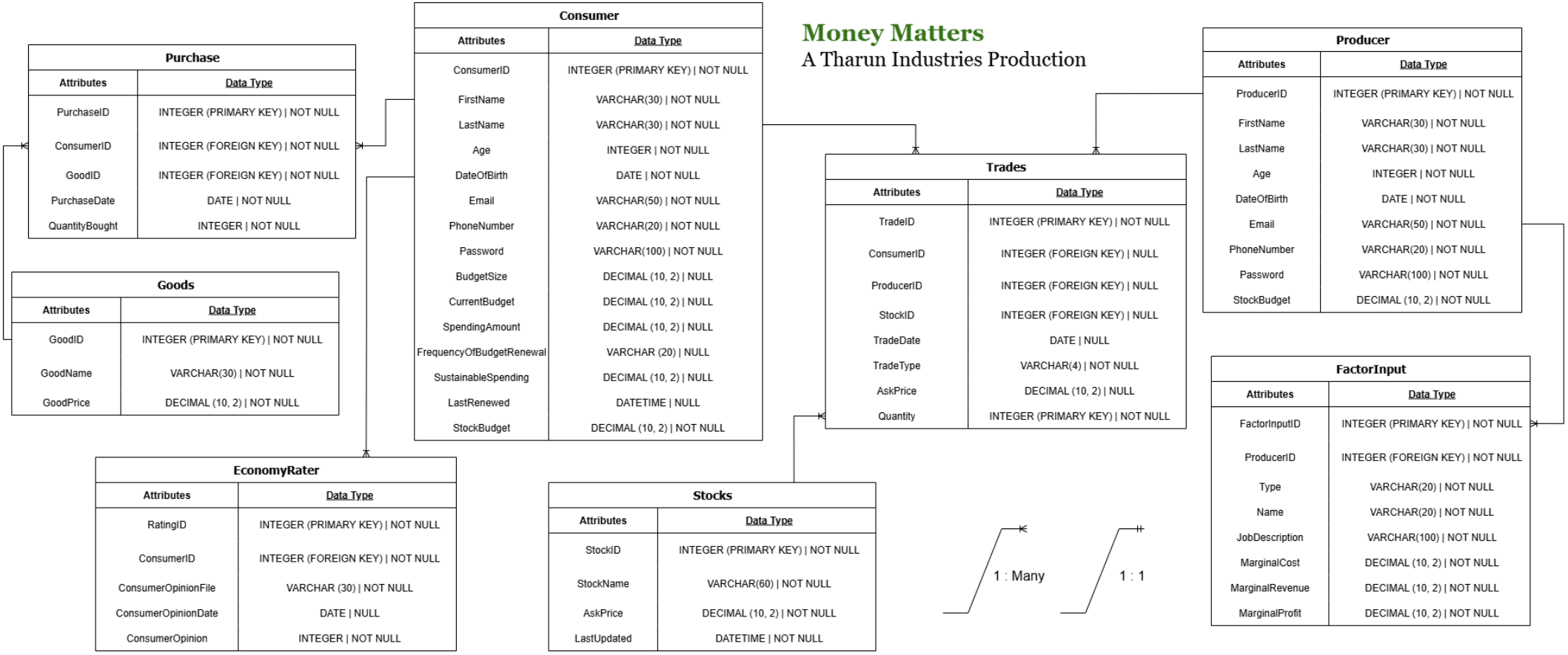
TradeType	AskPrice	Quantity

FactorInput Table (3NF)

FactorInputID	*ProducerID*	FactorInputType	FactorInputName	FactorInputDescription

MarginalCost	MarginalRevenue	MarginalProfit

This table is already in 3NF. This is because all attributes rely on the primary key, only the primary key and nothing but the key.


Money Matters
A Tharun Industries Production
Purchase
Attributes | Data Type
PurchaseID | INTEGER (PRIMARY KEY) | NOT NULL
ConsumerID | INTEGER (FOREIGN KEY) | NOT NULL
GoodID | INTEGER (FOREIGN KEY) | NOT NULL
PurchaseDate | DATE | NOT NULL
QuantityBought | INTEGER | NOT NULL
Consumer
Attributes | Data Type
ConsumerID | INTEGER (PRIMARY KEY) | NOT NULL
FirstName | VARCHAR(30) | NOT NULL
LastName | VARCHAR(30) | NOT NULL
Age | INTEGER | NOT NULL
DateOfBirth | DATE | NOT NULL
Email | VARCHAR(50) | NOT NULL
PhoneNumber | VARCHAR(20) | NOT NULL
Password | VARCHAR(100) | NOT NULL
BudgetSize | DECIMAL (10, 2) | NULL
CurrentBudget | DECIMAL (10, 2) | NULL
SpendingAmount | DECIMAL (10, 2) | NULL
FrequencyOfBudgetRenewal | VARCHAR (20) | NULL
SustainableSpending | DECIMAL (10, 2) | NULL
LastRenewed | DATETIME | NULL
StockBudget | DECIMAL (10, 2) | NOT NULL
Producer
Attributes | Data Type
ProducerID | INTEGER (PRIMARY KEY) | NOT NULL
FirstName | VARCHAR(30) | NOT NULL
LastName | VARCHAR(30) | NOT NULL
Age | INTEGER | NOT NULL
DateOfBirth | DATE | NOT NULL
Email | VARCHAR(50) | NOT NULL
PhoneNumber | VARCHAR(20) | NOT NULL
Password | VARCHAR(100) | NOT NULL
StockBudget | DECIMAL (10, 2) | NOT NULL
Trades
Attributes | Data Type
TradeID | INTEGER (PRIMARY KEY) | NOT NULL
ConsumerID | INTEGER (FOREIGN KEY) | NULL
ProducerID | INTEGER (FOREIGN KEY) | NULL
StockID | INTEGER (FOREIGN KEY) | NULL
TradeDate | DATE | NULL
TradeType | VARCHAR(4) | NOT NULL
AskPrice | DECIMAL (10, 2) | NULL
Quantity | INTEGER (PRIMARY KEY) | NOT NULL
Goods
Attributes | Data Type
GoodID | INTEGER (PRIMARY KEY) | NOT NULL
GoodName | VARCHAR(30) | NOT NULL
GoodPrice | DECIMAL (10, 2) | NOT NULL
EconomyRater
Attributes | Data Type
RatingID | INTEGER (PRIMARY KEY) | NOT NULL
ConsumerID | INTEGER (FOREIGN KEY) | NOT NULL
ConsumerOpinionFile | VARCHAR (30) | NOT NULL
ConsumerOpinionDate | DATE | NULL
ConsumerOpinion | INTEGER | NOT NULL
Stocks
Attributes | Data Type
StockID | INTEGER (PRIMARY KEY) | NOT NULL
StockName | VARCHAR(60) | NOT NULL
AskPrice | DECIMAL (10, 2) | NOT NULL
LastUpdated | DATETIME | NOT NULL
1 : Many
1 : 1
FactorInput
Attributes | Data Type
FactorInputID | INTEGER (PRIMARY KEY) | NOT NULL
ProducerID | INTEGER (FOREIGN KEY) | NOT NULL
Type | VARCHAR(20) | NOT NULL
Name | VARCHAR(20) | NOT NULL
JobDescription | VARCHAR(100) | NOT NULL
MarginalCost | DECIMAL (10, 2) | NOT NULL
MarginalRevenue | DECIMAL (10, 2) | NOT NULL
MarginalProfit | DECIMAL (10, 2) | NOT NULL

SQL Queries

DDL Scripts

```
CREATE TABLE Consumers (
	ConsumerID INT IDENTITY(1,1) NOT NULL,
	FirstName VARCHAR(30) NOT NULL,
	LastName VARCHAR(30) NOT NULL,
	Age INT NOT NULL,
	DateOfBirth Date NOT NULL,
	Email VARCHAR(75) NOT NULL,
	PhoneNumber VARCHAR(20) NOT NULL,
	Password VARCHAR(MAX) NOT NULL,
	BudgetSize DECIMAL(10, 2) DEFAULT 0.00 NULL,
	CurrentBudget DECIMAL(10, 2) DEFAULT 0.00 NULL,
	LastRenewed DATETIME DEFAULT GETDATE(),
	FrequencyOfRenewal VARCHAR(10) DEFAULT 'Pending' NULL,
	SustainableSpending DECIMAL(10, 2) NULL,
	StockBudget DECIMAL(10, 2) NOT NULL DEFAULT 10000.00,
	PRIMARY KEY (ConsumerID)
);
```

Create Consumer Table

```
CREATE TABLE EconomyRater (
	RatingID INT IDENTITY (1,1) NOT NULL,
	ConsumerID INT NOT NULL,
	ConsumerOpinionDate DATE default GETDATE(),
	ConsumerOpinionFile VARCHAR(30) NOT NULL,
	ConsumerOpinion INT NOT NULL,
	PRIMARY KEY (RatingID),
	FOREIGN KEY (ConsumerID) REFERENCES Consumers(ConsumerID)
);
```

Create EconomyRater Table

```
CREATE TABLE Goods (
	GoodID INT IDENTITY(1,1),
	GoodName VARCHAR(30) NOT NULL,
	GoodPrice DECIMAL(10, 2) NOT NULL,
	PRIMARY KEY (GoodID)
);
```

Create Goods Table

```
CREATE TABLE Purchase (
      PurchaseID INT IDENTITY(1,1),
      ConsumerID INT NOT NULL,
      GoodID INT NOT NULL,
      PurchaseDate DATETIME DEFAULT GETDATE(),
      QuantityBought INT NOT NULL,
      PRIMARY KEY (PurchaseID),
      FOREIGN KEY (ConsumerID) REFERENCES Consumers(ConsumerID),
      FOREIGN KEY (GoodID) REFERENCES Goods(GoodID)
);
```

Create Purchase Table

```
CREATE TABLE Producers (
      ProducerID INT IDENTITY(1,1) NOT NULL,
      FirstName VARCHAR(30) NOT NULL,
      LastName VARCHAR(30) NOT NULL,
      Age INT NOT NULL,
      DateOfBirth Date NOT NULL,
      Email VARCHAR(75) NOT NULL,
      PhoneNumber VARCHAR(20) NOT NULL,
      Password VARCHAR(MAX) NOT NULL,
      StockBudget DECIMAL(10,2) NOT NULL DEFAULT 10000.00,
      PRIMARY KEY (ProducerID),
)
```

Create Producer Table

```
CREATE TABLE Stocks (
      StockID INT IDENTITY(1,1),
      StockName VARCHAR(60) UNIQUE,
      AskPrice DECIMAL(10, 2) NOT NULL,
      LastUpdated DATETIME NOT NULL DEFAULT GETDATE(),
      PRIMARY KEY (StockID)
);
```

Create Stocks Table

```
CREATE TABLE Trades (
    TradeID INT IDENTITY(1,1) PRIMARY KEY,
    ConsumerID INT NULL,
    ProducerID INT NULL,
    StockID INT NOT NULL,
    TradeDate DATE DEFAULT GETDATE() NOT NULL,
    TradeType VARCHAR(4) NOT NULL,
    AskPrice DECIMAL(10, 2) NOT NULL,
    Quantity INT NOT NULL,
    FOREIGN KEY (ConsumerID) REFERENCES Consumers(ConsumerID),
    FOREIGN KEY (ProducerID) REFERENCES Producers(ProducerID),
    FOREIGN KEY (StockID) REFERENCES Stocks(StockID)
);
```

Create Trades Table

```
CREATE TABLE FactorInput (
      FactorInputID INT IDENTITY(1,1),
      ProducerID INT NOT NULL,
      FactorInputType VARCHAR(20) NOT NULL,
      FactorInputName VARCHAR(30) NOT NULL,
      FactorInputDescription VARCHAR(50) NOT NULL,
      MarginalCost DECIMAL (10, 2) NOT NULL,
      MarginalRevenue DECIMAL (10, 2) NOT NULL,
      MarginalProfit DECIMAL (10, 2) NOT NULL,
      PRIMARY KEY (FactorInputID),
      FOREIGN KEY (ProducerID) REFERENCES Producers(ProducerID)
);
```

Create FactorInput Table

DML Scripts

Consumers Table

```
INSERT INTO Consumers (FirstName, LastName, Age, DateOfBirth, Email,
PhoneNumber, Password)
VALUES (?, ?, ?, ?, ?, ?, ?)
```

Insert consumer data from signup page to the database

```
UPDATE Consumers
SET Password = ?
WHERE ConsumerID = ?
```

Whenever the user logins into their account, the program generates a new Vernam cipher, hence a new ciphertext password. It will update the consumer's password, which is stored in ciphertext, ensuring user's security.

```
UPDATE Consumers
SET StockBudget = ?
WHERE ConsumerID = ?
```

Whenever the user purchases or sells a stock, there is a change in the stock budget.

```
UPDATE Consumers
SET BudgetSize = ?
WHERE ConsumerID = ?
```

Updates the user's budget size, whenever the user wants to change their budget

```
UPDATE Consumers
SET CurrentBudget = ?
WHERE ConsumerID = ?
```

Updates the user's current budget, whenever the user purchases a good

```
UPDATE Consumers
SET FrequencyOfRenewal = ?
WHERE ConsumerID = ?
```

Updates the frequency of budget renewal, whenever the user wants to change how frequent they renew their budget

```
UPDATE Consumers
SET LastRenewed = ?
WHERE ConsumerID = ?
```

Updates the when the user's budget was last renewed

EconomyRater Table

```
INSERT INTO EconomyRater (ConsumerID, ConsumerOpinionFile,
ConsumerOpinion)
VALUES (?, ?, ?)
```

Inserts the consumer's current opinion of the economy, with the name of the JSON file that stores all the user's results.

```
UPDATE EconomyRater
SET ConsumerOpinion = ?
WHERE RatingID = ?
```

If a user submits the Economy Rating form multiple times in a day, their previous ratings for that day are updated to their current opinion.

Goods Table

```
INSERT INTO Goods (GoodName, GoodPrice)
VALUES (?, ?)
```

Inserts a new good to the database when the user includes it in their budget.

Purchase Table

```
INSERT INTO Purchase (ConsumerID, GoodID, QuantityBought)
VALUES (?, ?, ?)
```

Inserts a new purchase record when a user purchases a good.

Producers Table

```
INSERT INTO Producer (FirstName, LastName, Age, DateOfBirth, Email,
PhoneNumber, Password)
VALUES (?, ?, ?, ?, ?, ?, ?)
```

Insert consumer data from signup page to the database

```
UPDATE Producers
SET Password = ?
WHERE ProducerID = ?
```

Whenever the user logins into their account, the program generates a new Vernam cipher, hence a new ciphertext password. Then it updates the consumer's password, which is stored in ciphertext, ensuring user's security.

```
UPDATE Producer
SET StockBudget = ?
WHERE ProducerID = ?
```

Whenever the user purchases or sells a stock, there is a change in the stock budget.

FactorInput Table

```
INSERT INTO FactorInput (FactorInputType, ProducerID, FactorInputName,
FactorInputDescription, MarginalCost, MarginalRevenue, MarginalProfit)
VALUES (?, ?, ?, ?, ?, ?, ?)
```

Inserts a new factor input associated with a Producer into the FactorInput Table.

```
UPDATE FactorInput
SET MarginalCost = ?, MarginalRevenue = ?, MarginalProfit = ?
WHERE FactorInputName = ?
AND ProducerID = ?
```

Updates the marginal cost, revenue, and profit of a factor input for a given producer based on the factor input's name.

```
DELETE FROM FactorInput
WHERE FactorInputName = ?
AND ProducerID = ?
```

Deletes a factor input from the table for a given producer based on the factor input's name.

Stocks Table

```
INSERT INTO Stocks (StockName, AskPrice)
VALUES (?, ?)
```

Inserts a new stock into the Stocks table with the stock name and ask price.

```
UPDATE Stocks
SET AskPrice = ?, LastUpdated = ?
WHERE StockName = ?
```

Updates the ask price and last updated timestamp of a specific stock, identified by its name.

Trades Table

```
INSERT INTO Trades (ConsumerID, StockID, TradeType, AskPrice, Quantity)
VALUES (?, ?, ?, ?, ?)
```

Inserts a new trade to the Trades Table

```
INSERT INTO Trades (ProducerID, StockID, TradeType, AskPrice, Quantity)
VALUES (?, ?, ?, ?, ?)
```

Inserts a new trade to the Trades Table

DQL Scripts

Consumers Table

```
SELECT *
FROM Consumers
WHERE ConsumerID = ?
```

Once the user is logged in, all the user's data is stored in the consumer class.

```
SELECT ConsumerID
FROM Consumers
WHERE Email = ?
AND Password = ?
```

When logging into an account, the system checks if the entered email and password match stored email and password. If valid, the user gains access; else the user must retry.

```
SELECT Count(Email) AS Emails
FROM Consumers
WHERE Email = ?
```

When a user registers, the system checks the Consumer table to determine if the email already exists. If the email isn't already used, the account is created; otherwise, the user must choose a different email.

EconomyRater Table

```
SELECT AVG(ConsumerOpinion) AS AverageConsumerOpinion
FROM EconomyRater
WHERE ConsumerOpinionDate = ?
```

Calculates the average consumer opinion of all consumers who have completed the form at a specific date.

```
SELECT COUNT(RatingID) AS RecordNumber, RatingID
FROM EconomyRater
WHERE ConsumerID = ?
AND ConsumerOpinionDate = ?
GROUP BY RatingID
```

Calculates how many times a user has submitted the Rate The Economy Form in a day. If the user submits the form, more than 1 time in a specific day, the EconomyRater Table will update the previous rating on that day, else it will insert a new record.

```
SELECT ConsumerOpinion, ConsumerOpinionDate
FROM EconomyRater, Consumers
WHERE Consumers.ConsumerID = EconomyRater.ConsumerID
AND EconomyRater.ConsumerID = ?
ORDER BY ConsumerOpinionDate DESC
```

By retrieving all economy ratings from a specific consumer and ordering them from newest to oldest, I can use Java to access the user's most recent response.

```
SELECT *
FROM Consumers
WHERE
(
    SELECT
        SUM(CASE WHEN TradeType = 'BUY' THEN Quantity ELSE 0 END) -
        SUM(CASE WHEN TradeType = 'SELL' THEN Quantity ELSE 0 END)
    FROM Trades
    WHERE Consumers.ConsumerID = ?
    AND Trades.ConsumerID = Consumers.ConsumerID
) <= 0
AND StockBudget <= 0;
```

Selects a specific consumer whose net trade quantity is zero stock budget is zero or negative. This means that I can appropriately provide consumers more money to invest in stocks rather than being capped at £0

Goods Table

```
SELECT GoodID
FROM Goods
WHERE GoodName = ?
AND GoodPrice = ?
```

Retrieves the appropriate GoodID based on the good's name and price. This GoodID is then stored in the Purchase table as a foreign key.

```
SELECT COUNT(*) AS Count
FROM Goods
WHERE GoodName = ?
AND GoodPrice = ?
```

When a user enters a good in the budget tracker, the system checks the Good table to determine if the good already exists. If the good does not exist, the good will be inserted into the Goods Table.

Purchase Table

```
SELECT SUM(Goods.GoodPrice * Purchase.QuantityBought) AS TotalSpending
FROM Purchase, Goods, Consumers
WHERE Purchase.PurchaseDate >= ?
AND Purchase.PurchaseDate < ?
AND Goods.GoodID = Purchase.GoodID
AND Purchase.ConsumerID = Consumers.ConsumerID
AND Consumers.ConsumerID = ?
```

Calculates the total amount a specific consumer spent on purchases within a given date range (this will be used to calculate today's total spending) by multiplying each good's price by the quantity bought and then adds all the amounts as TotalSpending.

```
SELECT Purchase.QuantityBought, Goods.GoodPrice
FROM Purchase, Goods, Consumers
WHERE Purchase.GoodID = Goods.GoodID
AND Purchase.ConsumerID = Consumers.ConsumerID
AND Consumers.ConsumerID = ?
AND CONVERT(DATE, Purchase.PurchaseDate) = ?
ORDER BY Purchase.PurchaseDate
```

Retrieves the quantity purchased and price of goods for a specific consumer on a specific date. The results are ordered by the time of purchase, from earliest to latest on that date.

```
SELECT Goods.GoodName, Goods.GoodPrice, Purchase.QuantityBought,
Purchase.PurchaseDate
FROM Consumers, Goods, Purchase
WHERE Consumers.ConsumerID = Purchase.ConsumerID
AND Goods.GoodID = Purchase.GoodID
AND Consumers.ConsumerID = ?
ORDER BY Purchase.PurchaseDate DESC
```

Retrieves purchase details, including the good's name, price, quantity bought, and purchase date. These details are stored in the Goods Record.

Producers Table

```
SELECT *
FROM Producers
WHERE ProducerID = ?
```

Once the user is logged in, all the user's data is stored in the producer class.

```
SELECT ProducerID
FROM Producers
WHERE Email = ?
AND Password = ?
```

When logging into an account, the system checks if the entered email and password match stored email and password. If valid, the user gains access; else the user must retry.

```
SELECT Count(Email) AS Emails
FROM Producers
WHERE Email = ?
```

When a user registers, the system checks the Producer table to determine if the email already exists. If the email isn't already used, the account is created; otherwise, the user must choose a different email.

```
SELECT *
FROM Producers
WHERE
(
    SELECT
        SUM(CASE WHEN TradeType = 'BUY' THEN Quantity ELSE 0 END) -
        SUM(CASE WHEN TradeType = 'SELL' THEN Quantity ELSE 0 END)
    FROM Trades
    WHERE Producers.ProducerID = ?
    AND Trades.ProducerID = Producers.ProducerID
) <= 0
AND StockBudget <= 0;
```

Selects a specific producer whose net trade quantity is zero stock budget is zero or negative. This means that I can appropriately provide producers more money to invest in stocks rather than being capped at £0

FactorInput Table

```
SELECT *
FROM FactorInput
WHERE ProducerID = ?
```

Retrieves all factor inputs for a given producer from the FactorInput Table, which is used to create FactorInput records.

```
SELECT *
FROM FactorInput
WHERE ProducerID = ?
AND FactorInputName = ?
```

Retrieves all details of a specific factor input for a given producer,

```
SELECT COUNT (FactorInputName) AS NumberOfFactorInputs
FROM FactorInput
WHERE FactorInputType = ?
AND ProducerID = ?
```

Counts how many factor inputs of a given type belong to a specific producer. This will be used for the Cobb Douglas Function calculation.

```
SELECT SUM(MarginalCost) AS TotalCost
FROM FactorInput
WHERE ProducerID = ?
```

Calculates the total marginal cost of all factor inputs for a specific producer.

```
SELECT SUM(MarginalRevenue) AS TotalRevenue
FROM FactorInput
WHERE ProducerID = ?
```

Calculates the total revenue cost of all factor inputs for a specific producer.

```
SELECT SUM(MarginalProfit) AS TotalProfit
FROM FactorInput
WHERE ProducerID = ?
```

Calculates the total revenue cost of all factor inputs for a specific producer.

```
SELECT MarginalCost
FROM FactorInput
WHERE ProducerID = ?
```

Retrieves the marginal cost values of all factor inputs that belong to a specific producer.

```
SELECT MarginalRevenue
FROM FactorInput
WHERE ProducerID = ?
```

Retrieves the marginal revenue values of all factor inputs that belong to a specific producer.

Stocks Table

```
SELECT StockID
FROM Stocks
WHERE StockName = ?
```

Retrieves the StockID of a stock based on its name.

Trades Table

```
SELECT Trades.ConsumerID, Stocks.StockName, SUM(Trades.Quantity *
IIF(Trades.TradeType = 'BUY', 1, -1)) AS NetQuantity,
SUM(Trades.Quantity * IIF(Trades.TradeType = 'BUY', 1, -1) *
Trades.AskPrice) AS TotalAskPrice, AVG(Trades.AskPrice) AS
AverageAskPrice, AVG(Stocks.AskPrice) - AVG(Trades.AskPrice) AS Profit
FROM Trades, Stocks, Consumers
WHERE Stocks.StockID = Trades.StockID
AND Consumers.ConsumerID = ?
AND Consumers.ConsumerID = Trades.ConsumerID
GROUP BY Trades.ConsumerID, Stocks.StockName
HAVING SUM(Trades.Quantity * IIF(Trades.TradeType = 'BUY', 1, -1)) > 0
ORDER BY Trades.ConsumerID, Stocks.StockName
```

Retrieves all stocks held by a specific consumer, showing the net quantity, total amount spent, average purchase price, and profit for each, but only includes stocks where the consumer owns more than zero.

```
SELECT Trades.ProducerID, Stocks.StockName, SUM(Trades.Quantity *
IIF(Trades.TradeType = 'BUY', 1, -1)) AS NetQuantity,
SUM(Trades.Quantity * IIF(Trades.TradeType = 'BUY', 1, -1) *
Trades.AskPrice) AS TotalAskPrice, AVG(Trades.AskPrice) AS
AverageAskPrice, AVG(Stocks.AskPrice) - AVG(Trades.AskPrice) AS Profit
FROM Trades, Stocks, Producers
WHERE Stocks.StockID = Trades.StockID
AND Producers.ProducerID = ?
AND Producers.ProducerID = Trades.ProducerID
GROUP BY Trades.ProducerID, Stocks.StockName
HAVING SUM(Trades.Quantity * IIF(Trades.TradeType = 'BUY', 1, -1)) > 0
ORDER BY Trades.ProducerID, Stocks.StockName
```

Retrieves all stocks held by a specific producer, showing the net quantity, total amount spent, average purchase price, and profit for each, but only includes stocks where the producer owns more than zero.

Security and Integrity of Data

As I will be storing sensitive user information, such as their login information, and their financial data, I will be using a Vernam Cipher to encrypt user data. I would store the user's encryption key in a .txt file, where the user's ID is the name of the file, so that it is easy to look up the encryption key and apply the cipher to the user input, to verify whether it matches the actual password. However, I would store the password and other financial data as ciphertext in the database. This would ensure that if anyone looks at the database, the data would be meaningless.

To protect the integrity of data, I will check the data type from the front end, after every time a user submits an input, to reduce the risk of having data type or run time errors. Furthermore, I would implement a few input controls to restrict user input, such as drop-down menus, radio buttons and date pickers. These systems will occur in the frontend.

Algorithm Design

Vernam Cipher

As I will need to ensure the security of confidential user data, such as passwords, I have created a Vernam Cipher. This will be profound for the rest of the system to ensure that users are able to trust that their data is kept quite securely.

This algorithm converts plain text to its ASCII equivalent, and then creates a random Vernam key. Once the random Vernam Key has been created, it will use the logical XOR operator to turn the bits of the ASCII text to a 1 if the the bits of the ASCII text and Vernam key are different. Otherwise, it will get to a 0. Once all ASCII values have undergone the logical XOR operator, the cipher text has been created. Finally, the algorithm is able to convert the cipher text back to plain text, by applying the logical XOR operator on the cipher text, which gives the plain text but in ASCII. By grouping the ASCII values in 7 characters, the algorithm is able to convert the ASCII code to plain text.

```
FUNCTION convertPlainTextToBinary(plainText)
 binaryText ← ""

 FOR i ← 0 TO LENGTH(plainText) - 1
     ch ← plainText[i]
     binaryValue ← TO_BINARY(ASCII(ch))
     binaryText ← binaryText & binaryValue
 NEXT i

 RETURN binaryText

FUNCTION createVernamKey(plainText)
 keyLength ← LENGTH(plainText) * 7
 vernamKey ← ""
```

```
 FOR i ← 0 TO keyLength - 1
     bit ← RANDOMINT(0, 1)
     vernamKey ← vernamKey & bit
 NEXT i

 RETURN vernamKey

FUNCTION XORfunction(binaryText, vernamKey)
 XORText ← ""

 FOR i ← 0 TO LENGTH(binaryText) - 1
     IF binaryText[i] = vernamKey[i] THEN
         XORText ← XORText & "0"
     ELSE
         XORText ← XORText & "1"
 NEXT i

 RETURN XORText

FUNCTION convertCipherTextToPlainText(cipherText, vernamKey)
 plainText ← ""
 sevenBits ← ""
 xorResult ← XORfunction(cipherText, vernamKey)

 FOR i ← 0 TO LENGTH(xorResult) - 1
     sevenBits ← sevenBits & xorResult[i]

     IF LENGTH(sevenBits) = 7 THEN
         asciiValue ← BINARYTOINTEGER(sevenBits)
```

```
        ch ← CHARACTER(asciiValue)
        plainText ← plainText & ch
        sevenBits ← ""
NEXT i

RETURN plainText
```

Recursive Merge Sort Algorithm

I created a recursive merge sort algorithm. This will be very useful for most of my objectives, such as enabling consumers to sort economic news by their title, or date published, or for consumers to sort how they spent their budget, by date, price, or quantity bought.

This code will recursively half the input list into two halves until each sublist has only 1 element. It then calls the merge method to combine the sorted sublists back into a single sorted list. The merge method goes through the two sublists, comparing elements and adding the smaller one to the result list, and then appends any remaining elements once there are no elements in one of the sublists.

```
FUNCTION mergeSort(arr, comparator):
    IF size of arr < 2:
        RETURN

    mid ← size of arr / 2
    left ← sublist of arr from 0 to mid
    right ← sublist of arr from mid to end

    mergeSort(left, comparator)
    mergeSort(right, comparator)

    merge(arr, left, right, comparator)

FUNCTION merge(result, left, right, comparator):
    i ← 0
    j ← 0
    CLEAR result

    WHILE i < size(left) AND j < size(right):
```

```
        IF comparator(left[i], right[j]) ≤ 0:
            ADD left[i] TO result
            i ← i + 1
        ELSE:
            ADD right[j] TO result
            j ← j + 1

    WHILE i < size(left):
        ADD left[i] TO result
        i ← i + 1

    WHILE j < size(right):
        ADD right[j] TO result
        j ← j + 1
```

Recursive Binary Search Algorithm

I have created a recursive bubble search algorithm. This will be very useful for most of my objectives, such as enabling consumers to search for their desired piece of news, or for producers who want to search and compare the costs and revenue a particular factor inputs possesses.

The following algorithm compares the middle item in the array to the key. If the middle item is equal to the key, the searching algorithm stops and returns the index where the key was found. If the key is greater than the middle item, the algorithm will ignore any values less than and equal to the middle value. However, if the key is less than the middle item, the algorithm will ignore any values greater than and equal to the middle value. Then, the algorithm will compare the middle item of the shrunken array with the key and this process will continue until the key is found. In the case the key is not in the array, the algorithm will return "-1"

```
FUNCTION binarySearchInteger(arr, min, max, key):
    IF min > max:
        RETURN -1

    mid ← (min + max) / 2

    IF arr[mid] == key:
        RETURN mid
    ELSE IF arr[mid] < key:
        RETURN binarySearchInteger(arr, mid + 1, max, key)
    ELSE:
        RETURN binarySearchInteger(arr, min, mid - 1, key)

FUNCTION binarySearchObject(arr, min, max, key):
    IF min > max:
        RETURN -1
```

```
mid ← (min + max) / 2
comparison ← compare(key, arr[mid].toString())

IF comparison == 0:
    RETURN mid
ELSE IF comparison > 0:
    RETURN binarySearchObject(arr, mid + 1, max, key)
ELSE:
    RETURN binarySearchObject(arr, min, mid - 1, key)
```

Risky Purchases

This algorithm will calculate whether a good that a user purchases is considered to be an outlier, by checking whether the price of a given good bought is 1.5 times above the standard deviation of all goods bought.

The algorithm calculates the standard deviation of the price of a good, and groups purchases based on the name of the good. If the price of the good is above (1.5 x standard deviation) of total goods, it will be considered to be an outlier good and stored in the outlier good record.

```
FUNCTION calculateMean(values):
    sum ← 0
    FOR val IN values:
        sum ← sum + val
    RETURN sum / LENGTH(values)

FUNCTION calculateVariance(values, mean):
    varianceSum ← 0
    FOR val IN values:
        varianceSum ← varianceSum + ((val - mean) * (val - mean))
    RETURN varianceSum / LENGTH(values)

FUNCTION calculateStdDev(values):
    IF values IS EMPTY:
        RETURN 0

    mean ← calculateMean(values)
    variance ← calculateVariance(values, mean)
    RETURN squareroot(variance)
```

```
FUNCTION groupPurchases(purchases):
    totalsPerItem ← empty map

    FOR purchase IN purchases:
        item ← purchase.getGoodName()
        IF item NOT IN totalsPerItem:
            totalsPerItem[item] ← empty list
        ENDIF
        ADD purchase.getTotalPrice() TO totalsPerItem[item]
    ENDFOR

    RETURN totalsPerItem

FUNCTION calculateVolatility(purchases, totalsPerItem):
    volatility ← empty map

    FOR (item, totals) IN totalsPerItem:
        IF totals IS EMPTY:
            volatility[item] ← 0
        ELSE:
            volatility[item] ← calculateStdDev(totals)

    RETURN volatility

FUNCTION calculateOutliers(totalsPerItem, volatility):
    outlierGoods ← empty list

    FOR (goodName, totals) IN totalsPerItem:
        mean ← calculateMean(totals)
```

```
    stdDev ← volatility[goodName]

    FOR each price IN totals:
        TRY:
            z ← (price - mean) / stdDev
        CATCH division by zero:
            z ← 0
        FINALLY:
            IF absolute(z) > 1.5:
                ADD new OutlierGoods(goodName, price, z) TO outlierGoods

RETURN outlierGoods
```

Combinatorial Subset Search

This algorithm will generate every possible subsets of a list that also have certain constraints. In this case I will find every combination of factor inputs, which is the most profitable worker, under a maximum cost constraint.

This means that if I have a list of workers, and have a maximum cost constraint of £100, the algorithm will select the most profitable workers, where the cumulative costs of the most profitable workers are less or equal to £100

```
FUNCTION generateCombinations(factorInputs):
    allCombos ← empty list of list of FactorInput
    n ← length of factorInputs

    FOR size FROM 1 TO n:
        CALL combine(factorInputs, allCombos, empty list, 0, size)

    RETURN allCombos

FUNCTION combine(factorInputs, allCombos, current, start, size):
    IF length of current == size THEN
        ADD copy of current TO allCombos
        RETURN

    FOR i FROM start TO length of factorInputs - 1:
        ADD factorInputs[i] TO current
        CALL combine(factorInputs, allCombos, current, i + 1, size)
        REMOVE last element FROM current

FUNCTION totalProfit(combo):
    profit ← 0
```

```
    FOR factorInput IN combo:
        profit ← profit + factorInput.marginalProfit
    RETURN profit

FUNCTION totalCost(combo):
    cost ← 0
    FOR each factorInput IN combo:
        cost ← cost + factorInput.marginalCost

    RETURN cost

FUNCTION showBestCombination(allFactorInputs, maxCost):
    allCombos ← generateCombinations(allFactorInputs)
    bestProfit ← NULL
    bestCombo ← empty list

    FOR combo IN allCombos:
        cost ← totalCost(combo)

        IF cost ≤ maxCost THEN
            profit ← totalProfit(combo)

            IF profit > bestProfit THEN
                bestProfit ← profit
                bestCombo ← combo

    RETURN bestCombo
```

API Endpoints & Sample Data

For the following RSS.XML files, I will be only showing the data of 1 news article. However, the .XML files will have multiple pieces of data for each article. For all the .XML files, each article will be contained in the `<image>` attribute, which makes it easier to access data.

BBC News

https://feeds.bbci.co.uk/news/business/economy/rss.xml

To extract the relevant details, from BBC News, I have used the BBC News - Economy rss.xml file to extract the following attributes:

Title: `<title></title>`
Description: `<description></description>`
Published Date: `<pubDate></pubDate>`
Link: `<link></link>`
Image: `<media:thumbnail>`

```
<rss
      xmlns:dc="http://purl.org/dc/elements/1.1/"
      xmlns:content="http://purl.org/rss/1.0/modules/content/"
      xmlns:atom="http://www.w3.org/2005/Atom"
      xmlns:media="http://search.yahoo.com/mrss/" version="2.0">
      <channel>
            <title>
                  <![CDATA[ BBC News ]]>
            </title>
            <description>
                  <![CDATA[ BBC News - Economy ]]>
            </description>
            <link>https://www.bbc.co.uk/news/business/economy</link>
            <image>

<url>https://news.bbcimg.co.uk/nol/shared/img/bbc_news_120x60.gif</url>
                  <title>BBC News</title>

<link>https://www.bbc.co.uk/news/business/economy</link>
            </image>
            <generator>RSS for Node</generator>
            <lastBuildDate>Thu, 18 Sep 2025 11:50:28 GMT</lastBuildDate>
            <atom:link
href="https://feeds.bbci.co.uk/news/business/economy/rss.xml" rel="self"
```

```
type="application/rss+xml"/>
            <copyright>
                  <![CDATA[ Copyright: (C) British Broadcasting
Corporation, see
https://www.bbc.co.uk/usingthebbc/terms-of-use/#15metadataandrssfeeds
for terms and conditions of reuse. ]]>
            </copyright>
            <language>
                  <![CDATA[ en-gb ]]>
            </language>
            <ttl>15</ttl>
            <item>
                  <title>
                        <![CDATA[ UK 'not out of woods' on inflation,
says Bank as rates held ]]>
                  </title>
                  <description>
                        <![CDATA[ The Bank of England governor warns
"we're not out of the woods yet" in terms of rising inflation. ]]>
                  </description>

<link>https://www.bbc.com/news/articles/cge2q7wvyz3o?at_medium=RSS&at_ca
mpaign=rss</link>
                  <guid
isPermaLink="false">https://www.bbc.com/news/articles/cge2q7wvyz3o#0</gu
id>
                  <pubDate>Thu, 18 Sep 2025 11:39:17 GMT</pubDate>
                  <media:thumbnail width="240" height="134"
url="https://ichef.bbci.co.uk/ace/standard/240/cpsprodpb/2d9e/live/f92b0
2e0-93b3-11f0-af04-cd07e8e3d1e7.jpg"/>
            </item>
      </channel>
</rss>
```

The Telegraph

https://www.telegraph.co.uk/business/rss.xml

To extract the relevant details, from The Telegraph, I have used the Business rss.xml file from the The Telegraph to extract the following attributes:

Title: <title></title>
Description: <description></description>
Published Date: <pubDate></pubDate>
Link: <link></link>
Image: <enclosure>

```
<rss
      xmlns:dc="http://purl.org/dc/elements/1.1/"
      xmlns:ns0="http://www.w3.org/2005/Atom" version="2.0">
      <channel>
            <title>Business</title>
            <link>https://www.telegraph.co.uk/business/</link>
            <ns0:link
href="https://www.telegraph.co.uk/business/rss.xml" rel="self"
type="application/rss+xml"/>
            <description>www.telegraph.co.uk for the latest news from
the UK and around the world.</description>
            <language>en-GB</language>
            <lastBuildDate>Thu, 18 Sep 2025 09:36:15 GMT</lastBuildDate>
            <image>
                  <title>Telegraph Logo</title>

<url>https://www.telegraph.co.uk/content/dam/telegraph-logos/Micro%2089-
189w%20-%20Black.png</url>

<link>https://www.telegraph.co.uk/content/dam/telegraph-logos/Micro%2089
-189w%20-%20Black.png</link>
            </image>
            <ttl>1</ttl>
            <item>
                  <title>Fed cuts US interest rates after Trump pressure
as economy weakens</title>

<link>https://www.telegraph.co.uk/business/2025/09/17/federal-reserve-de
cision-interest-rates-trump-pressure/</link>
                  <description>Fed cuts US interest rates after Trump
pressure as economy weakens</description>
                  <pubDate>Wed, 17 Sep 2025 20:01:03 GMT</pubDate>
                  <guid
```

```
isPermaLink="false">b7e84c2c-789d-315a-a38c-1fa3354bc43e</guid>
                  <dc:creator>EMMA TAGGART</dc:creator>
                  <dc:creator>Hans van Leeuwen</dc:creator>
                  <category
domain="">topics:things/uk-economy</category>
                  <category
domain="">topics:organisations/ftse-100</category>
                  <category domain="">topics:things/pound</category>
                  <category
domain="">topics:organisations/dow-jones-industrial-average</category>
                  <category
domain="">topics:in-the-news/cost-of-living</category>
                  <category
domain="">topics:things/global-economy</category>
                  <category domain="">structure:business</category>
                  <category domain="">structure:live-priority</category>
                  <enclosure
url="https://www.telegraph.co.uk/content/dam/business/2025/09/17/TELEMMG
LPICT000420460610_17581283745200_trans_NvBQzQNjv4BqgsaO8O78rhmZrDxTlQBjd
GLvJF5WfpqnBZShRL_tOZw.jpeg" type="image/jpeg" length="99"/>
            </item>
      </channel>
</rss>
```


Tharun Nadeswaran - Money Matters
Student Code -
Centre Number -
100

The Economist

https://www.economist.com/finance-and-economics/rss.xml

To extract the relevant details, from The Economist, I have used the Finance And Economics rss.xml file from the The Economist to extract the following attributes:

Title: <title></title>
Description: <description></description>
Published Date: <pubDate></pubDate>
Link: <link></link>


```
<rss
      xmlns:content="http://purl.org/rss/1.0/modules/content/"
      xmlns:atom="http://www.w3.org/2005/Atom" version="2.0">
      <channel>
            <title>
                  <![CDATA[ Finance & economics ]]>
            </title>
            <description/>
            <link>https://www.economist.com/finance-and-economics</link>
            <pubDate>Thu, 18 Sep 2025 10:26:31 +0000</pubDate>
            <lastBuildDate>Thu, 18 Sep 2025 10:26:31
+0000</lastBuildDate>
            <atom:link
href="https://www.economist.com/finance-and-economics/rss.xml"
rel="self" type="application/rss+xml"/>
            <item>
                  <title>
                        <![CDATA[ Would an all-out trade war be better?
]]>
                  </title>
                  <description>
                        <![CDATA[ Donald Trump has so far avoided
retaliation, which might carry a cost of its own ]]>
                  </description>

<link>https://www.economist.com/finance-and-economics/2025/09/18/would-a
n-all-out-trade-war-be-better</link>
                  <guid
isPermaLink="false">1a3fb93a-8ca0-4f12-b109-174dd51f13a3</guid>
                  <pubDate>Thu, 18 Sep 2025 10:26:22 +0000</pubDate>
            </item>
      </channel>
</rss>
```

The Independent

https://www.independent.co.uk/news/business/rss

To extract the relevant details, from The Independent, I have used the The Independent Business rss.xml file to extract the following attributes:

Title: <title></title>
Description: <description></description>
Published Date: <pubDate></pubDate>
Link: <link></link>
Image: <media:content>

```
<rss
      xmlns:media="http://search.yahoo.com/mrss/"
      xmlns:dc="http://purl.org/dc/elements/1.1/"
      xmlns:atom="http://www.w3.org/2005/Atom" version="2.0">
      <channel>
            <title>The Independent Business</title>
            <link>https://www.independent.co.uk/news/business/rss</link>
            <description>Visit now for the latest news direct from The
Independent Business and updated throughout the day</description>
            <language>en-gb</language>
            <copyright>Copyright 2025, ESI Media Ltd</copyright>
            <pubDate>Thu, 18 Sep 2025 12:08:24 GMT</pubDate>
            <dc:date>2025-09-18T12:08:24Z</dc:date>
            <dc:language>en-gb</dc:language>
            <dc:rights>ESI Media Ltd or its affiliated companies. All
rights reserved. 2025</dc:rights>
            <link rel="self"
href="https://www.independent.co.uk/news/business/rss"
type="application/xml"/>
            <link rel="hub" href="https://pubsubhubbub.appspot.com/"/>
            <item>
                  <title>
                        <![CDATA[ Bank of England holds UK interest rates
at 4% amid inflation concerns as mortgage holders issued warning - live
]]>
                  </title>

<link>https://www.independent.co.uk/news/business/interest-rates-bank-of
-england-mortgage-ftse-100-latest-news-b2828851.html</link>
                  <description>
                        <![CDATA[ <p>MPC vote latest along with stock
```

```
market news plus business and finance updates from Thursday</p> ]]>
                  </description>
                  <pubDate>Thu, 18 Sep 2025 11:55:56 GMT</pubDate>
                  <guid isPermaLink="false">b2828223</guid>
                  <media:content
url="https://static.independent.co.uk/2025/08/07/16/75d753f66ba3a74227dd
1cad195be327Y29udGVudHNlYXJjaGFwaSwxNzU0NjY1MjE5-2.81157811.jpg?width=12
00&auto=webp&crop=3%3A2" type="image/jpeg" medium="image">
                        <media:credit>PA Wire</media:credit>
                        <media:text>
                              <![CDATA[ Bank of England holds interest
rates at 4% amid inflation concerns (John Walton/PA) - (PA Wire) ]]>
                        </media:text>
                  </media:content>
                  <dc:creator>
                        <![CDATA[ Karl Matchett ]]>
                  </dc:creator>
                  <dc:date>2025-09-18T11:55:56+00:00</dc:date>
                  <category>
                        <![CDATA[ Business ]]>
                  </category>
            </item>
      </channel>
</rss>
```


Tharun Nadeswaran - Money Matters
Student Code -
Centre Number -
103

CNBC

https://search.cnbc.com/rs/search/combinedcms/view.xml?partnerId=wrss01&id=10000664

To extract the relevant details, from CNBC, I have used the Finance CNBC rss.xml to extract the following attributes:

Title: <title></title>
Description: <description></description>
Published Date: <pubDate></pubDate>
Link: <link></link>

```
<rss
xmlns:metadata="http://search.cnbc.com/rss/2.0/modules/siteContentMetada
ta"
      xmlns:atom="http://www.w3.org/2005/Atom"
      xmlns:cnbc="http://cnbc.com/rss/elements/1.0/" version="2.0">
      <channel>
            <atom:link rel="self" type="application/rss+xml"
href="https://www.cnbc.com/id/10000664/device/rss/rss.html"/>
            <language>en-us</language>
            <ttl>60</ttl>
            <title>Finance</title>
            <feed_asset>
                  <id>10000664</id>
                  <type>franchise</type>
                  <subtype>section</subtype>
                  <seo>https://www.cnbc.com/finance/</seo>
            </feed_asset>
            <lastBuildDate>Thu, 18 Sep 2025 12:11 GMT</lastBuildDate>
            <pubDate>Thu, 18 Sep 2025 12:11 GMT</pubDate>
            <link>https://www.cnbc.com/finance/</link>
            <description>
                  <![CDATA[ Latest investing news and finance headlines
straight from Wall Street. ]]>
            </description>
            <item>

<link>https://www.cnbc.com/2025/09/18/american-express-platinum-card-ref
resh-895-fee-3500-perks.html</link>
                  <guid isPermaLink="false">108200293</guid>
                  <metadata:type>cnbcnewsstory</metadata:type>
                  <metadata:id>108200293</metadata:id>
                  <metadata:sponsored>false</metadata:sponsored>
```

Tharun Nadeswaran - Money Matters
Student Code -
Centre Number -
104

```
                <title>American Express unveils refreshed Platinum
card with $895 annual fee, upping the ante in luxury cards</title>
                <description>
                      <![CDATA[ American Express' move highlights an
arms race of sorts when it comes to catering to wealthy consumers.
JPMorgan and Citi released updated or new cards recently. ]]>
                </description>
                <pubDate>Thu, 18 Sep 2025 11:26:22 GMT</pubDate>
            </item>
      </channel>
</rss>
```

Frankfurter API

I have used the Frankfurter Public API, to access exchange rates. The API will return a .JSON payload, where I will extract the relevant data from.
Base URL = https://api.frankfurter.app

Exchange Rate for a single day
https://api.frankfurter.app/2025-09-10?base=GBP&symbols=USD

Query Parameters
date = single date to fetch data
base = the current currency code
symbol = the wanted currency code

Fetching data from the Frankfurter API, I will get the following .JSON payload.


```
{
    "amount": 1,
    "base": "GBP",
    "date": "2025-09-18",
    "rates": {
        "USD": 1.3615
    }
}
```


I will extract the rates -> wantedCurrency attribute, to get the exchange rate.

```
    "rates": {
        "wantedCurrency": value
    }
```

Exchange Rate for a series of days
https://api.frankfurter.app/2025-09-10..?base=GBP&symbols=USD

Query Parameters
date = starting date to fetch data
.. = retrieve all data from the starting dates until the present
base = the current currency code
symbol = the wanted currency code


```
{
    "amount": 1,
    "base": "GBP",
    "start_date": "2025-09-10",
    "end_date": "2025-09-18",
    "rates": {
        "2025-09-10": {
            "USD": 1.3534
        },
        "2025-09-11": {
            "USD": 1.3513
        },
        "2025-09-12": {
            "USD": 1.3542
        },
        "2025-09-15": {
            "USD": 1.3616
        },
        "2025-09-16": {
            "USD": 1.364
        },
        "2025-09-17": {
            "USD": 1.3645
        },
        "2025-09-18": {
            "USD": 1.3615
        }
    }
}
```

I will extract the rates -> date -> wantedCurrency attribute, to get the exchange rate. I will extract all the wantedCurrency attributes for all the dates in the .JSON payload.

```
    "rates": {
        "date": {
            "wantedCurrency": value
        }
    }
```

WorldBank API

Fetch Macro-Economic Indicators

https://api.worldbank.org/v2/countries/GB/indicators/NY.GDP.MKTP.CD?format=json&per_page=1000

I have used the WorldBank Public API, to access macro-economic indicators, from the given country code and indicator code. The API will return a .JSON payload, where I will extract the relevant data from.

Query Parameters
countries/GB = Country code (in this case it is United Kingdom)
indicators/NY.GDP.MKTP.CD = Indicator code (in this case it is NY.GDP.MKTP.CD (GDP, current US$))
format=json = Response format is JSON
per_page=1000 = Limit results per page to 1000 (for this sample data, I have only shown the first 3 results)


```
[
    {
        "page": 1,
        "pages": 22,
        "per_page": 3,
        "total": 65,
        "sourceid": "2",
        "lastupdated": "2025-07-01"
    },
    [
        {
            "indicator": {
                "id": "NY.GDP.MKTP.CD",
                "value": "GDP (current US$)"
            },
            "country": {
                "id": "GB",
                "value": "United Kingdom"
            },
            "countryiso3code": "GBR",
            "date": "2024",
            "value": 3643834188782.91,
            "unit": "",
            "obs_status": "",
            "decimal": 0
```

```
        },
        {
            "indicator": {
                "id": "NY.GDP.MKTP.CD",
                "value": "GDP (current US$)"
            },
            "country": {
                "id": "GB",
                "value": "United Kingdom"
            },
            "countryiso3code": "GBR",
            "date": "2023",
            "value": 3369861888972.88,
            "unit": "",
            "obs_status": "",
            "decimal": 0
        },
        {
            "indicator": {
                "id": "NY.GDP.MKTP.CD",
                "value": "GDP (current US$)"
            },
            "country": {
                "id": "GB",
                "value": "United Kingdom"
            },
            "countryiso3code": "GBR",
            "date": "2022",
            "value": 3114042471144.39,
            "unit": "",
            "obs_status": "",
            "decimal": 0
        }
    ]
]
```

Fetch Available Countries
https://api.worldbank.org/v2/country/?format=json&per_page=1000

I have used the WorldBank Public API, to available countries, so that the user can look up a specific country's macroeconomic data. The API will return a .JSON payload, where I will extract the relevant data from.

Query Parameters
format=json = Response format is JSON
per_page=1000 = Limit results per page to 1000 (for this sample data, I have only shown the first 3 results)


```
[
    {
        "page": 1,
        "pages": 99,
        "per_page": "3",
        "total": 296
    },
    [
        {
            "id": "ABW",
            "iso2Code": "AW",
            "name": "Aruba",
            "region": {
                "id": "LCN",
                "iso2code": "ZJ",
                "value": "Latin America & Caribbean "
            },
            "adminregion": {
                "id": "",
                "iso2code": "",
                "value": ""
            },
            "incomeLevel": {
                "id": "HIC",
                "iso2code": "XD",
                "value": "High income"
            },
            "lendingType": {
                "id": "LNX",
                "iso2code": "XX",
                "value": "Not classified"
            },
            "capitalCity": "Oranjestad",
```

```
            "longitude": "-70.0167",
            "latitude": "12.5167"
        },
        {
            "id": "AFE",
            "iso2Code": "ZH",
            "name": "Africa Eastern and Southern",
            "region": {
                "id": "NA",
                "iso2code": "NA",
                "value": "Aggregates"
            },
            "adminregion": {
                "id": "",
                "iso2code": "",
                "value": ""
            },
            "incomeLevel": {
                "id": "NA",
                "iso2code": "NA",
                "value": "Aggregates"
            },
            "lendingType": {
                "id": "",
                "iso2code": "",
                "value": "Aggregates"
            },
            "capitalCity": "",
            "longitude": "",
            "latitude": ""
        },
        {
            "id": "AFG",
            "iso2Code": "AF",
            "name": "Afghanistan",
            "region": {
                "id": "MEA",
                "iso2code": "ZQ",
                "value": "Middle East, North Africa, Afghanistan &
Pakistan"
            },
            "adminregion": {
                "id": "MNA",
                "iso2code": "XQ",
                "value": "Middle East, North Africa, Afghanistan &
Pakistan (excluding high income)"
```

```
            },
            "incomeLevel": {
                "id": "LIC",
                "iso2code": "XM",
                "value": "Low income"
            },
            "lendingType": {
                "id": "IDX",
                "iso2code": "XI",
                "value": "IDA"
            },
            "capitalCity": "Kabul",
            "longitude": "69.1761",
            "latitude": "34.5228"
        }
    ]
]
```

Yfinance Python Library

Using the Yfinance library, I instantiated a Ticker Object, of a certain stock, which will be useful to retrieve information of the company or the stock.

getInfoOfCompany():

Choose the fields to extract from the company, and return a dictionary with the selected company information.


```
{
    "symbol": "AAPL",
    "shortName": "Apple Inc.",
    "sector": "Technology",
    "industry": "Consumer Electronics",
    "fullTimeEmployees": 150000,
    "country": "United States",
    "website": "https://www.apple.com",
}
```

getStockData("2025-09-10")

Fetch historical stock prices and volumes, from the start date, which is set in the parameter to the latest date that has stock information.

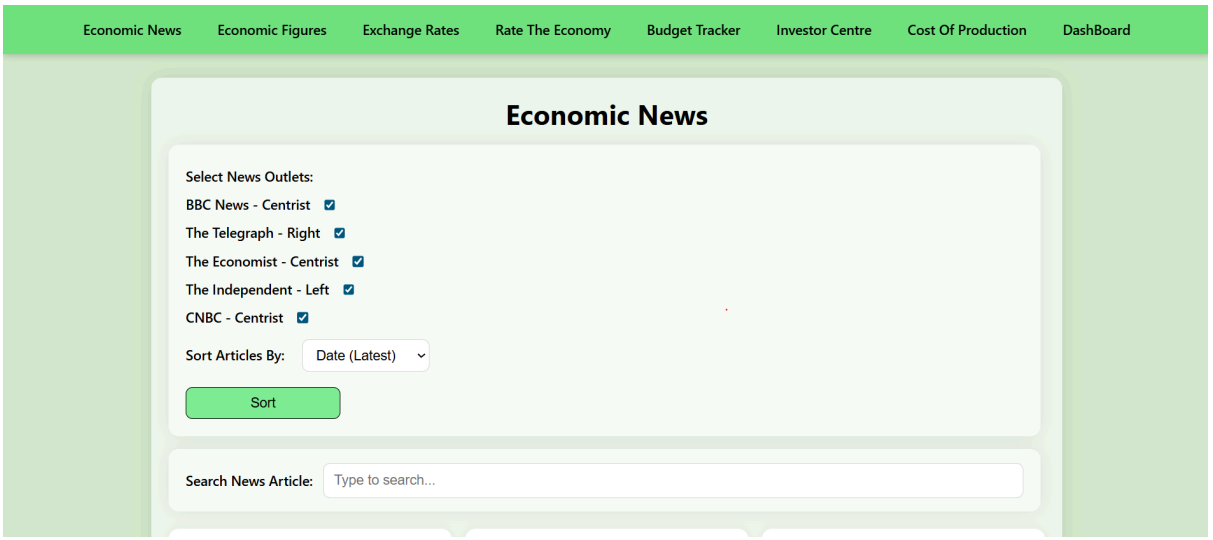

```
[
    {
        "Date": "10/09/25",
        "Open": 232.19,
        "High": 232.42,
        "Low": 225.95,
        "Close": 226.79,
        "Volume": 83440800,
    },
    {
        "Date": "11/09/25",
        "Open": 226.88,
        "High": 230.45,
        "Low": 226.65,
        "Close": 230.03,
        "Volume": 50208600,
    },
    {
        "Date": "12/09/25",
        "Open": 229.22,
        "High": 234.51,
        "Low": 229.02,
        "Close": 234.07,
        "Volume": 55824200,
    },
    {
        "Date": "15/09/25",
        "Open": 237.0,
        "High": 238.19,
        "Low": 235.03,
        "Close": 236.7,
        "Volume": 42699500,
    },
    {
        "Date": "16/09/25",
        "Open": 237.18,
        "High": 241.22,
        "Low": 236.32,
        "Close": 238.15,
        "Volume": 63421100,
    },
]
```

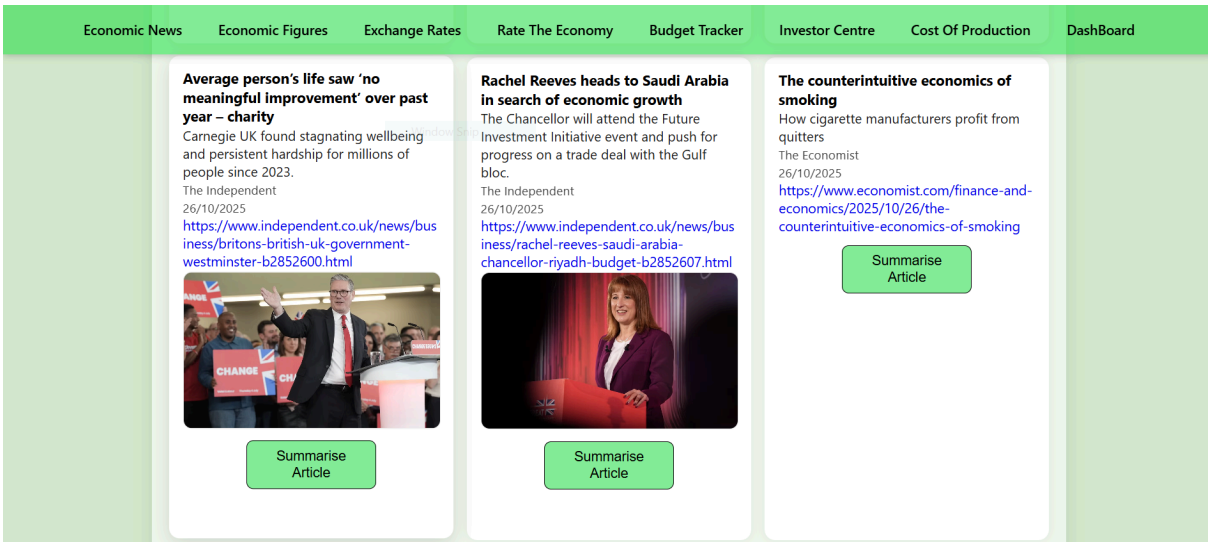
User Interface Designs

After considering my client's feedback and improving the earlier drafts, these are the versions that will be used to build the actual website.

Objective 1

This page will display several news articles, from different news sources, where users can see relevant details from the articles and view a summary of each news article. Users are able to search and sort economic news by the source, date or title.





Slow growth raises stakes even higher for the Budget

The Budget somehow has to provide certainty, boost consumer and business confidence, and fill a large fiscal gap.

BBC News

13/11/2025

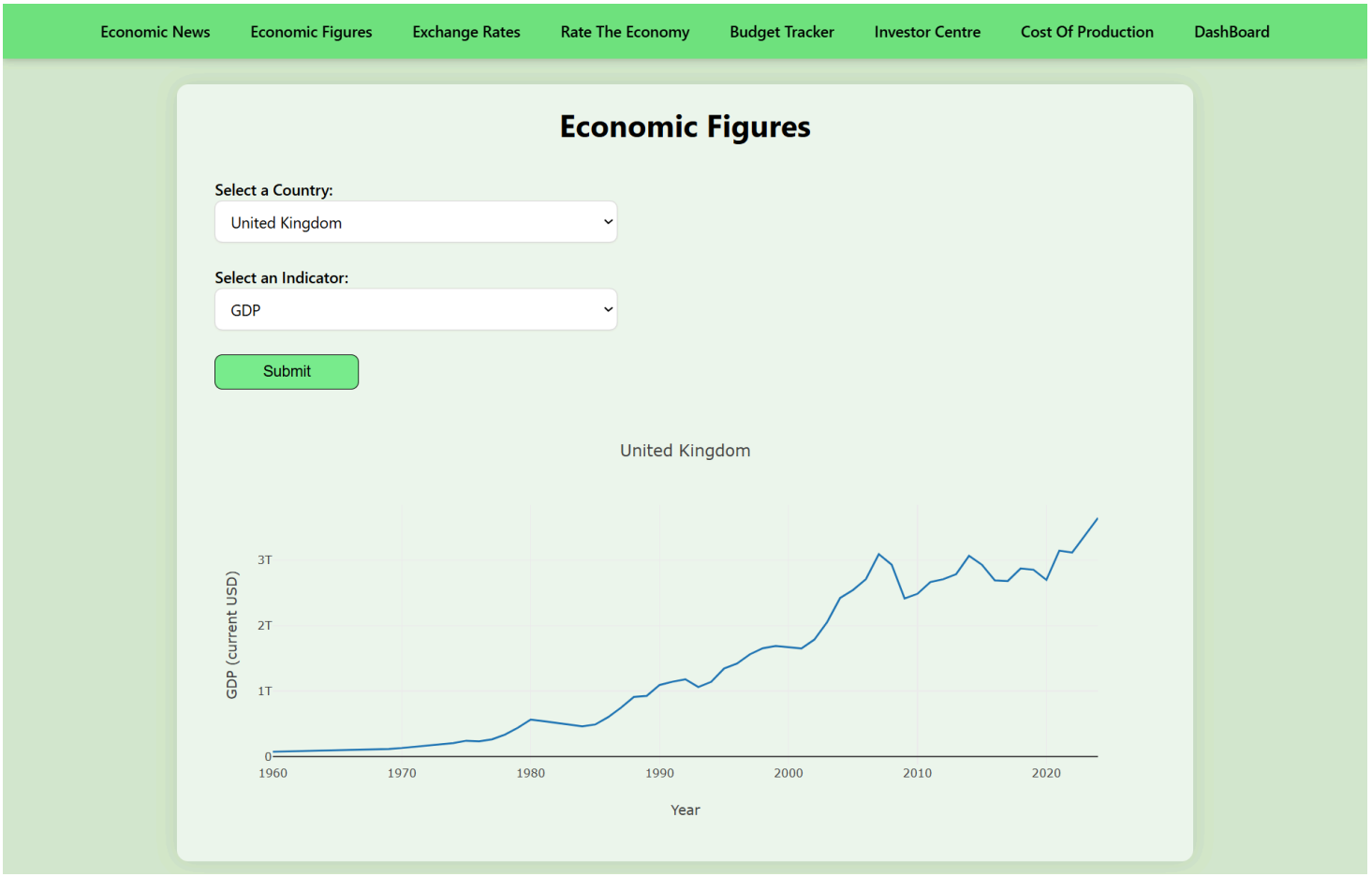
https://www.bbc.com/news/articles/c93x5xw22n6o?at_medium=RSS&at_campaign=rss



The UK's economic growth has stagnated at lower-than-expected levels, with July-to-September GDP shrinking by 0.1%. This contraction is attributed to Jaguar Land Rover's cyber-attack which led to vehicle production delays. The subsequent September decrease was disproportionate due to the worst monthly fall on record outside of the pandemic, affecting consumer and business investment. Cautious consumers and uncertain businesses are hindering economic growth. Key to aiding this slowdown is the Budget's aim to provide more certainty in taxation, potentially shielding workers from rising income tax rates. However, such measures may face practical challenges due to their financial implications.

Objective 2

Users can search for a range of macro-economic indicators and view graphs that show trends in exchange rates and other key data. The graphs are interactive, allowing users to hover over a specific year to see the exact values.



Exchange Rates

Select the Base Currency:

Australian Dollar

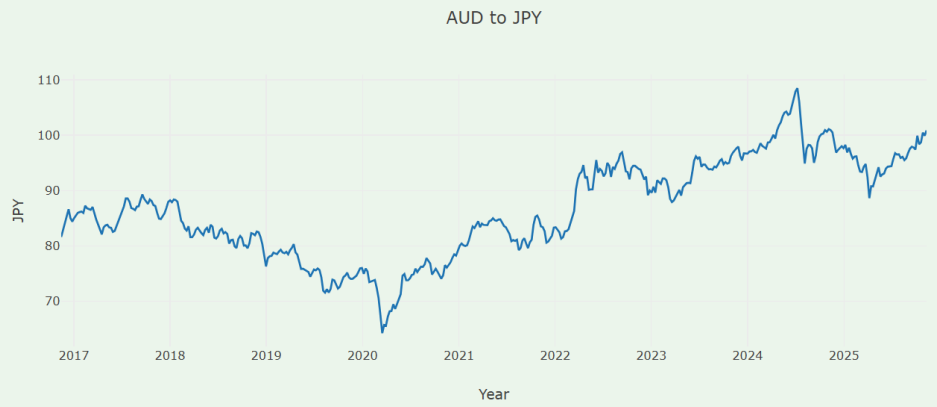
Select the Wanted Currency:

Japanese Yen

Select a Time Period:

14/11/2025

Submit



Currency Converter

Date: 14/11/2016

1.00 AUD = 81.44 JPY

Enter AUD:

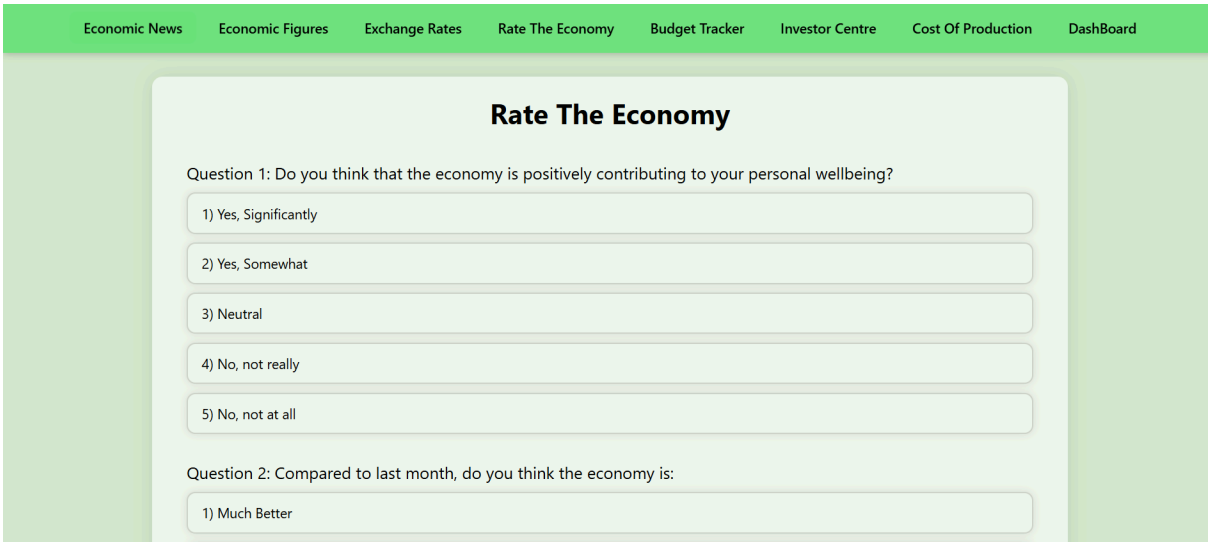
132.44

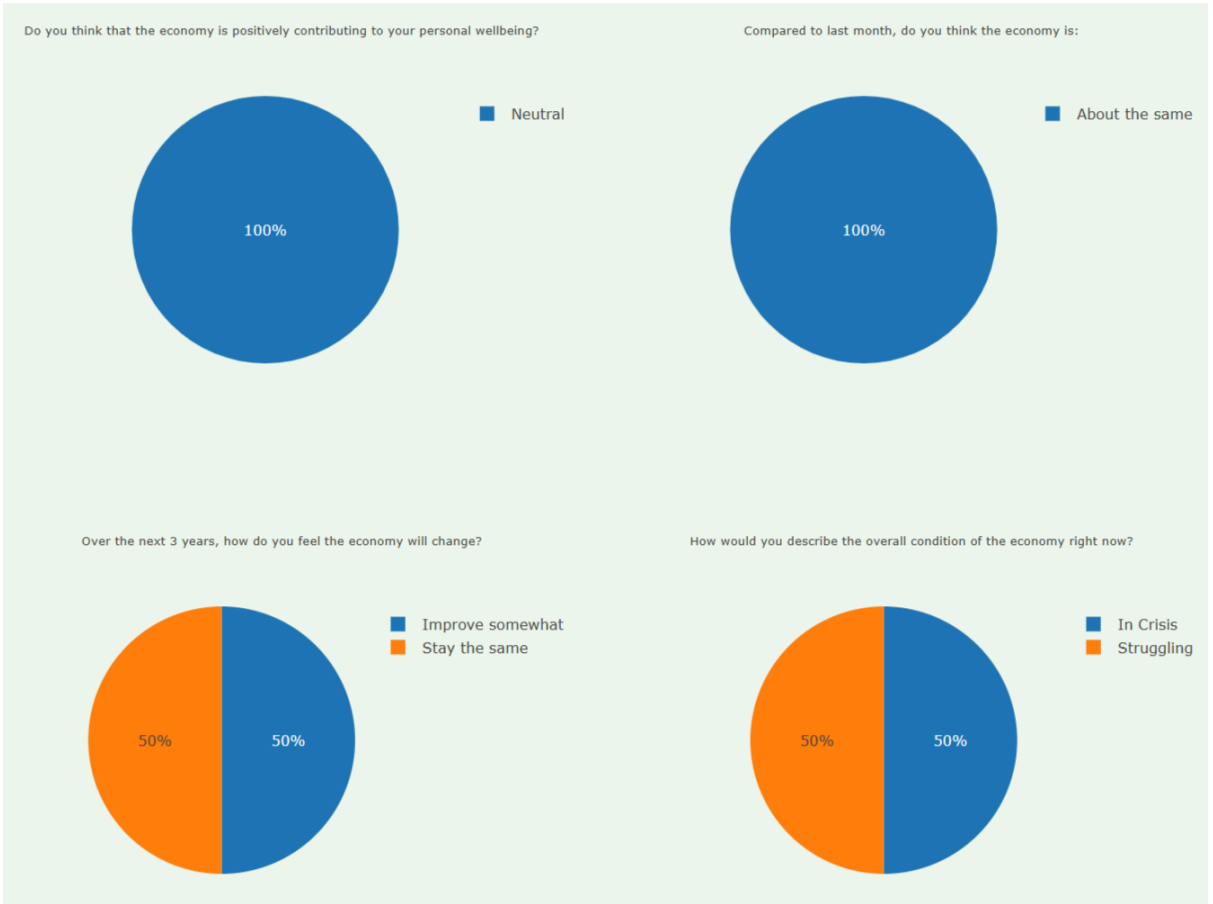
Convert Currency

132.44 AUD = 10785.91 JPY

Objective 3

Users can answer questions to help inform themselves, and others, about their opinions on the economy. After submitting their answers, they can then view how other people have responded and see what the public thinks about the state of the economy at selected dates, where their choices are shown through pie charts.

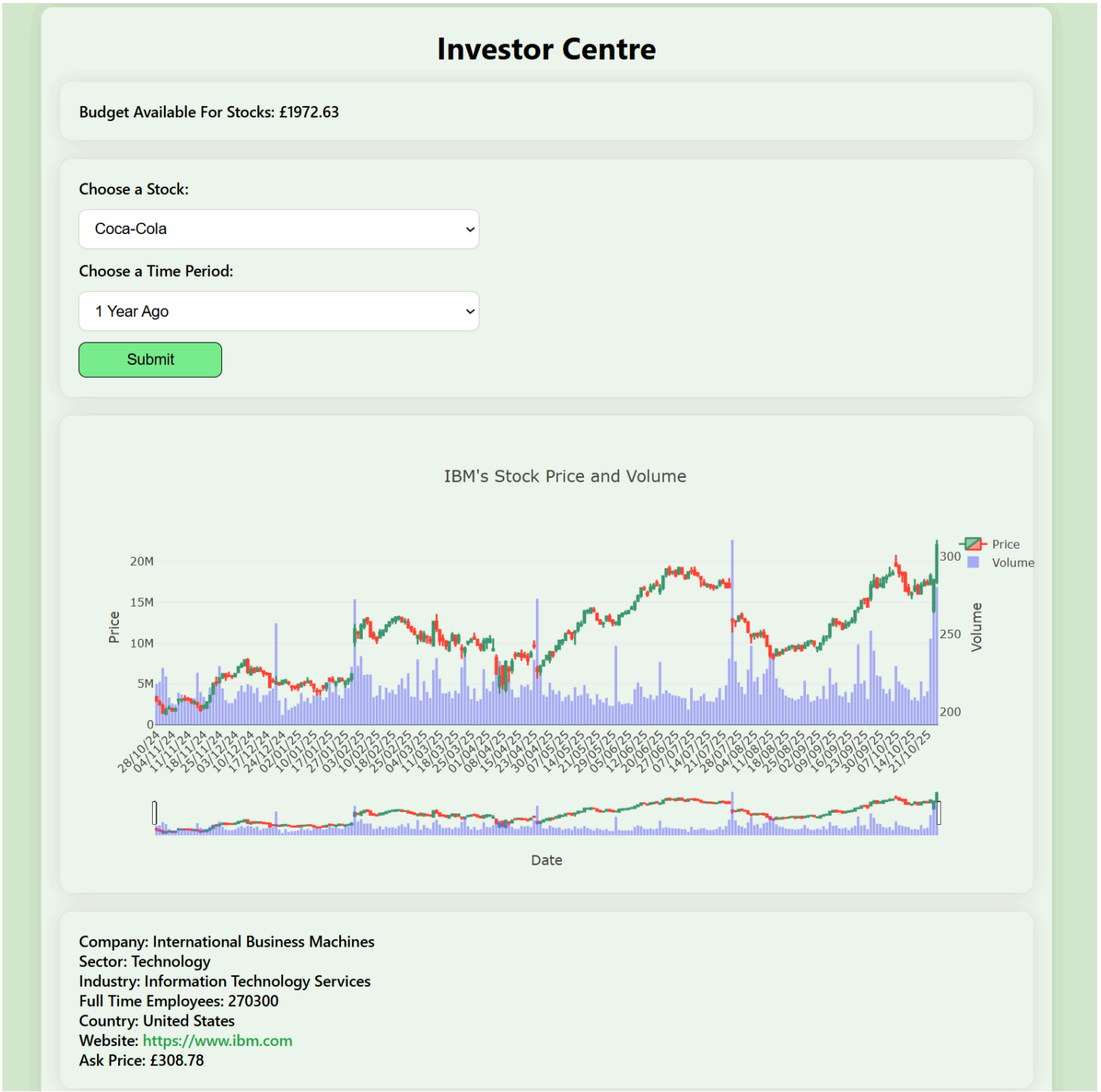




Tharun Nadeswaran - Money Matters
Student Code -
Centre Number -
120

Objective 4

Users can select different stocks and view candlestick graphs that display key stock data, including trends such as the SMA and information about any stocks they own. They can also buy and sell stocks, and over time they can choose to sell their investments to make either a profit or a loss.



Purchase Stocks

Buy Stock From IBM: £308.78

Quantity:

Submit

Sell Stocks

Sell Stock From IBM: £308.78

Quantity:

Submit

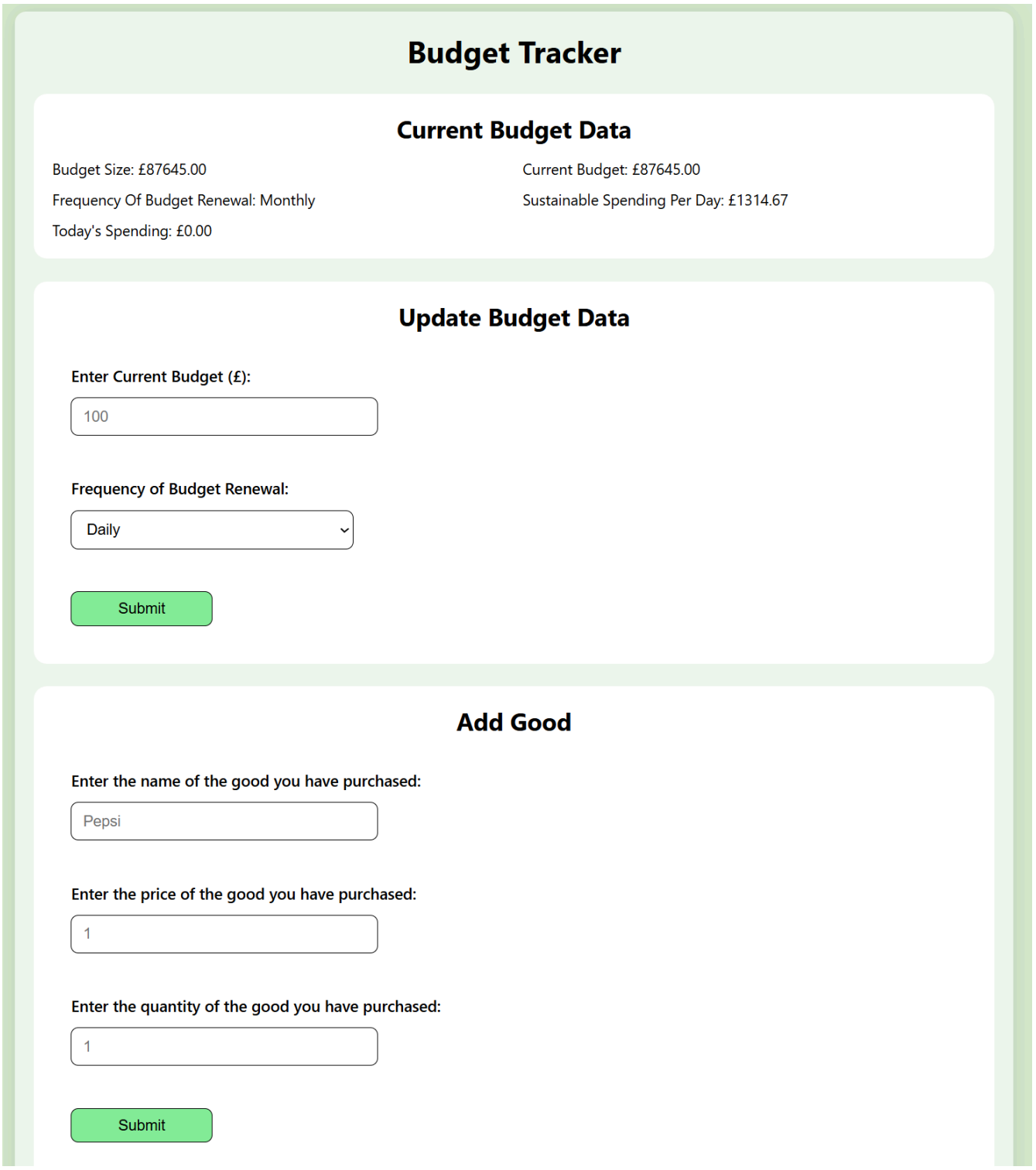
Owned Stocks

Stock: Coca-Cola
Quantity: 100
Bought Stock Price: £6570.00
Average Bought Stock Price: £65.70
Current Stock Price: £69.36
Potential Profit If One Stock Sold: £3.64
Potential Total Profit: £364.00

Stock: Microsoft
Quantity: 3
Bought Stock Price: £1569.33
Average Bought Stock Price: £523.11
Current Stock Price: £535.00
Potential Profit If One Stock Sold: £11.89
Potential Total Profit: £35.67

Objective 5

This page displays the user's budget along with all of their purchases. Users can sort their purchases by price or by the quantity bought. They can also view any risky purchases they've made, track their spending throughout the week, and see projections for their future budget.



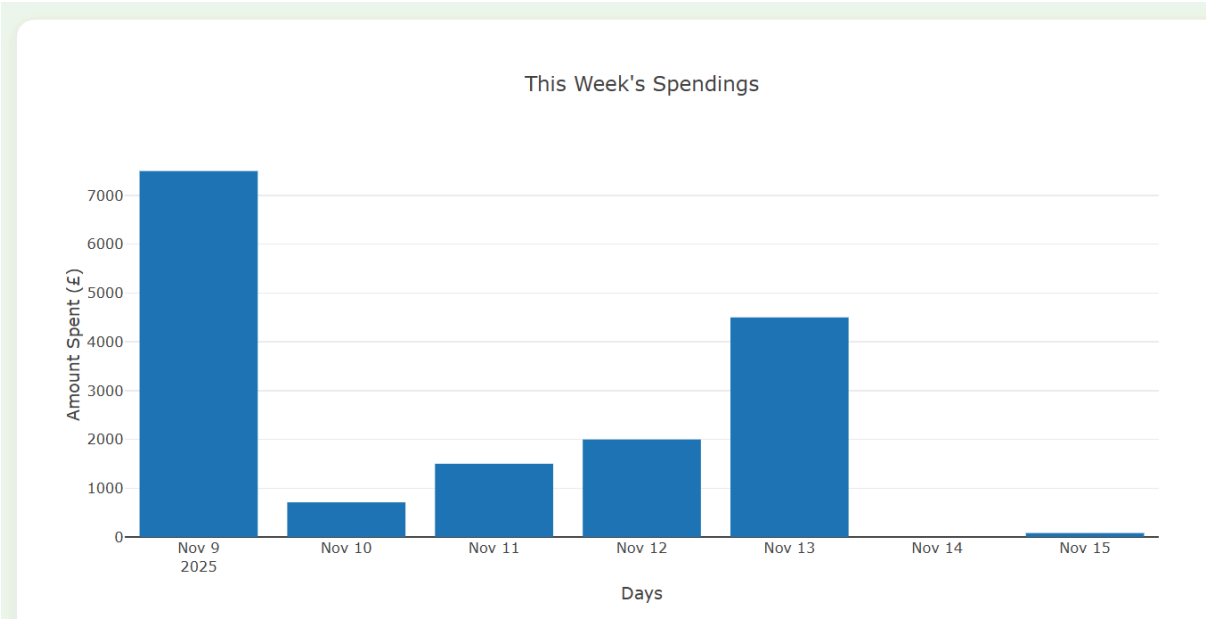
Goods Purchased

Date (Latest) Submit

Date	Name	Price (£)	Quantity	Total Price (£)
09/09/2025	Pespi	2.00	4	8.00
09/09/2025	Pespi	1.00	2	2.00
09/09/2025	Pespi	2.00	1	2.00
09/09/2025	Pespi	2.00	1	2.00
09/09/2025	Pespi	2.00	1	2.00
09/09/2025	Car	999.00	1	999.00
09/09/2025	KTM	2.00	4	8.00
06/09/2025	KTM	1500.00	1	1500.00

Risky Purchases

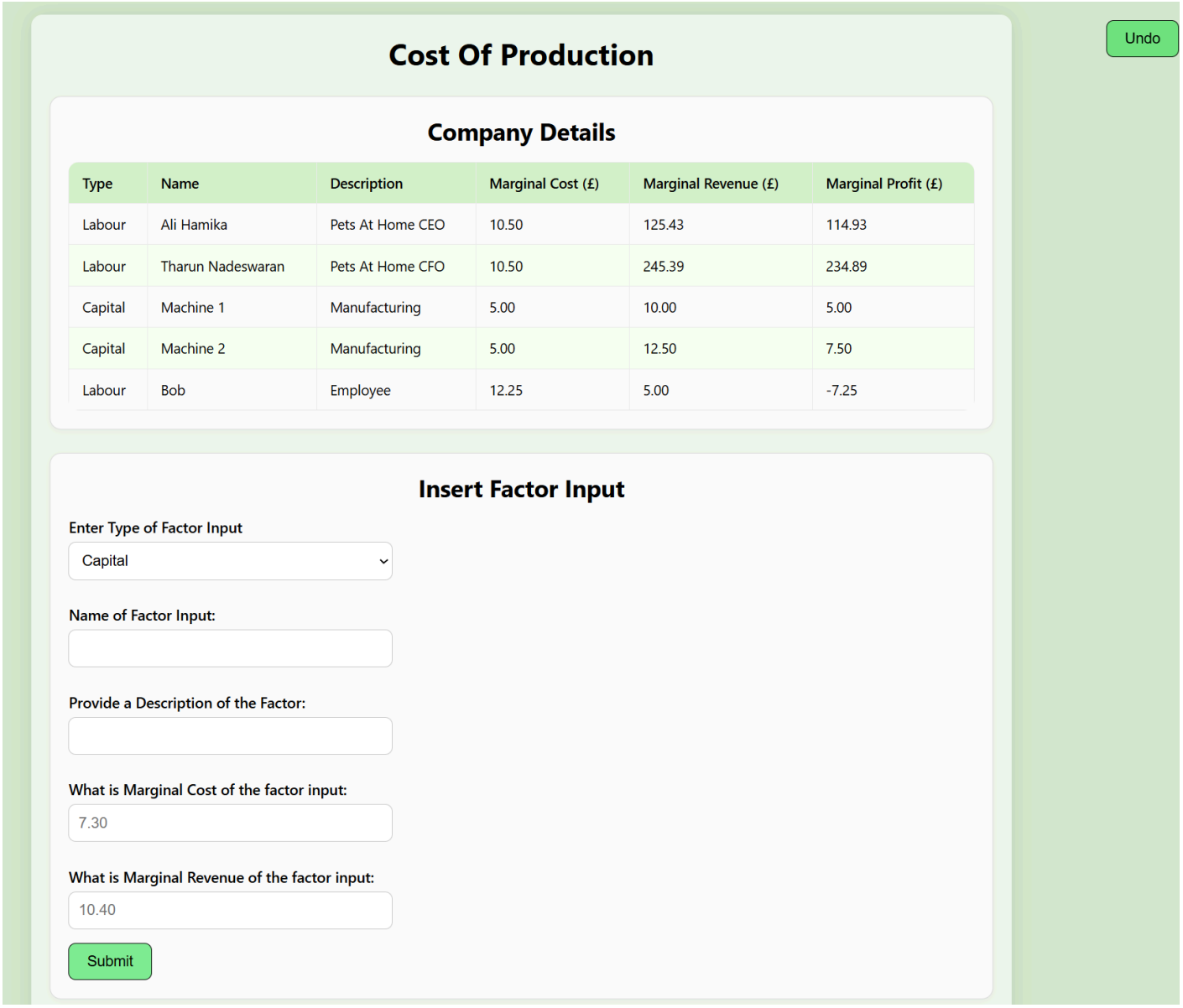
Pespi at £8.00 is a risky purchase.




Future Budget
Budget Components (£)
40k
35k
30k
25k
27/10/2025
27/11/2025
27/12/2025
27/01/2026
27/02/2026
27/03/2026
27/04/2026
27/05/2026
27/06/2026
27/07/2026
27/08/2026
27/09/2026
Months
Essentials
Leisure
Savings

Objective 6

This page displays the firm's cost of production, where producers can edit, delete, or hire factor inputs such as labour and capital. Producers can also identify which workers are the most profitable under a given cost constraint. Users can use an undo button to reverse any changes made to factor inputs, and they can view a graph comparing costs and revenue.



Current Data

Total Labour: 3

Total Capital: 2

Total Cost: £43.25

Average Cost: £8.65

Total Revenue: £398.32

Average Revenue: £79.66

Total Profit: £355.07

Average Profit: £71.01

Addition to GDP: 10.33

Optimal Workers

Enter The Maximum Cost (this will be used to find the optimal workers):

Submit

Update Factor Input

Select Factor Input

Labour - Ali Hamika

Change Marginal Cost of the factor input:

7.30

Change Marginal Revenue of the factor input:

10.40

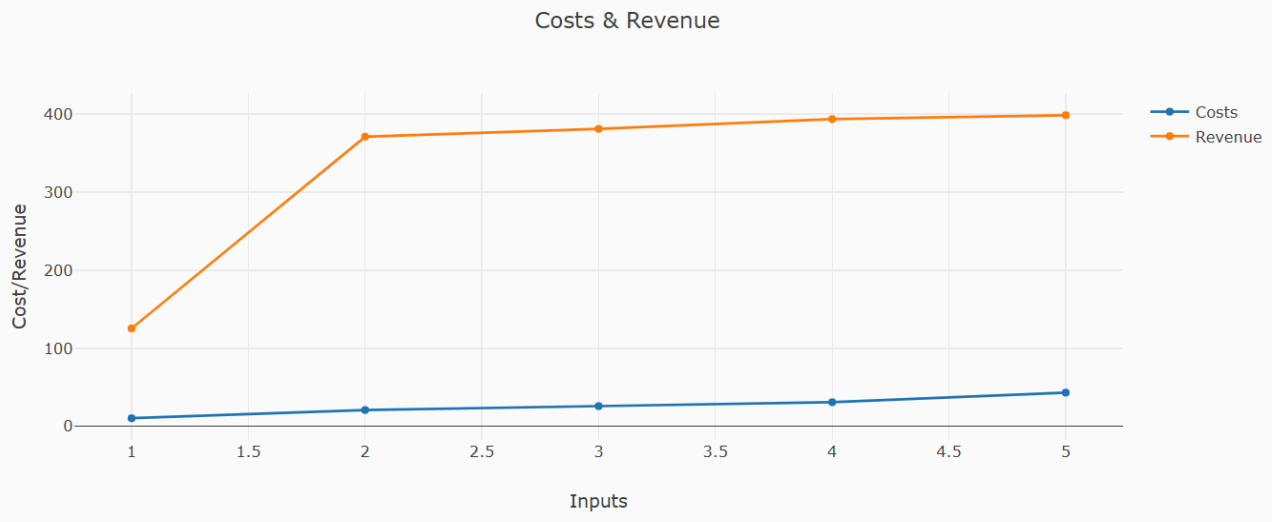
Submit

Fire Factor Input

Fire Factor Input

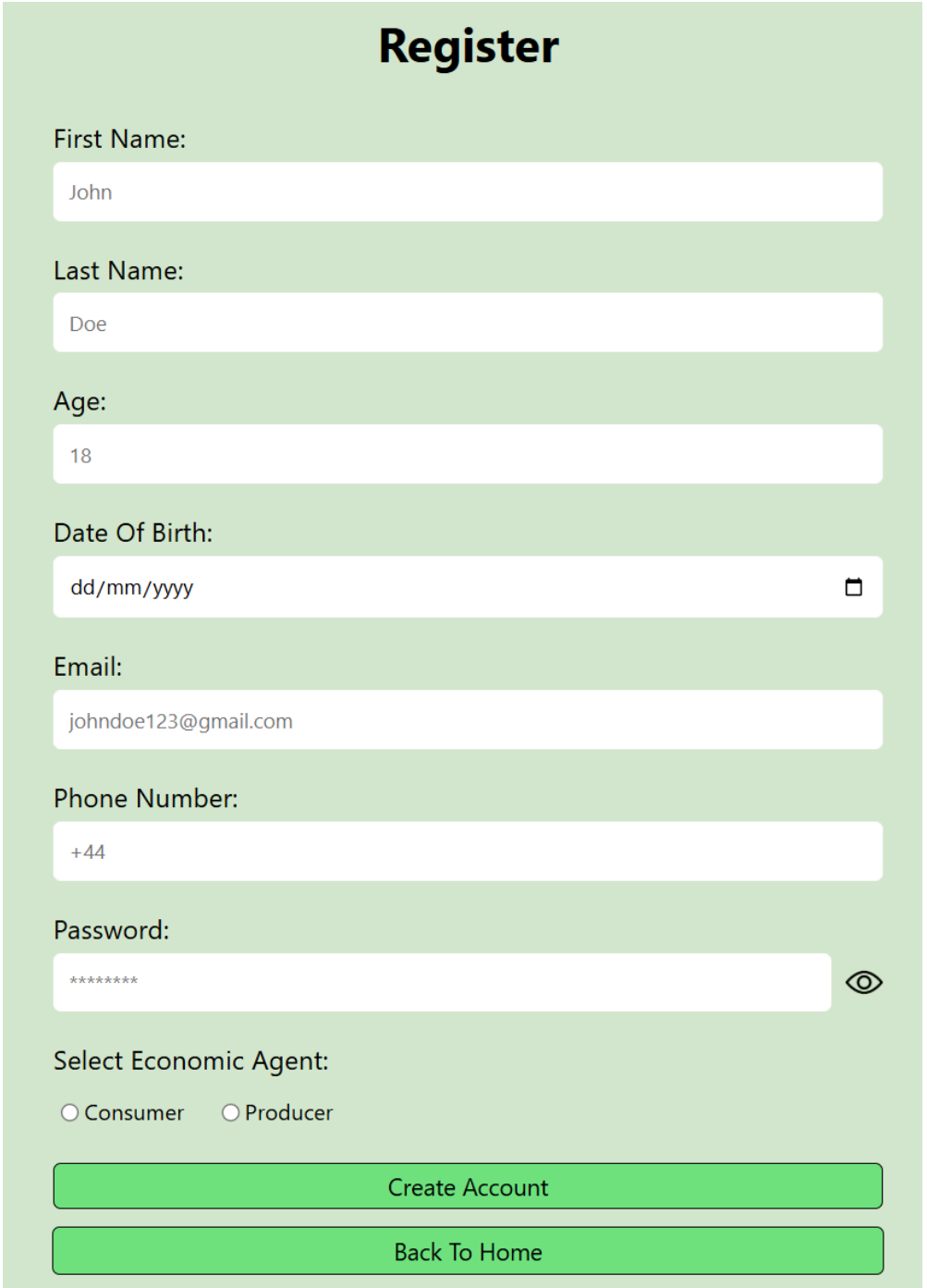
Labour - Ali Hamika

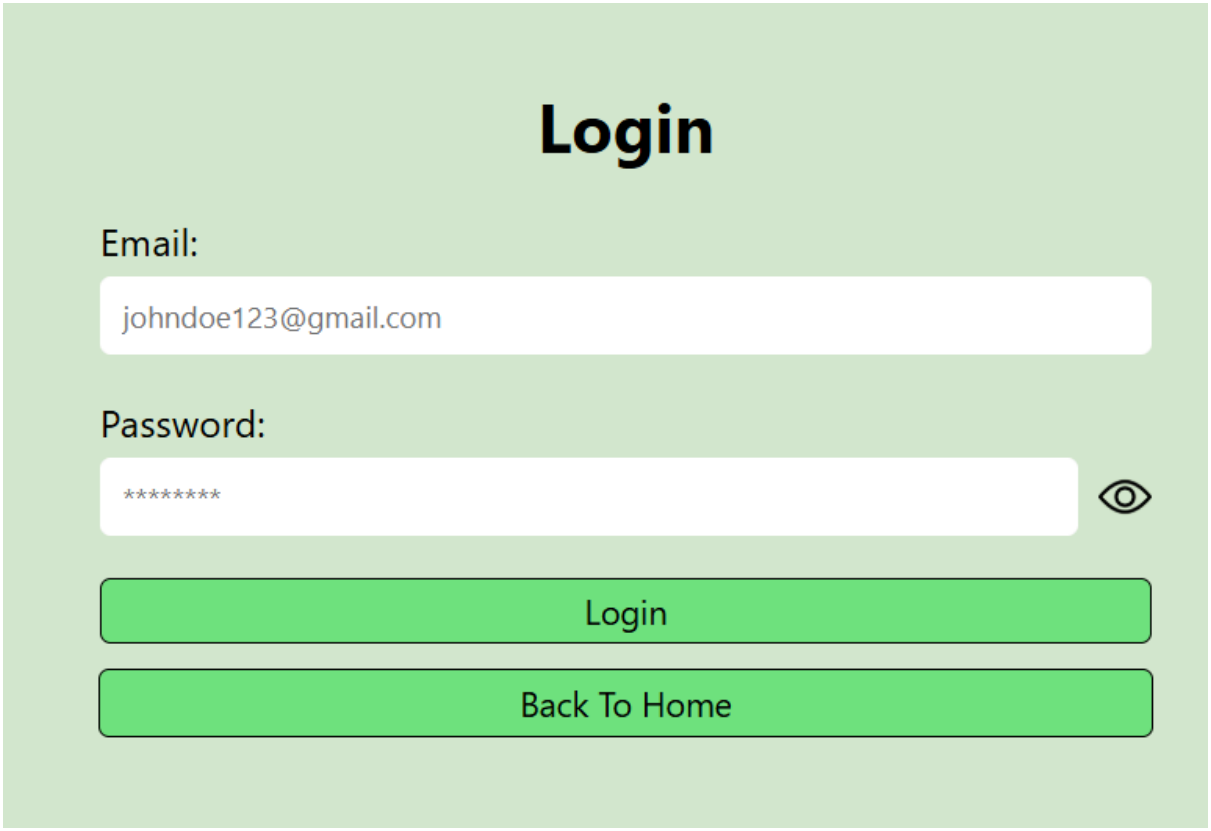
Submit



Objective 7

This page allows users to log in to their account or register for a new one. During registration, users can enter their personal details to create a secure profile, and returning users can sign in using their credentials. The system checks if the user's password is strong enough by following a few constraints. Once an account has been made, users can login to their account.

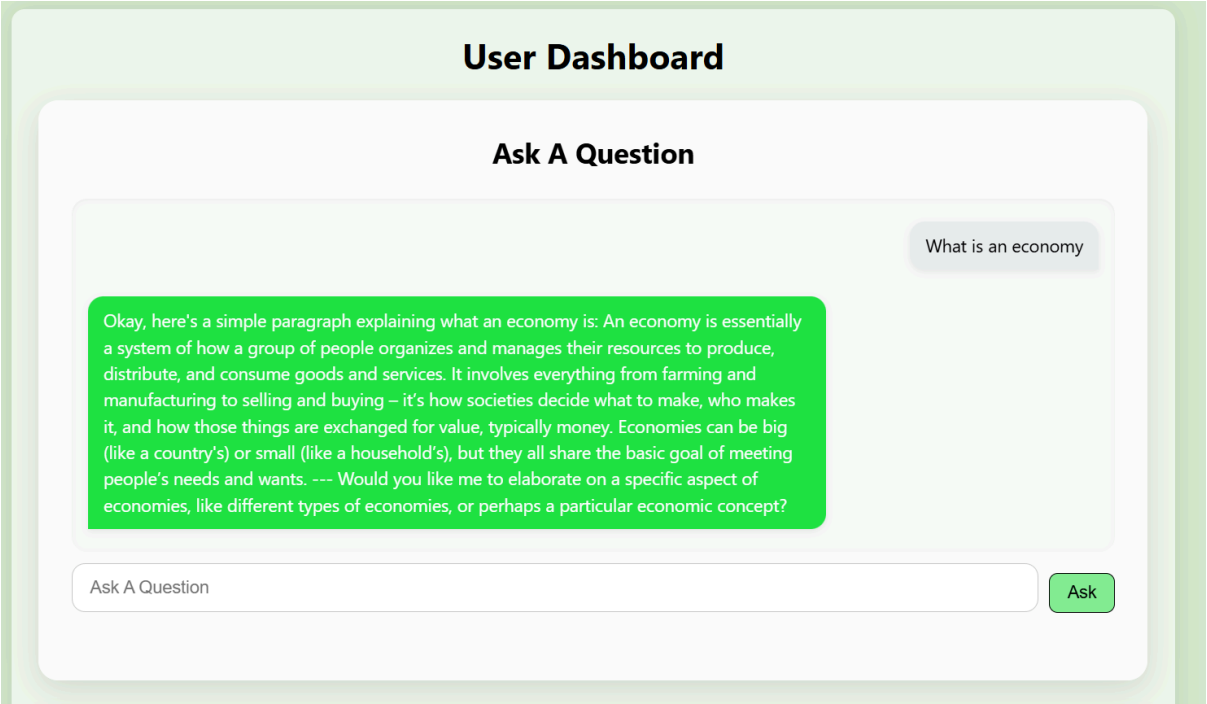


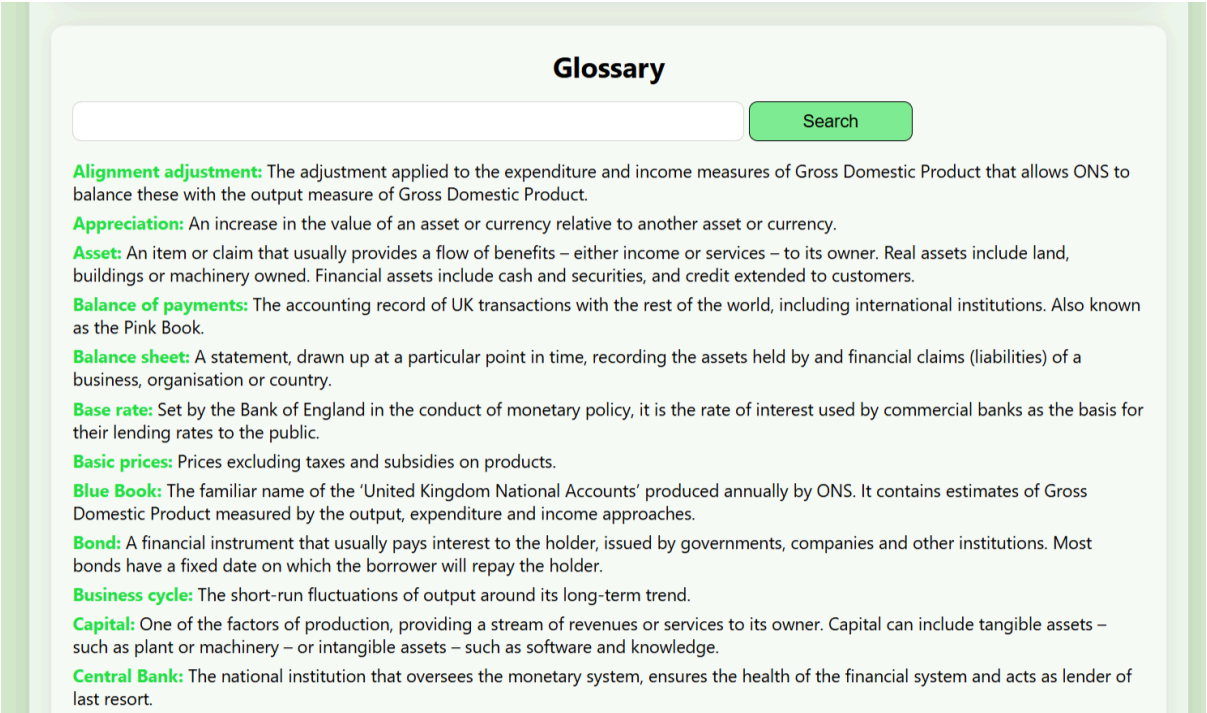


Tharun Nadeswaran - Money Matters
Student Code -
Centre Number -
130

Objective 9

Users can ask the AI questions about the economy or any finance-related topic. They can also search for specific terms in the glossary, and if a term isn't found, the system returns an error message to let them know it doesn't exist.



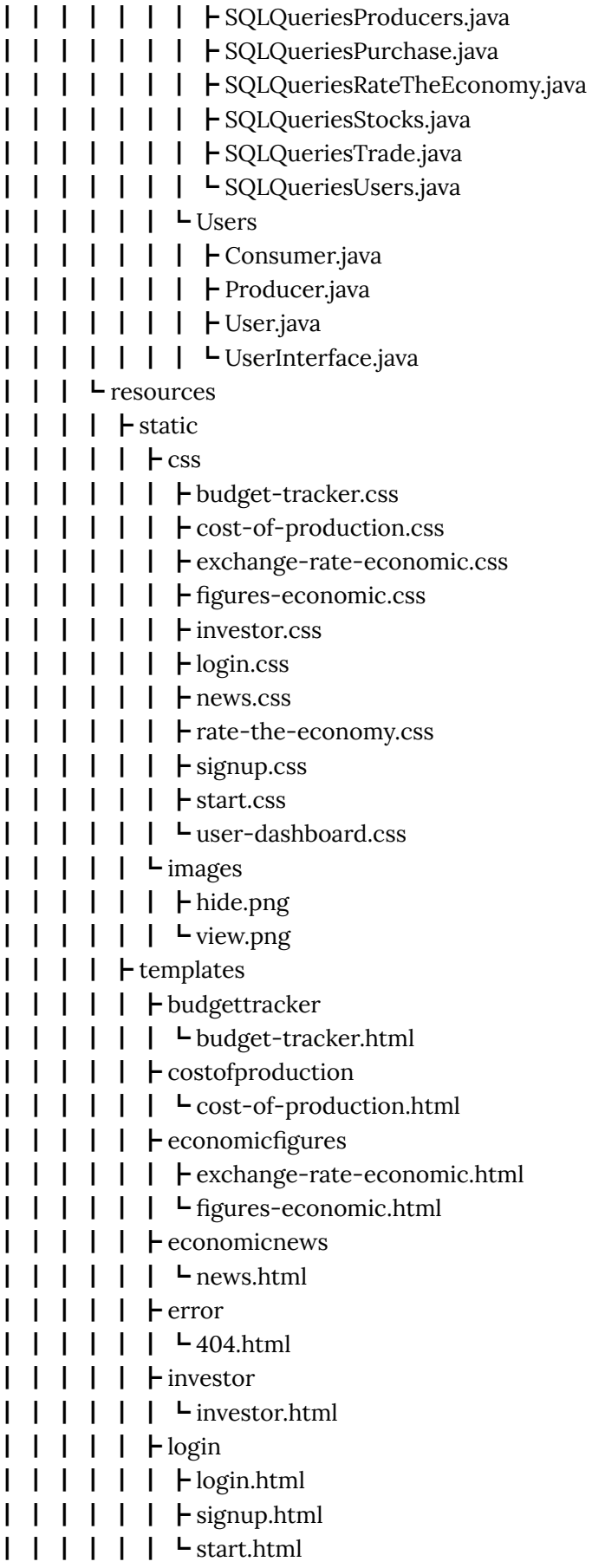


File Structure

As my project will be a dynamic website, it will heavily rely on having an organised file structure. Given that I'm using Java with the Maven archetype and Spring Boot as the primary framework, the project already benefits from a clear structure. Additionally, I have created other packages to further manage data. The template folder stores the relevant HTML files. This will allow me to better maintain my code, and allows me to easily scale up the code, once I implement more features, without it being difficult to analyse how each section of code interacts with the overall system.

```
MoneyMatters
├ .vscode
|  ├ launch.json
|  └ settings.json
├ MoneyMatters
|  ├ src
|  |  ├ main
|  |  |  ├ java
|  |  |  |  └ com
|  |  |  |  |  └ mycompany
|  |  |  |  |  |  ├ Algorithms
|  |  |  |  |  |  |  ├ BinarySearch.java
|  |  |  |  |  |  |  ├ FileHandling.java
|  |  |  |  |  |  |  ├ FileHandlingJSON.java
|  |  |  |  |  |  |  ├ Glossary.java
|  |  |  |  |  |  |  ├ MergeSort.java
|  |  |  |  |  |  |  └ VernamCipher.java
|  |  |  |  |  |  ├ Controllers
|  |  |  |  |  |  |  ├ BudgetTrackerController.java
|  |  |  |  |  |  |  ├ CostOfProductionController.java
|  |  |  |  |  |  |  ├ EconomicFiguresController.java
|  |  |  |  |  |  |  ├ EconomicNewsController.java
|  |  |  |  |  |  |  ├ ExchangeRateController.java
|  |  |  |  |  |  |  ├ InvestorController.java
|  |  |  |  |  |  |  ├ LoginController.java
|  |  |  |  |  |  |  ├ RateTheEconomyController.java
|  |  |  |  |  |  |  └ UserDashboardController.java
|  |  |  |  |  |  ├ EconomicFigures
|  |  |  |  |  |  |  ├ Countries.java
|  |  |  |  |  |  |  ├ ExchangeRateAPI.java
|  |  |  |  |  |  |  ├ GDPFetcher.java
|  |  |  |  |  |  |  ├ GDPPerCapitaFetcher.java
|  |  |  |  |  |  |  ├ GiniIndexFetcher.java
|  |  |  |  |  |  |  ├ GovernmentDebtFetcher.java
|  |  |  |  |  |  |  ├ InflationFetcher.java
|  |  |  |  |  |  |  ├ InterestRateFetcher.java
```

```
|  |  |  |  |  |  |  |  ├ MacroIndicatorFetcher.java
|  |  |  |  |  |  |  |  ├ MacroIndicators.java
|  |  |  |  |  |  |  |  ├ NetTradeFetcher.java
|  |  |  |  |  |  |  |  ├ TaxRevenueGDPFetcher.java
|  |  |  |  |  |  |  |  ├ TaxRevenueLCUFetcher.java
|  |  |  |  |  |  |  |  ├ TotalReservesFetcher.java
|  |  |  |  |  |  |  |  ├ TradeFetcher.java
|  |  |  |  |  |  |  |  ├ UnemploymentFetcher.java
|  |  |  |  |  |  |  |  └ WorldBankAPI.java
|  |  |  |  |  |  |  ├ Logic
|  |  |  |  |  |  |  |  ├ BudgetTrackerLogic.java
|  |  |  |  |  |  |  |  ├ CostOfProductionLogic.java
|  |  |  |  |  |  |  |  ├ InvestorLogic.java
|  |  |  |  |  |  |  |  ├ RateTheEconomyLogic.java
|  |  |  |  |  |  |  |  └ WebScraperLogic.java
|  |  |  |  |  |  |  ├ MoneyMatters
|  |  |  |  |  |  |  |  └ MoneyMattersApplication.java
|  |  |  |  |  |  |  ├ PythonFlaskAPI
|  |  |  |  |  |  |  |  ├ __pycache__
|  |  |  |  |  |  |  |  |  ├ BudgetTrackerMaths.cpython-311.pyc
|  |  |  |  |  |  |  |  |  ├ CostOfProductionMaths.cpython-311.pyc
|  |  |  |  |  |  |  |  |  ├ InvestmentData.cpython-311.pyc
|  |  |  |  |  |  |  |  |  └ OllamaAI.cpython-311.pyc
|  |  |  |  |  |  |  |  ├ APIQueue.java
|  |  |  |  |  |  |  |  ├ APIWorker.java
|  |  |  |  |  |  |  |  ├ BudgetTrackerMaths.py
|  |  |  |  |  |  |  |  ├ CostOfProductionMaths.py
|  |  |  |  |  |  |  |  ├ InvestmentData.py
|  |  |  |  |  |  |  |  ├ OllamaAI.py
|  |  |  |  |  |  |  |  ├ PythonFlaskAPI.java
|  |  |  |  |  |  |  |  └ PythonFlaskAPI.py
|  |  |  |  |  |  |  ├ Records
|  |  |  |  |  |  |  |  ├ CompanyRecord.java
|  |  |  |  |  |  |  |  ├ DefinitionRecord.java
|  |  |  |  |  |  |  |  ├ EconomicNewsRecord.java
|  |  |  |  |  |  |  |  ├ FactorInput.java
|  |  |  |  |  |  |  |  ├ GoodRecord.java
|  |  |  |  |  |  |  |  ├ OutlierGoods.java
|  |  |  |  |  |  |  |  ├ Purchase.java
|  |  |  |  |  |  |  |  └ RateTheEconomyQuestions.java
|  |  |  |  |  |  |  ├ Repository
|  |  |  |  |  |  |  |  ├ SQLQueries.java
|  |  |  |  |  |  |  |  ├ SQLQueriesBudgetTracker.java
|  |  |  |  |  |  |  |  ├ SQLQueriesConsumers.java
|  |  |  |  |  |  |  |  ├ SQLQueriesDDL.java
|  |  |  |  |  |  |  |  ├ SQLQueriesFactorInput.java
|  |  |  |  |  |  |  |  ├ SQLQueriesGoods.java
```



Tharun Nadeswaran - Money Matters
Student Code -
Centre Number -
134

```
|   |   |   |   |   ├ ratetheeconomy
|   |   |   |   |   |   └ rate-the-economy.html
|   |   |   |   |   └ userdashboard
|   |   |   |   |       └ user-dashboard.html
|   |   |   |   └ application.properties
|   |   └ test
|   |   |   └ java
|   |   |   |   └ com
|   |   |   |       └ mycompany
|   |   |   |           └ MoneyMatters
|   |   |   |               ├ ConsumerTableTest.java
|   |   |   |               ├ CostRestraintTest.java
|   |   |   |               ├ DDLTest.java
|   |   |   |               ├ MoneyMattersApplicationTests.java
|   |   |   |               ├ PythonFlaskAPITest.java
|   |   |   |               ├ VernamCipherTest.java
|   |   |   |               └ WorldBankAPITest.java
|   ├ target
|   |   ├ classes
|   |   |   ├ com
|   |   |   |   └ mycompany
|   |   |   |       ├ Algorithms
|   |   |   |       |   ├ BinarySearch.class
|   |   |   |       |   ├ FileHandling.class
|   |   |   |       |   ├ FileHandlingJSON.class
|   |   |   |       |   ├ Glossary.class
|   |   |   |       |   ├ MergeSort.class
|   |   |   |       |   ├ VernamCipher$1.class
|   |   |   |       |   └ VernamCipher.class
|   |   |   |       ├ Controllers
|   |   |   |       |   ├ BudgetTrackerController.class
|   |   |   |       |   ├ CostOfProductionController.class
|   |   |   |       |   ├ EconomicFiguresController.class
|   |   |   |       |   ├ EconomicNewsController.class
|   |   |   |       |   ├ ExchangeRateController.class
|   |   |   |       |   ├ InvestorController.class
|   |   |   |       |   ├ LoginController.class
|   |   |   |       |   ├ RateTheEconomyController.class
|   |   |   |       |   └ UserDashboardController.class
|   |   |   |       ├ EconomicFigures
|   |   |   |       |   ├ Countries.class
|   |   |   |       |   ├ ExchangeRateAPI.class
|   |   |   |       |   ├ GDPFetcher.class
|   |   |   |       |   ├ GDPPerCapitaFetcher.class
|   |   |   |       |   ├ GiniIndexFetcher.class
|   |   |   |       |   ├ GovernmentDebtFetcher.class
|   |   |   |       |   ├ InflationFetcher.class
```

```
|   |   |   |   |   |   ├ InterestRateFetcher.class
|   |   |   |   |   |   ├ MacroIndicatorFetcher.class
|   |   |   |   |   |   ├ MacroIndicators.class
|   |   |   |   |   |   ├ NetTradeFetcher.class
|   |   |   |   |   |   ├ TaxRevenueGDPFetcher.class
|   |   |   |   |   |   ├ TaxRevenueLCUFetcher.class
|   |   |   |   |   |   ├ TotalReservesFetcher.class
|   |   |   |   |   |   ├ TradeFetcher.class
|   |   |   |   |   |   ├ UnemploymentFetcher.class
|   |   |   |   |   |   └ WorldBankAPI.class
|   |   |   |   |   ├ Logic
|   |   |   |   |   |   ├ BudgetTrackerLogic.class
|   |   |   |   |   |   ├ CostOfProductionLogic.class
|   |   |   |   |   |   ├ InvestorLogic$1.class
|   |   |   |   |   |   ├ InvestorLogic$2.class
|   |   |   |   |   |   ├ InvestorLogic.class
|   |   |   |   |   |   ├ RateTheEconomyLogic$1.class
|   |   |   |   |   |   ├ RateTheEconomyLogic$2.class
|   |   |   |   |   |   ├ RateTheEconomyLogic.class
|   |   |   |   |   |   └ WebScraperLogic.class
|   |   |   |   |   ├ MoneyMatters
|   |   |   |   |   |   └ MoneyMattersApplication.class
|   |   |   |   |   ├ PythonFlaskAPI
|   |   |   |   |   |   ├ __pycache__
|   |   |   |   |   |   |   ├ BudgetTrackerMaths.cpython-311.pyc
|   |   |   |   |   |   |   ├ CostOfProductionMaths.cpython-311.pyc
|   |   |   |   |   |   |   ├ InvestmentData.cpython-311.pyc
|   |   |   |   |   |   |   └ OllamaAI.cpython-311.pyc
|   |   |   |   |   |   ├ APIQueue.class
|   |   |   |   |   |   ├ APIWorker.class
|   |   |   |   |   |   ├ BudgetTrackerMaths.py
|   |   |   |   |   |   ├ CostOfProductionMaths.py
|   |   |   |   |   |   ├ InvestmentData.py
|   |   |   |   |   |   ├ OllamaAI.py
|   |   |   |   |   |   ├ PythonFlaskAPI$1.class
|   |   |   |   |   |   ├ PythonFlaskAPI$2.class
|   |   |   |   |   |   ├ PythonFlaskAPI$3.class
|   |   |   |   |   |   ├ PythonFlaskAPI$4.class
|   |   |   |   |   |   ├ PythonFlaskAPI.class
|   |   |   |   |   |   └ PythonFlaskAPI.py
|   |   |   |   |   ├ Records
|   |   |   |   |   |   ├ CompanyRecord.class
|   |   |   |   |   |   ├ DefinitionRecord.class
|   |   |   |   |   |   ├ EconomicNewsRecord.class
|   |   |   |   |   |   ├ FactorInput.class
|   |   |   |   |   |   ├ GoodRecord.class
|   |   |   |   |   |   ├ OutlierGoods.class
```

```
|   |   |   |   |   |   ├ Purchase.class
|   |   |   |   |   |   └ RateTheEconomyQuestions.class
|   |   |   |   |   ├ Repository
|   |   |   |   |   |   ├ SQLQueries.class
|   |   |   |   |   |   ├ SQLQueriesBudgetTracker.class
|   |   |   |   |   |   ├ SQLQueriesConsumers.class
|   |   |   |   |   |   ├ SQLQueriesDDL.class
|   |   |   |   |   |   ├ SQLQueriesFactorInput.class
|   |   |   |   |   |   ├ SQLQueriesGoods.class
|   |   |   |   |   |   ├ SQLQueriesProducers.class
|   |   |   |   |   |   ├ SQLQueriesPurchase.class
|   |   |   |   |   |   ├ SQLQueriesRateTheEconomy.class
|   |   |   |   |   |   ├ SQLQueriesStocks$1.class
|   |   |   |   |   |   ├ SQLQueriesStocks.class
|   |   |   |   |   |   ├ SQLQueriesTrade.class
|   |   |   |   |   |   └ SQLQueriesUsers.class
|   |   |   |   |   └ Users
|   |   |   |   |   |   ├ Consumer.class
|   |   |   |   |   |   ├ Producer.class
|   |   |   |   |   |   ├ User.class
|   |   |   |   |   |   └ UserInterface.class
|   |   |   ├ static
|   |   |   |   ├ css
|   |   |   |   |   ├ budget-tracker.css
|   |   |   |   |   ├ cost-of-production.css
|   |   |   |   |   ├ exchange-rate-economic.css
|   |   |   |   |   ├ figures-economic.css
|   |   |   |   |   ├ investor.css
|   |   |   |   |   ├ login.css
|   |   |   |   |   ├ news.css
|   |   |   |   |   ├ rate-the-economy.css
|   |   |   |   |   ├ signup.css
|   |   |   |   |   ├ start.css
|   |   |   |   |   └ user-dashboard.css
|   |   |   |   └ images
|   |   |   |   |   ├ hide.png
|   |   |   |   |   └ view.png
|   |   |   ├ templates
|   |   |   |   ├ budgettracker
|   |   |   |   |   └ budget-tracker.html
|   |   |   |   ├ costofproduction
|   |   |   |   |   └ cost-of-production.html
|   |   |   |   ├ economicfigures
|   |   |   |   |   ├ exchange-rate-economic.html
|   |   |   |   |   └ figures-economic.html
|   |   |   |   ├ economicnews
|   |   |   |   |   └ news.html
```

```
|   |   |   |   ├ error
|   |   |   |   |   └ 404.html
|   |   |   |   ├ investor
|   |   |   |   |   └ investor.html
|   |   |   |   ├ login
|   |   |   |   |   ├ login.html
|   |   |   |   |   ├ signup.html
|   |   |   |   |   └ start.html
|   |   |   |   ├ ratetheeconomy
|   |   |   |   |   └ rate-the-economy.html
|   |   |   |   └ userdashboard
|   |   |   |   |   └ user-dashboard.html
|   |   |   └ application.properties
|   |   ├ generated-sources
|   |   |   └ annotations
|   |   ├ generated-test-sources
|   |   |   └ test-annotations
|   |   ├ maven-status
|   |   |   └ maven-compiler-plugin
|   |   |   |   ├ compile
|   |   |   |   |   └ default-compile
|   |   |   |   |   |   ├ createdFiles.lst
|   |   |   |   |   |   └ inputFiles.lst
|   |   |   |   └ testCompile
|   |   |   |   |   └ default-testCompile
|   |   |   |   |   |   ├ createdFiles.lst
|   |   |   |   |   |   └ inputFiles.lst
|   |   ├ surefire-reports
|   |   |   ├ 2025-10-27T13-29-11_139-jvmRun1.dump
|   |   |   ├ 2025-10-27T13-29-11_139-jvmRun1.dumpstream
|   |   |   ├ com.mycompany.MoneyMatters.DatabaseTest.txt
|   |   |   ├
com.mycompany.MoneyMatters.DatabaseTests.ConsumerTableTe
st.txt
|   |   |   ├
com.mycompany.MoneyMatters.DDLTests.DatabaseTest.txt
|   |   |   ├ com.mycompany.MoneyMatters.MathsTest.txt
|   |   |   ├
com.mycompany.MoneyMatters.MoneyMattersApplicationTests.t
xt
|   |   |   ├
com.mycompany.MoneyMatters.PythonFlaskAPITest.txt
|   |   |   ├ com.mycompany.MoneyMatters.VernamCipherTest.txt
|   |   |   ├ com.mycompany.MoneyMatters.WorldBankAPITest.txt
|   |   |   ├
TEST-com.mycompany.MoneyMatters.DatabaseTest.xml
```

```
|  |  |  ├ TEST-com.mycompany.MoneyMatters.DatabaseTests.ConsumerTableTest.xml
|  |  |  ├ TEST-com.mycompany.MoneyMatters.DDLTests.DatabaseTest.xml
|  |  |  ├ TEST-com.mycompany.MoneyMatters.MathsTest.xml
|  |  |  ├ TEST-com.mycompany.MoneyMatters.MoneyMattersApplicationTests.xml
|  |  |  ├ TEST-com.mycompany.MoneyMatters.PythonFlaskAPITest.xml
|  |  |  ├ TEST-com.mycompany.MoneyMatters.VernamCipherTest.xml
|  |  |  └ TEST-com.mycompany.MoneyMatters.WorldBankAPITest.xml
|  |  └ test-classes
|  |  |  └ com
|  |  |  |  └ mycompany
|  |  |  |  |  └ MoneyMatters
|  |  |  |  |  |  ├ ConsumerTableTest.class
|  |  |  |  |  |  ├ CostRestraintTest.class
|  |  |  |  |  |  ├ DDLTest.class
|  |  |  |  |  |  ├ MoneyMattersApplicationTests.class
|  |  |  |  |  |  ├ PythonFlaskAPITest.class
|  |  |  |  |  |  ├ VernamCipherTest.class
|  |  |  |  |  |  └ WorldBankAPITest.class
|  ├ VernamKeys
|  |  └ tharunnadeswaran2207@gmail.com.json
|  └ pom.xml
├ SQLCreateTableQueries
|  ├ ConsumersTable.txt
|  ├ EconomyRaterTable.txt
|  ├ FactorInputTable.txt
|  ├ GoodsTable.txt
|  ├ ProducersTable.txt
|  ├ PurchaseTable.txt
|  ├ StocksTable.txt
|  └ TradesTable.txt
├ SQLSelectTableQueries
|  ├ SelectNoNetQuantityConsumer.txt
|  └ SelectNoNetQuantityProducer.txt
├ UserOpinions
|  ├ 09-09-2025
|  |  └ userOpinion1.json
|  ├ 27-10-2025
|  |  ├ userOpinion1.json
```

```
|  |  └ userOpinion2.json
|  └ 29-08-2025
|  |  └ userOpinion1.json
├ VernamKeys
|  ├ 31.json
|  ├ consumer.json
|  ├ littenpokemon222@gmail.com.json
|  ├ producer.json
|  ├ sad.json
|  └ tharunnadeswaran2207@gmail.com.json
├ glossaryOfEconomicTerms.json
├ rateTheEconomyQuestions.json
└ tickerInfo.json
```

Furthermore, I have chosen to store certain data in a .JSON file, as well as a database, as it resembles a dictionary structure, making it easy to append, write and read data, unlike .TXT files. For example, I have stored users' Vernam Keys, the words and definitions of economic terms, ticker information from stock markets and user's opinion of the economy. I have only used .TXT files to store DDL scripts.

The Prototype

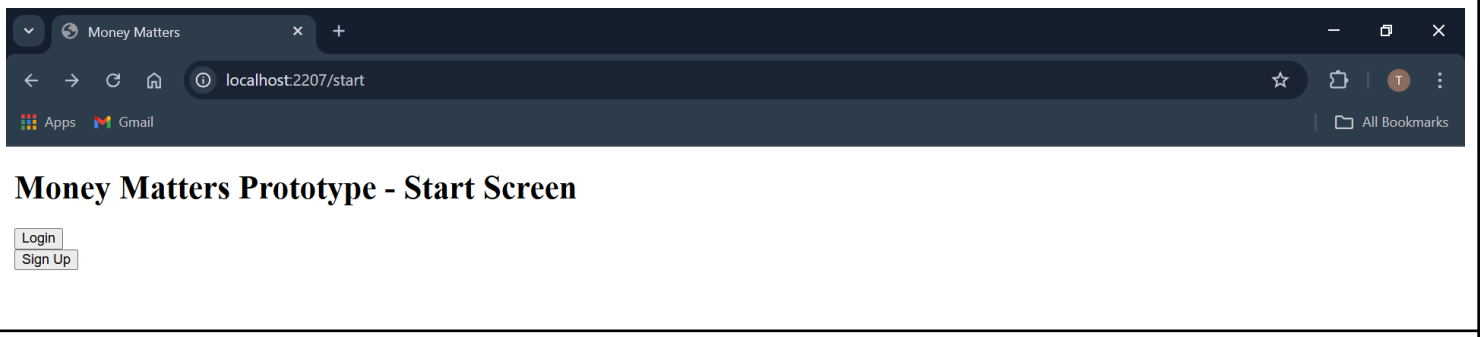
I have created a prototype using Java with the Maven archetype and Spring Boot as the primary framework, with VS Code as my IDE. I also learnt the basics of HTML, so that I can display a website a user could use to see and enter data, which is integrated with Thymeleaf (a Java library that is able to render dynamic HTML projects). As this is a prototype, I have not implemented any CSS nor Javascript, but it serves as a proof of concept to demonstrate my ability to connect the frontend with the backend, with basic database management. This will be very significant for the rest of my project, as it provides the foundation of how I am able to incorporate other objectives effectively.

This prototype serves as a login screen, which allows users to sign up or login, by inputting their personal information - reaching the goals of Objective 8. The user's data will be stored in the database, in the relevant tables, Consumer Table, and Producer Table. Once the user has gained access to the system, the system allows the user to view a list of economic news from BBC or The Telegraph, via the Web Scraper I have implemented, or be able to use the budget tracker feature, fulfilling Objective 5. I also ensured that only consumers are able to manage their budgets, meaning that only producers will be able to view economic news.

However, there are a few critiques with this prototype. For example, users can enter the wrong type of data, such as letters instead of digits when entering a budget. This can cause errors in the database as the system doesn't know how to handle incorrect data types, which will lead to data inconsistency. Also, I have not implemented a system, so that only unique emails can be used to create an account. This means that if someone has an existing account, but makes an account with the same account, it will lead to data being inconsistent, as the new user with the same email may not have access to their account, as their password is not the same as the old one.

After reviewing my prototype with my end user, we both agreed that this project is going towards the right direction. He really liked how users were able to easily view the main gist of economic news in the hub, and users are able to track their spending.

See Appendix B for the code of the Prototype.



User is on the start screen



User goes on the sign up page



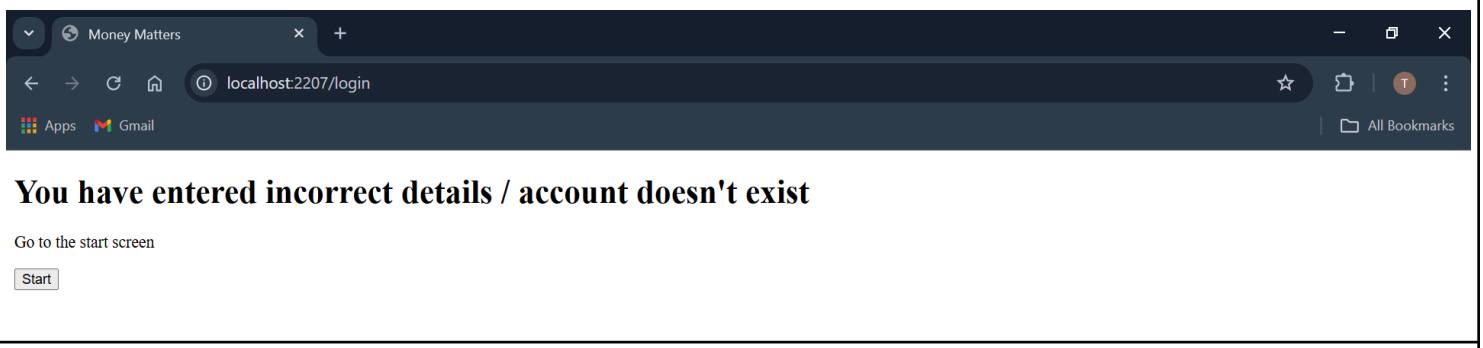
User enters their details on the signup page



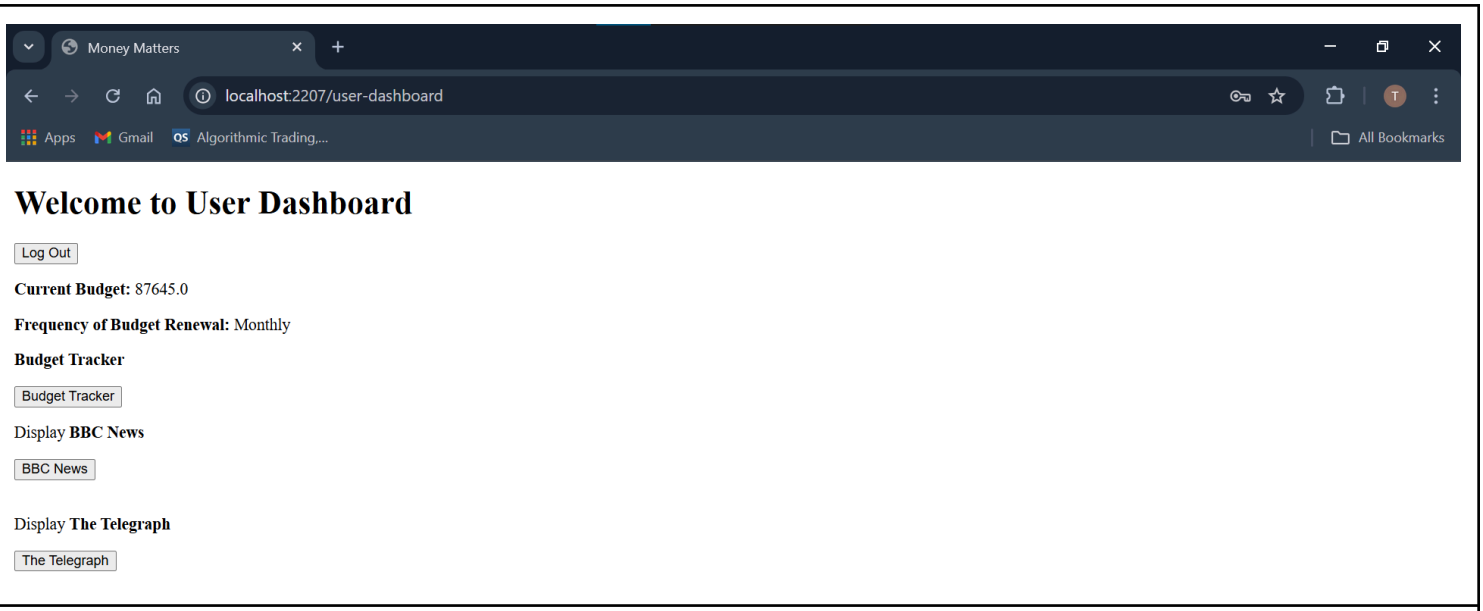
User is then redirected to the login page



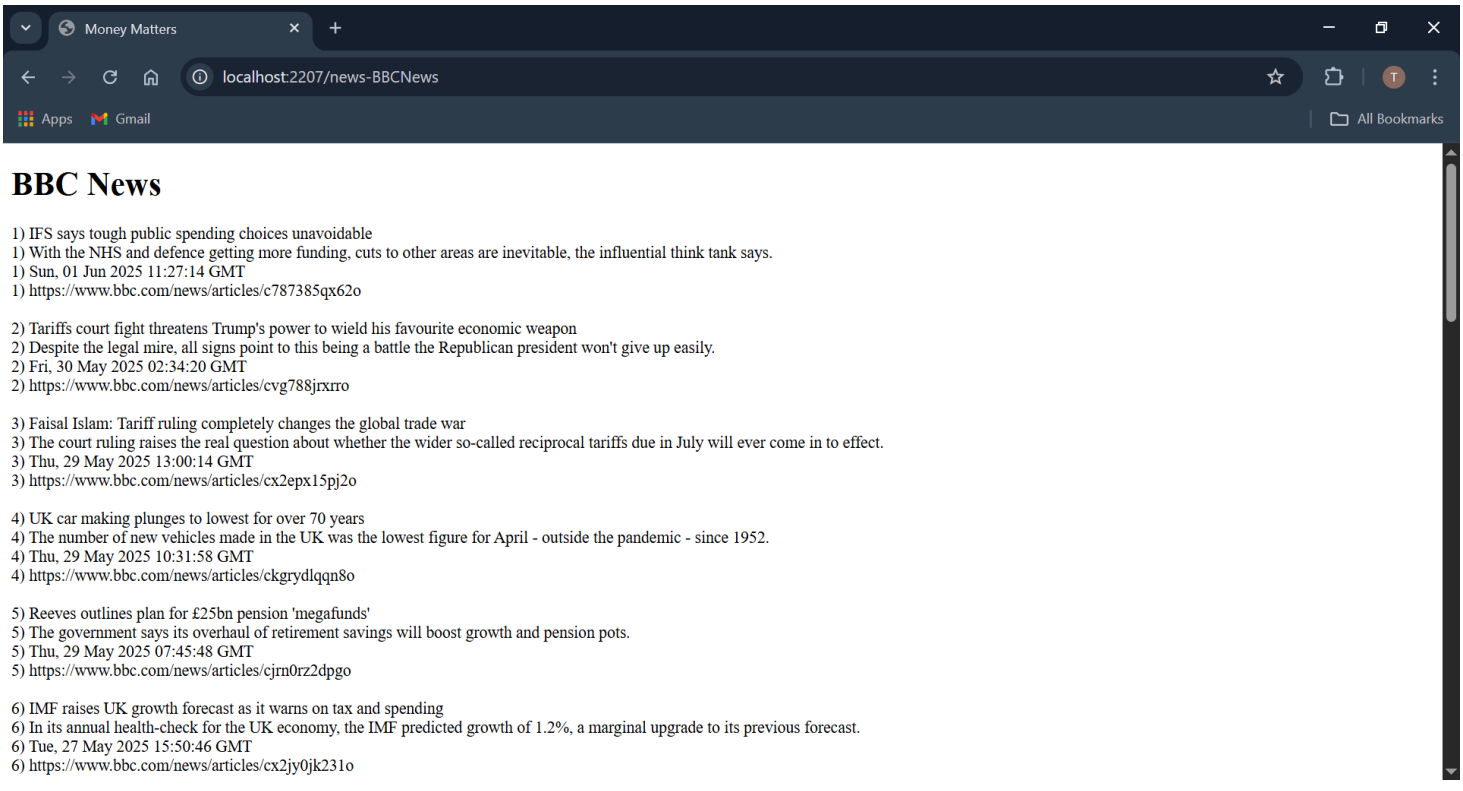
User enters their details



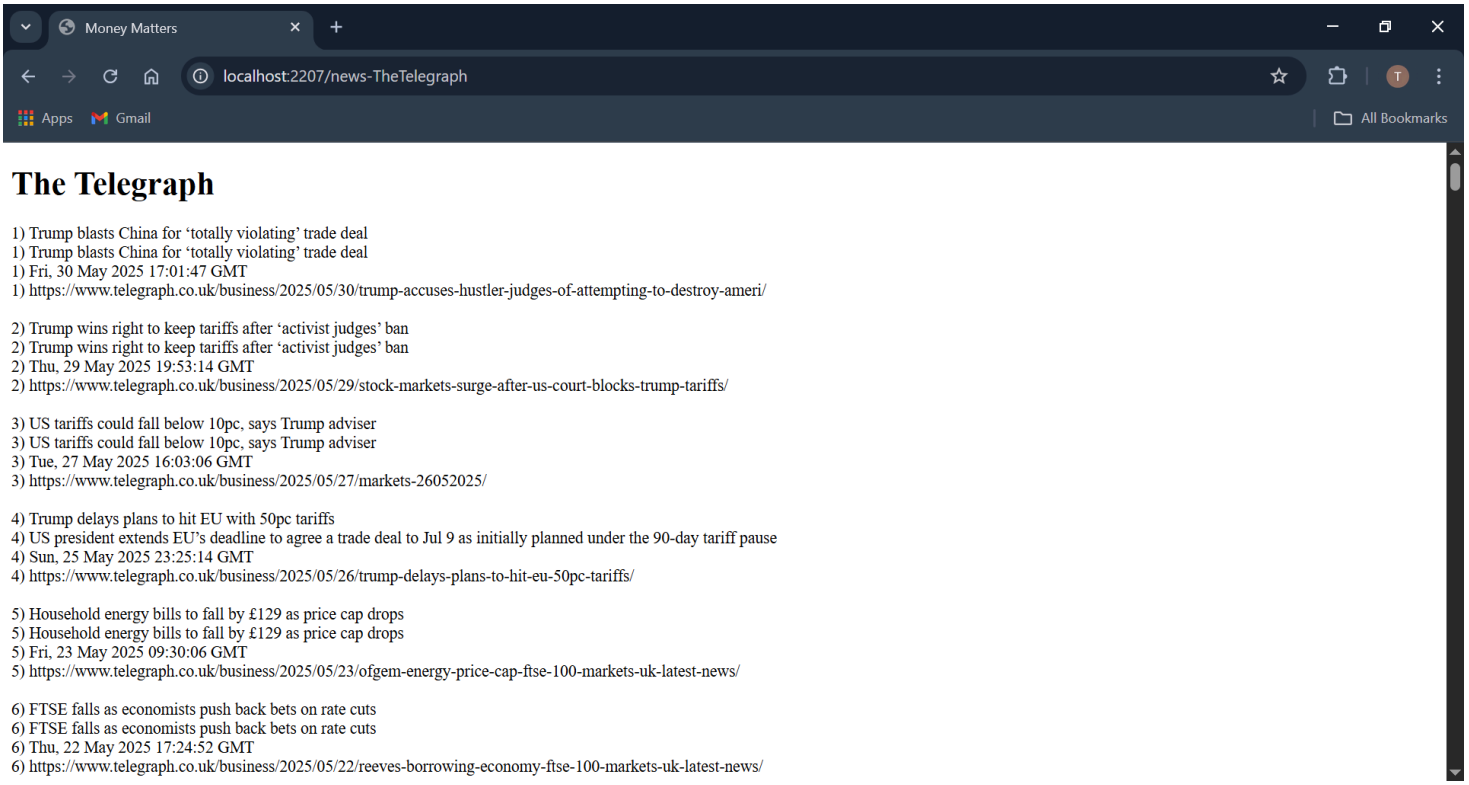
If the user enters incorrect details, or the account doesn't exist, they get redirected to this page



Provided that the user enters correct details, they are sent to the user dashboard



Users view a list of Economic News from BBC News



Users view a list of Economic News from The Telegraph



Consumers are able to input the size of their budget and select how frequent it gets renewed

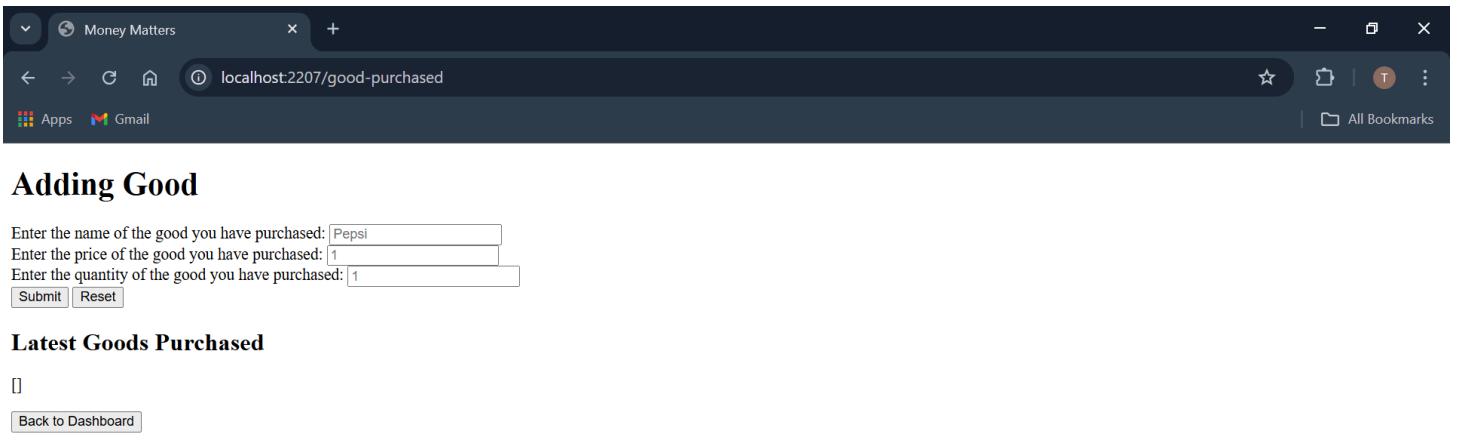
Tharun Nadeswaran - Money Matters
Student Code -
Centre Number -
145



The consumer enters their details



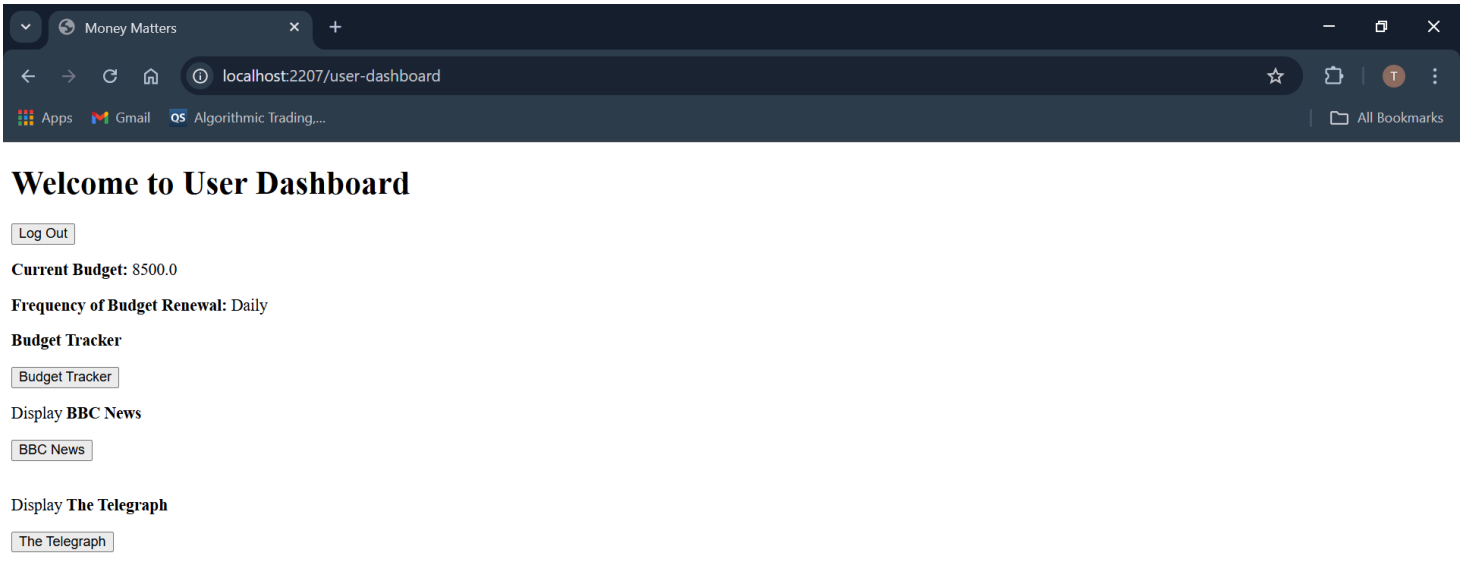
The consumer is redirected to the user dashboard, with their current budget and frequency of budget renewal being updated



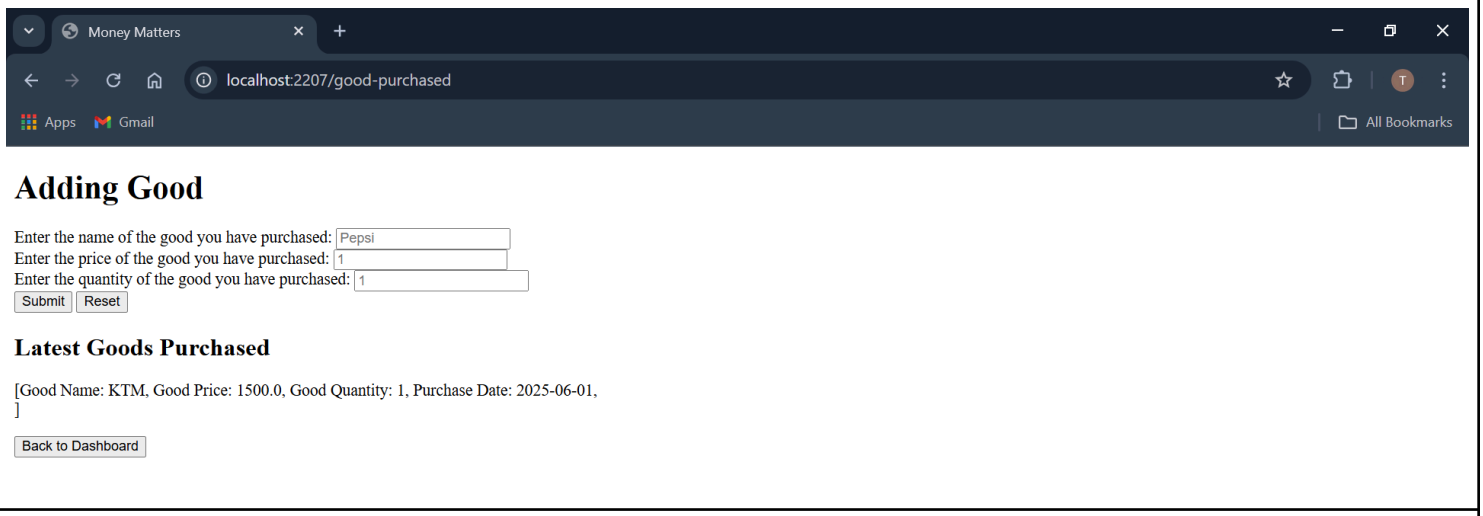
Consumers can enter details, on a good they have bought



Consumers enter details



Consumers are redirected to the home page, showing a reduced budget, based on the quantity and price of the goods.



When the consumer returns to the purchase good screen, they can see their latest purchases



Users will be redirected to this screen, if there is an unknown URL

Technical Solution

See References for the guides I have used to complete parts of the system.

Contents

MoneyMatters Package

File	Details	Skills	Page №
MoneyMattersApplication.java	Contains main method		**170**
application.properties	Contains server port and application name		**170**
pom.xml	Contains Springboot and other dependencies and configures the project.		**170**

Algorithms Package

File	Details	Skills	Page №
BinarySearch.java	Handles Recursive Binary Search for Integers and Objects	**ArrayList Operations** **Recursive Binary Search**	**174**
FileHandling.java	Creates folders, and files Deletes, reads, write, appends to .txt files	**File I/O Handling (Create, Read, Write, Append, Delete)**	**176**
FileHandlingJSON.java	Creates, write, reads .json files Reads the whole file, and converts it to a list of objects,	**LinkedHashMap and ArrayList Operations to store parsed JSON data.**	**180**

	or read a specific attribute	**Inherits from FileHandling: Inheritance, Overriding, Polymorphism** **JSON Tree Traversal**	
Glossary.java	Selects words and definition from .json file and stores it as a HashMap	**LinkedHashMap and ArrayList Operations** **Reads data from a JSON file** **Composition by using FileHandlingJSON**	**184**
MergeSort.java	Handles Recursive Merge Sort for Strings and Dates	**Works with user-defined records: EconomicNewsRecord and Purchase** **ArrayList Operations** **Recursive Merge Sort**	**186**
VernamCipher.java	Creates a random vernam key, which is the same length as the password, and stores the cipher in a .json file	**Composition by using FileHandlingJSON** **LinkedHashMap Operations** **Entropy - random key generation**	**189**

Controllers Package

File	Details	Skills	Page №
BudgetTrackerController.java	Retrieves user budget details, purchases, sustainable spending, future budgets based on inflation rates, and risky purchases. Users can update their budget, record new purchases, and filter spending history by (date, price, quantity)	**Cross-table parameterised SQL** **Aggregate SQL Functions** **Calls Flask API** **Complex Business Model Handles budget management, spending projections, and risk detection** **Composition** **Dynamically creates GoodRecord and Purchase objects** **Use recursive merge sort** **Uses SQL repositories**	**192**
CostOfProductionController.java	Producers add, update, delete, and optimise their factor inputs (labour and capital) Undo feature using a stack to make changes to factor input's data	**Stack Undo Mechanism** **ArrayList and LinkedHashMap Operations** **Cobb-Douglas Production Function** **Cross-table parameterised SQL**	**199**

		Aggregate SQL Functions **Optimal Factor Input** **Dynamically creates FactorInput objects** **Calls Python Flask API** **Composition** **Uses SQL repositories**	
EconomicFiguresController.java	Loads available countries from WorldBankAPI Once a user selects a country and a macro-economic indicators such as GDP, inflation, or unemployment, it will display relevant graphs, showing the change of indicators over time.	**ArrayList and LinkedHashMap Operations** **Use recursive bubble sort**	**206**
ExchangeRateController.java	Displays available currencies and dates Frankfurter API. Once selected, users can view graphs showing the changes in exchange and can convert currency	**Spring Boot REST API, using GET and POST Requests**	**210**

	from one to another.		
EconomicNewsController.java	Economic News is displayed to the user so that they can sort, filter and use A.I. to summarise the news.	**Calls Flask API to summarise news** **Dynamically creates EconomicNewsRecord** **Concurrent Processing (Multithreading) when calling Flask API** **Uses recursive merge sort** **ArrayList Operations**	**212**
InvestorController.java	Users are able to view graphs that show a change of the price of stocks, over a certain period of time, which is based on real life data from the python yfinance library. Users are able to buy and sell stock, where their prices can change over time, meaning that users can make a loss or profit.	**ArrayList LinkedHashMap Operations** **Encapsulates logic while interacting with multiple user types (Consumer and Producer)** **Calls Python Flask API** **Cross-table parameterised SQL** **Aggregate SQL Functions** **Uses SQL repositories**	**217**
LoginController.java	Users can either make a new	**Handles multiple types of users**	**225**

	account or login to their current account. Encrypts user's password using a random vernam key.	**(Producers or Consumers)** **Uses Vernam Cipher** **Dynamically creates Producer and Consumer objects** **Uses SQL repositories**	
RateTheEconomyController.java	Display questions and answers, which is fetched from a JSON file.	**Uses SQL repositories** **Files I/O Operations**	**232**
UserDashboardController.java	Loads terms and definitions from a JSON File. Users are able to search for specific terms, and use the A.I. bot to answer any questions about economic or finance.	**ArrayList LinkedHashMap Operations** **Uses recursive binary search** **Calls Flask API for A.I. generated answers**	**236**

EconomicFigures Package

File	Details	Skills	Page №
WorldBankAPI.java	Setups subclasses to fetch data from the World Bank API	**Calling APIs and parsing JSON** **Abstract Class**	**239**
MacroIndicators.java	Abstract class for fetching macroeconomic indicator data from the World Bank API	**Calling APIs and parsing JSON** **Abstract Methods**	**240**
MacroIndicatorFetcher.java	Creates a specific MacroIndicators subclass, where the object is based on the given World Bank indicator code.	**Dynamically creates different subclasses of MacroIndicators**	**243**
Countries.java	Fetches country data from the World Bank API	**Calling APIs and parsing JSON**	**245**
GDPFetcher.java	Configures how GDP values should be formatted	**Dynamic generation of objects based on complex user-defined use of OOP model** **Polymorphism**	**247**
GDPPerCapitaFetcher.java	Configures how GDP per Capita values should be formatted	**Dynamic generation of objects based on complex user-defined use of OOP model** **Polymorphism**	**248**

GiniIndexFetcher.java	Configures how Gini Index values should be formatted	**Dynamic generation of objects based on complex user-defined use of OOP model** **Polymorphism**	**249**
GovernmentDebtFetcher.java	Configures how Government debt values should be formatted	**Dynamic generation of objects based on complex user-defined use of OOP model** **Polymorphism**	**250**
InflationFetcher.java	Configures how Inflation values should be formatted	**Dynamic generation of objects based on complex user-defined use of OOP model** **Polymorphism**	**251**
InterestRateFetcher.java	Configures how Interest rates should be formatted	**Dynamic generation of objects based on complex user-defined use of OOP model** **Polymorphism**	**252**
NetTradeFetcher.java	Configures how Net Trade values should be formatted	**Dynamic generation of objects based on complex user-defined use of OOP model** **Polymorphism**	**253**

TaxRevenueGDPFetcher.java	Configures how Tax Revenue (GDP) values should be formatted	**Dynamic generation of objects based on complex user-defined use of OOP model** **Polymorphism**	**254**
TaxRevenueLCUFetcher.java	Configures how Tax Revenue (LCU) values should be formatted	**Dynamic generation of objects based on complex user-defined use of OOP model** **Polymorphism**	**255**
TotalReservesFetcher.java	Configures how Total Reserves values should be formatted	**Dynamic generation of objects based on complex user-defined use of OOP model** **Polymorphism**	**256**
TradeFetcher.java	Configures how Trade values should be formatted	**Dynamic generation of objects based on complex user-defined use of OOP model** **Polymorphism**	**257**
UnemploymentFetcher.java	Configures how Unemployment values should be formatted	**Dynamic generation of objects based on complex user-defined use of OOP model** **Polymorphism**	**258**

ExchangeRateAPI.java	Fetches the exchange rate for a single day between two currencies or a series of exchange rates between two currencies across multiple dates.	**Dynamic generation of objects based on complex user-defined use of OOP model**	**259**

Logic Package

File	Details	Skills	Page №
BudgetTrackerLogic.java	Tracking budgets, analysing purchases, and detecting outliers	**ArrayList LinkedHashMap Operations** **z-score outlier detection (calculates mean, variance, standard deviation), volatility calculation, dynamically grouping of purchases** **Calls Flask API** **Dynamically creates OutlierGoods objects** **Uses SQL repositories**	**263**
CostOfProductionLogic.java	Calculates cumulative marginal costs and revenue for a given producer and calculate the most profitable workers with given cost constraint	**ArrayList LinkedHashMap Operations** **Dynamically creates FactorInput objects** **Profit maximisation logic constrained by cost, combination generation (Optimisation)** **Recursive**	**270**

		Functions **Uses SQL repositories**	
InvestorLogic.java	Reads a JSON file of companies and their ticker symbols, and through the Python Flask API, the stock data is separated in Java into components: dates, opens, highs, lows, closes, and volumes for analysis.	**ArrayList** **LinkedHashMap** **Operations** **Dynamically creates Company Record objects** **Calls Flask API**	**274**
RateTheEconomy.java	Questions answers and answer weights are loaded from a JSON File and stored in an ArrayList of HashMaps. Converts a consumer's answers into a structured format, writes them to JSON files in a date-based folder hierarchy and updates the database with total scores. Collects and counts all user answers for a given date, providing frequency distributions per question.	**ArrayList** **LinkedHashMap** **Operations** **Uses SQL repositories** **File and Folder Handling, especially JSON Files**	**278**
WebScraperLogic.java	Scrapes news from RSS feeds and returns a list of	**ArrayList** **Operations** **Dynamically**	**285**

	EconomicNews Record	**creates EconomicNewsRecord** **Parses XML RSS Feeds.** **Extracts fields like title, description, pubDate, link, and image.** **XML Tree Traversal**	

PythonFlaskAPI Package

File	Details	Skills	Page №
BudgetTrackerMaths.py	Calculates the Maths elements of the Budget Tracker	**Matrix and Vector Calculations** **Python Class with static methods** **Projecting future budget**	**290**
CostOfProductionMaths.py	Calculates the Cobb Douglas Production Function	**Calculates Cobb-Douglas Production Function (with randomness) and Total Factor Productivity (TFP)** **Logarithmic transformations, exponents calculations**	**292**
InvestorData.py	Fetches historical stock data from yfinance and the company's information	**Calls Yahoo Finance API via yfinance to fetch real-time and historical stock data.** **Computes SMA, rolling mean, and linear regression to estimate trend gradient.** **Dictionaries**	**293**
OllamaAI.py	Uses A.I. models to generate answers to user questions or	**Downloads, parses, and processes natural**	**296**

	summarise economic news	**language articles for summarisation.** **API calls to Ollama AI.** **Uses newspaper to fetch article content over HTTP**	
PythonFlaskAPI.py	Sets up FlaskAPI to communicate with Java backend	**Flask server has multiple endpoints to clients and interacts with logic modules and external APIs** **Calls Ollama AI for natural language answers and summaries; fetches stock data from Yahoo Finance and returns JSON** **Dynamically creates classes and calls methods based on user input**	**298**
PythonFlaskAPI.java	Parses data through java to python and back to java with json payloads	**Complex Client-Server Model**	**302**
APIWorker.java	Worker Threads will continuously	**Queue Operation**	**312**

	process tasks from a shared queue. Each worker runs in its own thread and executes tasks until it is shut down.	**Multithreading** **Encapsulation** **ArrayList Operations**	
APIQueue.java	Contain worker threads and a shared task queue.	**Queue Operation** **Multithreading** **Encapsulation** **ArrayList Operations**	**314**

Records Package

File	Details	Skills	Page №
CompanyRecord.java	Contains a record class the stores Company data	**Creates Record**	**316**
DefinitionRecord.java	Contains a record class the stores Definition data	**Creates Record**	**316**
GoodRecord.java	Contains a record class the stores Goods data	**Creates Record**	**316**
FactorInput.java	Contains a record class the stores Factor Input data	**Creates Class**	**317**
OutlierGoods.java	Contains a record class the stores Outlier data	**Creates Class**	**320**
Purchase.java	Contains a record class the stores Purchase data	**Creates Class**	**322**
RateTheEconomyQuestions.java	Contains a record class the stores Rate The Economy questions	**Creates Class**	**324**

Repository Package

File	Details	Skills	Page №
SQLQueries.java	Setups subclasses to perform SQL Queries	**Abstract Class**	**325**
SQLQueriesDDL.java	Creates Tables, in an order, that prevents any referential integrity issues	**Inherits methods and attributes from SQLQueries** **DDL Scripts** **Concurrency** **Uses FileHandling to read SQL queries from text files**	**326**
SQLQueriesUsers.java	Abstract class that act as an interface by containing Consumer and Producer abstract methods	**Abstract Class**	**332**
SQLQueriesConsumers.java	Contains SQL Queries for the Consumer Table	**Inherits methods and attributes from SQLQueriesUsers** **DML and DQL Scripts** **Uses FileHandling to read SQL queries from text files**	**333**
SQLQueriesProducers.java	Contains SQL Queries for the Producer Table	**Inherits methods and attributes from SQLQueriesUsers** **DML and DQL**	**343**

		Scripts **Uses FileHandling to read SQL queries from text files**	
SQLQueriesBudgetTracker.java	Contains SQL Queries for the Budget Tracker Table	**DML and DQL Scripts** **Cross-Table Parameterised SQL**	**350**
SQLQueriesFactorInput.java	Contains SQL Queries for the Factor Input Table	**DML and DQL Scripts**	**353**
SQLQueriesGoods.java	Contains SQL Queries for the Goods Table	**DML and DQL Scripts** **Cross-Table Parameterised SQL**	**362**
SQLQueriesPurchase.java	Contains SQL Queries for the Purchase Table	**DML and DQL Scripts** **Cross-Table Parameterised SQL**	**365**
SQLQueriesRateTheEconomy.java	Contains SQL Queries for the Rate The Economy Table	**DML and DQL Scripts**	**367**
SQLQueriesStocks.java	Contains SQL Queries for the Stock Table	**DML and DQL Scripts**	**372**
SQLQueriesTrade.java	Contains SQL Queries for the Trades Table	**DML and DQL Scripts** **Cross-Table Parameterised SQL**	**376**

Users Package

File	Details	Skills	Page №
UserInterface.java	Interface defines what methods every user class must have, ensuring consistency	**Interface**	**380**
User.java	Abstract Class provides shared attributes and methods, reducing duplication	**Abstract Class**	**381**
Consumer.java	Inherits attributes and methods from User	**Inheritance** **Polymorphism**	**384**
Producer.java	Inherits attributes and methods from User	**Inheritance** **Polymorphism**	**388**

JSON Files

File	Details	Skills	Page №
userOpinion1.json	Contains a JSON file of the user's opinions from the Rate The Economy. Stored as a list of objects, where each object contains a question, answer and weight	**Parsing JSON**	**390**
tharunnadeswaran2207@gmail.com.json	Contains a JSON file of the user's vernam key	**Parsing JSON**	**392**
glossaryOfEconomicTerms.json	Contains a list of objects, where each object consists a word and a definition	**Parsing JSON**	**393**
rateTheEconomyQuestions.json	Shows all the questions available from the Rate The Economy. Stored as a list of objects, where each object contains a question, a dictionary of answers and weights	**Parsing JSON**	**395**
tickerInfo.json	Contains a list of objects, where each object contains a company name and a ticker symbol	**Parsing JSON**	**399**

MoneyMatters Package

MoneyMatterApplication.java

```java
package com.mycompany.MoneyMatters;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.annotation.ComponentScan;
import com.mycompany.Repository.SQLQueriesDDL;

@SpringBootApplication
@ComponentScan(basePackages = "com.mycompany.*")
public class MoneyMattersApplication {
    public static void main(String[] args) {
        SQLQueriesDDL.createTables();
        SpringApplication.run(MoneyMattersApplication.class, args);
    }
}
```

application.properties

```
spring.application.name=MoneyMatters
server.port=2207
```

pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.5.5</version>
        <relativePath/>
    </parent>
    <groupId>com.mycompany</groupId>
    <artifactId>MoneyMatters</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <name>MoneyMatters</name>
    <description>NEA Project</description>
    <url/>
    <licenses>
        <license/>
    </licenses>
    <developers>
        <developer/>
    </developers>
    <scm>
        <connection/>
        <developerConnection/>
        <tag/>
        <url/>
    </scm>
    <properties>
```

```
        <java.version>21</java.version>
    </properties>
    <dependencies>

        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-actuator</artifactId>
        </dependency>

        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>

        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-test</artifactId>
            <scope>test</scope>
        </dependency>

        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-thymeleaf</artifactId>
        </dependency>

        <dependency>
            <groupId>org.jsoup</groupId>
            <artifactId>jsoup</artifactId>
            <version>1.16.1</version>
        </dependency>
```

```
        <dependency>
            <groupId>com.microsoft.sqlserver</groupId>
            <artifactId>mssql-jdbc</artifactId>
        </dependency>

        <dependency>
            <groupId>com.fasterxml.jackson.core</groupId>
            <artifactId>jackson-databind</artifactId>
        </dependency>

        <dependency>
            <groupId>org.mockito</groupId>
            <artifactId>mockito-core</artifactId>
            <scope>test</scope>
        </dependency>

    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
        </plugins>
    </build>
</project>
```

Algorithms Package

BinarySearch.java

```java
package com.mycompany.Algorithms;

import java.util.ArrayList;

public class BinarySearch {
    public static int binarySearchInteger(int[] arr, int min, int max, int key) {
        if (min > max) {
            return -1;
        }

        // Find midpoint
        int mid = (min + max) / 2;

        // return the index, if equal
        if (arr[mid] == key) {
            return mid;
        }

        // If the middle element is less than the key, search in the right half
        else if (arr[mid] < key) {
            return binarySearchInteger(arr, mid + 1, max, key);
        }

        // Else, search in the left half
        else {
            return binarySearchInteger(arr, min, mid - 1, key);
        }
```

```
    }

    public static int binarySearchObject(ArrayList<?> arr, int min, int max, String key) {
        if (min > max) {
            return -1;
        }

        // Find midpoint
        int mid = (min + max) / 2;

        // Compare the key with the middle element (converted to String)
        int comparison = key.compareTo(arr.get(mid).toString());

        // return the index, if equal
        if (comparison == 0) {
            return mid;
        }

        // If the middle element is less than the key, search the right half
        else if (comparison > 0) {
            return binarySearchObject(arr, mid + 1, max, key);
        }

        // Else, search the left half
        else {
            return binarySearchObject(arr, min, mid - 1, key);
        }
    }
}
```

FileHandling.java

```
package com.mycompany.Algorithms;

import java.io.BufferedReader;
import java.io.File;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;

public class FileHandling {

    private final String fileExtension = ".txt";

    public File createFolder(String rootDirectory) {
        try {
            File myFolder = new File(rootDirectory);

            // Creates folder if it doesn't exist
            if (!myFolder.exists()) {
                if (myFolder.mkdir()) {
                    System.out.println("Folder created: " + rootDirectory);
                }

                else {
                    System.out.println("Folder exists: " + rootDirectory);
                }
            }

            return myFolder;
        }
```

```
        catch (Exception ex) {
            System.out.println("Error with folder: " + rootDirectory);
        }

        return null;
    }

    public File createFile(String rootDirectory) {
        try {
            // Appends ".txt" extension
            File myFile = new File(rootDirectory + fileExtension);

            if (myFile.createNewFile()) {
                System.out.println("File created: " + myFile.getName());
            }

            else {
                System.out.println("File already exists.");
            }

            return myFile;
        }

        catch (IOException ex) {
            ex.printStackTrace();
        }

        return null;
    }
```

```
    // Deletes a file
    public void deleteFile(File file) {
        if (file.delete()) {
            System.out.println("Deleted the file: " + file.getName());
        }

        else {
            System.out.println("Failed to delete the file.");
        }
    }

    public String readFile(File file) {
        StringBuilder sb = new StringBuilder();
        try (BufferedReader reader = new BufferedReader(new FileReader(file))) {
            String line;
            // Read each line of the file and adds to StringBuilder
            while ((line = reader.readLine()) != null) {
                sb.append(line);
            }
        }

        catch (IOException ex) {
            ex.printStackTrace();
        }

        return sb.toString();
    }

    // Writes contents to a file
    public void writeFile(File file, Object text) {
        try (FileWriter writer = new FileWriter(file, false)) {
```

```
            writer.write(text.toString());
        }

        catch (IOException ex) {
            ex.printStackTrace();
        }
    }

    // Appends contents to a file
    public void appendFile(File file, Object text) {
        try (FileWriter writer = new FileWriter(file, true)) {
            writer.write(text.toString());
        }

        catch (IOException ex) {
            ex.printStackTrace();
        }
    }
}
```

FileHandlingJSON.java

```java
package com.mycompany.Algorithms;

import java.io.File;
import java.io.IOException;
import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;

import com.fasterxml.jackson.core.type.TypeReference;
import com.fasterxml.jackson.databind.JsonNode;
import com.fasterxml.jackson.databind.ObjectMapper;

// Extends FileHandling Class to for specific JSON operations.
public class FileHandlingJSON extends FileHandling {

    private final String fileExtension = ".json";

    @Override
    public File createFile(String rootDirectory) {
        try {
            // Appends ".json" extension
            File myFile = new File(rootDirectory + fileExtension);

            if (myFile.createNewFile()) {
                System.out.println("File created: " + myFile.getName());
            }

            else {
                System.out.println("File already exists.");
```

```
            }
            return myFile;
        }

        catch (IOException e) {
            System.out.println("An error occurred while creating the JSON file.");
        }

        return null;
    }

    @Override
    public void writeFile(File file, Object text) {
        ObjectMapper objectMapper = new ObjectMapper();
        try {
            // Serialise object to JSON and write it to file
            objectMapper.writerWithDefaultPrettyPrinter().writeValue(file, text);
        }

        catch (Exception ex) {
            ex.printStackTrace();
        }
    }

    // Reads JSON files and converts it into an LinkedHashMap.
    public <T> LinkedHashMap<String, T> readFile(File file, TypeReference<LinkedHashMap<String, T>> typeReference) {
        ObjectMapper objectMapper = new ObjectMapper();
        try {
            LinkedHashMap<String, T> contents = objectMapper.readValue(file, typeReference);
            if (contents != null) {
                return contents;
```

```
            }
        }

        catch (Exception ex) {
            ex.printStackTrace();
        }

        return null;
    }

    // Reads JSON files and converts it into an ArrayList of objects.
    public <T> ArrayList<T> readFileList(File file, TypeReference<List<T>> typeReference) {
        ObjectMapper objectMapper = new ObjectMapper();
        try {
            ArrayList<T> contents = new ArrayList<>(objectMapper.readValue(file, typeReference));
            return contents;
        }

        catch (Exception ex) {
            ex.printStackTrace();
        }

        return null;
    }

    // Reads a specific field from all objects in a JSON array file and returns them as strings.
    public ArrayList<String> readFieldNameString(File file, String fieldName) {
        ObjectMapper objectMapper = new ObjectMapper();

        try {
            // Parse JSON file into a tree
```

```
            JsonNode rootNode = objectMapper.readTree(file);
            ArrayList<String> contents = new ArrayList<>();

            // Loop through each object in the JSON array and extract the field value
            for (JsonNode node : rootNode) {
                contents.add(node.get(fieldName).asText());
            }

            return contents;
        }

        catch (Exception ex) {
            ex.printStackTrace();
        }

        return null;
    }
}
```

Glossary.java

```java
package com.mycompany.Algorithms;

import java.io.File;
import java.util.ArrayList;
import java.util.LinkedHashMap;

public class Glossary {

    private static final FileHandlingJSON FileHandlingJSON = new FileHandlingJSON();

    // Reads glossaryOfEconomicTerms.json and stores each word-definition pair in a LinkedHashMap.
    public static LinkedHashMap<String, String> storeTermsToLinkedHashMap() {
        File glossary = new File("glossaryOfEconomicTerms.json");
        LinkedHashMap<String, String> wordsDefintionsPairs = new LinkedHashMap<>();

        // Extract "word" and "definition" fields and put them in the map
        try {
            ArrayList<String> words = FileHandlingJSON.readFieldNameString(glossary, "word");
            ArrayList<String> definitions = FileHandlingJSON.readFieldNameString(glossary, "definition");

            for (int i = 0; i < words.size(); i++) {
                wordsDefintionsPairs.put(words.get(i), definitions.get(i));
            }
        }

        catch (Exception ex) {
            System.out.println("Error reading glossary JSON file");
        }
```

```
        return wordsDefintionsPairs;
    }

    // Returns all words from the glossary as an ArrayList.
    public static ArrayList<String> storeWordsToArrayList() {
        return new ArrayList<>(storeTermsToLinkedHashMap().keySet());
    }

    // Returns all words in lowercase and without spaces, for searching.
    public static ArrayList<String> storePlainWordsToArrayList() {
        ArrayList<String> storeWordsToArrayList = storeWordsToArrayList();
        ArrayList<String> storePlainWordsToArrayList = new ArrayList<>();

        for (String word : storeWordsToArrayList) {
            storePlainWordsToArrayList.add(word.replace(" ", "").toLowerCase());
        }

        return storePlainWordsToArrayList;
    }

    // Returns all definitions from the glossary as an ArrayList.
    public static ArrayList<String> storeDefinitionsToArrayList() {
        return new ArrayList<>(storeTermsToLinkedHashMap().values());
    }
}
```

MergeSort.java

```java
package com.mycompany.Algorithms;

import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.util.ArrayList;
import java.util.Comparator;
import com.mycompany.Records.EconomicNewsRecord;
import com.mycompany.Records.Purchase;

public class MergeSort {

    public static <T> void mergeSort(ArrayList<T> arr, Comparator<T> comparator) {
        if (arr.size() < 2) {
            return;
        }

        int mid = arr.size() / 2;

        // Split array into left and right halves
        ArrayList<T> left = new ArrayList<>(arr.subList(0, mid));
        ArrayList<T> right = new ArrayList<>(arr.subList(mid, arr.size()));

        // Recursively sort the halves
        mergeSort(left, comparator);
        mergeSort(right, comparator);

        // Merge the sorted halves
        merge(arr, left, right, comparator);
    }
```

```
    private static <T> void merge(ArrayList<T> result, ArrayList<T> left, ArrayList<T> right, Comparator<T>
comparator) {
        int i = 0, j = 0;
        result.clear(); // Clear the result list before merging

        // Merge elements in order
        while (i < left.size() && j < right.size()) {
            if (comparator.compare(left.get(i), right.get(j)) <= 0) {
                result.add(left.get(i++));
            }

            else {
                result.add(right.get(j++));
            }
        }

        // Add any remaining elements from left
        while (i < left.size()) {
            result.add(left.get(i++));
        }

        // Add any remaining elements from right
        while (j < right.size()) {
            result.add(right.get(j++));
        }
    }

    //Sorts EconomicNewsRecord objects alphabetically by their title.
    public static void mergeSortEconomicNewsByTitle(ArrayList<EconomicNewsRecord> arr) {
        mergeSort(arr, Comparator.comparing(EconomicNewsRecord::title));
```

```
    }

    // Sorts EconomicNewsRecord by their publication date (oldest to newest)
    public static void mergeSortEconomicNewsByDate(ArrayList<EconomicNewsRecord> arr) {
        mergeSort(arr, Comparator.comparing(MergeSort::parseNewsDate));
    }

    private static LocalDate parseNewsDate(EconomicNewsRecord record) {
        return LocalDate.parse(record.pubDate(), DateTimeFormatter.ofPattern("dd/MM/yyyy"));
    }

    // Sorts Purchase by purchase date (oldest to newest)
    public static void mergeSortPurchasesByDate(ArrayList<Purchase> arr) {
        mergeSort(arr, Comparator.comparing(MergeSort::parsePurchaseDate));
    }

    private static LocalDate parsePurchaseDate(Purchase purchase) {
        return LocalDate.parse(purchase.getPurchaseDate());
    }

    //Sorts Purchase by price (lowest to highest).
    public static void mergeSortPurchasesByPrice(ArrayList<Purchase> arr) {
        mergeSort(arr, Comparator.comparingDouble(Purchase::getGoodPrice));
    }

    //Sorts Purchase by quantity purchased (lowest to highest).
    public static void mergeSortPurchasesByQuantity(ArrayList<Purchase> arr) {
        mergeSort(arr, Comparator.comparingDouble(Purchase::getGoodQuantity));
    }
}
```

VernamCipher.java

```
package com.mycompany.Algorithms;

import java.io.File;
import java.util.LinkedHashMap;
import java.util.Random;
import com.fasterxml.jackson.core.type.TypeReference;

public class VernamCipher {

    private static final FileHandlingJSON FileHandlingJSON = new FileHandlingJSON();

    public static String convertPlainTextToBinary(String plainText) {
        StringBuilder binaryText = new StringBuilder();
        char[] plainTextAsCharArray = plainText.toCharArray();

        for (int i = 0; i < plainTextAsCharArray.length; i++) {
            String binaryValue = Integer.toBinaryString(plainTextAsCharArray[i]);
            binaryText.append(binaryValue);
        }

        return binaryText.toString();
    }

    // Generates a random Vernam key of the same length as the ASCII version of the plain text. Stores the generated
key in a JSON file, which is named as the user's email.
    public static String createVernamKey(String email, String plainText) {
        Random random = new Random();
        int keyLength = plainText.length() * 7;
        StringBuilder vernamKey = new StringBuilder();
```

```
        // Generate random vernam key
        for (int i = 0; i < keyLength; i++) {
            int bit = random.nextInt(2);
            vernamKey.append(bit);
        }

        // Store the key to a JSON file
        storeVernamKeysInJSON(email, vernamKey.toString());
        return vernamKey.toString();
    }

    // XOR Operation between the ASCII text and the Vernam key.
    public static String XORfunction(String binaryText, String vernamKey) {
        StringBuilder resultingText = new StringBuilder();
        char[] binaryTextAsCharArray = binaryText.toCharArray();
        char[] vernamKeyAsCharArray = vernamKey.toCharArray();

        for (int i = 0; i < binaryTextAsCharArray.length; i++) {
            if (binaryTextAsCharArray[i] == vernamKeyAsCharArray[i]) {
                resultingText.append(0);
            }

            else {
                resultingText.append(1);
            }
        }

        return resultingText.toString();
    }
```

```
    // Stores Vernam Key in JSON File
    public static void storeVernamKeysInJSON(String email, String vernamKey) {
        String folderName = "VernamKeys";

        // Creates VernamKeys folder, if folder doesn't exist.
        FileHandlingJSON.createFolder(folderName);
        File vernamKeyFile = FileHandlingJSON.createFile(folderName + File.separator + email);

        LinkedHashMap<String, String> vernamKeyLinkedHashMap = new LinkedHashMap<>();
        vernamKeyLinkedHashMap.put("vernamKey", vernamKey);

        // Write the key to the file
        FileHandlingJSON.writeFile(vernamKeyFile, vernamKeyLinkedHashMap);
    }

    // Loads the Vernam key for a given user's email from the JSON file.
    public static String loadVernamKey(String email) {
        String folderName = "VernamKeys";
        String fileName = email + ".json";
        File vernamKeyFile = new File(folderName + File.separator + fileName);

        LinkedHashMap<String, String> vernamKeyMap = FileHandlingJSON.readFile(vernamKeyFile, new
TypeReference<LinkedHashMap<String, String>>() {});
        return vernamKeyMap.get("vernamKey");
    }
}
```

Controllers Package

BudgetTrackerController.java

```
package com.mycompany.Controllers;

import java.time.LocalDate;
import java.time.Year;
import java.time.format.DateTimeFormatter;
import java.util.ArrayList;
import java.util.Collections;
import java.util.LinkedHashMap;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.servlet.ModelAndView;
import com.mycompany.Algorithms.MergeSort;
import com.mycompany.EconomicFigures.MacroIndicatorFetcher;
import com.mycompany.EconomicFigures.MacroIndicators;
import com.mycompany.Logic.BudgetTrackerLogic;
import com.mycompany.PythonFlaskAPI.PythonFlaskAPI;
import com.mycompany.Records.GoodRecord;
import com.mycompany.Records.Purchase;
import com.mycompany.Repository.SQLQueriesBudgetTracker;
import com.mycompany.Repository.SQLQueriesConsumers;
import com.mycompany.Repository.SQLQueriesGoods;
import com.mycompany.Repository.SQLQueriesPurchase;
import com.mycompany.Users.Consumer;
import com.mycompany.Users.Producer;
```

```
import jakarta.servlet.http.HttpSession;

@RestController
public class BudgetTrackerController {

    private static final SQLQueriesConsumers SQLQueriesConsumers = new SQLQueriesConsumers();

    public static ModelAndView showData(ModelAndView mv, Consumer consumer) {
        mv.addObject("budgetSize", consumer.getBudgetSize());
        mv.addObject("currentBudget", consumer.getCurrentBudget());
        mv.addObject("frequencyOfBudgetRenewal", consumer.getFrequencyOfBudgetRenewal());

        // Calculate income distribution (Essentials, Leisure, Savings)
        LinkedHashMap<String, Double> incomeComponents =
PythonFlaskAPI.calculateIncomeComponents(consumer.getBudgetSize());
        mv.addObject("incomeComponents", incomeComponents);

        if (consumer.getFrequencyOfBudgetRenewal().equals("Monthly")) {
            mv.addObject("sustainableSpending",
PythonFlaskAPI.sustainableSpending(incomeComponents.get("Essentials")));
        }

        // Fetch UK inflation data to project consumer's future budget
        MacroIndicators UKInflationRate = MacroIndicatorFetcher.instantiateIndicator("GBR", "FP.CPI.TOTL.ZG");
        mv.addObject("futureBudget", PythonFlaskAPI.futureBudget(incomeComponents,
UKInflationRate.getData().get(Year.now().minusYears(1)).doubleValue()));

        mv.addObject("todaySpending", SQLQueriesPurchase.showTodaysSpending(consumer));

        // Display future budget over the next 12 months
        ArrayList<String> upcomingMonthsArrayList = new ArrayList<>();
```

```
        for (int i = 0; i < 12; i++) {
            DateTimeFormatter df = DateTimeFormatter.ofPattern("dd/MM/yyyy");
            LocalDate date = LocalDate.now().plusMonths(i);
            upcomingMonthsArrayList.add(date.format(df));
        }

        mv.addObject("months", upcomingMonthsArrayList);

        // Add spending trend data and outlier purchases
        mv.addObject("dateTotalPricePairs", BudgetTrackerLogic.getDateTotalPricePairs(consumer));
        mv.addObject("riskyPurchases", BudgetTrackerLogic.getOutliers(consumer));

        return mv;
    }

    @GetMapping("budget-tracker")
    public ModelAndView showBudgetTracker(HttpSession session) {
        ModelAndView mv = new ModelAndView("budgettracker/budget-tracker");

        if (session.getAttribute("user") instanceof Consumer consumer) {
            showData(mv, consumer);

            BudgetTrackerLogic.saveNextRenewedDate(consumer);

            ArrayList<Purchase> allConsumerPurchases = SQLQueriesBudgetTracker.getConsumerPurchase(consumer);
            MergeSort.mergeSortPurchasesByDate(allConsumerPurchases);
            Collections.reverse(allConsumerPurchases);
            mv.addObject("consumerPurchases", allConsumerPurchases);
        }

        // If logged in as a Producer
```

```
        else if (session.getAttribute("user") instanceof Producer producer) {
            mv.setViewName("userdashboard/user-dashboard");
            mv.addObject("windowsAlert", true);
            mv.addObject("windowsAlertMessage", "Unable To Access. Need To Be A Consumer.");
        }

        // If not logged in
        else {
            mv.setViewName("login/start");
        }

        return mv;
    }

    @PostMapping("budget-tracker")
    public ModelAndView storeBudgetData(@RequestParam String formName,
                @RequestParam (required = false) Double budgetSize,
                @RequestParam (required = false) String frequencyOfRenewal,
                @RequestParam (required = false) String goodName,
                @RequestParam (required = false) Double goodPrice,
                @RequestParam (required = false) Integer goodQuantity,
                @RequestParam (required = false) String sort,
                HttpSession session) {

        ModelAndView mv = new ModelAndView("redirect:budget-tracker");

        if (session.getAttribute("user") instanceof Consumer consumer) {

            // Update budget details
            switch (formName) {
                case "userBudget":
```

```
                    if (budgetSize != null) {
                        SQLQueriesConsumers.updateBudgetSize(budgetSize, consumer);
                        SQLQueriesConsumers.updateCurrentBudget(budgetSize, consumer);
                        SQLQueriesConsumers.updateFrequencyOfBudgetRenewal(frequencyOfRenewal, consumer);
                        SQLQueriesConsumers.updateLastRenewed(LocalDate.now(), consumer);

                        // Update sustainable spending based on new budget
                        LinkedHashMap<String, Double> incomeComponents =
PythonFlaskAPI.calculateIncomeComponents(consumer.getBudgetSize());

SQLQueriesConsumers.updateSustainableSpending(PythonFlaskAPI.sustainableSpending(incomeComponents.get("Essentials")),
consumer);
                    }
                    break;

                case "userGoods":
                    double currentBudget = consumer.getCurrentBudget();
                    double totalCost = goodPrice * goodQuantity;

                    // Prevent purchase if insufficient budget
                    if (currentBudget < totalCost) {
                        mv.setViewName("redirect:budget-tracker");
                        mv.addObject("windowsAlert", true);
                        mv.addObject("windowsAlertMessage", "Insufficient budget for this purchase!");
                        return mv;
                    }

                    if (!SQLQueriesGoods.doesGoodIDExist(goodName, goodPrice)) {
                        SQLQueriesGoods.insertGood(goodName, goodPrice);
                    }
```

```
                    GoodRecord good = new GoodRecord(SQLQueriesGoods.getGoodID(goodName, goodPrice), goodName,
goodPrice);
                    SQLQueriesPurchase.insertPurchase(consumer, good, goodQuantity);
                    currentBudget -= totalCost;

                    SQLQueriesConsumers.updateCurrentBudget(currentBudget, consumer);
                    consumer.setCurrentBudget(currentBudget);
                    break;

                case "filterBudget":
                    mv.setViewName("budgettracker/budget-tracker");
                    ArrayList<Purchase> allConsumerPurchases = SQLQueriesBudgetTracker.getConsumerPurchase(consumer);
                    switch (sort) {
                        case "date (latest)":
                            MergeSort.mergeSortPurchasesByDate(allConsumerPurchases);
                            Collections.reverse(allConsumerPurchases);
                            break;
                        case "date (earliest)":
                            MergeSort.mergeSortPurchasesByDate(allConsumerPurchases);
                            break;
                        case "most expensive":
                            MergeSort.mergeSortPurchasesByPrice(allConsumerPurchases);
                            Collections.reverse(allConsumerPurchases);
                            break;
                        case "least expensive":
                            MergeSort.mergeSortPurchasesByPrice(allConsumerPurchases);
                            break;
                        case "most bought":
                            MergeSort.mergeSortPurchasesByQuantity(allConsumerPurchases);
                            Collections.reverse(allConsumerPurchases);
                            break;
```

```
                    case "least bought":
                        MergeSort.mergeSortPurchasesByQuantity(allConsumerPurchases);
                        break;
                    default:
                        break;
                }

                mv.addObject("selectedSort", sort);
                mv.addObject("consumerPurchases", allConsumerPurchases);
                break;

            default:
                break;
        }

        consumer = SQLQueriesConsumers.getData(consumer.getConsumerID());
        session.setAttribute("user", consumer);
        showData(mv, consumer);
    }

    return mv;
  }
}
```

CostOfProductionController.java

```java
package com.mycompany.Controllers;

import java.text.DecimalFormat;
import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.Map;
import java.util.Stack;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.servlet.ModelAndView;
import com.mycompany.Logic.CostOfProductionLogic;
import com.mycompany.PythonFlaskAPI.PythonFlaskAPI;
import com.mycompany.Records.FactorInput;
import com.mycompany.Repository.SQLQueriesFactorInput;
import com.mycompany.Users.Consumer;
import com.mycompany.Users.Producer;
import jakarta.servlet.http.HttpSession;

@RestController
public class CostOfProductionController {

    // Stack to store previous states of FactorInput for undo
    private final Stack<LinkedHashMap<String, FactorInput>> undoStack = new Stack<>();

    public ModelAndView showData(ModelAndView mv, Producer producer) {
        DecimalFormat df = new DecimalFormat("0.00");
        Integer numberOfLabour;
```

```
        Integer numberOfCapital;
        Integer totalNumberOfFactorInputs;
        Double totalCost;
        Double averageCost;
        Double totalRevenue;
        Double averageRevenue;
        Double totalProfit;
        Double averageProfit;
        Double cobbDouglasFunction;

        // Fetch all factor inputs from a given producer
        LinkedHashMap<String, FactorInput> allFactorInputsLinkedHashMap =
SQLQueriesFactorInput.getAllFactorInputTable(producer);

        try {
            // Fetch counts of labour and capital inputs
            numberOfLabour = SQLQueriesFactorInput.getNumberOfFactorInputs("Labour", producer);
            numberOfCapital = SQLQueriesFactorInput.getNumberOfFactorInputs("Capital", producer);
            totalNumberOfFactorInputs = numberOfLabour + numberOfCapital;

            // Fetch totals and averages
            totalCost = SQLQueriesFactorInput.getTotalCost(producer.getProducerID());
            averageCost = SQLQueriesFactorInput.getAverageCost(producer.getProducerID());
            totalRevenue = SQLQueriesFactorInput.getTotalRevenue(producer.getProducerID());
            averageRevenue = SQLQueriesFactorInput.getAverageRevenue(producer.getProducerID());
            totalProfit = SQLQueriesFactorInput.getTotalProfit(producer.getProducerID());
            averageProfit = SQLQueriesFactorInput.getAverageProfit(producer.getProducerID());

            // Perform Cobb-Douglas production function
            cobbDouglasFunction = Double.valueOf(PythonFlaskAPI.performCobbDouglasFunction(totalRevenue, totalCost,
numberOfCapital, numberOfLabour));
```

```
}

catch (Exception ex) {
    numberOfLabour = 0;
    numberOfCapital = 0;
    totalCost = 0.0;
    averageCost = 0.0;
    totalRevenue = 0.0;
    averageRevenue = 0.0;
    totalProfit = 0.0;
    averageProfit = 0.0;
    cobbDouglasFunction = 0.0;
}

mv.addObject("numberOfLabour", numberOfLabour);
mv.addObject("numberOfCapital", numberOfCapital);
mv.addObject("totalCost", df.format(totalCost));
mv.addObject("averageCost", df.format(averageCost));
mv.addObject("totalRevenue", df.format(totalRevenue));
mv.addObject("averageRevenue", df.format(averageRevenue));
mv.addObject("totalProfit", df.format(totalProfit));
mv.addObject("averageProfit", df.format(averageProfit));

try {
    mv.addObject("cobbDouglasFunction", df.format(cobbDouglasFunction));
}

catch (Exception ex){
    System.out.println("Error when parsing the cobbDouglasFunction");
}
```

```
        mv.addObject("allFactorInputs", allFactorInputsLinkedHashMap);
        mv.addObject("cumulativeMarginalCosts", CostOfProductionLogic.getCumulativeMarginalCosts(producer));
        mv.addObject("cumulativeMarginalRevenue", CostOfProductionLogic.getCumulativeMarginalRevenue(producer));

        return mv;
    }

    @GetMapping("cost-of-production")
    public ModelAndView showCostOfProduction(HttpSession session) {
        ModelAndView mv = new ModelAndView("costofproduction/cost-of-production");

        if (session.getAttribute("user") instanceof Producer producer) {
            showData(mv, producer);
        }

        else if (session.getAttribute("user") instanceof Consumer consumer) {
            mv.setViewName("userdashboard/user-dashboard");
            mv.addObject("windowsAlert", true);
            mv.addObject("windowsAlertMessage", "Unable To Access. Need To Be A Producer.");
        }

        else {
            mv.setViewName("login/start");
        }

        return mv;
    }

    @PostMapping("cost-of-production")
    public ModelAndView getCostOfProduction(@RequestParam String formName,
                                            @RequestParam (required = false) String factorInputType,
```

```
                        @RequestParam (required = false) String factorInputName,
                        @RequestParam (required = false) String factorInputDescription,
                        @RequestParam (required = false) Double marginalCost,
                        @RequestParam (required = false) Double marginalRevenue,
                        @RequestParam (required = false) String factorInputDelete,
                        @RequestParam (required = false) Double maximumCost,
                        HttpSession session) {

        ModelAndView mv = new ModelAndView("costofproduction/cost-of-production");
        LinkedHashMap<String, FactorInput> tempLinkedHashMap = new LinkedHashMap<>();

        if (session.getAttribute("user") instanceof Producer producer) {

            // Insert new factor input
            if (formName.equals("factorInputInsert")) {
                FactorInput factorInput = new FactorInput(factorInputType, factorInputName, factorInputDescription,
marginalCost, marginalRevenue);
                tempLinkedHashMap.put("Insert", factorInput);
                SQLQueriesFactorInput.insertDataIntoFactorInputTable(factorInput, producer);
            }

            // Update existing factor input (push previous state to undo stack)
            else if (formName.equals("factorInputUpdate")) {
                tempLinkedHashMap.put("Update", SQLQueriesFactorInput.getFactorInputByName(producer,
factorInputName));
                SQLQueriesFactorInput.updateDataIntoFactorInputTable(marginalCost, marginalRevenue, factorInputName,
producer);
                undoStack.add(tempLinkedHashMap);
            }

            // Delete factor input (save deleted record to undo stack)
```

```
            else if (formName.equals("factorInputDelete")) {
                tempLinkedHashMap.put("Delete", SQLQueriesFactorInput.getFactorInputByName(producer,
factorInputDelete));
                SQLQueriesFactorInput.deleteDataFromFactorInputTable(factorInputDelete, producer);
                undoStack.add(tempLinkedHashMap);
            }

            // Compute optimal factor input combination under budget constraint
            else if (formName.equals("optimalFactorInput")) {
                LinkedHashMap<String, FactorInput> allFactorInputsLinkedHashMap =
SQLQueriesFactorInput.getAllFactorInputTable(producer);
                ArrayList<FactorInput> allFactorInputsArrayList = new
ArrayList<>(allFactorInputsLinkedHashMap.values());
                mv.addObject("bestCombination", CostOfProductionLogic.showBestCombination(allFactorInputsArrayList,
maximumCost));
            }

            // Undo last change
            else if (formName.equals("undo")) {
                if (!undoStack.isEmpty()) {
                    Map<String, FactorInput> lastAction = undoStack.pop();
                    String action = lastAction.keySet().iterator().next();
                    FactorInput factorInput = lastAction.values().iterator().next();

                    switch (action) {
                        case "Update": // change to old cost/revenue values
                            SQLQueriesFactorInput.updateDataIntoFactorInputTable(factorInput.getMarginalCost(),
factorInput.getMarginalRevenue(), factorInput.getFactorInputName(), producer);
                            break;
                        case "Insert": // remove the inserted record
                            SQLQueriesFactorInput.deleteDataFromFactorInputTable(factorInput.getFactorInputName(),
```

```
producer);
                            break;
                        case "Delete": // restore deleted record
                            SQLQueriesFactorInput.insertDataIntoFactorInputTable(factorInput, producer);
                            break;
                        default:
                            break;
                    }
                }

                else {
                    // Stack is empty
                    mv.addObject("windowsAlert", true);
                    mv.addObject("windowsAlertMessage", "Stack is Empty, Nothing to undo");
                }
            }

            showData(mv, producer);
        }

        return mv;
    }
}
```

EconomicFiguresController.java

```java
package com.mycompany.Controllers;

import java.math.BigDecimal;
import java.time.Year;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.LinkedHashMap;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.servlet.ModelAndView;
import com.mycompany.Algorithms.BinarySearch;
import com.mycompany.EconomicFigures.Countries;
import com.mycompany.EconomicFigures.MacroIndicatorFetcher;
import com.mycompany.EconomicFigures.MacroIndicators;

@RestController
public class EconomicFiguresController {

    @GetMapping("figures-economic")
    public ModelAndView showEconomicFigures() {
        ModelAndView mv = new ModelAndView("economicfigures/figures-economic");

        // Instantiate the Countries object to get all country data
        Countries countries = new Countries();
        LinkedHashMap<String, String> countryCodes = countries.getCountryData();

        mv.addObject("countryCodes", countryCodes);
```

```
        return mv;
    }

    @PostMapping("figures-economic")
    public ModelAndView getEconomicFigures(@RequestParam String countryCode,
                                           @RequestParam String indicatorCode) {

        ModelAndView mv = new ModelAndView("economicfigures/figures-economic");

        Countries countriesObj = new Countries();
        LinkedHashMap<String, String> countryCodes = countriesObj.getCountryData();
        mv.addObject("countryCodes", countryCodes);

        // Convert TreeMap values and keys to ArrayLists for index-based access
        ArrayList<String> codes = new ArrayList<>(countryCodes.values());
        ArrayList<String> countries = new ArrayList<>(countryCodes.keySet());

        // Find the selected country and add it to the model
        int indexOfCountryCode = BinarySearch.binarySearchObject(codes, 0, codes.size(), countryCode);
        mv.addObject("country", countries.get(indexOfCountryCode));
        mv.addObject("selectedCountry", countries.get(indexOfCountryCode));

        // Fetch macroeconomic data for the selected country and indicator
        MacroIndicators macroIndicators = MacroIndicatorFetcher.instantiateIndicator(countryCode, indicatorCode);
        HashMap<Year, BigDecimal> yearDataPairs = macroIndicators.getData();

        mv.addObject("yearDataPairs", yearDataPairs);
        mv.addObject("indicatorCode", indicatorCode);

        switch (indicatorCode) {
```

```
case "NY.GDP.MKTP.CD":
    mv.addObject("selectedIndicator", "GDP");
    break;
case "NY.GDP.PCAP.CD":
    mv.addObject("selectedIndicator", "GDP Per Capita");
    break;
case "FP.CPI.TOTL.ZG":
    mv.addObject("selectedIndicator", "Inflation (CPI)");
    break;
case "SL.UEM.TOTL.ZS":
    mv.addObject("selectedIndicator", "Real Interest Rate");
    break;
case "FR.INR.RINR":
    mv.addObject("selectedIndicator", "Total Unemployment");
    break;
case "SI.POV.GINI":
    mv.addObject("selectedIndicator", "Gini Index");
    break;
case "GC.DOD.TOTL.GD.ZS":
    mv.addObject("selectedIndicator", "Total Central Government Debt");
    break;
case "BN.GSR.GNFS.CD":
    mv.addObject("selectedIndicator", "Net Trade In Goods & Services");
    break;
case "NE.TRD.GNFS.ZS":
    mv.addObject("selectedIndicator", "Trade (% of GDP)");
    break;
case "GC.TAX.TOTL.CN":
    mv.addObject("selectedIndicator", "Tax revenue (current LCU)");
    break;
case "GC.TAX.TOTL.GD.ZS":
```

```
                mv.addObject("selectedIndicator", "Tax revenue (% of GDP)");
                break;
            case "FI.RES.TOTL.CD":
                mv.addObject("selectedIndicator", "Total reserves - includes gold");
                break;
            default:
                break;
        }

        return mv;
    }
}
```

ExchangeRateController.java

```
package com.mycompany.Controllers;

import java.time.LocalDate;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.servlet.ModelAndView;
import com.mycompany.EconomicFigures.ExchangeRateAPI;

@RestController
public class ExchangeRateController {

    @GetMapping("exchange-rate-economic")
    public ModelAndView showExchangeRate() {
        ModelAndView mv = new ModelAndView("economicfigures/exchange-rate-economic");

        // Provide list of available currency codes and dates
        mv.addObject("avaliableCurrencies", ExchangeRateAPI.addCurrencyCodesPairs());
        mv.addObject("avaliableDates", ExchangeRateAPI.addDates());

        return mv;
    }

    // Fetch exchange rate between two currencies
    @PostMapping("exchange-rate-economic")
    public ModelAndView getExchangeRate(@RequestParam String baseCurrency,
                    @RequestParam String wantedCurrency,
                    @RequestParam LocalDate date) {
```

```
        ModelAndView mv = new ModelAndView("economicfigures/exchange-rate-economic");

        // Date can't be in the future (use yesterday if so)
        if (date.isAfter(LocalDate.now().minusDays(1))) {
            date = LocalDate.now().minusDays(1);
        }

        // If base and target currencies are the same; rate is 1
        if (baseCurrency.equals(wantedCurrency)) {
            mv.addObject("convertedCurrency", 1);
        }

        else {
            // Fetch conversion rate
            mv.addObject("convertedCurrency", ExchangeRateAPI.compareTwoCurrencies(baseCurrency, wantedCurrency,
date));
        }

        mv.addObject("seriesOfCurrencies", ExchangeRateAPI.seriesOfCurrencies(baseCurrency, wantedCurrency, date));

        mv.addObject("date", date);
        mv.addObject("baseCurrency", baseCurrency);
        mv.addObject("wantedCurrency", wantedCurrency);
        mv.addObject("avaliableCurrencies", ExchangeRateAPI.addCurrencyCodesPairs());
        mv.addObject("avaliableDates", ExchangeRateAPI.addDates());
        mv.addObject("selectedBaseCurrency", baseCurrency);
        mv.addObject("selectedWantedCurrency", wantedCurrency);

        return mv;
    }
}
```

EconomicNewsController.java

```java
package com.mycompany.Controllers;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.Map;
import java.util.concurrent.Future;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.servlet.ModelAndView;
import com.mycompany.Algorithms.MergeSort;
import com.mycompany.Logic.WebScraperLogic;
import com.mycompany.PythonFlaskAPI.APIQueue;
import com.mycompany.PythonFlaskAPI.PythonFlaskAPI;
import com.mycompany.Records.EconomicNewsRecord;

@RestController
public class EconomicNewsController {

    private static final APIQueue taskManager = new APIQueue(3);

    // Store scraped economic news articles
    private final ArrayList<EconomicNewsRecord> economicNews = WebScraperLogic.scrapeNews();

    @GetMapping("news")
    public ModelAndView showNews() {
        ModelAndView mv = new ModelAndView("economicnews/news");
```

```
        // Sort articles by date (latest first)
        MergeSort.mergeSortEconomicNewsByDate(economicNews);
        Collections.reverse(economicNews);

        mv.addObject("economicNews", economicNews);

        // Default selected sources is all sources
        // 0 - BBCRSSFeed
        // 1 - TheTelegraphRSSFeed
        // 2 - TheEconomistRSSFeed
        // 3 - TheIndependentRSSFeed
        // 4 - CNBCRSSFeed
        ArrayList<Integer> selectedSources = new ArrayList<>(List.of(0, 1, 2, 3, 4));
        mv.addObject("selectedSources", selectedSources

        mv.addObject("selectedSort", "date (latest)");

        return mv;
    }

    // Handle filtering and AI summarisation from the news page
    @PostMapping("news")
    public Object summariseBBCNewsArticle(
        @RequestParam String formName,
        @RequestParam(required = false) String sort,
        @RequestParam(required = false) String link,
        @RequestParam(required = false) ArrayList<Integer> newsSource) {

        // Filter and sort news articles
        if (formName.equals("filterNews")) {
```

```
            ModelAndView mv = new ModelAndView("economicnews/news");
            ArrayList<EconomicNewsRecord> reducedEconomicNews = new ArrayList<>();

            // Group news by source
            ArrayList<ArrayList<EconomicNewsRecord>> economicNewsBySource =
WebScraperLogic.sortEconomicsNewsBySource(economicNews);

            if (newsSource == null) {
                mv.addObject("economicNews", reducedEconomicNews);
                mv.addObject("selectedSources", new ArrayList<>());
                mv.addObject("selectedSort", sort);
                return mv;
            }

            // Collect news from the selected sources
            for (int i = 0; i < newsSource.size(); i++) {
                reducedEconomicNews.addAll(economicNewsBySource.get(newsSource.get(i)));
            }

            mv.addObject("selectedSources", newsSource);

            // Apply sorting option selected by user
            switch (sort) {
                case "date (earliest)":
                    MergeSort.mergeSortEconomicNewsByDate(reducedEconomicNews);
                    break;
                case "date (latest)":
                    MergeSort.mergeSortEconomicNewsByDate(reducedEconomicNews);
                    Collections.reverse(reducedEconomicNews);
                    break;
                case "title (a-z)":
```

```
                MergeSort.mergeSortEconomicNewsByTitle(reducedEconomicNews);
                break;
            case "title (z-a)":
                MergeSort.mergeSortEconomicNewsByTitle(reducedEconomicNews);
                Collections.reverse(reducedEconomicNews);
                break;
            default:
                break;
        }

        mv.addObject("selectedSort", sort);
        mv.addObject("economicNews", reducedEconomicNews);

        return mv;
    }

    // Summarise news articles using Python Flask API
    else if (formName.equals("summarise")) {
        try {
            Future<Map<String, String>> future = taskManager.submitTask(() -> {
                return Map.of("aiReply", PythonFlaskAPI.summariseNewsArticle(link));
            });

            // Wait for the AI summarisation to complete
            Map<String, String> result = future.get();
            return result;
        }

        catch (Exception ex) {

        }
```

```
        }

        return null;
    }
}
```

InvestorController.java

```
package com.mycompany.Controllers;

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.LinkedHashMap;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.servlet.ModelAndView;
import com.mycompany.Logic.InvestorLogic;
import com.mycompany.PythonFlaskAPI.PythonFlaskAPI;
import com.mycompany.Repository.SQLQueriesConsumers;
import com.mycompany.Repository.SQLQueriesProducers;
import com.mycompany.Repository.SQLQueriesStocks;
import com.mycompany.Repository.SQLQueriesTrade;
import com.mycompany.Users.Consumer;
import com.mycompany.Users.Producer
import jakarta.servlet.http.HttpSession;

@RestController
public class InvestorController {

    private static final SQLQueriesConsumers SQLQueriesConsumers = new SQLQueriesConsumers();
    private static final SQLQueriesProducers SQLQueriesProducers = new SQLQueriesProducers();

    public ModelAndView showData(ModelAndView mv, HttpSession session) {
        // Updates and inserts all stocks data to the database
        SQLQueriesStocks.insertAllStock();
```

```
mv.addObject("allTickerInfo", InvestorLogic.getAllTickerInfo());

// Time ranges for candlestick chart
LinkedHashMap<String, String> timePeriods = new LinkedHashMap<>();
timePeriods.put("3 Months Ago", LocalDate.now().minusMonths(3).toString());
timePeriods.put("6 Months Ago", LocalDate.now().minusMonths(6).toString());
timePeriods.put("1 Year Ago", LocalDate.now().minusYears(1).toString());
timePeriods.put("5 Years Ago", LocalDate.now().minusYears(5).toString());
timePeriods.put("10 Years Ago", LocalDate.now().minusYears(10).toString());
mv.addObject("timePeriods", timePeriods);

// If user is a Consumer
if (session.getAttribute("user") instanceof Consumer consumer) {
    SQLQueriesConsumers.needToUpdateStockBudget(consumer);
    mv.addObject("stockBudget", consumer.getStockBudget());

    // Owned stocks
    ArrayList<String> stockNames = new ArrayList<>();
    ArrayList<Integer> netQuantities = new ArrayList<>();
    ArrayList<Double> totalAskPrices = new ArrayList<>();
    ArrayList<Double> averageAskPrices = new ArrayList<>();
    ArrayList<Double> currentAskPrices = new ArrayList<>();
    ArrayList<Double> potentialProfit = new ArrayList<>();

    // Retrieve user's stock
    for (ArrayList<Object> row : SQLQueriesTrade.getCurrentUserStock(consumer, null)) {
        stockNames.add((String) row.get(0));
        netQuantities.add(((Number) row.get(1)).intValue());
        totalAskPrices.add(((Number) row.get(2)).doubleValue());
        averageAskPrices.add(((Number) row.get(3)).doubleValue());
```

```
                currentAskPrices.add(PythonFlaskAPI.getAskPrice(InvestorLogic.getAllTickerInfo().get((String)
row.get(0))));
                potentialProfit.add(((Number) row.get(4)).doubleValue());
            }

            mv.addObject("allStockNames", stockNames);
            mv.addObject("allNetQuantities", netQuantities);
            mv.addObject("allTotalAskPrices", totalAskPrices);
            mv.addObject("allAverageAskPrices", averageAskPrices);
            mv.addObject("allCurrentAskPrices", currentAskPrices);
            mv.addObject("allPotentialProfit", potentialProfit);

            // Update user data
            consumer = SQLQueriesConsumers.getData(consumer.getConsumerID());
            session.setAttribute("user", consumer);
        }

        // If user is a Producer
        else if (session.getAttribute("user") instanceof Producer producer) {
            SQLQueriesProducers.needToUpdateStockBudget(producer);
            mv.addObject("stockBudget", producer.getStockBudget());

            ArrayList<String> stockNames = new ArrayList<>();
            ArrayList<Integer> netQuantities = new ArrayList<>();
            ArrayList<Double> totalAskPrices = new ArrayList<>();
            ArrayList<Double> averageAskPrices = new ArrayList<>();
            ArrayList<Double> currentAskPrices = new ArrayList<>();
            ArrayList<Double> potentialProfit = new ArrayList<>();

            // Retrieve user's stock
            for (ArrayList<Object> row : SQLQueriesTrade.getCurrentUserStock(null, producer)) {
```

```
                stockNames.add((String) row.get(0));
                netQuantities.add(((Number) row.get(1)).intValue());
                totalAskPrices.add(((Number) row.get(2)).doubleValue());
                averageAskPrices.add(((Number) row.get(3)).doubleValue());
                currentAskPrices.add(PythonFlaskAPI.getAskPrice(InvestorLogic.getAllTickerInfo().get((String)
row.get(0))));
                potentialProfit.add(((Number) row.get(4)).doubleValue());
            }

            mv.addObject("allStockNames", stockNames);
            mv.addObject("allNetQuantities", netQuantities);
            mv.addObject("allTotalAskPrices", totalAskPrices);
            mv.addObject("allAverageAskPrices", averageAskPrices);
            mv.addObject("allCurrentAskPrices", currentAskPrices);
            mv.addObject("allPotentialProfit", potentialProfit);

            // Update user data
            producer = SQLQueriesProducers.getData(producer.getProducerID());
            session.setAttribute("user", producer);
        }

        return mv;
    }

    @GetMapping("investor")
    public ModelAndView showInvestorCentre(HttpSession session) {
        ModelAndView mv = new ModelAndView("investor/investor");

        // Only logged-in Consumers or Producers can access
        if (session.getAttribute("user") instanceof Consumer consumer || session.getAttribute("user") instanceof
Producer producer) {
```

```
            showData(mv, session);
        }

        else {
            mv.setViewName("login/start");
        }

        return mv;
    }

    @PostMapping("investor")
    public ModelAndView postInvestorCentre(@RequestParam String formName,
                        @RequestParam(required = false) String tickerSymbol,
                        @RequestParam(required = false) String timeSpan,
                        @RequestParam(required = false) Integer quantityBuy,
                        @RequestParam(required = false) Integer quantitySell,
                        HttpSession session) {

        ModelAndView mv = new ModelAndView("investor/investor");

        // Logic for Consumers and Producers
        if (session.getAttribute("user") instanceof Consumer consumer || session.getAttribute("user") instanceof
Producer producer) {

            // If user selects a company, load chart and load details
            if (formName.equals("selectCompany")) {
                mv.addObject("companyInformation", InvestorLogic.getCompanyInfo(tickerSymbol));
                mv.addObject("stockPriceComponents", InvestorLogic.getStockPriceDataComponents(tickerSymbol,
timeSpan));
                mv.addObject("companyName", InvestorLogic.getAllTickerInfoReversed().get(tickerSymbol));
                mv.addObject("askPrice", PythonFlaskAPI.getAskPrice(tickerSymbol));
```

```
                session.setAttribute("currentTickerSymbol", tickerSymbol);

                // SMA Information
                LinkedHashMap<String, Object> SMA10Day = PythonFlaskAPI.calculateSMA(tickerSymbol, 10);
                LinkedHashMap<String, Object> SMA50Day = PythonFlaskAPI.calculateSMA(tickerSymbol, 50);
                LinkedHashMap<String, Object> SMA100Day = PythonFlaskAPI.calculateSMA(tickerSymbol, 100);

                mv.addObject("SMA10DayTrend", SMA10Day.get("trend"));
                mv.addObject("SMA10DayGradient", SMA10Day.get("gradient"));
                mv.addObject("SMA50DayTrend", SMA50Day.get("trend"));
                mv.addObject("SMA50DayGradient", SMA50Day.get("gradient"));
                mv.addObject("SMA100DayTrend", SMA100Day.get("trend"));
                mv.addObject("SMA100DayGradient", SMA100Day.get("gradient"));

                mv.addObject("graphContainerDiv", "graphContainerDiv");
                mv.addObject("companyDetailsDiv", "companyDetailsDiv");
                mv.addObject("purchaseStockDiv", "purchaseStockDiv");
                mv.addObject("sellStockDiv", "sellStockDiv");
            }
        }

        // Trading logic for Consumers
        if (session.getAttribute("user") instanceof Consumer consumer) {
            String currentTickerSymbol = (String) session.getAttribute("currentTickerSymbol");

            if (formName.equals("purchaseStock")) {
                // Deduct cost from stock budget and insert buy trade. Updates the Database
                SQLQueriesConsumers.updateStockBudget(consumer, consumer.getStockBudget() - (quantityBuy *
PythonFlaskAPI.getAskPrice(currentTickerSymbol)));
                SQLQueriesTrade.insertTrade(consumer, null,
SQLQueriesStocks.getStockID(InvestorLogic.getAllTickerInfoReversed().get(currentTickerSymbol)), "BUY",
```

```
PythonFlaskAPI.getAskPrice(currentTickerSymbol), quantityBuy);
                mv.setViewName("redirect:investor");
            }
            else if (formName.equals("sellStock")) {
                // Add profit to stock budget and insert sell trade. Updates the Database
                SQLQueriesConsumers.updateStockBudget(consumer, consumer.getStockBudget() + (quantitySell *
PythonFlaskAPI.getAskPrice(currentTickerSymbol)));
                SQLQueriesTrade.insertTrade(consumer, null,
SQLQueriesStocks.getStockID(InvestorLogic.getAllTickerInfoReversed().get(currentTickerSymbol)), "SELL",
PythonFlaskAPI.getAskPrice(currentTickerSymbol), quantitySell);
                mv.setViewName("redirect:investor");
            }

            consumer = SQLQueriesConsumers.getData(consumer.getConsumerID());
            session.setAttribute("user", consumer);
        }

        // Trading logic for Producers
        else if (session.getAttribute("user") instanceof Producer producer) {
            String currentTickerSymbol = (String) session.getAttribute("currentTickerSymbol");

            if (formName.equals("purchaseStock")) {
                // Deduct cost from stock budget and insert buy trade. Updates the Database
                SQLQueriesProducers.updateStockBudget(producer, producer.getStockBudget() - (quantityBuy *
PythonFlaskAPI.getAskPrice(currentTickerSymbol)));
                SQLQueriesTrade.insertTrade(null, producer,
SQLQueriesStocks.getStockID(InvestorLogic.getAllTickerInfoReversed().get(currentTickerSymbol)), "BUY",
PythonFlaskAPI.getAskPrice(currentTickerSymbol), quantityBuy);
                mv.setViewName("redirect:investor");
            }
```

```
            else if (formName.equals("sellStock")) {
                // Add profit to stock budget and insert sell trade. Updates the Database
                SQLQueriesProducers.updateStockBudget(producer, producer.getStockBudget() + (quantitySell *
PythonFlaskAPI.getAskPrice(currentTickerSymbol)));
                SQLQueriesTrade.insertTrade(null, producer,
SQLQueriesStocks.getStockID(InvestorLogic.getAllTickerInfoReversed().get(currentTickerSymbol)), "SELL",
PythonFlaskAPI.getAskPrice(currentTickerSymbol), quantitySell);
                mv.setViewName("redirect:investor");
            }

            producer = SQLQueriesProducers.getData(producer.getProducerID());
            session.setAttribute("user", producer);
        }

        mv.addObject("selectedTickerSymbol", tickerSymbol);
        mv.addObject("selectedTimeSpan", timeSpan);
        showData(mv, session);

        return mv;
    }
}
```

LoginController.java

```java
package com.mycompany.Controllers;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.servlet.ModelAndView;
import org.springframework.web.servlet.mvc.support.RedirectAttributes;
import com.mycompany.Algorithms.VernamCipher;
import com.mycompany.Repository.SQLQueriesConsumers;
import com.mycompany.Repository.SQLQueriesProducers;
import com.mycompany.Users.Consumer;
import com.mycompany.Users.Producer;
import jakarta.servlet.http.HttpSession;

@RestController
public class LoginController {

    private static final SQLQueriesConsumers SQLQueriesConsumers = new SQLQueriesConsumers();
    private static final SQLQueriesProducers SQLQueriesProducers = new SQLQueriesProducers();

    @GetMapping("/")
    public ModelAndView redirectToStartScreen() {
        ModelAndView mv = new ModelAndView("redirect:start");
        return mv;
    }

    // Show start screen
```

```java
@GetMapping("start")
public ModelAndView showStartScreen(HttpSession session) {
    //invalidate any existing session
    session.invalidate();
    ModelAndView mv = new ModelAndView("login/start");
    return mv;
}

// Show signup form
@GetMapping("signup")
public ModelAndView showSignupScreen(HttpSession session) {
    ModelAndView mv = new ModelAndView("login/signup");
    return mv;
}

// Handle signup form and store user data to database
@PostMapping("signup")
public ModelAndView storeSignupData(@RequestParam String firstName,
                @RequestParam String lastName,
                @RequestParam Integer age,
                @RequestParam String dateOfBirth,
                @RequestParam String email,
                @RequestParam String phoneNumber,
                @RequestParam String password,
                @RequestParam String typeOfEconomicAgent) {

    ModelAndView mv = new ModelAndView("login/login");

    // Check if password is strong
    char[] passwordAsCharArray = password.toCharArray();
    boolean length = false;
```

```
boolean uppercase = false;
boolean lowercase = false;
boolean digit = false;
boolean special = false;

if (password.length() > 8) {
    length = true;
}

for (char ch : passwordAsCharArray) {
    if (Character.isUpperCase(ch)) {
        uppercase = true;
    }

    else if (Character.isLowerCase(ch)) {
        lowercase = true;
    }

    else if (Character.isDigit(ch)) {
        digit = true;
    }

    else {
        special = true;
    }
}

if ((length && uppercase && lowercase && digit && special) == false) {
    mv.setViewName("login/signup");
    mv.addObject("windowsAlert", true);
    mv.addObject("windowsAlertMessage", "Password is not strong. Password must contain at least:\n Uppercase
```

```
Character\n Lowercase Character\n Digit\n Special Character\n More Than 8 Characters");
            return mv;
        }

        // Check if email already exists for either Consumers or Producers
        if (SQLQueriesConsumers.doesEmailExist(email) > 0 || SQLQueriesProducers.doesEmailExist(email) > 0){
            mv.setViewName("login/signup");
            mv.addObject("windowsAlert", true);
            mv.addObject("windowsAlertMessage", "Email Already Exists");
            return mv;
        }

        // Encrypt password using Vernam cipher
        String vernamKey = VernamCipher.createVernamKey(email, password);
        String passwordInCipherText = VernamCipher.XORfunction(VernamCipher.convertPlainTextToBinary(password),
vernamKey);

        // Store user depending on type of user (Consumer or Producer)
        if (typeOfEconomicAgent.equals("Consumer")) {
            Consumer consumer = new Consumer(firstName, lastName, age, dateOfBirth, email, phoneNumber,
passwordInCipherText);
            SQLQueriesConsumers.insertUserDataIntoTable(consumer);
        }

        else if (typeOfEconomicAgent.equals("Producer")) {
            Producer producer = new Producer(firstName, lastName, age, dateOfBirth, email, phoneNumber,
passwordInCipherText);
            SQLQueriesProducers.insertUserDataIntoTable(producer);
        }

        return mv;
```

```
}

// Show login screen
@GetMapping("/login")
public ModelAndView showLoginScreen(HttpSession session) {
    ModelAndView mv = new ModelAndView("login/login");
    return mv;
}

// Handle login form and authenticate user, by checking database
@PostMapping("/login")
public ModelAndView authenticateLogin(@RequestParam String email,
                @RequestParam String password,
                HttpSession session,
                RedirectAttributes redirectAttributes) throws Exception {
    ModelAndView mv = new ModelAndView("redirect:user-dashboard");

    String vernamKey = VernamCipher.loadVernamKey(email);

    // If email not found, return error
    if (vernamKey == null) {
        mv.setViewName("login/login");
        mv.addObject("windowsAlert", true);
        mv.addObject("windowsAlertMessage", "Invalid Email");
        return mv;
    }

    // Encrypt entered password for comparison with stored one
    String passwordAsPlainText = password;
    password = VernamCipher.XORfunction(VernamCipher.convertPlainTextToBinary(password), vernamKey);
```

```
        // Try to authenticate as Consumer or Producer
        int consumerID = SQLQueriesConsumers.authenticateLogin(email, password);
        int producerID = SQLQueriesProducers.authenticateLogin(email, password);

        if (consumerID > 0) {
            Consumer consumer = SQLQueriesConsumers.getData(consumerID);
            session.setAttribute("user", consumer);

            // Change Vernam key in JSON and update stored password in database
            String newVernamKey = VernamCipher.createVernamKey(email, passwordAsPlainText);
            String newPasswordInCipherText =
VernamCipher.XORfunction(VernamCipher.convertPlainTextToBinary(passwordAsPlainText), newVernamKey);
            SQLQueriesConsumers.updatePassword(newPasswordInCipherText, consumerID);
        }

        else if (producerID > 0) {
            Producer producer = SQLQueriesProducers.getData(producerID);
            session.setAttribute("user", producer);

            // Change Vernam key in JSON and update stored password in database
            String newVernamKey = VernamCipher.createVernamKey(email, passwordAsPlainText);
            String newPasswordInCipherText =
VernamCipher.XORfunction(VernamCipher.convertPlainTextToBinary(passwordAsPlainText), newVernamKey);
            SQLQueriesProducers.updatePassword(newPasswordInCipherText, producerID);
        }

        // Invalid password
        else {
            mv.setViewName("redirect:login");
            redirectAttributes.addFlashAttribute("windowsAlert", true);
            redirectAttributes.addFlashAttribute("windowsAlertMessage", "Invalid Password");
```

```
        }

        return mv;
    }
}
```

RateTheEconomyController.java

```
package com.mycompany.Controllers;

import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.util.ArrayList;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.servlet.ModelAndView;
import com.mycompany.Logic.RateTheEconomyLogic;
import com.mycompany.Repository.SQLQueriesRateTheEconomy;
import com.mycompany.Users.Consumer;
import com.mycompany.Users.Producer;
import jakarta.servlet.http.HttpSession;

@RestController
public class RateTheEconomyController {

    public static void loadData(ModelAndView mv, Consumer consumer) {
        // Load all questions and their multiple answers
        mv.addObject("allQuestionsAndAnswerPairs", RateTheEconomyLogic.loadJSONToArrayList());

        // Load available dates for which user opinions exist
        mv.addObject("avaliableDates", RateTheEconomyLogic.loadUserOpinionDates());

        // Load latest consumer opinion and the date it was submitted
        mv.addObject("consumerOpinionDate",
SQLQueriesRateTheEconomy.getLatestConsumerOpinion(consumer).get("consumerOpinionDate"));
```

```
        mv.addObject("consumerOpinion",
SQLQueriesRateTheEconomy.getLatestConsumerOpinion(consumer).get("consumerOpinion"));
    }

    @GetMapping("rate-the-economy")
    public ModelAndView showQuestions(HttpSession session) {
        ModelAndView mv = new ModelAndView();

        // If logged-in user is a Consumer
        if (session.getAttribute("user") instanceof Consumer consumer) {
            mv.setViewName("ratetheeconomy/rate-the-economy");
            loadData(mv, consumer);
        }

        // If logged-in user is a Producer
        else if (session.getAttribute("user") instanceof Producer producer) {
            mv.setViewName("userdashboard/user-dashboard");
            mv.addObject("windowsAlert", true);
            mv.addObject("windowsAlertMessage", "Unable To Access. Need To Be A Consumer.");
        }

        // If no user is logged in
        else {
            mv.setViewName("login/start");
        }

        return mv;
    }

    @PostMapping("rate-the-economy")
    public ModelAndView storeUserInput(HttpSession session,
```

```
                @RequestParam String formName,
                @RequestParam (required = false) String answer1, @RequestParam (required = false) String answer2,
                @RequestParam (required = false) String answer3, @RequestParam (required = false) String answer4,
                @RequestParam (required = false) String answer5, @RequestParam (required = false) String answer6,
                @RequestParam (required = false) String answer7, @RequestParam (required = false) String answer8,
                @RequestParam (required = false) String answer9, @RequestParam (required = false) String answer10,
                @RequestParam (required = false) String answer11, @RequestParam (required = false) String
selectedDate) {

        ModelAndView mv = new ModelAndView("ratetheeconomy/rate-the-economy");

        // Only Consumers can submit
        if (session.getAttribute("user") instanceof Consumer consumer) {
            loadData(mv, consumer);

            // User selects a previous date to view opinions
            if (formName.equals("chooseDateForm")) {
                mv.addObject("allUserOpinions", RateTheEconomyLogic.getAllUserOpinions(selectedDate));

                DateTimeFormatter df1 = DateTimeFormatter.ofPattern("dd-MM-yyyy"); // Input date format
                DateTimeFormatter df2 = DateTimeFormatter.ofPattern("yyyy-MM-dd"); // Format for database queries

                // Calculate average opinion for the selected date
                mv.addObject("averageConsumerOpinion",
SQLQueriesRateTheEconomy.averageConsumerOpinion(LocalDate.parse(selectedDate, df1).format(df2)));
            }

            else if (formName.equals("rateTheEconomyForm")) {
                ArrayList<String> answers = new ArrayList<>();
                answers.add(answer1);
                answers.add(answer2);
```

```
                answers.add(answer3);
                answers.add(answer4);
                answers.add(answer5);
                answers.add(answer6);
                answers.add(answer7);
                answers.add(answer8);
                answers.add(answer9);
                answers.add(answer10);
                answers.add(answer11);

                ArrayList<ArrayList<Object>> userOpinion = RateTheEconomyLogic.storeUserOpinion(answers);
                RateTheEconomyLogic.storeUserOpinionToAppropiateLocations(userOpinion, consumer);

                mv.setViewName("redirect:rate-the-economy");
            }
        }

        return mv;
    }
}
```

UserDashBoardController.java

```java
package com.mycompany.Controllers;

import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.Map;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.servlet.ModelAndView;
import com.mycompany.Algorithms.BinarySearch;
import com.mycompany.Algorithms.Glossary;
import com.mycompany.PythonFlaskAPI.PythonFlaskAPI;
import com.mycompany.Users.Consumer;
import com.mycompany.Users.Producer;
import jakarta.servlet.http.HttpSession;

@RestController
public class UserDashboardController {

    // Store glossary data - terms and definitions
    private final LinkedHashMap<String, String> wordsDefinitionsPairs = Glossary.storeTermsToLinkedHashMap();
    private final ArrayList<String> wordsArrayList = Glossary.storeWordsToArrayList();
    private final ArrayList<String> plainWordsArrayList = Glossary.storePlainWordsToArrayList();
    private final ArrayList<String> definitionsArrayList = Glossary.storeDefinitionsToArrayList();

    public ModelAndView showData(ModelAndView mv) {
        mv.addObject("wordsDefinitionsPairs", wordsDefinitionsPairs);
        return mv;
```

```
    }

    @GetMapping("/user-dashboard")
    public ModelAndView showUserDashboard(HttpSession session) {
        ModelAndView mv = new ModelAndView("userdashboard/user-dashboard");

        // If no valid user in session
        if (!(session.getAttribute("user") instanceof Consumer consumer || session.getAttribute("user") instanceof
Producer producer)) {
            mv.setViewName("login/start");
            session.invalidate();
        }

        showData(mv);
        return mv;
    }

    @PostMapping("/user-dashboard")
    public Object postUserDashboard(@RequestParam String formName,
        @RequestParam(required = false) String glossarySearchInput,
        @RequestParam(required = false) String userPrompt) {

        if (formName.equals("glossarySearch")) {
            ModelAndView mv = new ModelAndView("userdashboard/user-dashboard");

            // Normalise search input (remove spaces, lowercase)
            glossarySearchInput = glossarySearchInput.replace(" ", "").toLowerCase();

            // Perform binary search on glossary word list
            int index = BinarySearch.binarySearchObject(plainWordsArrayList, 0, plainWordsArrayList.size(),
glossarySearchInput);
```

```
            LinkedHashMap<String, String> wordsDefinitionsPair = new LinkedHashMap<>();

            // If word not found, show error message
            if (index == -1) {
                wordsDefinitionsPair.put("Word Not Found", "");
            }

            // If found, show word with definition
            else {
                wordsDefinitionsPair.put(wordsArrayList.get(index), definitionsArrayList.get(index));
            }

            mv.addObject("wordsDefinitionsPairs", wordsDefinitionsPair);

            return mv;
        }

        // User asked AI a question
        else if (formName.equals("askAIBot")) {
            return Map.of("aiReply", PythonFlaskAPI.aiAnswer(userPrompt));
        }

        return null;
    }
}
```

EconomicFigures Package

WorldBankAPI.java

```java
package com.mycompany.EconomicFigures;

import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;

// Abstract class for World Bank API requests.
public abstract class WorldBankAPI {
    protected String url = "https://api.worldbank.org/v2/";
    protected final HttpClient client = HttpClient.newHttpClient();

    // Subclasses override this with more specific endpoints
    protected HttpRequest request = HttpRequest.newBuilder().uri(URI.create(url)).GET().build();
}
```

MacroIndicators.java

```
package com.mycompany.EconomicFigures;

import java.math.BigDecimal;
import java.net.URI;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.net.http.HttpResponse.BodyHandlers;
import java.text.DecimalFormat;
import java.time.Year;
import java.util.HashMap;
import com.fasterxml.jackson.databind.JsonNode;
import com.fasterxml.jackson.databind.ObjectMapper;

public abstract class MacroIndicators extends WorldBankAPI {
    protected Year year;
    protected BigDecimal value;
    protected String indicatorCode;
    protected String countryCode;

    // Sets up the HTTP request
    public MacroIndicators(String countryCode, String indicatorCode) {
        this.countryCode = countryCode;
        this.indicatorCode = indicatorCode;

        url = "https://api.worldbank.org/v2/countries/" + this.countryCode + "/indicators/" + this.indicatorCode +
"?format=json&per_page=1000";
        request = HttpRequest.newBuilder().uri(URI.create(url)).GET().build();
    }
```

```
// Each subclass defines how to format value for display
protected abstract DecimalFormat configureFormat();

// Fetches data from the World Bank API
public HashMap<Year, BigDecimal> getData() {
    HashMap<Year, BigDecimal> yearDataPairs = new HashMap<>();

    try {
        HttpResponse<String> response = client.send(request, BodyHandlers.ofString());
        ObjectMapper mapper = new ObjectMapper();
        JsonNode root = mapper.readTree(response.body());
        JsonNode dataArray = root.get(1);
        DecimalFormat df = configureFormat();

        if (dataArray != null && dataArray.isArray()) {
            for (JsonNode dataNode : dataArray) {
                boolean skip = false;
                year = Year.of(dataNode.get("date").asInt());

                if (dataNode.get("value").isNull()) {
                    skip = true;
                }

                if (!skip) {
                    value = new BigDecimal(df.format(new BigDecimal(dataNode.get("value").asText())));
                    yearDataPairs.put(year, value);
                }
            }
        }
    }
```

```
        catch (Exception ex) {
            System.out.println("Error fetching macro indicator data");
        }

        return yearDataPairs;
    }
}
```

MacroIndicatorFetcher.java

```java
package com.mycompany.EconomicFigures;

public class MacroIndicatorFetcher {

    // Method that creates and returns a specific MacroIndicators fetcher
    public static MacroIndicators instantiateIndicator(String countryCode, String indicatorCode) {
        switch (indicatorCode) {
            // GDP (current US$)
            case "NY.GDP.MKTP.CD":
                return new GDPFetcher(countryCode, indicatorCode);

            // GDP per capita (current US$)
            case "NY.GDP.PCAP.CD":
                return new GDPPerCapitaFetcher(countryCode, indicatorCode);

            // Inflation, consumer prices (annual %)
            case "FP.CPI.TOTL.ZG":
                return new InflationFetcher(countryCode, indicatorCode);

            // Unemployment (% of total labour force)
            case "SL.UEM.TOTL.ZS":
                return new UnemploymentFetcher(countryCode, indicatorCode);

            // Real interest rate (%)
            case "FR.INR.RINR":
                return new InterestRateFetcher(countryCode, indicatorCode);

            // Gini index (income inequality)
            case "SI.POV.GINI":
```

```
                return new GiniIndexFetcher(countryCode, indicatorCode);

            // Government debt (% of GDP)
            case "GC.DOD.TOTL.GD.ZS":
                return new GovernmentDebtFetcher(countryCode, indicatorCode);

            // Net trade in goods and services (current US$)
            case "BN.GSR.GNFS.CD":
                return new NetTradeFetcher(countryCode, indicatorCode);

            // Trade (% of GDP)
            case "NE.TRD.GNFS.ZS":
                return new TradeFetcher(countryCode, indicatorCode);

            // Tax revenue (current LCU)
            case "GC.TAX.TOTL.CN":
                return new TaxRevenueLCUFetcher(countryCode, indicatorCode);

            // Tax revenue (% of GDP)
            case "GC.TAX.TOTL.GD.ZS":
                return new TaxRevenueGDPFetcher(countryCode, indicatorCode);

            // Total reserves (current US$)
            case "FI.RES.TOTL.CD":
                return new TotalReservesFetcher(countryCode, indicatorCode);

            default:
                return null;
        }
    }
}
```

Countries.java

```java
package com.mycompany.EconomicFigures;

import java.net.URI;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.net.http.HttpResponse.BodyHandlers;
import java.util.LinkedHashMap;
import com.fasterxml.jackson.databind.JsonNode;
import com.fasterxml.jackson.databind.ObjectMapper;

public class Countries extends WorldBankAPI {

    public Countries () {
        url = "https://api.worldbank.org/v2/country/?format=json&per_page=300";
        request = HttpRequest.newBuilder().uri(URI.create(url)).GET().build();
    }

    // Fetches country data from the World Bank API
    public LinkedHashMap<String, String> getCountryData() {
        LinkedHashMap<String, String> countryCodes = new LinkedHashMap<>();

        try {
            HttpResponse<String> response = client.send(request, BodyHandlers.ofString());
            ObjectMapper mapper = new ObjectMapper();
            JsonNode root = mapper.readTree(response.body());
            JsonNode dataArray = root.get(1);

            if (dataArray != null && dataArray.isArray()) {
                for (JsonNode dataNode : dataArray) {
```

```
                    // Extract country name and country ID and store them in TreeMap
                    countryCodes.put(dataNode.get("name").asText(), dataNode.get("id").asText());
                }
            }
        }

        catch (Exception ex) {
            System.out.println("Error");
        }

        return countryCodes;
    }
}
```

GDPFetcher.java

```java
package com.mycompany.EconomicFigures;

import java.text.DecimalFormat;

// Configures how GDP values should be formatted when displayed
public class GDPFetcher extends MacroIndicators {

    public GDPFetcher(String countryCode, String indicatorCode) {
        super(countryCode, indicatorCode);
    }

    @Override
    protected DecimalFormat configureFormat() {
        DecimalFormat df = new DecimalFormat();
        df.setGroupingUsed(false); // Disable commas or spaces in numbers
        return df;
    }
}
```

GDPPerCapitaFetcher.java

```java
package com.mycompany.EconomicFigures;

import java.text.DecimalFormat;

// Configures how GDP per capita values should be formatted when displayed
public class GDPPerCapitaFetcher extends MacroIndicators {

    public GDPPerCapitaFetcher(String countryCode, String indicatorCode) {
        super(countryCode, indicatorCode);
    }

    @Override
    protected DecimalFormat configureFormat() {
        DecimalFormat df = new DecimalFormat();
        df.applyPattern(".00"); // Format numbers with two decimal places
        return df;
    }
}
```

GiniIndexFetcher.java

```java
package com.mycompany.EconomicFigures;

import java.text.DecimalFormat;

// Configures how Gini Index values should be formatted when displayed
public class GiniIndexFetcher extends MacroIndicators {

    public GiniIndexFetcher(String countryCode, String indicatorCode) {
        super(countryCode, indicatorCode);
    }

    @Override
    protected DecimalFormat configureFormat() {
        DecimalFormat df = new DecimalFormat();
        df.applyPattern("##.#0"); // Format numbers with one decimal place (e.g., 35.70)
        return df;
    }
}
```

GovernmentDebtFetcher.java

```java
package com.mycompany.EconomicFigures;

import java.text.DecimalFormat;

// Configures how government debt values should be formatted when displayed
public class GovernmentDebtFetcher extends MacroIndicators {

    public GovernmentDebtFetcher(String countryCode, String indicatorCode) {
        super(countryCode, indicatorCode);
    }

    @Override
    protected DecimalFormat configureFormat() {
        DecimalFormat df = new DecimalFormat();
        df.applyPattern(".#0"); // Format numbers with two decimal place
        return df;
    }
}
```

InflationFetcher.java

```java
package com.mycompany.EconomicFigures;

import java.text.DecimalFormat;

// Configures how inflation values should be formatted when displayed
public class InflationFetcher extends MacroIndicators {

    public InflationFetcher(String countryCode, String indicatorCode) {
        super(countryCode, indicatorCode);
    }

    @Override
    protected DecimalFormat configureFormat() {
        DecimalFormat df = new DecimalFormat();
        df.applyPattern(".0"); // Format numbers with one decimal place
        return df;
    }
}
```

InterestRateFetcher.java

```java
package com.mycompany.EconomicFigures;

import java.text.DecimalFormat;

// Configures how interest rate values should be formatted when displayed
public class InterestRateFetcher extends MacroIndicators {

    public InterestRateFetcher(String countryCode, String indicatorCode) {
        super(countryCode, indicatorCode);
    }

    @Override
    protected DecimalFormat configureFormat() {
        DecimalFormat df = new DecimalFormat();
        df.applyPattern("##.##"); // Format numbers with up to two decimal places
        return df;
    }
}
```

NetTraderFetcher.java

```java
package com.mycompany.EconomicFigures;

import java.text.DecimalFormat;

// Configures how Net Trade values should be formatted when displayed
public class NetTradeFetcher extends MacroIndicators {

    public NetTradeFetcher(String countryCode, String indicatorCode) {
        super(countryCode, indicatorCode);
    }

    @Override
    protected DecimalFormat configureFormat() {
        DecimalFormat df = new DecimalFormat();
        df.applyPattern(".#"); // Format with one decimal place
        return df;
    }
}
```

TaxRevenueGDPFetcher.java

```java
package com.mycompany.EconomicFigures;

import java.text.DecimalFormat;

// Configures how Tax Revenue (as a % of GDP) values should be formatted when displayed
public class TaxRevenueGDPFetcher extends MacroIndicators {

    public TaxRevenueGDPFetcher(String countryCode, String indicatorCode) {
        super(countryCode, indicatorCode);
    }

    @Override
    protected DecimalFormat configureFormat() {
        DecimalFormat df = new DecimalFormat();
        df.applyPattern(".00"); // Format with two decimal place
        return df;
    }
}
```

TaxRevenueLCUFetcher.java

```java
package com.mycompany.EconomicFigures;

import java.text.DecimalFormat;

// Configures how Tax Revenue values (in local currency) should be formatted when displayed
public class TaxRevenueLCUFetcher extends MacroIndicators {

    public TaxRevenueLCUFetcher(String countryCode, String indicatorCode) {
        super(countryCode, indicatorCode);
    }

    @Override
    protected DecimalFormat configureFormat() {
        DecimalFormat df = new DecimalFormat();
        df.setGroupingUsed(false); // Disable commas or spaces in numbers
        return df;
    }
}
```

TotalReservesFetcher.java

```java
package com.mycompany.EconomicFigures;

import java.text.DecimalFormat;

// Configures how Total Reserves values should be formatted when displayed
public class TotalReservesFetcher extends MacroIndicators {

    public TotalReservesFetcher(String countryCode, String indicatorCode) {
        super(countryCode, indicatorCode);
    }

    @Override
    protected DecimalFormat configureFormat() {
        DecimalFormat df = new DecimalFormat();
        df.applyPattern(".00"); // Format numbers with two decimal places
        return df;
    }
}
```

TradeFetcher.java

```java
package com.mycompany.EconomicFigures;

import java.text.DecimalFormat;

// Configures how Trade values should be formatted when displayed
public class TradeFetcher extends MacroIndicators {

    public TradeFetcher(String countryCode, String indicatorCode) {
        super(countryCode, indicatorCode);
    }

    @Override
    protected DecimalFormat configureFormat() {
        DecimalFormat df = new DecimalFormat();
        df.applyPattern(".#"); // Format numbers with two decimal place
        return df;
    }
}
```

UnemploymentFetcher.java

```java
package com.mycompany.EconomicFigures;

import java.text.DecimalFormat;

// Configures how Unemployment values should be formatted when displayed
public class UnemploymentFetcher extends MacroIndicators {

    public UnemploymentFetcher(String countryCode, String indicatorCode) {
        super(countryCode, indicatorCode);
    }

    @Override
    protected DecimalFormat configureFormat() {
        DecimalFormat df = new DecimalFormat();
        df.applyPattern("0.#"); // Format numbers with one decimal place
        return df;
    }
}
```

ExchangeRateAPI

```
package com.mycompany.EconomicFigures;

import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.net.http.HttpResponse.BodyHandlers;
import java.time.LocalDate;
import java.util.Iterator;
import java.util.LinkedHashMap;
import java.util.TreeMap;
import com.fasterxml.jackson.databind.JsonNode;
import com.fasterxml.jackson.databind.ObjectMapper;

public class ExchangeRateAPI {
    public static TreeMap<String, String> addCurrencyCodesPairs () {
        TreeMap<String, String> currencyCodesPairs = new TreeMap<>();
        currencyCodesPairs.put("Australian Dollar", "AUD");
        currencyCodesPairs.put("Bulgarian Lev", "BGN");
        currencyCodesPairs.put("Brazilian Real", "BRL");
        currencyCodesPairs.put("Canadian Dollar", "CAD");
        currencyCodesPairs.put("Swiss Franc", "CHF");
        currencyCodesPairs.put("Chinese Renminbi Yuan", "CNY");
        currencyCodesPairs.put("Czech Koruna", "CZK");
        currencyCodesPairs.put("Danish Krone", "DKK");
        currencyCodesPairs.put("Euro", "EUR");
        currencyCodesPairs.put("British Pound", "GBP");
        currencyCodesPairs.put("Hong Kong Dollar", "HKD");
        currencyCodesPairs.put("Hungarian Forint", "HUF");
```

```
        currencyCodesPairs.put("Indonesian Rupiah", "IDR");
        currencyCodesPairs.put("Israeli New Sheqel", "ILS");
        currencyCodesPairs.put("Indian Rupee", "INR");
        currencyCodesPairs.put("Japanese Yen", "JPY");
        currencyCodesPairs.put("South Korean Won", "KRW");
        currencyCodesPairs.put("Mexican Peso", "MXN");
        currencyCodesPairs.put("Malaysian Ringgit", "MYR");
        currencyCodesPairs.put("Norwegian Krone", "NOK");
        currencyCodesPairs.put("New Zealand Dollar", "NZD");
        currencyCodesPairs.put("Philippine Peso", "PHP");
        currencyCodesPairs.put("Polish Złoty", "PLN");
        currencyCodesPairs.put("Romanian Leu", "RON");
        currencyCodesPairs.put("Swedish Krona", "SEK");
        currencyCodesPairs.put("Singapore Dollar", "SGD");
        currencyCodesPairs.put("Thai Baht", "THB");
        currencyCodesPairs.put("Turkish Lira", "TRY");
        currencyCodesPairs.put("United States Dollar", "USD");
        currencyCodesPairs.put("South African Rand", "ZAR");
        return currencyCodesPairs;
    }

    public static LinkedHashMap<String, LocalDate> addDates() {
        LinkedHashMap<String, LocalDate> dates = new LinkedHashMap<>();
        LocalDate desiredDate = LocalDate.now().minusDays(1);
        dates.put("Now", desiredDate);
        dates.put("A Day Ago", desiredDate.minusDays(1));
        dates.put("A Week Ago", desiredDate.minusWeeks(1));
        dates.put("A Month Ago", desiredDate.minusMonths(1));
        dates.put("6 Months Ago", desiredDate.minusMonths(6));
        dates.put("1 Year Ago", desiredDate.minusYears(1));
        dates.put("5 Years Ago", desiredDate.minusYears(5));
```

```
        dates.put("10 Years Ago", desiredDate.minusYears(10));
        return dates;
    }

    // Fetches the exchange rate for a single day between two currencies.
    public static String compareTwoCurrencies(String baseCurrency, String wantedCurrency, LocalDate date) {
        String convertedCurrency = null;

        StringBuilder url = new StringBuilder("https://api.frankfurter.app/");
        HttpClient client = HttpClient.newHttpClient;
        url.append(date);
        url.append("?base=").append(baseCurrency);
        url.append("&symbols=").append(wantedCurrency);
        HttpRequest request = HttpRequest.newBuilder().uri(URI.create(url.toString())).GET().build();
        try {
            HttpResponse<String> response = client.send(request, BodyHandlers.ofString());
            ObjectMapper mapper = new ObjectMapper();
            JsonNode root = mapper.readTree(response.body());
            convertedCurrency = root.get("rates").get(wantedCurrency).asText();
        }

        catch (Exception ex) {
            System.out.println("Error");
        }

        return convertedCurrency;
    }

    // Fetches a series of exchange rates between two currencies across multiple dates.
    public static LinkedHashMap<String, Double> seriesOfCurrencies(String baseCurrency, String wantedCurrency,
LocalDate date) {
```

```
        StringBuilder url = new StringBuilder("https://api.frankfurter.app/");
        HttpClient client = HttpClient.newHttpClient();
        LinkedHashMap<String, Double> dataSeries = new LinkedHashMap<>();
        url.append(date);
        url.append("..");
        url.append("?base=").append(baseCurrency);
        url.append("&symbols=").append(wantedCurrency);
        System.out.println(url.toString());

        HttpRequest request = HttpRequest.newBuilder().uri(URI.create(url.toString())).GET().build();
        try {
            HttpResponse<String> response = client.send(request, BodyHandlers.ofString());
            ObjectMapper mapper = new ObjectMapper();
            JsonNode root = mapper.readTree(response.body());
            JsonNode ratesNode = root.get("rates");

            Iterator<String> dates = ratesNode.fieldNames();
            while (dates.hasNext()) {
                String nextDate = dates.next();
                JsonNode rateNode = ratesNode.get(nextDate).get(wantedCurrency);
                dataSeries.put(nextDate, rateNode.asDouble());
            }
        }

        catch (Exception ex) {
            System.out.println("Error");
        }

        return dataSeries;
    }
}
```

Logic Package

BudgetTrackerLogic.java

```
package com.mycompany.Logic;

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.Map;
import com.mycompany.PythonFlaskAPI.APIQueue;
import com.mycompany.PythonFlaskAPI.PythonFlaskAPI;
import com.mycompany.Records.OutlierGoods;
import com.mycompany.Records.Purchase;
import com.mycompany.Repository.SQLQueriesBudgetTracker;
import com.mycompany.Repository.SQLQueriesConsumers;
import com.mycompany.Users.Consumer;

public class BudgetTrackerLogic {
    private static final APIQueue taskManager = new APIQueue(3);
    private static final SQLQueriesConsumers SQLQueriesConsumers = new SQLQueriesConsumers();

    // Updates the consumer's next budget renewal date based on their renewal frequency
    public static void saveNextRenewedDate(Consumer consumer) {
        LocalDate lastRenewed = consumer.getLastRenewed();
        LocalDate nextRenewed;
        String frequencyOfBudgetRenewal = consumer.getFrequencyOfBudgetRenewal();

        // Determine next renewal date based on frequency
        switch (frequencyOfBudgetRenewal) {
```

```
            case "Daily":
                nextRenewed = LocalDate.now().plusDays(1);
                break;
            case "Weekly":
                nextRenewed = LocalDate.now().plusWeeks(1);
                break;
            case "Monthly":
                nextRenewed = LocalDate.now().plusMonths(1);
                break;
            default:
                nextRenewed = LocalDate.now().plusDays(1);
                break;
        }

        // If today is past the last renewal, update budget and renewal date in database
        if (LocalDate.now().equals(lastRenewed) || LocalDate.now().isAfter(lastRenewed)) {
            SQLQueriesConsumers.updateCurrentBudget(consumer.getBudgetSize(), consumer);
            SQLQueriesConsumers.updateLastRenewed(nextRenewed, consumer);
        }
    }

    // Reduces the total cost of a purchase from the current budget
    public static double reduceCurrentBudget(double currentBudget, int goodQuantity, double goodPrice) {
        return currentBudget - (goodQuantity * goodPrice);
    }

    // Returns a mapping of the last 7 days to total purchase prices
    public static LinkedHashMap<LocalDate, Double> getDateTotalPricePairs(Consumer consumer) {
        LinkedHashMap<LocalDate, Double> dateTotalPricePairs = new LinkedHashMap<>();
        for (int i = 6; i >= 0; i--) {
            int daysAgo = i;
```

```
            ArrayList<ArrayList<Double>> totalPriceOfPurchasesLastWeek =
SQLQueriesBudgetTracker.getLastWeekPurchases(consumer, i);

            taskManager.submitTask(() -> {
                if (!totalPriceOfPurchasesLastWeek.isEmpty()) {
                    dateTotalPricePairs.put(LocalDate.now().minusDays(daysAgo),
PythonFlaskAPI.getLastWeekPurchases(totalPriceOfPurchasesLastWeek));
                }

                else {
                    dateTotalPricePairs.put(LocalDate.now().minusDays(daysAgo), 0.0);
                }
            });
        }

        return dateTotalPricePairs;
    }

    // Calculates mean of a list of doubles
    public static double calculateMean(ArrayList<Double> values) {
        double sum = 0.0;
        for (double val : values){
            sum += val;
        }

        return sum / values.size();
    }

    // Calculates variance given a mean
    public static double calculateVariance(ArrayList<Double> values, double mean) {
        double varianceSum = 0.0;
```

```
        for (double val : values) {
            varianceSum += (val - mean) * (val - mean);
        }

        return varianceSum / values.size();
    }

    // Calculates standard deviation of a list of doubles
    public static double calculateStdDev(ArrayList<Double> values) {
        if (values.isEmpty()) return 0.0;

        double mean = calculateMean(values);
        double variance = calculateVariance(values, mean);

        return Math.sqrt(variance);
    }

    // Groups purchases by item name. Organises all purchases by product to analyse spending patterns and detect
anomalies
    public static LinkedHashMap<String, ArrayList<Double>> groupPurchases(ArrayList<Purchase> purchases) {
        LinkedHashMap<String, ArrayList<Double>> totalsPerItem = new LinkedHashMap<>();

        // Iterates over each purchase made by the consumer
        for (Purchase purchase : purchases) {
            String item = purchase.getGoodName();
            totalsPerItem.putIfAbsent(item, new ArrayList<>());
            totalsPerItem.get(item).add(purchase.getTotalPrice());
        }

        return totalsPerItem;
    }
```

```
    // Groups purchases by item name
    public static LinkedHashMap<String, Double> calculateVolatility(ArrayList<Purchase> purchases,
LinkedHashMap<String, ArrayList<Double>> totalsPerItem) {
        LinkedHashMap<String, Double> volatility = new LinkedHashMap<>();

        // Iterate through each item and its list of total purchase prices
        for (Map.Entry<String, ArrayList<Double>> entry : totalsPerItem.entrySet()) {
            String item = entry.getKey();
            ArrayList<Double> totals = entry.getValue();

            // If there are no purchase prices recorded, set volatility to 0. Else, calculate standard deviation of
the list of prices
            if (totals.isEmpty()) {
                volatility.put(item, 0.0);
            }

            else {
                volatility.put(item, calculateStdDev(totals));
            }
        }

        return volatility;
    }

    // Detects outlier purchases using z-score (> 1.5 considered risky)
    public static ArrayList<OutlierGoods> calculateOutliers(LinkedHashMap<String, ArrayList<Double>> totalsPerItem,
LinkedHashMap<String, Double> volatility) {
        ArrayList<OutlierGoods> outlierGoods = new ArrayList<>();

        // Iterate over each item and its list of purchase totals
```

```
for (Map.Entry<String, ArrayList<Double>> entry : totalsPerItem.entrySet()) {
    String goodName = entry.getKey();
    ArrayList<Double> totals = entry.getValue();

    // Calculate mean of purchase totals for this item
    double mean = calculateMean(totals);

    // Retrieve standard deviation for this item
    double stdDev = volatility.get(goodName);

    for (double price : totals) {
        double z = 0;
        try {
            z = (price - mean) / stdDev;
        }

        // Division by 0 error
        catch (Exception ex) {
            z = 0;
        }

        finally {
            // Mark as outlier if z-score exceeds threshold
            if (Math.abs(z) > 1.5) {
                outlierGoods.add(new OutlierGoods(goodName, price, z));
            }
        }
    }
}

return outlierGoods;
```

```
    }

    // Fetches outliers for a consumer based on their purchase history
    public static ArrayList<OutlierGoods> getOutliers(Consumer consumer) {
        ArrayList<Purchase> purchases = SQLQueriesBudgetTracker.getConsumerPurchase(consumer);
        LinkedHashMap<String, ArrayList<Double>> totalsPerItem = groupPurchases(purchases);
        LinkedHashMap<String, Double> volatility = calculateVolatility(purchases, totalsPerItem);
        return calculateOutliers(totalsPerItem, volatility);
    }
}
```

CostOfProductionLogic.java

```java
package com.mycompany.Logic;

import java.util.ArrayList;
import java.util.LinkedHashMap;
import com.mycompany.Records.FactorInput;
import com.mycompany.Repository.SQLQueriesFactorInput;
import com.mycompany.Users.Producer;

public class CostOfProductionLogic {

    // Calculates cumulative marginal costs for a given producer
    public static LinkedHashMap<Integer, Double> getCumulativeMarginalCosts(Producer producer) {
        ArrayList<Double> allMarginalCosts = SQLQueriesFactorInput.getAllMarginalCosts(producer);
        LinkedHashMap<Integer, Double> cumulativeMarginalCosts = new LinkedHashMap<>();
        double currentCosts = 0;

        for (int i = 0; i < allMarginalCosts.size(); i++) {
            currentCosts += allMarginalCosts.get(i);
            cumulativeMarginalCosts.put(i + 1, currentCosts);
        }

        return cumulativeMarginalCosts;
    }

    // Calculates cumulative marginal revenue for a given producer
    public static LinkedHashMap<Integer, Double> getCumulativeMarginalRevenue(Producer producer) {
        ArrayList<Double> allMarginalRevenue = SQLQueriesFactorInput.getAllMarginalRevenue(producer);
        LinkedHashMap<Integer, Double> cumulativeMarginalRevenue = new LinkedHashMap<>();
        double currentRevenue = 0;
```

```
        // Sum marginal revenues cumulatively
        for (int i = 0; i < allMarginalRevenue.size(); i++) {
            currentRevenue += allMarginalRevenue.get(i);
            cumulativeMarginalRevenue.put(i + 1, currentRevenue);
        }

        return cumulativeMarginalRevenue;
    }

    // Generates all possible combinations of factor inputs
    public static ArrayList<ArrayList<FactorInput>> generateCombinations(ArrayList<FactorInput> factorInputs) {
        ArrayList<ArrayList<FactorInput>> allCombos = new ArrayList<>();

        int n = factorInputs.size();

        for (int size = 1; size <= n; size++) {
            combine(factorInputs, allCombos, new ArrayList<>(), 0, size);
        }

        return allCombos;
    }

    // Generate combinations of a specific size
    private static void combine(ArrayList<FactorInput> factorInputs, ArrayList<ArrayList<FactorInput>> allCombos,
ArrayList<FactorInput> current, int start, int size) {
        if (current.size() == size) {
            allCombos.add(new ArrayList<>(current));
            return;
        }
```

```
        for (int i = start; i < factorInputs.size(); i++) {
            current.add(factorInputs.get(i));
            combine(factorInputs, allCombos, current, i + 1, size);
            current.remove(current.size() - 1);
        }
    }

    // Calculates total profit for a given combination of factor inputs
    public static double totalProfit(ArrayList<FactorInput> combo) {
        double profit = 0;
        for (FactorInput factorInput : combo) {
            profit += factorInput.getMarginalProfit();
        }
        return profit;
    }

    // Calculates total cost for a given combination of factor inputs
    public static double totalCost(ArrayList<FactorInput> combo) {
        double cost = 0;
        for (FactorInput factorInput : combo) {
            cost += factorInput.getMarginalCost();
        }
        return cost;
    }

    // Determines the best combination of factor inputs that maximises profit within a cost constraint
    public static ArrayList<FactorInput> showBestCombination(ArrayList<FactorInput> allFactorInputsArrayList, double
maxCost) {
        ArrayList<ArrayList<FactorInput>> allCombos = generateCombinations(allFactorInputsArrayList);

        double bestProfit = Double.NEGATIVE_INFINITY;
```

```
        ArrayList<FactorInput> bestCombo = new ArrayList<>();

        // Evaluate each combination
        for (ArrayList<FactorInput> combo : allCombos) {
            double cost = totalCost(combo);

            // Only consider combinations within the max cost constraint
            if (cost <= maxCost) {
                double profit = totalProfit(combo);

                // Update the best combination if profit is higher
                if (profit > bestProfit) {
                    bestProfit = profit;
                    bestCombo = combo;
                }
            }
        }

        return bestCombo;
    }
}
```

InvestorLogic.java

```
package com.mycompany.Logic;

import java.io.File;
import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import com.fasterxml.jackson.core.type.TypeReference;
import com.mycompany.Algorithms.FileHandlingJSON;
import com.mycompany.PythonFlaskAPI.PythonFlaskAPI;
import com.mycompany.Records.CompanyRecord;

public class InvestorLogic {

    private static final File tickerInfo = new File("tickerInfo.json");
    private static final FileHandlingJSON JSONHandling = new FileHandlingJSON();

    // Retrieves all company names and their ticker symbols from JSON
    public static LinkedHashMap<String, String> getAllTickerInfo() {
        ArrayList<LinkedHashMap<String, String>> allTickerInfoAsJSON = JSONHandling.readFileList(tickerInfo, new
TypeReference<List<LinkedHashMap<String, String>>>() {});
        LinkedHashMap<String, String> allTickerInfo = new LinkedHashMap<>();

        for (LinkedHashMap<String, String> entry : allTickerInfoAsJSON) {
            allTickerInfo.put(entry.get("companyName"), entry.get("tickerSymbol"));
        }

        return allTickerInfo;
    }
```

```java
// Retrieves all ticker symbols and their company names (reversed map)
public static LinkedHashMap<String, String> getAllTickerInfoReversed() {
    ArrayList<LinkedHashMap<String, String>> allTickerInfoAsJSON =
        JSONHandling.readFileList(tickerInfo, new TypeReference<List<LinkedHashMap<String, String>>>() {});

    LinkedHashMap<String, String> allTickerInfoReversed = new LinkedHashMap<>();

    for (LinkedHashMap<String, String> entry : allTickerInfoAsJSON) {
        allTickerInfoReversed.put(entry.get("tickerSymbol"), entry.get("companyName"));
    }

    return allTickerInfoReversed;
}

// Fetches company information
public static CompanyRecord getCompanyInfo(String tickerSymbol) {
    LinkedHashMap<String, String> companyData = PythonFlaskAPI.getCompanyInformation(tickerSymbol);
    String fullTimeEmployees;

    try {
        fullTimeEmployees = companyData.get("fullTimeEmployees");
    }

    catch (Exception ex) {
        fullTimeEmployees = "Unknown";
    }

    return new CompanyRecord(
        companyData.get("symbol"),
        companyData.get("shortName"),
        companyData.get("sector"),
```

```
            companyData.get("industry"),
            fullTimeEmployees,
            companyData.get("country"),
            companyData.get("website")
        );
    }

    // Retrieves stock price data for a ticker and splits it into separate components
    public static ArrayList<ArrayList<?>> getStockPriceDataComponents(String tickerSymbol, String timeSpan) {
        ArrayList<LinkedHashMap<String, String>> allStockPriceData = PythonFlaskAPI.getStockPrice(tickerSymbol,
timeSpan);

        ArrayList<ArrayList<?>> stockPriceDataComponents = new ArrayList<>();
        ArrayList<String> dates = new ArrayList<>();
        ArrayList<Double> opens = new ArrayList<>();
        ArrayList<Double> highs = new ArrayList<>();
        ArrayList<Double> lows = new ArrayList<>();
        ArrayList<Double> closes = new ArrayList<>();
        ArrayList<Long> volumes = new ArrayList<>();

        for (LinkedHashMap<String, String> dataPoint : allStockPriceData) {
            dates.add(dataPoint.get("Date"));
            opens.add(Double.valueOf(dataPoint.get("Open")));
            highs.add(Double.valueOf(dataPoint.get("High")));
            lows.add(Double.valueOf(dataPoint.get("Low")));
            closes.add(Double.valueOf(dataPoint.get("Close")));
            volumes.add(Long.valueOf(dataPoint.get("Volume")));
        }

        stockPriceDataComponents.add(dates);
        stockPriceDataComponents.add(opens);
```

```
        stockPriceDataComponents.add(highs);
        stockPriceDataComponents.add(lows);
        stockPriceDataComponents.add(closes);
        stockPriceDataComponents.add(volumes);

        return stockPriceDataComponents;
    }
}
```

RateTheEconomyLogic.java

```
package com.mycompany.Logic;

import java.io.File;
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import com.fasterxml.jackson.core.type.TypeReference;
import com.fasterxml.jackson.databind.JsonNode;
import com.fasterxml.jackson.databind.ObjectMapper;
import com.mycompany.Algorithms.FileHandlingJSON;
import com.mycompany.Records.RateTheEconomyQuestions;
import com.mycompany.Repository.SQLQueriesRateTheEconomy;
import com.mycompany.Users.Consumer;

public class RateTheEconomyLogic {

    private static final FileHandlingJSON FileHandlingJSON = new FileHandlingJSON();

    // Loads the full list of RateTheEconomyQuestions objects from JSON into an ArrayList
    public static ArrayList<RateTheEconomyQuestions> loadJSONToArrayList() {
        File questionAnswerPairingsFile = new File("rateTheEconomyQuestions.json");
        return FileHandlingJSON.readFileList(questionAnswerPairingsFile, new
TypeReference<List<RateTheEconomyQuestions>>() {});
    }

    // Loads only the questions from the JSON file into an ArrayList of Strings
    public static ArrayList<String> loadQuestionsToArrayList() {
```

```
        File questionAnswerPairingsFile = new File("rateTheEconomyQuestions.json");
        return FileHandlingJSON.readFieldNameString(questionAnswerPairingsFile, "question");
    }

    // Loads the answer-to-weight mappings for each question from the JSON file
    public static ArrayList<LinkedHashMap<String, Integer>> loadAnswerWeightPairingsToArrayList() {
        ArrayList<LinkedHashMap<String, Integer>> allAnswerWeights = new ArrayList<>();
        File questionAnswerPairingsFile = new File("rateTheEconomyQuestions.json");
        ObjectMapper objectMapper = new ObjectMapper();

        try {
            JsonNode rootNode = objectMapper.readTree(questionAnswerPairingsFile);
            for (JsonNode questionNode : rootNode) {
                // Extract the answers node for each question
                JsonNode answersNode = questionNode.get("answers");
                LinkedHashMap<String, Integer> currentAnswerWeights = objectMapper.convertValue(answersNode, new
TypeReference<LinkedHashMap<String, Integer>>() {});
                allAnswerWeights.add(currentAnswerWeights);
            }
        }

        catch (Exception ex) {
            ex.printStackTrace();
        }

        return allAnswerWeights;
    }

    // Stores a user's answers along with the question and corresponding weight
    public static ArrayList<ArrayList<Object>> storeUserOpinion(ArrayList<String> answers) {
        ArrayList<ArrayList<Object>> userOpinion = new ArrayList<>();
```

```
        ArrayList<String> questions = loadQuestionsToArrayList();
        ArrayList<LinkedHashMap<String, Integer>> answerWeightPairings = loadAnswerWeightPairingsToArrayList();

        for (int i = 0; i < questions.size(); i++) {
            ArrayList<Object> newRow = new ArrayList<>();
            newRow.add(questions.get(i));
            newRow.add(answers.get(i));
            newRow.add(answerWeightPairings.get(i).get(answers.get(i)));
            userOpinion.add(newRow);
        }

        return userOpinion;
    }

    // Stores the user's opinion to a JSON file
    public static void storeUserOpinionToAppropiateLocations(ArrayList<ArrayList<Object>> userOpinion, Consumer
consumer) {
        String folderName1 = "UserOpinions";
        DateTimeFormatter dateFormatter = DateTimeFormatter.ofPattern("dd-MM-yyyy");
        String folderName2 = LocalDate.now().format(dateFormatter);

        FileHandlingJSON.createFolder(folderName1);
        FileHandlingJSON.createFolder(folderName1 + File.separator + folderName2);

        String fileName = "userOpinion" + consumer.getConsumerID();
        File userOpinionFile = FileHandlingJSON.createFile(folderName1 + File.separator + folderName2 + File.separator
+ fileName);

        // Convert the 2D ArrayList of user opinions to a list of LinkedHashMaps
        ArrayList<LinkedHashMap<String, Object>> userOpinionArrayList = new ArrayList<>();
        for (ArrayList<Object> row : userOpinion) {
```

```
            LinkedHashMap<String, Object> tempLinkedHashMap = new LinkedHashMap<>();
            tempLinkedHashMap.put("question", row.get(0));
            tempLinkedHashMap.put("answer", row.get(1));
            tempLinkedHashMap.put("weight", row.get(2));
            userOpinionArrayList.add(tempLinkedHashMap);
        }

        // Write the user's opinion to JSON
        FileHandlingJSON.writeFile(userOpinionFile, userOpinionArrayList);

        // Calculate total score and update the database
        int totalScore = getTotalScore(userOpinionFile);
        int ratingID = SQLQueriesRateTheEconomy.checkNumberOfRatings(consumer);
        if (ratingID == -1) {
            SQLQueriesRateTheEconomy.insertDataIntoEconomyRaterTable(consumer, fileName, totalScore);
        }

        else {
            SQLQueriesRateTheEconomy.updateConsumerOpinion(ratingID, totalScore);
        }
    }

    // Calculates the total score of a user's answers as a percentage of the maximum score (57)
    public static int getTotalScore(File userOpinionFile) {
        ObjectMapper objectMapper = new ObjectMapper();
        double totalScore = 0;

        try {
            JsonNode rootNode = objectMapper.readTree(userOpinionFile);
            if (rootNode.isArray()) {
                for (JsonNode node : rootNode) {
```

```
                totalScore += node.get("weight").asInt();
            }
        }
    }

    catch (Exception ex) {
        ex.printStackTrace();
    }

    return (int) ((totalScore / 57) * 100);
}

// Returns the available dates for which user opinions have been stored
public static LinkedHashMap<String, String> loadUserOpinionDates() {
    File folder = new File("UserOpinions");
    File[] files = folder.listFiles();
    LinkedHashMap<String, String> availableDates = new LinkedHashMap<>();

    for (File file : files) {
        if (file.isDirectory()) {
            availableDates.put(file.getName(), file.getName());
        }
    }

    return availableDates;
}

// Get all user answers for a given date and counts frequencies for each answer
public static LinkedHashMap<String, LinkedHashMap<String, Integer>> getAllUserOpinions(String date) {
    ArrayList<String> questions = loadQuestionsToArrayList();
    LinkedHashMap<String, LinkedHashMap<String, Integer>> allUserOpinions = new LinkedHashMap<>();
```

```
for (String question : questions) {
    allUserOpinions.put(question, new LinkedHashMap<>());
}

File folder = new File("UserOpinions" + File.separator + date);
File[] files = folder.listFiles();
if (files == null) {
    return allUserOpinions;
}

ObjectMapper objectMapper = new ObjectMapper();

for (int i = 0; i < files.length; i++) {
    File userOpinionFile = files[i];

    try {
        JsonNode rootNode = objectMapper.readTree(userOpinionFile);

        for (int j = 0; j < rootNode.size(); j++) {
            JsonNode node = rootNode.get(j);
            String question = node.get("question").asText();
            String answer = node.get("answer").asText();

            LinkedHashMap<String, Integer> answerCounts = allUserOpinions.get(question);
            answerCounts.put(answer, answerCounts.getOrDefault(answer, 0) + 1);
        }
    }

    catch (Exception ex) {
        ex.printStackTrace();
```

```
            }
        }

        return allUserOpinions;
    }
}
```

WebScraperLogic.java

```
package com.mycompany.Logic;

import java.io.IOException;
import java.io.InputStream;
import java.net.URL;
import java.time.LocalDate;
import java.time.ZonedDateTime;
import java.time.format.DateTimeFormatter;
import java.util.ArrayList;
import java.util.LinkedHashMap;
import org.jsoup.Jsoup;
import org.jsoup.nodes.Document;
import org.jsoup.nodes.Element;
import org.jsoup.parser.Parser;
import org.jsoup.select.Elements;
import com.mycompany.Records.EconomicNewsRecord;

public class WebScraperLogic {

    private static final FileHandlingJSON JSONHandling = new FileHandlingJSON();

    // RSS feed URLs for different news sources
    private final static String BBCRSSFeed = "https://feeds.bbci.co.uk/news/business/economy/rss.xml";
    private final static String TheTelegraphRSSFeed = "https://www.telegraph.co.uk/business/rss.xml";
    private final static String TheEconomistRSSFeed = "https://www.economist.com/finance-and-economics/rss.xml";
    private final static String TheIndependentRSSFeed = "https://www.independent.co.uk/news/business/rss";
    private final static String CNBCRSSFeed =
"https://search.cnbc.com/rs/search/combinedcms/view.xml?partnerId=wrss01&id=10000664";
```

```java
// Scrapes news from RSS feeds and returns a list of EconomicNewsRecord
public static ArrayList<EconomicNewsRecord> scrapeNews() {
    ArrayList<EconomicNewsRecord> economicNews = new ArrayList<>();
    LinkedHashMap<String, String> RSSFeeds = new LinkedHashMap<>();
    RSSFeeds.put("BBC News", BBCRSSFeed);
    RSSFeeds.put("The Telegraph", TheTelegraphRSSFeed);
    RSSFeeds.put("The Economist", TheEconomistRSSFeed);
    RSSFeeds.put("The Independent", TheIndependentRSSFeed);
    RSSFeeds.put("CNBC", CNBCRSSFeed);

    ArrayList<String> RSSFeedsSource = new ArrayList<>(RSSFeeds.keySet());
    ArrayList<String> RSSFeedsLinks = new ArrayList<>(RSSFeeds.values());

    int economicNewsID = 1;

    // Iterate over all RSS feed URLs
    for (int i = 0; i < RSSFeedsLinks.size(); i++) {
        String URL = RSSFeedsLinks.get(i);

        try {
            InputStream inputLink = new URL(URL).openStream();
            Document doc = Jsoup.parse(inputLink, "UTF-8", "", Parser.xmlParser());
            Elements headlines = doc.select("item");

            // Limit to max 50 items per feed
            int maxSize = Math.min(headlines.size(), 50);

            // Iterates over each news item
            for (int j = 0; j < maxSize; j++) {
                Element headline = headlines.get(j);
                String title = headline.select("title").text();
```

```
                    String description = headline.select("description").text();

                    // Parse publication date and format it
                    LocalDate pubDate = ZonedDateTime.parse(headline.select("pubDate").text(),
DateTimeFormatter.RFC_1123_DATE_TIME).toLocalDate();
                    DateTimeFormatter df = DateTimeFormatter.ofPattern("dd/MM/yyyy");
                    String pubDateFormatted = pubDate.format(df);

                    String link = headline.select("link").text();
                    String image = null;
                    String source = RSSFeedsSource.get(i);

                    // Extract Image based on source
                    switch (URL) {
                        case BBCRSSFeed:
                            image = headline.select("media\\:thumbnail").attr("url");
                            break;
                        case TheTelegraphRSSFeed:
                            image = headline.select("enclosure").attr("url");
                            break;
                        case TheIndependentRSSFeed:
                            image = headline.select("media\\:content").attr("url");
                            break;
                        default:
                            break;
                    }

                    // Add the news item to the list
                    economicNews.add(new EconomicNewsRecord(economicNewsID, title, description, pubDateFormatted,
link, image, source));
                    economicNewsID++;
```

```
                }
            }

            catch (IOException e) {
                System.out.println("An error occurred in scraping the news from " + URL);
            }
        }

        return economicNews;
    }

    // Sorts a list of EconomicNewsRecord by their news source
    public static ArrayList<ArrayList<EconomicNewsRecord>> sortEconomicsNewsBySource(ArrayList<EconomicNewsRecord>
economicNews) {
        ArrayList<ArrayList<EconomicNewsRecord>> economicNewsBySource = new ArrayList<>();
        ArrayList<EconomicNewsRecord> BBCNews = new ArrayList<>();
        ArrayList<EconomicNewsRecord> TheTelegraph = new ArrayList<>();
        ArrayList<EconomicNewsRecord> TheEconomist = new ArrayList<>();
        ArrayList<EconomicNewsRecord> TheIndependent = new ArrayList<>();
        ArrayList<EconomicNewsRecord> CNBC = new ArrayList<>();

        // Group news items by source
        for (EconomicNewsRecord record : economicNews) {
            switch (record.source()) {
                case "BBC News":
                    BBCNews.add(record);
                    break;
                case "The Telegraph":
                    TheTelegraph.add(record);
                    break;
                case "The Economist":
```

```
                    TheEconomist.add(record);
                    break;
                case "The Independent":
                    TheIndependent.add(record);
                    break;
                case "CNBC":
                    CNBC.add(record);
                    break;
                default:
                    break;
            }
        }

        economicNewsBySource.add(BBCNews);
        economicNewsBySource.add(TheTelegraph);
        economicNewsBySource.add(TheEconomist);
        economicNewsBySource.add(TheIndependent);
        economicNewsBySource.add(CNBC);

        return economicNewsBySource;
    }
}
```

PythonFlaskAPI Package

BudgetTrackerMaths.py

```
import math
import numpy as np
from datetime import date

class BudgetTrackerMaths:
    @staticmethod
    def calculateIncomeComponents(budget):
        # Essentials = 45%, Leisure = 30%, Savings = 25%
        desiredIncomeComponents = {}

        essentials = budget * 0.45
        leisure = budget * 0.3
        savings = budget * 0.25

        desiredIncomeComponents.update({"Essentials": essentials})
        desiredIncomeComponents.update({"Leisure": leisure})
        desiredIncomeComponents.update({"Savings": savings})

        return desiredIncomeComponents

    @staticmethod
    def futureBudget(incomeComponents, growthRate):
        growthRate = growthRate / 100

        # Calculate equivalent monthly growth rate
        monthlyGrowthRate = (1 + growthRate) ** (1/12) - 1
```

```
        incomeComponents = np.array(list(incomeComponents.values()))

        # Create months 0-11 as a column vector
        months = np.arange(0, 12).reshape(-1, 1)

        # Growth factor for each month
        growthRates = (1 + monthlyGrowthRate) ** months
        futureBudgets = incomeComponents * growthRates

        return futureBudgets

    @staticmethod
    def sustainableSpendingPerDay(essentials):
        return essentials / 30

    @staticmethod
    def calculateBudgetOverAWeek(prices, quantity):
        pricesMatrix = np.array(prices)
        quantitiesMatrix = np.array(quantity)
        return (pricesMatrix * quantitiesMatrix).sum()
```

CostOfProductionMaths.py

```python
import math
import random

class CostOfProductionMaths:

    @staticmethod
    def totalFactorProductivity(totalRevenue, totalCost):
        return (math.log1p(totalRevenue) / math.log1p(totalCost))

    @staticmethod
    def cobbDouglasProductionFunction(totalRevenue, totalCost, K, L):
        # Random alpha and beta
        alpha = random.uniform(0.3, 0.4)
        beta  = 1 - alpha

        tfp = CostOfProductionMaths.totalFactorProductivity(totalRevenue, totalCost)
        K = (K + 1) ** alpha * math.log1p(K)
        L = (L + 1) ** beta  * math.sqrt(L)

        return tfp * K * L
```

InvestmentData.py

```
import yfinance as yf
from datetime import datetime, date, timedelta
import numpy as np

class InvestmentData:
    def __init__(self, tickerSymbol):
        self.tickerSymbol = tickerSymbol
        self.ticker = yf.Ticker(self.tickerSymbol)
        self.info = self.ticker.info

    def getInfoOfCompany(self):
        # Extract company information
        fields = ["symbol", "shortName", "sector", "industry", "fullTimeEmployees", "country", "website"]
        companyInfo = {}

        for field in fields:
            companyInfo[field] = self.ticker.info.get(field)

        return companyInfo

    def getStockData(self, startDate):
        currentStockData = []
        startDate = datetime.strptime(startDate, "%Y-%m-%d").date()
        endDate = date.today() - timedelta(days=1)

        # Fetch historical stock data
        data = self.ticker.history(start=startDate, end=endDate)

        for i in range(0, len(data)):
```

```
            currentStockDataInstance = {}
            currentStockDataInstance["Date"] = data.index[i].strftime("%d/%m/%y")
            currentStockDataInstance["Open"] = round(float(data["Open"].iloc[i]), 2)
            currentStockDataInstance["High"] = round(float(data["High"].iloc[i]), 2)
            currentStockDataInstance["Low"] = round(float(data["Low"].iloc[i]), 2)
            currentStockDataInstance["Close"] = round(float(data["Close"].iloc[i]), 2)
            currentStockDataInstance["Volume"] = int(data["Volume"].iloc[i])

            # Add the day's data to the main list
            currentStockData.append(currentStockDataInstance)

        return currentStockData

    # Retrieve the current ask price
    def getAskPrice(self):
        return self.info.get("ask")

    def getSMATrend(self, window):
        end_date = date.today()
        start_date = end_date - timedelta(days=window * 3)
        data = self.ticker.history(start=start_date, end=end_date)

        if data.empty or len(data) < window:
            return {"error": "Not enough data"}

        # Calculate SMA
        data["SMA"] = data["Close"].rolling(window=window).mean().dropna()
        data = data.dropna(subset=["SMA"])

        # Use Linear Regression to calculate the gradient of the SMA line
        y = data["SMA"].to_numpy()
```

```
x = np.arange(len(y))

coefficients = np.polyfit(x, y, 1)
gradient = coefficients[0]

# Calculate the SMA trend based on the gradient
if gradient > 0.25:
    trend = "Increasing"
elif gradient < 0.25:
    trend = "Decreasing"
else:
    trend = "Stable"

return {
    "trend": trend,
    "gradient": float(gradient)
}
```

OllamaAI.py

```
import ollama
from newspaper import Article

class OllamaAI:

    @staticmethod
    def aiAnswers(prompt):
        # "gemma3:4b" model to generate a natural language answer to a user prompt
        model = "gemma3:4b"

        prompt = prompt + ". In a simple paragraph"

        # Send the prompt to Ollama chat model
        response = ollama.chat(
            model=model,
            messages=[
                {"role": "user", "content": prompt}
            ],
        )

        return response["message"]["content"]

    @staticmethod
    def summariseNewsArticle(link):
        # "granite3.1-moe:1b" model to summarise a news article
        model = "granite3.1-moe:1b"

        # Download and parse the article text from the URL
        article = Article(link, language="en")
```

```
        article.download()
        article.parse()

        prompt = "Summarise the following article clearly and completely and end with a full sentence, explaining key
economic changes and predict their impact on everyday life.\n\n" + article.text

        # Send the prompt to Ollama chat model
        response = ollama.chat(
            model=model,
            messages=[
                {"role": "user", "content": prompt}
            ]
        )

        return response["message"]["content"]
```

PythonFlaskAPI.py

```
from flask import Flask, request, jsonify
from BudgetTrackerMaths import BudgetTrackerMaths
from CostOfProductionMaths import CostOfProductionMaths
from InvestmentData import InvestmentData
from OllamaAI import OllamaAI

FlaskAPI = Flask(__name__)

# Breakdown income into components (essential, leisure, savings)
@FlaskAPI.route("/calculate-Income-Components", methods=["POST"])
def calculateIncomeComponents():
    data = request.json
    income = data.get("income")
    incomeComponents = BudgetTrackerMaths.calculateIncomeComponents(income)
    return jsonify(incomeComponents)

# Calculates future budget based on income components and inflation
@FlaskAPI.route("/future-budget", methods=["POST"])
def futureBudget():
    data = request.json
    incomeComponents = data.get("incomeComponents")
    growthRate = data.get("growthRate")
    futureBudget = BudgetTrackerMaths.futureBudget(incomeComponents, growthRate)
    return jsonify(futureBudget.tolist())

# Calculates sustainable daily spending based on essential
@FlaskAPI.route("/sustainable-spending", methods=["POST"])
def sustainableSpending():
    data = request.json
```

```
    essentials = data.get("essentials")
    sustainableSpending = BudgetTrackerMaths.sustainableSpendingPerDay(essentials)
    return jsonify(sustainableSpending)

# Calculates total purchases over the last week based on prices and quantities
@FlaskAPI.route("/get-last-week-purchases", methods=["POST"])
def getLastWeekPurchases():
    data = request.json
    prices = data.get("prices")
    quantities = data.get("quantities")
    return jsonify(BudgetTrackerMaths.calculateBudgetOverAWeek(prices, quantities))

# Calculates total output using a Cobb-Douglas function
@FlaskAPI.route("/perform-cobb-douglas-function", methods=["POST"])
def performCobbDouglasFunction():
    data = request.json
    totalRevenue = data.get("totalRevenue")
    totalCost = data.get("totalCost")
    numberOfLabour = data.get("numberOfLabour")
    numberOfCapital = data.get("numberOfCapital")
    totalOutput = CostOfProductionMaths.cobbDouglasProductionFunction(totalRevenue, totalCost, numberOfCapital,
numberOfLabour)
    return jsonify(totalOutput)

# AI-generated answer for a user question
@FlaskAPI.route("/ai-answer", methods=["POST"])
def aiAnswers():
    data = request.json
    userPrompt = data.get("userPrompt")
    aiReply = OllamaAI.aiAnswers(userPrompt)
    return jsonify(aiReply)
```

```python
# Summarises a news article given its link
@FlaskAPI.route("/summarise-news-article", methods=["POST"])
def summariseNewsArticle():
    data = request.json
    link = data.get("link")
    aiReply = OllamaAI.summariseNewsArticle(link)
    return jsonify(aiReply)

# Fetches company information for a ticker symbol
@FlaskAPI.route("/company-information", methods=["POST"])
def companyInformation():
    data = request.json
    stock = data.get("tickerSymbol")
    stock = InvestmentData(stock)
    return jsonify(stock.getInfoOfCompany())

# Retrieves stock price data for a ticker and start date
@FlaskAPI.route("/stock-data", methods=["POST"])
def getStockPrice():
    data = request.json
    tickerSymbol = data.get("tickerSymbol")
    startDate = data.get("startDate")
    stock = InvestmentData(tickerSymbol)
    return jsonify(stock.getStockData(startDate))

# Retrieves current ask price for a stock ticker
@FlaskAPI.route("/stock-ask-price", methods=["POST"])
def getAskPrice():
    data = request.json
    tickerSymbol = data.get("tickerSymbol")
```

```
    stock = InvestmentData(tickerSymbol)
    return jsonify(stock.getAskPrice())

# Calculate SMA
@FlaskAPI.route("/calculate-SMA", methods=["POST"])
def getSMA():
    data = request.json
    tickerSymbol = data.get("tickerSymbol")
    window = int(data.get("window"))
    stock = InvestmentData(tickerSymbol)
    return jsonify(stock.getSMATrend(window))

if __name__ == '__main__':
    FlaskAPI.run(debug=True, port=5000)
```

PythonFlaskAPI.java

```java
package com.mycompany.PythonFlaskAPI;

import java.util.LinkedHashMap;
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.util.ArrayList;
import com.fasterxml.jackson.databind.ObjectMapper;
import com.fasterxml.jackson.core.type.TypeReference;

public class PythonFlaskAPI {
    private static final HttpClient client = HttpClient.newHttpClient();
    private static final ObjectMapper mapper = new ObjectMapper();

    // Breakdown income into components (essential, leisure, savings)
    public static LinkedHashMap<String, Double> calculateIncomeComponents(double income) {
        try {
            LinkedHashMap<String, Double> payload = new LinkedHashMap<>();
            payload.put("income", income);

            HttpRequest request = HttpRequest.newBuilder()
                .uri(new URI("http://localhost:5000/calculate-Income-Components"))
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(mapper.writeValueAsString(payload)))
                .build();

            HttpResponse<String> response = client.send(request, HttpResponse.BodyHandlers.ofString());
            return mapper.readValue(response.body(), new TypeReference<LinkedHashMap<String, Double>>() {});
```

```
        }

        catch (Exception ex) {
            ex.printStackTrace();
        }

        return null;
    }

    // Calculates total output using a Cobb-Douglas function
    public static String performCobbDouglasFunction(double totalRevenue, double totalCost, int numberOfLabour, int
numberOfCapital) {
        try {
            LinkedHashMap<String, Double> payload = new LinkedHashMap<>();
            payload.put("totalRevenue", totalRevenue);
            payload.put("totalCost", totalCost);
            payload.put("numberOfLabour", (double) numberOfLabour);
            payload.put("numberOfCapital", (double) numberOfCapital);

            HttpRequest request = HttpRequest.newBuilder()
                .uri(new URI("http://localhost:5000/perform-cobb-douglas-function"))
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(mapper.writeValueAsString(payload)))
                .build();

            HttpResponse<String> response = client.send(request, HttpResponse.BodyHandlers.ofString());
            return response.body();
        }

        catch (Exception ex) {
            ex.printStackTrace();
```

```
        }

        return null;
    }

    // AI-generated answer for a user question
    public static String aiAnswer(String userPrompt) {
        try {
            LinkedHashMap<String, String> payload = new LinkedHashMap<>();
            payload.put("userPrompt", userPrompt);

            HttpRequest request = HttpRequest.newBuilder()
                .uri(new URI("http://localhost:5000/ai-answer"))
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(mapper.writeValueAsString(payload)))
                .build();

            HttpResponse<String> response = client.send(request, HttpResponse.BodyHandlers.ofString());
            return response.body();
        }

        catch (Exception ex) {
            ex.printStackTrace();
        }

        return null;
    }

    // Summarises a news article given its link
    public static String summariseNewsArticle(String link) {
        try {
```

```
            LinkedHashMap<String, String> payload = new LinkedHashMap<>();
            payload.put("link", link);

            HttpRequest request = HttpRequest.newBuilder()
                .uri(new URI("http://localhost:5000/summarise-news-article"))
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(mapper.writeValueAsString(payload)))
                .build();

            HttpResponse<String> response = client.send(request, HttpResponse.BodyHandlers.ofString());
            return response.body();
        }

        catch (Exception ex) {
            ex.printStackTrace();
        }

        return null;
    }

    // Calculates future budget based on income components and inflation
    public static String futureBudget(LinkedHashMap<String, Double> incomeComponents, double growthRate) {
        try {
            LinkedHashMap<String, Object> payload = new LinkedHashMap<>();
            payload.put("incomeComponents", incomeComponents);
            payload.put("growthRate", growthRate);

            HttpRequest request = HttpRequest.newBuilder()
                .uri(new URI("http://localhost:5000/future-budget"))
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(mapper.writeValueAsString(payload)))
```

```
            .build();

        HttpResponse<String> response = client.send(request, HttpResponse.BodyHandlers.ofString());
        return response.body();
    }

    catch (Exception ex) {
        ex.printStackTrace();
    }

    return null;
}

// Calculates sustainable daily spending based on essential
public static Double sustainableSpending(Double essentials) {
    try {
        LinkedHashMap<String, Double> payload = new LinkedHashMap<>();
        payload.put("essentials", essentials);

        HttpRequest request = HttpRequest.newBuilder()
            .uri(new URI("http://localhost:5000/sustainable-spending"))
            .header("Content-Type", "application/json")
            .POST(HttpRequest.BodyPublishers.ofString(mapper.writeValueAsString(payload)))
            .build();

        HttpResponse<String> response = client.send(request, HttpResponse.BodyHandlers.ofString());
        return Double.valueOf(response.body());
    }

    catch (Exception ex) {
        ex.printStackTrace();
```

```
        }

        return null;
    }

    // Fetches company information for a ticker symbol
    public static LinkedHashMap<String, String> getCompanyInformation(String tickerSymbol) {
        try {
            LinkedHashMap<String, String> payload = new LinkedHashMap<>();
            payload.put("tickerSymbol", tickerSymbol);

            HttpRequest request = HttpRequest.newBuilder()
                .uri(new URI("http://localhost:5000/company-information"))
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(mapper.writeValueAsString(payload)))
                .build();

            HttpResponse<String> response = client.send(request, HttpResponse.BodyHandlers.ofString());
            return mapper.readValue(response.body(), new TypeReference<LinkedHashMap<String, String>>() {});
        }

        catch (Exception ex) {
            ex.printStackTrace();
        }

        return null;
    }

    // Retrieves stock price data for a ticker and start date
    public static ArrayList<LinkedHashMap<String, String>> getStockPrice(String tickerSymbol, String startDate) {
        try {
```

```
            LinkedHashMap<String, String> payload = new LinkedHashMap<>();
            payload.put("tickerSymbol", tickerSymbol);
            payload.put("startDate", startDate);

            HttpRequest request = HttpRequest.newBuilder()
                .uri(new URI("http://localhost:5000/stock-data"))
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(mapper.writeValueAsString(payload)))
                .build();

            HttpResponse<String> response = client.send(request, HttpResponse.BodyHandlers.ofString());
            return mapper.readValue(response.body(), new TypeReference<ArrayList<LinkedHashMap<String, String>>>()
{});
        }

        catch (Exception ex) {
            ex.printStackTrace();
        }

        return null;
    }

    // Retrieves the current ask price of a stock
    public static Double getAskPrice(String tickerSymbol) {
        try {
            LinkedHashMap<String, String> payload = new LinkedHashMap<>();
            payload.put("tickerSymbol", tickerSymbol);

            HttpRequest request = HttpRequest.newBuilder()
                .uri(new URI("http://localhost:5000/stock-ask-price"))
                .header("Content-Type", "application/json")
```

```
            .POST(HttpRequest.BodyPublishers.ofString(mapper.writeValueAsString(payload)))
            .build();

        HttpResponse<String> response = client.send(request, HttpResponse.BodyHandlers.ofString());
        return Double.valueOf(response.body());
    }

    catch (Exception ex) {
        ex.printStackTrace();
    }

    return null;
}

// Calculates total purchases over the last week based on prices and quantities
public static Double getLastWeekPurchases(ArrayList<ArrayList<Double>> priceQuantityPairs) {
    try {
        LinkedHashMap<String, Object> payload = new LinkedHashMap<>();
        payload.put("prices", priceQuantityPairs.get(0));
        payload.put("quantities", priceQuantityPairs.get(1));

        HttpRequest request = HttpRequest.newBuilder()
            .uri(new URI("http://localhost:5000/get-last-week-purchases"))
            .header("Content-Type", "application/json")
            .POST(HttpRequest.BodyPublishers.ofString(mapper.writeValueAsString(payload)))
            .build();

        HttpResponse<String> response = client.send(request, HttpResponse.BodyHandlers.ofString());
        return Double.valueOf(response.body());
    }
```

```
        catch (Exception ex) {
            ex.printStackTrace();
        }

        return null;
    }

    // Calculates SMA
    public static LinkedHashMap<String, Object> calculateSMA(String tickerSymbol, int window) {
        try {
            LinkedHashMap<String, String> payload = new LinkedHashMap<>();
            payload.put("tickerSymbol", tickerSymbol);
            payload.put("window", Integer.toString(window));

            HttpRequest request = HttpRequest.newBuilder()
                .uri(new URI("http://localhost:5000/calculate-SMA"))
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(mapper.writeValueAsString(payload)))
                .build();

            HttpResponse<String> response = client.send(request, HttpResponse.BodyHandlers.ofString());
            return mapper.readValue(response.body(), new TypeReference<LinkedHashMap<String, Object>>() {});
        }

        catch (Exception ex) {
            ex.printStackTrace();
        }

        return null;
    }
}
```

APIWorker.java

```java
package com.mycompany.PythonFlaskAPI;

import java.util.concurrent.BlockingQueue;

public class APIWorker extends Thread {

    // Shared queue of tasks to be executed by this worker
    private final BlockingQueue<Runnable> taskQueue;

    // Flag to indicate whether the worker should keep running
    private volatile boolean running = true;

    // Assigns the shared task queue to the worker.
    public APIWorker(BlockingQueue<Runnable> taskQueue) {
        this.taskQueue = taskQueue;
    }

    // Continuously takes tasks from the queue and executes them.
    @Override
    public void run() {
        while (running) {
            try {
                // Take a task from the queue
                Runnable task = taskQueue.take();
                task.run();
            }

            catch (Exception ex) {
                Thread.currentThread().interrupt();
```

```
                ex.printStackTrace();
                break;
            }
        }
    }

    // Shut down the worker
    public void shutdown() {
        running = false;
        this.interrupt();
    }
}
```

APIQueue.java

```java
package com.mycompany.PythonFlaskAPI;

import java.util.ArrayList;
import java.util.concurrent.BlockingQueue;
import java.util.concurrent.Callable;
import java.util.concurrent.Future;
import java.util.concurrent.FutureTask;
import java.util.concurrent.LinkedBlockingQueue;

public class APIQueue {

    // Shared queue for tasks submitted to the pool
    private final BlockingQueue<Runnable> taskQueue = new LinkedBlockingQueue<>();
    private final ArrayList<APIWorker> workers = new ArrayList<>();

    public APIQueue(int numberOfThreads) {
        for (int i = 0; i < numberOfThreads; i++) {
            // Create a new worker with access to the shared task queue
            APIWorker worker = new APIWorker(taskQueue);
            worker.start();
            workers.add(worker);
        }
    }

    // For Procedures
    public void submitTask(Runnable task) {
        try {
            // Add the task to the queue
            taskQueue.put(task);
```

```
        }
        catch (Exception ex) {
            Thread.currentThread().interrupt();
            ex.printStackTrace();
        }
    }

    // For Functions
    public <V> Future<V> submitTask(Callable<V> task) {
        FutureTask<V> futureTask = new FutureTask<>(task);
        try {
            taskQueue.put(futureTask);
        }

        catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            e.printStackTrace();
        }
        return futureTask;
    }

    // Shuts down all worker threads
    public void shutdown() {
        for (APIWorker worker : workers) {
            worker.shutdown();
        }
    }
}
```

Records Package

CompanyRecord.java

```
package com.mycompany.Records;

public record CompanyRecord(String symbol, String shortName, String sector, String industry, String fullTimeEmployees,
String country, String website) {

}
```

DefinitionRecord.java

```
package com.mycompany.Records;

public record DefinitionRecord(String word, String definition) {

}
```

GoodRecord.java

```
package com.mycompany.Records;

public record GoodRecord(int goodID, String goodName, double goodPrice) {

}
```

FactorInput.java

```
package com.mycompany.Records;

public class FactorInput {
    private int factorInputID;
    private String factorInputType;
    private String factorInputName;
    private String factorInputDescription;
    private double marginalCost;
    private double marginalRevenue;
    private double marginalProfit;

    public FactorInput() {

    }

    public FactorInput(String factorInputType, String factorInputName, String factorInputDescription, double
marginalCost, double marginalRevenue) {
        this.factorInputType = factorInputType;
        this.factorInputName = factorInputName;
        this.factorInputDescription = factorInputDescription;
        this.marginalCost = marginalCost;
        this.marginalRevenue = marginalRevenue;
        this.marginalProfit = marginalRevenue - marginalCost;
    }

    public FactorInput(int factorInputID, String factorInputType, String factorInputName, String
factorInputDescription, double marginalCost, double marginalRevenue) {
        this.factorInputID = factorInputID;
        this.factorInputType = factorInputType;
```

```
        this.factorInputName = factorInputName;
        this.factorInputDescription = factorInputDescription;
        this.marginalCost = marginalCost;
        this.marginalRevenue = marginalRevenue;
        this.marginalProfit = marginalRevenue - marginalCost;
    }

    public int getFactorInputID() {
        return factorInputID;
    }

    public void setFactorInputID(int factorInputID) {
        this.factorInputID = factorInputID;
    }

    public String getFactorInputType() {
        return factorInputType;
    }

    public void setFactorInputType(String factorInputType) {
        this.factorInputType = factorInputType;
    }

    public String getFactorInputName() {
        return factorInputName;
    }

    public void setFactorInputName(String factorInputName) {
        this.factorInputName = factorInputName;
    }
```

```
public String getFactorInputDescription() {
    return factorInputDescription;
}

public void setFactorInputDescription(String factorInputDescription) {
    this.factorInputDescription = factorInputDescription;
}

public double getMarginalCost() {
    return marginalCost;
}

public void setMarginalCost(double marginalCost) {
    this.marginalCost = marginalCost;
}

public double getMarginalRevenue() {
    return marginalRevenue;
}

public void setMarginalRevenue(double marginalRevenue) {
    this.marginalRevenue = marginalRevenue;
}

public double getMarginalProfit() {
    return marginalProfit;
}

public void setMarginalProfit(double marginalProfit) {
    this.marginalProfit = marginalProfit;
}
```

```
    @Override
    public String toString() {
        return "[factorInputID=" + factorInputID + ", factorInputType=" + factorInputType
                + ", factorInputName=" + factorInputName + ", factorInputDescription=" + factorInputDescription
                + ", marginalCost=" + marginalCost + ", marginalRevenue=" + marginalRevenue + ", marginalProfit="
                + marginalProfit + "]";
    }
}
```

OutlierGoods.java

```
package com.mycompany.Records;

public class OutlierGoods {

    private String goodName;
    private double price;
    private double outlierInfo;

    public OutlierGoods(String goodName, double price, double outlierInfo) {
        this.goodName = goodName;
        this.price = price;
        this.outlierInfo = outlierInfo;
    }

    public String getGoodName() {
        return goodName;
    }

    public double getPrice() {
        return price;
```

```
    }

    public double getOutlierInfo() {
        return outlierInfo;
    }
}
```

Purchase.java

```
package com.mycompany.Records;

public class Purchase {
    private String purchaseDate;
    private String goodName;
    private double goodPrice;
    private int goodQuantity;
    private double totalPrice;

    public Purchase(String goodName, double goodPrice, int goodQuantity, String purchaseDate) {
        this.purchaseDate = purchaseDate;
        this.goodName = goodName;
        this.goodPrice = goodPrice;
        this.goodQuantity = goodQuantity;
        this.totalPrice = goodPrice * goodQuantity;
    }

    public String getPurchaseDate() {
        return purchaseDate;
    }

    public void setPurchaseDate(String purchaseDate) {
        this.purchaseDate = purchaseDate;
    }

    public String getGoodName() {
        return goodName;
    }
```

```
    public void setGoodName(String goodName) {
        this.goodName = goodName;
    }

    public double getGoodPrice() {
        return goodPrice;
    }

    public void setGoodPrice(double goodPrice) {
        this.goodPrice = goodPrice;
    }

    public int getGoodQuantity() {
        return goodQuantity;
    }

    public void setGoodQuantity(int goodQuantity) {
        this.goodQuantity = goodQuantity;
    }

    public double getTotalPrice() {
        return totalPrice;
    }

    public void setTotalPrice(double totalPrice) {
        this.totalPrice = totalPrice;
    }
}
```

RateTheEconomyQuestions.java

```java
package com.mycompany.Records;

import java.util.LinkedHashMap;

public class RateTheEconomyQuestions {
    private String question;
    private LinkedHashMap<String, Integer> answers;

    public RateTheEconomyQuestions() {
    }

    public String getQuestion() {
        return question;
    }

    public void setQuestion(String question) {
        this.question = question;
    }

    public LinkedHashMap<String, Integer> getAnswers() {
        return answers;
    }

    public void setAnswers(LinkedHashMap<String, Integer> answers) {
        this.answers = answers;
    }
}
```

SQLQueries Package

SQLQueries.java

```
package com.mycompany.Repository;

public abstract class SQLQueries {

    protected static final String url =
"jdbc:sqlserver://localhost\\SQLEXPRESS:1433;databaseName=MONEYMATTERSDB;encrypt=true;trustServerCertificate=true;";
    protected static final String databaseUser = "user"; //not real user
    protected static final String databasePassword = "password"; //not real password

    public String getUrl() {
        return url;
    }

    public String getDatabaseUser() {
        return databaseUser;
    }

    public String getDatabasePassword() {
        return databasePassword;
    }
}
```

SQLQueriesDDL.java

```java
package com.mycompany.Repository;

import java.io.File;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;

import org.springframework.stereotype.Repository;

import com.mycompany.Algorithms.FileHandling;
import com.mycompany.PythonFlaskAPI.APIQueue;

@Repository
public class SQLQueriesDDL extends SQLQueries {

    private static final APIQueue taskManager = new APIQueue(4);

    // Creates the Consumers table
    private static void createConsumerTable() {
        File file = new File("SQLCreateTableQueries/ConsumersTable.txt");

        FileHandling createTableTXTGetter = new FileHandling();
        String query = createTableTXTGetter.readFile(file);

        try (Connection conn = DriverManager.getConnection(url, databaseUser, databasePassword);
            PreparedStatement pstmt = conn.prepareStatement(query)) {
            pstmt.executeUpdate();
        }
```

```
        catch (Exception ex) {
            ex.printStackTrace();
        }
    }

    // Creates the Producers table
    private static void createProducerTable() {
        File file = new File("SQLCreateTableQueries/ProducersTable.txt");
        FileHandling createTableTXTGetter = new FileHandling();
        String query = createTableTXTGetter.readFile(file);

        try (Connection conn = DriverManager.getConnection(url, databaseUser, databasePassword);
            PreparedStatement pstmt = conn.prepareStatement(query)) {
            pstmt.executeUpdate();
        }

        catch (Exception ex) {
            ex.printStackTrace();
        }
    }

    // Creates the EconomyRater table
    private static void createEconomyRaterTable() {
        File file = new File("SQLCreateTableQueries/EconomyRaterTable.txt");
        FileHandling createTableTXTGetter = new FileHandling();
        String query = createTableTXTGetter.readFile(file);

        try (Connection conn = DriverManager.getConnection(url, databaseUser, databasePassword);
            PreparedStatement pstmt = conn.prepareStatement(query)) {
            pstmt.executeUpdate();
        }
```

```
        catch (Exception ex) {
            ex.printStackTrace();
        }
    }

    // Creates the Goods table
    private static void createGoodsTable() {
        File file = new File("SQLCreateTableQueries/GoodsTable.txt");
        FileHandling createTableTXTGetter = new FileHandling();
        String query = createTableTXTGetter.readFile(file);

        try (Connection conn = DriverManager.getConnection(url, databaseUser, databasePassword);
            PreparedStatement pstmt = conn.prepareStatement(query)) {
            pstmt.executeUpdate();
        }

        catch (Exception ex) {
            ex.printStackTrace();
        }
    }

    // Creates the Purchase table
    private static void createPurchaseTable() {
        File file = new File("SQLCreateTableQueries/PurchaseTable.txt");
        FileHandling createTableTXTGetter = new FileHandling();
        String query = createTableTXTGetter.readFile(file);

        try (Connection conn = DriverManager.getConnection(url, databaseUser, databasePassword);
            PreparedStatement pstmt = conn.prepareStatement(query)) {
            pstmt.executeUpdate();
```

```
        }

        catch (Exception ex) {
            ex.printStackTrace();
        }
    }

    // Creates the FactorInput table
    private static void createFactorInputTable() {
        File file = new File("SQLCreateTableQueries/FactorInputTable.txt");
        FileHandling createTableTXTGetter = new FileHandling();
        String query = createTableTXTGetter.readFile(file);

        try (Connection conn = DriverManager.getConnection(url, databaseUser, databasePassword);
            PreparedStatement pstmt = conn.prepareStatement(query)) {
            pstmt.executeUpdate();
        }

        catch (Exception ex) {
            ex.printStackTrace();
        }
    }

    // Creates the Stocks table
    private static void createStocksTable() {
        File file = new File("SQLCreateTableQueries/StocksTable.txt");
        FileHandling createTableTXTGetter = new FileHandling();
        String query = createTableTXTGetter.readFile(file);

        try (Connection conn = DriverManager.getConnection(url, databaseUser, databasePassword);
            PreparedStatement pstmt = conn.prepareStatement(query)) {
```

```
            pstmt.executeUpdate();
        }

        catch (Exception ex) {
            ex.printStackTrace();
        }
    }

    // Creates the Trades table
    private static void createTradesTable() {
        File file = new File("SQLCreateTableQueries/TradesTable.txt");
        FileHandling createTableTXTGetter = new FileHandling();
        String query = createTableTXTGetter.readFile(file);

        try (Connection conn = DriverManager.getConnection(url, databaseUser, databasePassword);
            PreparedStatement pstmt = conn.prepareStatement(query)) {
            pstmt.executeUpdate();
        }

        catch (Exception ex) {
            ex.printStackTrace();
        }
    }

    // Public method to create all tables in the correct order
    public static void createTables() {
        taskManager.submitTask(() -> {
            createConsumerTable();
            createEconomyRaterTable();
            createGoodsTable();
            createPurchaseTable();
```

```
            createProducerTable();
            createFactorInputTable();
            createStocksTable();
            createTradesTable();
        });
    }
}
```

SQLQueriesUsers.java

```java
package com.mycompany.Repository;

import org.springframework.stereotype.Repository;
import com.mycompany.Users.User;

@Repository
public abstract class SQLQueriesUsers<T extends User> extends SQLQueries {

    // Inserts a new user record into the users table
    public abstract void insertUserDataIntoTable(T user);

    // Retrieves a user record by ID
    public abstract T getData(int userID);

    // Authenticates a user login by checking email and password
    public abstract int authenticateLogin(String email, String password);

    // Checks if an email already exists in the database
    public abstract int doesEmailExist(String email);

    // Updates the user's password
    public abstract void updatePassword(String password, int userID);

    // Updates the user's stock budget
    public abstract void updateStockBudget(T user, Double stockBudget);

    // Checks if the user needs a stock budget update and updates
    public abstract void needToUpdateStockBudget(T user);
}
```

SQLQueriesConsumers.java

```
package com.mycompany.Repository;

import java.io.File;
import java.sql.Connection;
import java.sql.Date;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.time.LocalDate;

import org.springframework.stereotype.Repository;

import com.mycompany.Algorithms.FileHandling;
import com.mycompany.Users.Consumer;

@Repository
public class SQLQueriesConsumers extends SQLQueriesUsers<Consumer>{

    // Inserts a new consumer record into the Consumers table
    @Override
    public void insertUserDataIntoTable(Consumer consumer) {
        String query = "INSERT INTO Consumers (FirstName, LastName, Age, DateOfBirth, Email, PhoneNumber, Password)
VALUES (?, ?, ?, ?, ?, ?, ?)";

        try (Connection conn = DriverManager.getConnection(url, databaseUser, databasePassword);
            PreparedStatement pstmt = conn.prepareStatement(query)) {

            pstmt.setString(1, consumer.getFirstName());
```

```
            pstmt.setString(2, consumer.getLastName());
            pstmt.setInt(3, consumer.getAge());
            pstmt.setDate(4, Date.valueOf(consumer.getDateOfBirth()));
            pstmt.setString(5, consumer.getEmail());
            pstmt.setString(6, consumer.getPhoneNumber());
            pstmt.setString(7, consumer.getPassword());

            pstmt.executeUpdate();
        }

        catch (SQLException e) {
            System.out.println("Connection failed: " + e.getMessage());
        }
    }

    // Retrieves a consumer record by ID
    @Override
    public  Consumer getData(int consumerID) {
        String query = "SELECT * FROM Consumers WHERE ConsumerID = ?";

        try (Connection conn = DriverManager.getConnection(url, databaseUser, databasePassword);
            PreparedStatement pstmt = conn.prepareStatement(query)) {

            pstmt.setInt(1, consumerID);
            ResultSet rs = pstmt.executeQuery();

            if (rs.next()) {
                Consumer consumer = new Consumer(
                    rs.getInt("ConsumerID"),
                    rs.getString("FirstName"),
                    rs.getString("LastName"),
```

```
                    rs.getInt("Age"),
                    rs.getString("DateOfBirth"),
                    rs.getString("Email"),
                    rs.getString("PhoneNumber"),
                    rs.getString("Password"),
                    rs.getDouble("BudgetSize"),
                    rs.getDouble("CurrentBudget"),
                    rs.getDate("LastRenewed").toLocalDate(),
                    rs.getString("FrequencyOfRenewal"),
                    rs.getDouble("SustainableSpending"),
                    rs.getDouble("StockBudget")
                );

                return consumer;
            }

            else {
                System.out.println("No user found with ID: " + consumerID);
            }

        }

        catch (SQLException e) {
            System.out.println("Error retrieving user: " + e.getMessage());
        }

        return null;
    }

    // Authenticates a consumer login by checking email and password
    @Override
```

```
public int authenticateLogin(String email, String password) {
    String query = "SELECT ConsumerID FROM Consumers WHERE Email = ? AND Password = ?";

    try (Connection conn = DriverManager.getConnection(url, databaseUser, databasePassword);
        PreparedStatement pstmt = conn.prepareStatement(query)) {

        pstmt.setString(1, email);
        pstmt.setString(2, password);

        ResultSet rs = pstmt.executeQuery();

        if (rs.next()) {
            return rs.getInt("ConsumerID");
        }

        else {
            System.out.println("Invalid email or password.");
            return -1;
        }
    }

    catch (SQLException e) {
        System.out.println("Login failed: " + e.getMessage());
        return -1;
    }
}

// Checks if an email already exists in the database
@Override
public int doesEmailExist (String email) {
    String query = "SELECT Count(Email) AS Emails FROM Consumers WHERE Email = ?";
```

```
        try (Connection conn = DriverManager.getConnection(url, databaseUser, databasePassword);
            PreparedStatement pstmt = conn.prepareStatement(query)) {

            pstmt.setString(1, email);
            ResultSet rs = pstmt.executeQuery();

            if (rs.next()) {
                return rs.getInt("Emails");
            }
        }

        catch (SQLException e) {
            System.out.println("Connection failed: " + e.getMessage());
        }

        return 0;
    }

    // Updates the consumer's password
    @Override
    public void updatePassword(String password, int consumerID) {
        String query = "UPDATE Consumers SET Password = ? WHERE ConsumerID = ?";

        try (Connection conn = DriverManager.getConnection(url, databaseUser, databasePassword);
            PreparedStatement pstmt = conn.prepareStatement(query)) {

            pstmt.setString(1, password);
            pstmt.setInt(2, consumerID);
            pstmt.executeUpdate();
        }
```

```
        catch (SQLException e) {
            System.out.println("Connection failed: " + e.getMessage());
        }
    }

    // Updates the consumer's stock budget
    public void updateStockBudget(Consumer consumer,  Double stockBudget) {
        String query = "UPDATE Consumers SET StockBudget = ? WHERE ConsumerID = ?";

        try (Connection conn = DriverManager.getConnection(url, databaseUser, databasePassword);
            PreparedStatement pstmt = conn.prepareStatement(query)) {

            pstmt.setDouble(1, stockBudget);
            pstmt.setInt(2, consumer.getConsumerID());
            pstmt.executeUpdate();
        }

        catch (SQLException e) {
            System.out.println("Connection failed: " + e.getMessage());
        }
    }

    // Updates the consumer's overall budget size
    public void updateBudgetSize(Double budgetSize, Consumer consumer) {
        String query = "UPDATE Consumers SET BudgetSize = ? WHERE ConsumerID = ?";

        try (Connection conn = DriverManager.getConnection(url, databaseUser, databasePassword);
            PreparedStatement pstmt = conn.prepareStatement(query)) {

            pstmt.setDouble(1, budgetSize);
```

```
            pstmt.setInt(2, consumer.getConsumerID());
            pstmt.executeUpdate();
        }

        catch (SQLException e) {
            System.out.println("Connection failed: " + e.getMessage());
        }
    }

    // Updates the consumer's current budget
    public  void updateCurrentBudget(Double currentBudget, Consumer consumer) {
        String query = "UPDATE Consumers SET CurrentBudget = ? WHERE ConsumerID = ?";

        try (Connection conn = DriverManager.getConnection(url, databaseUser, databasePassword);
            PreparedStatement pstmt = conn.prepareStatement(query)) {

            pstmt.setDouble(1, currentBudget);
            pstmt.setInt(2, consumer.getConsumerID());
            pstmt.executeUpdate();
        }

        catch (SQLException e) {
            System.out.println("Connection failed: " + e.getMessage());
        }
    }

    // Updates the frequency of budget renewal for a consumer
    public void updateFrequencyOfBudgetRenewal(String frequencyOfRenewal, Consumer consumer) {
        String query = "UPDATE Consumers SET FrequencyOfRenewal = ? WHERE ConsumerID = ?";

        try (Connection conn = DriverManager.getConnection(url, databaseUser, databasePassword);
```

```
            PreparedStatement pstmt = conn.prepareStatement(query)) {

            pstmt.setString(1, frequencyOfRenewal);
            pstmt.setInt(2, consumer.getConsumerID());
            pstmt.executeUpdate();
        }

        catch (SQLException e) {
            System.out.println("Connection failed: " + e.getMessage());
        }
    }

    // Updates the last renewed date for a consumer
    public void updateLastRenewed(LocalDate lastRenewed, Consumer consumer) {
        String query = "UPDATE Consumers SET LastRenewed = ? WHERE ConsumerID = ?";

        try (Connection conn = DriverManager.getConnection(url, databaseUser, databasePassword);
            PreparedStatement pstmt = conn.prepareStatement(query)) {

            pstmt.setDate(1, Date.valueOf(lastRenewed));
            pstmt.setInt(2, consumer.getConsumerID());
            pstmt.executeUpdate();
        }

        catch (SQLException e) {
            System.out.println("Connection failed: " + e.getMessage());
        }
    }

    // Updates sustainable spending for a consumer
    public void updateSustainableSpending(Double sustainableSpending, Consumer consumer) {
```

```
        String query = "UPDATE Consumers SET SustainableSpending = ? WHERE ConsumerID = ?";

        try (Connection conn = DriverManager.getConnection(url, databaseUser, databasePassword);
            PreparedStatement pstmt = conn.prepareStatement(query)) {

            pstmt.setDouble(1, sustainableSpending);
            pstmt.setInt(2, consumer.getConsumerID());
            pstmt.executeUpdate();
        }

        catch (SQLException e) {
            System.out.println("Connection failed: " + e.getMessage());
        }
    }

    // Checks if the consumer needs a stock budget update and updates
    @Override
    public void needToUpdateStockBudget(Consumer consumer) {
        File file = new File("SQLSelectTableQueries/SelectNoNetQuantityConsumer.txt");

        FileHandling createTableTXTGetter = new FileHandling();
        String query = createTableTXTGetter.readFile(file);

        try (Connection conn = DriverManager.getConnection(url, databaseUser, databasePassword);
            PreparedStatement pstmt = conn.prepareStatement(query)) {

            pstmt.setInt(1, consumer.getConsumerID());
            ResultSet rs = pstmt.executeQuery();

            if (rs.next()) {
                // Update stock budget to default if condition is met
```

```
                updateStockBudget(consumer, 1000.00);
            }
        }

        catch (Exception ex) {
            ex.printStackTrace();
        }
    }
}
```

SQLQueriesProducers.java

```
package com.mycompany.Repository;

import java.io.File;
import java.sql.Connection;
import java.sql.Date;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;

import org.springframework.stereotype.Repository;

import com.mycompany.Algorithms.FileHandling;
import com.mycompany.Users.Producer;

@Repository
public class SQLQueriesProducers extends SQLQueriesUsers<Producer> {

    // Inserts a new producer into the Producers table
    @Override
    public void insertUserDataIntoTable(Producer producer) {
        String query = "INSERT INTO Producers (FirstName, LastName, Age, DateOfBirth, Email, PhoneNumber, Password)
VALUES (?, ?, ?, ?, ?, ?, ?)";

        try (Connection conn = DriverManager.getConnection(url, databaseUser, databasePassword);
            PreparedStatement pstmt = conn.prepareStatement(query)) {

            pstmt.setString(1, producer.getFirstName());
```

```
            pstmt.setString(2, producer.getLastName());
            pstmt.setInt(3, producer.getAge());
            pstmt.setDate(4, Date.valueOf(producer.getDateOfBirth()));
            pstmt.setString(5, producer.getEmail());
            pstmt.setString(6, producer.getPhoneNumber());
            pstmt.setString(7, producer.getPassword());

            pstmt.executeUpdate();
        }

        catch (SQLException e) {
            System.out.println("Connection failed: " + e.getMessage());
        }
    }

    // Retrieves a producer by their ID
    @Override
    public Producer getData(int producerID) {
        String query = "SELECT * FROM Producers WHERE ProducerID = ?";

        try (Connection conn = DriverManager.getConnection(url, databaseUser, databasePassword);
            PreparedStatement pstmt = conn.prepareStatement(query)) {

            pstmt.setInt(1, producerID);
            ResultSet rs = pstmt.executeQuery();

            if (rs.next()) {
                Producer producer = new Producer(
                    rs.getInt("ProducerID"),
                    rs.getString("FirstName"),
                    rs.getString("LastName"),
```

```
                    rs.getInt("Age"),
                    rs.getString("DateOfBirth"),
                    rs.getString("Email"),
                    rs.getString("PhoneNumber"),
                    rs.getString("Password"),
                    rs.getDouble("StockBudget")
                );

                return producer;
            }
        }

        catch (SQLException e) {
            System.out.println("Error retrieving user: " + e.getMessage());
        }

        return null;
    }

    // Authenticates login by email and password
    @Override
    public int authenticateLogin(String email, String password) {
        String query = "SELECT * FROM Producers WHERE Email = ? AND Password = ?";

        try (Connection conn = DriverManager.getConnection(url, databaseUser, databasePassword);
            PreparedStatement pstmt = conn.prepareStatement(query)) {

            pstmt.setString(1, email);
            pstmt.setString(2, password);

            ResultSet rs = pstmt.executeQuery();
```

```
            if (rs.next()) {
                int producerID = rs.getInt("ProducerID");
                return producerID;
            }
        }

        catch (SQLException e) {
            System.out.println("Login failed: " + e.getMessage());
        }

        return -1;
    }

    // Checks if an email already exists in the Producers table
    @Override
    public int doesEmailExist (String email) {
        String query = "SELECT Count(Email) AS Emails FROM Producers WHERE Email = ?";

        try (Connection conn = DriverManager.getConnection(url, databaseUser, databasePassword);
            PreparedStatement pstmt = conn.prepareStatement(query)) {

            pstmt.setString(1, email);

            ResultSet rs = pstmt.executeQuery();
            if (rs.next()) {
                return rs.getInt("Emails");
            }
        }

        catch (SQLException e) {
```

```
            System.out.println("Connection failed: " + e.getMessage());
        }

        return 0;
    }

    // Updates the password for a producer
    @Override
    public void updatePassword(String password, int producerID) {
        String query = "UPDATE Producers SET Password = ? WHERE ProducerID = ?";

        try (Connection conn = DriverManager.getConnection(url, databaseUser, databasePassword);
            PreparedStatement pstmt = conn.prepareStatement(query)) {

            pstmt.setString(1, password);
            pstmt.setInt(2, producerID);
            pstmt.executeUpdate();
        }

        catch (SQLException e) {
            System.out.println("Connection failed: " + e.getMessage());
        }
    }

    // Updates the stock budget for a producer
    @Override
    public void updateStockBudget(Producer producer, Double stockBudget) {
        String query = "UPDATE Producers SET StockBudget = ? WHERE ProducerID = ?";

        try (Connection conn = DriverManager.getConnection(url, databaseUser, databasePassword);
            PreparedStatement pstmt = conn.prepareStatement(query)) {
```

```
            pstmt.setDouble(1, stockBudget);
            pstmt.setInt(2, producer.getProducerID());
            pstmt.executeUpdate();
        }

        catch (SQLException e) {
            System.out.println("Connection failed: " + e.getMessage());
        }
    }

    // Checks if a producer needs a stock budget update based on external query
    @Override
    public void needToUpdateStockBudget(Producer producer) {
        File file = new File("SQLSelectTableQueries/SelectNoNetQuantityProducer.txt");

        FileHandling createTableTXTGetter = new FileHandling();
        String query = createTableTXTGetter.readFile(file);

        try (Connection conn = DriverManager.getConnection(url, databaseUser, databasePassword);
            PreparedStatement pstmt = conn.prepareStatement(query)) {

            pstmt.setInt(1, producer.getProducerID());
            ResultSet rs = pstmt.executeQuery();

            if (rs.next()) {
                // Update stock budget if condition is met
                updateStockBudget(producer, 1000.00);
            }
        }
        catch (Exception ex) {
```

```
            ex.printStackTrace();
        }
    }
}
```

SQLQueriesBudgetTracker.java

```
package com.mycompany.Repository;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.util.ArrayList;

import org.springframework.stereotype.Repository;

import com.mycompany.Records.Purchase;
import com.mycompany.Users.Consumer;

@Repository
public class SQLQueriesBudgetTracker extends SQLQueries {

    // Fetches the purchase prices and quantities for a consumer on a specific day (daysToSubtract from today)
    public static ArrayList<ArrayList<Double>> getLastWeekPurchases(Consumer consumer, int daysToSubtract) {
        String query = "SELECT Purchase.QuantityBought, Goods.GoodPrice " +
                       "FROM Purchase, Goods, Consumers " +
                       "WHERE Purchase.GoodID = Goods.GoodID AND Purchase.ConsumerID = Consumers.ConsumerID " +
                       "AND Consumers.ConsumerID = ? " +
                       "AND CONVERT(DATE, Purchase.PurchaseDate) = ? " +
                       "ORDER BY Purchase.PurchaseDate";

        try (Connection conn = DriverManager.getConnection(url, databaseUser, databasePassword);
```

```
            PreparedStatement pstmt = conn.prepareStatement(query)) {

            DateTimeFormatter df = DateTimeFormatter.ofPattern("yyyy-MM-dd");
            pstmt.setInt(1, consumer.getConsumerID());
            pstmt.setString(2, LocalDate.now().minusDays(daysToSubtract).format(df));

            ArrayList<ArrayList<Double>> priceQuantityPairs = new ArrayList<>();
            ArrayList<Double> prices = new ArrayList<>();
            ArrayList<Double> quantities = new ArrayList<>();

            ResultSet rs = pstmt.executeQuery();
            while (rs.next()) {
                prices.add(rs.getDouble("GoodPrice"));
                quantities.add(rs.getDouble("QuantityBought"));
            }

            priceQuantityPairs.add(prices);
            priceQuantityPairs.add(quantities);

            return priceQuantityPairs;
        }

        catch (SQLException ex) {
            System.out.println("Connection failed: " + ex.getMessage());
        }

        return null;
    }

    // Retrieves all purchases of a consumer, ordered by purchase date descending
    public static ArrayList<Purchase> getConsumerPurchase(Consumer consumer) {
```

```
        String query = "SELECT Goods.GoodName, Goods.GoodPrice, Purchase.QuantityBought, Purchase.PurchaseDate " +
                       "FROM Consumers, Goods, Purchase " +
                       "WHERE Consumers.ConsumerID = Purchase.ConsumerID AND Goods.GoodID = Purchase.GoodID " +
                       "AND Consumers.ConsumerID = ? " +
                       "ORDER BY Purchase.PurchaseDate DESC";

        ArrayList<Purchase> consumerPurchases = new ArrayList<>();

        try (Connection conn = DriverManager.getConnection(url, databaseUser, databasePassword);
            PreparedStatement pstmt = conn.prepareStatement(query)) {

            pstmt.setInt(1, consumer.getConsumerID());
            ResultSet rs = pstmt.executeQuery();
            // Map ResultSet rows to Purchase objects
            while (rs.next()) {
                consumerPurchases.add(new Purchase(
                    rs.getString("GoodName"),
                    rs.getDouble("GoodPrice"),
                    rs.getInt("QuantityBought"),
                    rs.getDate("PurchaseDate").toString()
                ));
            }
        }

        catch (SQLException e) {
            System.out.println("Connection failed: " + e.getMessage());
        }

        return consumerPurchases;
    }
}
```

SQLQueriesFactorInput.java

```
package com.mycompany.Repository;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
import java.util.ArrayList;
import java.util.LinkedHashMap;
import org.springframework.stereotype.Repository;
import com.mycompany.Records.FactorInput;
import com.mycompany.Users.Producer;

@Repository
public class SQLQueriesFactorInput extends SQLQueries {

    // Inserts a new FactorInput record into the database for a given producer
    public static void insertDataIntoFactorInputTable(FactorInput factorInput, Producer producer) {
        String query = "INSERT INTO FactorInput (FactorInputType, ProducerID, FactorInputName, FactorInputDescription,
MarginalCost, MarginalRevenue, MarginalProfit) VALUES (?, ?, ?, ?, ?, ?, ?)";

        try (Connection conn = DriverManager.getConnection(url, databaseUser, databasePassword);
            PreparedStatement pstmt = conn.prepareStatement(query, Statement.RETURN_GENERATED_KEYS)) {

                pstmt.setString(1, factorInput.getFactorInputType());
                pstmt.setInt(2, producer.getProducerID());
                pstmt.setString(3, factorInput.getFactorInputName());
                pstmt.setString(4, factorInput.getFactorInputDescription());
```

```
            pstmt.setDouble(5, factorInput.getMarginalCost());
            pstmt.setDouble(6, factorInput.getMarginalRevenue());
            pstmt.setDouble(7, factorInput.getMarginalProfit());

            pstmt.executeUpdate();
        }

        catch (SQLException e) {
            System.out.println("Connection failed: " + e.getMessage());
        }
    }

    // Retrieves all FactorInput records for a given producer
    public static LinkedHashMap<String, FactorInput> getAllFactorInputTable(Producer producer) {
        String query = "SELECT * FROM FactorInput WHERE ProducerID = ?";
        LinkedHashMap<String, FactorInput> factorInputs = new LinkedHashMap<>();

        try (Connection conn = DriverManager.getConnection(url, databaseUser, databasePassword);
            PreparedStatement pstmt = conn.prepareStatement(query)) {

            pstmt.setInt(1, producer.getProducerID());
            ResultSet rs = pstmt.executeQuery();

            while (rs.next()) {
                FactorInput factorInput = new FactorInput(
                    rs.getInt("FactorInputID"),
                    rs.getString("FactorInputType"),
                    rs.getString("FactorInputName"),
                    rs.getString("FactorInputDescription"),
                    rs.getDouble("MarginalCost"),
                    rs.getDouble("MarginalRevenue")
```

```
                    );

                    factorInputs.put(factorInput.getFactorInputType() + " - " + factorInput.getFactorInputName(),
factorInput);
                }

                return factorInputs;
            }

            catch (SQLException e) {
                System.out.println("Connection failed: " + e.getMessage());
            }

            return null;
        }

        // Retrieve a single FactorInput by name for a producer
        public static FactorInput getFactorInputByName(Producer producer, String factorInputName) {
            String query = "SELECT * FROM FactorInput WHERE ProducerID = ? AND FactorInputName = ?";

            try (Connection conn = DriverManager.getConnection(url, databaseUser, databasePassword);
                PreparedStatement pstmt = conn.prepareStatement(query)) {

                    pstmt.setInt(1, producer.getProducerID());
                    pstmt.setString(2, factorInputName);
                    ResultSet rs = pstmt.executeQuery();
                    if (rs.next()) {
                        return new FactorInput(
                            rs.getInt("FactorInputID"),
                            rs.getString("FactorInputType"),
                            rs.getString("FactorInputName"),
```

```
                        rs.getString("FactorInputDescription"),
                        rs.getDouble("MarginalCost"),
                        rs.getDouble("MarginalRevenue")
                    );
                }
            }

            catch (SQLException e) {
                System.out.println("Connection failed: " + e.getMessage());
            }

            return null;
        }

        // Count the number of FactorInputs of a specific type for a producer
        public static int getNumberOfFactorInputs(String factorInputType, Producer producer) {
            String query = "SELECT COUNT (FactorInputName) AS NumberOfFactorInputs FROM FactorInput WHERE FactorInputType
    = ? AND ProducerID = ?";

            try (Connection conn = DriverManager.getConnection(url, databaseUser, databasePassword);
                PreparedStatement pstmt = conn.prepareStatement(query)) {

                    pstmt.setString(1, factorInputType);
                    pstmt.setInt(2, producer.getProducerID());
                    ResultSet rs = pstmt.executeQuery();
                    if (rs.next()) {
                        return rs.getInt("NumberOfFactorInputs");
                    }
            }

            catch (SQLException e) {
```

```
            System.out.println("Connection failed: " + e.getMessage());
        }

        return 0;
    }

    // Get total cost for a producer
    public static double getTotalCost(int producerID) {
        String query = "SELECT SUM(MarginalCost) AS TotalCost FROM FactorInput WHERE ProducerID = ?";

        try (Connection conn = DriverManager.getConnection(url, databaseUser, databasePassword);
            PreparedStatement pstmt = conn.prepareStatement(query)) {

                pstmt.setInt(1, producerID);
                ResultSet rs = pstmt.executeQuery();
                if (rs.next()) {
                    return rs.getDouble("TotalCost");
                }
        }

        catch (SQLException e) {
            System.out.println("Connection failed: " + e.getMessage());
        }

        return Integer.MIN_VALUE;
    }

    // Get total revenue for a producer
    public static double getTotalRevenue(int producerID) {
        String query = "SELECT SUM(MarginalRevenue) AS TotalRevenue FROM FactorInput WHERE ProducerID = ?";
```

```
    try (Connection conn = DriverManager.getConnection(url, databaseUser, databasePassword);
         PreparedStatement pstmt = conn.prepareStatement(query)) {

            pstmt.setInt(1, producerID);
            ResultSet rs = pstmt.executeQuery();
            if (rs.next()) {
                return rs.getDouble("TotalRevenue");
            }
    }

    catch (SQLException e) {
        System.out.println("Connection failed: " + e.getMessage());
    }

    return Integer.MIN_VALUE;
}

// Get total profit for a producer
public static double getTotalProfit(int producerID) {
    String query = "SELECT SUM(MarginalProfit) AS TotalProfit FROM FactorInput WHERE ProducerID = ?";

    try (Connection conn = DriverManager.getConnection(url, databaseUser, databasePassword);
         PreparedStatement pstmt = conn.prepareStatement(query)) {

            pstmt.setInt(1, producerID);
            ResultSet rs = pstmt.executeQuery();
            if (rs.next()) {
                return rs.getDouble("TotalProfit");
            }
    }
```

```
        catch (SQLException e) {
            System.out.println("Connection failed: " + e.getMessage());
        }

        return Integer.MIN_VALUE;
    }

    // Update FactorInput's costs and revenue, and recalculate profit
    public static void updateDataIntoFactorInputTable(double marginalCost, double marginalRevenue, String
factorInputName, Producer producer) {
        String query = "UPDATE FactorInput SET MarginalCost = ?, MarginalRevenue = ?, MarginalProfit = ? WHERE
FactorInputName = ? AND ProducerID = ?";

        try (Connection conn = DriverManager.getConnection(url, databaseUser, databasePassword);
             PreparedStatement pstmt = conn.prepareStatement(query, Statement.RETURN_GENERATED_KEYS)) {

                pstmt.setDouble(1, marginalCost);
                pstmt.setDouble(2, marginalRevenue);
                pstmt.setDouble(3, marginalRevenue - marginalCost); // Profit = Revenue - Cost
                pstmt.setString(4, factorInputName);
                pstmt.setInt(5, producer.getProducerID());
                pstmt.executeUpdate();
        }

        catch (SQLException e) {
            System.out.println("Connection failed: " + e.getMessage());
        }
    }

    // Delete a FactorInput record for a producer
    public static int deleteDataFromFactorInputTable(String factorInputName, Producer producer) {
```

```
    String query = "DELETE FROM FactorInput WHERE FactorInputName = ? AND ProducerID = ?";

    try (Connection conn = DriverManager.getConnection(url, databaseUser, databasePassword);
         PreparedStatement pstmt = conn.prepareStatement(query)) {

            pstmt.setString(1, factorInputName);
            pstmt.setInt(2, producer.getProducerID());
            pstmt.executeUpdate();
    }

    catch (SQLException e) {
        System.out.println("Connection failed: " + e.getMessage());
    }

    return -1;
}

// Get all marginal costs for a producer
public static ArrayList<Double> getAllMarginalCosts(Producer producer) {
    String query = "SELECT MarginalCost FROM FactorInput WHERE ProducerID = ?";
    ArrayList<Double> allMarginalCosts = new ArrayList<>();

    try (Connection conn = DriverManager.getConnection(url, databaseUser, databasePassword);
         PreparedStatement pstmt = conn.prepareStatement(query)) {

            pstmt.setInt(1, producer.getProducerID());
            ResultSet rs = pstmt.executeQuery();
            while (rs.next()) {
                allMarginalCosts.add(rs.getDouble("MarginalCost"));
            }
    }
```

```
        catch (SQLException e) {
            System.out.println("Connection failed: " + e.getMessage());
        }

        return allMarginalCosts;
    }

    // Get all marginal revenues for a producer
    public static ArrayList<Double> getAllMarginalRevenue(Producer producer) {
        String query = "SELECT MarginalRevenue FROM FactorInput WHERE ProducerID = ?";
        ArrayList<Double> allMarginalRevenue = new ArrayList<>();

        try (Connection conn = DriverManager.getConnection(url, databaseUser, databasePassword);
             PreparedStatement pstmt = conn.prepareStatement(query)) {

                pstmt.setInt(1, producer.getProducerID());
                ResultSet rs = pstmt.executeQuery();
                while (rs.next()) {
                    allMarginalRevenue.add(rs.getDouble("MarginalRevenue"));
                }
        }

        catch (SQLException e) {
            System.out.println("Connection failed: " + e.getMessage());
        }

        return allMarginalRevenue;
    }
}
```

SQLQueriesGoods.java

```java
package com.mycompany.Repository;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import org.springframework.stereotype.Repository;

@Repository
public class SQLQueriesGoods extends SQLQueries {

    // Inserts a new good into the Goods table
    public static void insertGood(String goodName, Double goodPrice) {
        String query = "INSERT INTO Goods (GoodName, GoodPrice) VALUES (?, ?)";

        try (Connection conn = DriverManager.getConnection(url, databaseUser, databasePassword);
            PreparedStatement pstmt = conn.prepareStatement(query)) {

            pstmt.setString(1, goodName);
            pstmt.setDouble(2, goodPrice);

            pstmt.executeUpdate();
        }

        catch (SQLException e) {
            System.out.println("Connection failed: " + e.getMessage());
        }
    }
```

```java
// Checks if a good with the given name and price already exists
public static boolean doesGoodIDExist(String goodName, Double goodPrice) {
    String query = "SELECT COUNT(*) AS Count FROM Goods WHERE GoodName = ? AND GoodPrice = ?";

    try (Connection conn = DriverManager.getConnection(url, databaseUser, databasePassword);
        PreparedStatement pstmt = conn.prepareStatement(query)) {

        pstmt.setString(1, goodName);
        pstmt.setDouble(2, goodPrice);

        ResultSet rs = pstmt.executeQuery();
        if (rs.next()) {
            int count = rs.getInt("Count");
            return count >= 1;
        }
    }

    catch (SQLException e) {
        System.out.println("Connection failed: " + e.getMessage());
    }

    return false;
}

// Retrieves the GoodID for a specific good by name and price
public static int getGoodID(String goodName, Double goodPrice) {
    String query = "SELECT GoodID FROM Goods WHERE GoodName = ? AND GoodPrice = ?";

    try (Connection conn = DriverManager.getConnection(url, databaseUser, databasePassword);
        PreparedStatement pstmt = conn.prepareStatement(query)) {
```

```
            pstmt.setString(1, goodName);
            pstmt.setDouble(2, goodPrice);

            ResultSet rs = pstmt.executeQuery();

            if (rs.next()) {
                int goodID = rs.getInt("GoodID");
                return goodID;
            }
        }

        catch (SQLException e) {
            System.out.println("Connection failed: " + e.getMessage());
        }

        return -1;
    }
}
```

SQLQueriesPurchase.java

```java
package com.mycompany.Repository;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.time.LocalDate;
import org.springframework.stereotype.Repository;
import com.mycompany.Records.GoodRecord;
import com.mycompany.Users.Consumer;

@Repository
public class SQLQueriesPurchase extends SQLQueries {

    // Inserts a new purchase record into the Purchase table
    public static void insertPurchase(Consumer consumer, GoodRecord good, Integer quantity) {
        String query = "INSERT INTO Purchase (ConsumerID, GoodID, QuantityBought) VALUES (?, ?, ?)";

        try (Connection conn = DriverManager.getConnection(url, databaseUser, databasePassword);
            PreparedStatement pstmt = conn.prepareStatement(query)) {

            pstmt.setInt(1, consumer.getConsumerID());
            pstmt.setInt(2, good.goodID());
            pstmt.setInt(3, quantity);
            pstmt.executeUpdate();
        }
        catch (SQLException e) {
            System.out.println("Connection failed: " + e.getMessage());
```

```
        }
    }

    // Calculates the total spending for a consumer today
    public static Double showTodaysSpending(Consumer consumer) {
        String query = "SELECT SUM(Goods.GoodPrice * Purchase.QuantityBought) AS TotalSpending " +
                       "FROM Purchase, Goods, Consumers " +
                       "WHERE Purchase.PurchaseDate >= ? AND Purchase.PurchaseDate < ? " +
                       "AND Goods.GoodID = Purchase.GoodID " +
                       "AND Purchase.ConsumerID = Consumers.ConsumerID " +
                       "AND Consumers.ConsumerID = ?";

        try (Connection conn = DriverManager.getConnection(url, databaseUser, databasePassword);
             PreparedStatement pstmt = conn.prepareStatement(query)) {

            pstmt.setDate(1, java.sql.Date.valueOf(LocalDate.now()));
            pstmt.setDate(2, java.sql.Date.valueOf(LocalDate.now().plusDays(1)));
            pstmt.setInt(3, consumer.getConsumerID());

            ResultSet rs = pstmt.executeQuery();
            if (rs.next()) {
                return rs.getDouble("TotalSpending");
            }
        }
        catch (SQLException e) {
            System.out.println("Connection failed: " + e.getMessage());
        }

        return null;
    }
}
```

SQLQueriesRateTheEconomy.java

```java
package com.mycompany.Repository;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.time.LocalDate;
import java.util.LinkedHashMap;
import org.springframework.stereotype.Repository;
import com.mycompany.Users.Consumer;

@Repository
public class SQLQueriesRateTheEconomy extends SQLQueries {

    // Inserts a consumer rating into the EconomyRater table
    public static void insertDataIntoEconomyRaterTable(Consumer consumer, String fileName, int consumerOpinion) {
        String query = "INSERT INTO EconomyRater (ConsumerID, ConsumerOpinionFile, ConsumerOpinion) VALUES (?, ?, ?)";

        try (Connection conn = DriverManager.getConnection(url, databaseUser, databasePassword);
            PreparedStatement pstmt = conn.prepareStatement(query)) {

            pstmt.setInt(1, consumer.getConsumerID());
            pstmt.setString(2, fileName);
            pstmt.setInt(3, consumerOpinion);

            pstmt.executeUpdate();
        }
```

```
        catch (SQLException e) {
            System.out.println("Connection failed: " + e.getMessage());
        }
    }

    // Returns the average consumer opinion for a given date
    public static double averageConsumerOpinion(String date) {
        String query = "SELECT AVG(ConsumerOpinion) AS AverageConsumerOpinion FROM EconomyRater WHERE
ConsumerOpinionDate = ?";

        try (Connection conn = DriverManager.getConnection(url, databaseUser, databasePassword);
            PreparedStatement pstmt = conn.prepareStatement(query)) {

            pstmt.setString(1, date);

            ResultSet rs = pstmt.executeQuery();
            if (rs.next()) {
                return rs.getDouble("AverageConsumerOpinion");
            }
        }

        catch (SQLException e) {
            System.out.println("Connection failed: " + e.getMessage());
        }

        return -1;
    }

    // Checks if a consumer has already rated today; returns the RatingID if exists
    public static int checkNumberOfRatings(Consumer consumer) {
        String query = "SELECT COUNT(RatingID) AS RecordNumber, RatingID FROM EconomyRater WHERE ConsumerID = ? AND
```

```
ConsumerOpinionDate = ? GROUP BY RatingID";

        try (Connection conn = DriverManager.getConnection(url, databaseUser, databasePassword);
            PreparedStatement pstmt = conn.prepareStatement(query)) {

            pstmt.setInt(1, consumer.getConsumerID());
            pstmt.setString(2, LocalDate.now().toString());

            ResultSet rs = pstmt.executeQuery();
            if (rs.next()) {
                if (rs.getInt("RecordNumber") == 1) {
                    return rs.getInt("RatingID");
                }
            }
        }

        catch (SQLException e) {
            System.out.println("Connection failed: " + e.getMessage());
        }

        return -1;
    }

    // Updates an existing consumer opinion in the EconomyRater table
    public static void updateConsumerOpinion(int ratingID, int consumerOpinion) {
        String query = "UPDATE EconomyRater SET ConsumerOpinion = ? WHERE RatingID = ?";

        try (Connection conn = DriverManager.getConnection(url, databaseUser, databasePassword);
            PreparedStatement pstmt = conn.prepareStatement(query)) {

            pstmt.setInt(1, consumerOpinion);
```

```
            pstmt.setInt(2, ratingID);

            pstmt.executeUpdate();
        }

        catch (SQLException e) {
            System.out.println("Connection failed: " + e.getMessage());
        }
    }

    // Retrieves the latest consumer opinion for a given consumer
    public static LinkedHashMap<String, String> getLatestConsumerOpinion(Consumer consumer) {
        String query = "SELECT ConsumerOpinion, ConsumerOpinionDate FROM EconomyRater, Consumers " +
                       "WHERE Consumers.ConsumerID = EconomyRater.ConsumerID AND EconomyRater.ConsumerID = ? " +
                       "ORDER BY ConsumerOpinionDate DESC";

        try (Connection conn = DriverManager.getConnection(url, databaseUser, databasePassword);
            PreparedStatement pstmt = conn.prepareStatement(query)) {

            pstmt.setInt(1, consumer.getConsumerID());

            ResultSet rs = pstmt.executeQuery();
            if (rs.next()) {
                LinkedHashMap<String, String> latestConsumerOpinionMap = new LinkedHashMap<>();
                latestConsumerOpinionMap.put("consumerOpinionDate", rs.getString("ConsumerOpinionDate"));
                latestConsumerOpinionMap.put("consumerOpinion", rs.getString("ConsumerOpinion"));
                return latestConsumerOpinionMap;
            }
        }

        catch (SQLException e) {
```

```
            System.out.println("Connection failed: " + e.getMessage());
        }

        return new LinkedHashMap<>();
    }
}
```

SQLQueriesStocks.java

```java
package com.mycompany.Repository;

import java.io.File;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import org.springframework.stereotype.Repository;
import com.fasterxml.jackson.core.type.TypeReference;
import com.mycompany.Algorithms.FileHandlingJSON;
import com.mycompany.PythonFlaskAPI.APIQueue;
import com.mycompany.PythonFlaskAPI.PythonFlaskAPI;

@Repository
public class SQLQueriesStocks extends SQLQueries {

    private static final FileHandlingJSON JSONHandling = new FileHandlingJSON();
    private static final APIQueue taskManager = new APIQueue(5);

    // Inserts a new stock into the Stocks table. If the stock already exists, calls updateStock to update the price
    public static void insertStock(String stockName, Double askPrice) {
        String query = "INSERT INTO Stocks (StockName, AskPrice) VALUES (?, ?)";

        try (Connection conn = DriverManager.getConnection(url, databaseUser, databasePassword);
```

```
            PreparedStatement pstmt = conn.prepareStatement(query)) {

            pstmt.setString(1, stockName);
            pstmt.setDouble(2, askPrice);
            pstmt.executeUpdate();
        }

        catch (SQLException e) {
            // If insertion fails, update the existing stock price
            updateStock(stockName, askPrice);
        }
    }

    // Reads all stock info from a JSON file and inserts them into the Stocks table
    public static void insertAllStock() {
        File tickerInfo = new File("tickerInfo.json");

        // Read JSON array of stock information
        ArrayList<LinkedHashMap<String, String>> allTickerInfoAsJSON = JSONHandling.readFileList(tickerInfo, new
TypeReference<List<LinkedHashMap<String, String>>>() {});

        // Loop through all entries and insert each stock
        for (int i = 0; i < allTickerInfoAsJSON.size(); i++) {
            final String companyName = allTickerInfoAsJSON.get(i).get("companyName");
            final String tickerSymbol = allTickerInfoAsJSON.get(i).get("tickerSymbol");

            taskManager.submitTask(() -> {
                Double askPrice = PythonFlaskAPI.getAskPrice(tickerSymbol);
                insertStock(companyName, askPrice);
            });
        }
```

```
}

// Updates an existing stock's ask price and last updated timestamp
public static void updateStock(String stockName, Double askPrice) {
    String query = "UPDATE Stocks SET AskPrice = ?, LastUpdated = ? WHERE StockName = ?";

    try (Connection conn = DriverManager.getConnection(url, databaseUser, databasePassword);
        PreparedStatement pstmt = conn.prepareStatement(query)) {

        pstmt.setDouble(1, askPrice);
        pstmt.setTimestamp(2, java.sql.Timestamp.valueOf(LocalDateTime.now()));
        pstmt.setString(3, stockName);

        pstmt.executeUpdate();
    }

    catch (SQLException e) {
        System.out.println("Connection failed: " + e.getMessage());
    }
}

// Retrieves the StockID of a stock by its name
public static Integer getStockID(String stockName) {
    String query = "SELECT StockID FROM Stocks WHERE StockName = ?";

    try (Connection conn = DriverManager.getConnection(url, databaseUser, databasePassword);
        PreparedStatement pstmt = conn.prepareStatement(query)) {

        pstmt.setString(1, stockName);

        ResultSet rs = pstmt.executeQuery();
```

```
            if (rs.next()) {
                return rs.getInt("StockID");
            }
        }

        catch (SQLException e) {
            System.out.println("Connection failed: " + e.getMessage());
        }

        return null;
    }
}
```

SQLQueriesTrade.java

```
package com.mycompany.Repository;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;

import org.springframework.stereotype.Repository;

import com.mycompany.Users.Consumer;
import com.mycompany.Users.Producer;

@Repository
public class SQLQueriesTrade extends SQLQueries {

    //Inserts a trade (buy or sell) into the Trades table. Can handle trades for either a consumer or a producer.
    public static void insertTrade(Consumer consumer, Producer producer, Integer stockID, String tradeType, Double
askPrice, Integer quantity) {
        int userID;
        String query;

        // Determine whether this trade is for a producer or a consumer
        if (consumer == null) {
            userID = producer.getProducerID();
            query = "INSERT INTO Trades (ProducerID, StockID, TradeType, AskPrice, Quantity) VALUES (?, ?, ?, ?, ?)";
        }
```

```
        else {
            userID = consumer.getConsumerID();
            query = "INSERT INTO Trades (ConsumerID, StockID, TradeType, AskPrice, Quantity) VALUES (?, ?, ?, ?, ?)";
        }

        // Execute the trade insertion
        try (Connection conn = DriverManager.getConnection(url, databaseUser, databasePassword);
            PreparedStatement pstmt = conn.prepareStatement(query)) {

            pstmt.setInt(1, userID);
            pstmt.setInt(2, stockID);
            pstmt.setString(3, tradeType);
            pstmt.setDouble(4, askPrice);
            pstmt.setInt(5, quantity);
            pstmt.executeUpdate();
        }

        catch (SQLException e) {
            System.out.println("Connection failed: " + e.getMessage());
        }
    }

    //Retrieves the current stock holdings for a user (consumer or producer). Calculates net quantity, total cost,
average ask price, and profit per stock.
    public static ArrayList<ArrayList<Object>> getCurrentUserStock(Consumer consumer, Producer producer) {
        int userID;
        String query;
        ArrayList<ArrayList<Object>> trades = new ArrayList<>();

        // Build the correct query depending on user type
        if (consumer == null) {
```

```
            userID = producer.getProducerID();
            query = "SELECT Stocks.StockName, " +
                    "SUM(Trades.Quantity * IIF(Trades.TradeType = 'BUY', 1, -1)) AS NetQuantity, " +
                    "SUM(Trades.Quantity * IIF(Trades.TradeType = 'BUY', 1, -1) * Trades.AskPrice) AS TotalAskPrice, "
+
                    "AVG(Trades.AskPrice) AS AverageAskPrice, " +
                    "AVG(Stocks.AskPrice) - AVG(Trades.AskPrice) AS Profit " +
                    "FROM Trades, Stocks, Producers " +
                    "WHERE Stocks.StockID = Trades.StockID AND Producers.ProducerID = ? AND Producers.ProducerID =
Trades.ProducerID " +
                    "GROUP BY Trades.ProducerID, Stocks.StockName " +
                    "HAVING SUM(Trades.Quantity * IIF(Trades.TradeType = 'BUY', 1, -1)) > 0 " +
                    "ORDER BY Trades.ProducerID, Stocks.StockName";
        }

        else {
            userID = consumer.getConsumerID();
            query = "SELECT Stocks.StockName, " +
                    "SUM(Trades.Quantity * IIF(Trades.TradeType = 'BUY', 1, -1)) AS NetQuantity, " +
                    "SUM(Trades.Quantity * IIF(Trades.TradeType = 'BUY', 1, -1) * Trades.AskPrice) AS TotalAskPrice, "
+
                    "AVG(Trades.AskPrice) AS AverageAskPrice, " +
                    "AVG(Stocks.AskPrice) - AVG(Trades.AskPrice) AS Profit " +
                    "FROM Trades, Stocks, Consumers " +
                    "WHERE Stocks.StockID = Trades.StockID AND Consumers.ConsumerID = ? AND Consumers.ConsumerID =
Trades.ConsumerID " +
                    "GROUP BY Trades.ConsumerID, Stocks.StockName " +
                    "HAVING SUM(Trades.Quantity * IIF(Trades.TradeType = 'BUY', 1, -1)) > 0 " +
                    "ORDER BY Trades.ConsumerID, Stocks.StockName";
        }
```

```
        // Execute query and populate results
        try (Connection conn = DriverManager.getConnection(url, databaseUser, databasePassword);
            PreparedStatement pstmt = conn.prepareStatement(query)) {

            pstmt.setInt(1, userID);
            ResultSet rs = pstmt.executeQuery();

            while (rs.next()) {
                ArrayList<Object> tempArrayList = new ArrayList<>();
                tempArrayList.add(rs.getString("StockName"));
                tempArrayList.add(rs.getInt("NetQuantity"));
                tempArrayList.add(rs.getDouble("TotalAskPrice"));
                tempArrayList.add(rs.getDouble("AverageAskPrice"));
                tempArrayList.add(rs.getDouble("Profit"));
                trades.add(tempArrayList);
            }
        }

        catch (SQLException e) {
            System.out.println("Connection failed: " + e.getMessage());
        }

        return trades;
    }
}
```

Users Package

UserInterface.java

```
package com.mycompany.Users;
public interface UserInterface {

    public String getFirstName();
    public void setFirstName(String firstName);

    public String getLastName();
    public void setLastName(String lastName);

    public Integer getAge();
    public void setAge(Integer age);

    public String getDateOfBirth();
    public void setDateOfBirth(String dateOfBirth);

    public String getEmail();
    public void setEmail(String email);

    public String getPassword();
    public void setPassword(String password);

    public String getPhoneNumber();
    public void setPhoneNumber(String phoneNumber);

    public Double getStockBudget();
    public void setStockBudget(Double stockBudget);
}
```

User.java

```
package com.mycompany.Users;
public abstract class User implements UserInterface {
    protected String firstName;
    protected String lastName;
    protected Integer age;
    protected String dateOfBirth;
    protected String email;
    protected String phoneNumber;
    protected String password;
    protected Double stockBudget;

    @Override
    public String getFirstName() {
        return firstName;
    }

    @Override
    public void setFirstName(String firstName) {
        this.firstName = firstName;
    }

    @Override
    public String getLastName() {
        return lastName;
    }

    @Override
    public void setLastName(String lastName) {
        this.lastName = lastName;
```

```
    }

    @Override
    public Integer getAge() {
        return age;
    }

    @Override
    public void setAge(Integer age) {
        this.age = age;
    }

    @Override
    public String getDateOfBirth() {
        return dateOfBirth;
    }

    @Override
    public void setDateOfBirth(String dateOfBirth) {
        this.dateOfBirth = dateOfBirth;
    }

    @Override
    public String getEmail() {
        return email;
    }

    @Override
    public void setEmail(String email) {
        this.email = email;
    }
```

```
    @Override
    public String getPhoneNumber() {
        return phoneNumber;
    }

    @Override
    public void setPhoneNumber(String phoneNumber) {
        this.phoneNumber = phoneNumber;
    }

    @Override
    public String getPassword() {
        return password;
    }

    @Override
    public void setPassword(String password) {
        this.password = password;
    }

    @Override
    public Double getStockBudget() {
        return stockBudget;
    }

    @Override
    public void setStockBudget(Double stockBudget) {
        this.stockBudget = stockBudget;
    }
}
```

Consumer.java

```
package com.mycompany.Users;

import java.time.LocalDate;

public class Consumer extends User {

    private Integer consumerID;
    private Double budgetSize;
    private Double currentBudget;
    private LocalDate lastRenewed;
    private String frequencyOfBudgetRenewal;
    private Double sustainableSpending;

    public Integer getConsumerID() {
        return consumerID;
    }

    public void setConsumerID(Integer consumerID) {
        this.consumerID = consumerID;
    }

    public Double getBudgetSize() {
        return budgetSize;
    }

    public void setBudgetSize(Double budgetSize) {
        this.budgetSize = budgetSize;
    }
```

```
public String getFrequencyOfBudgetRenewal() {
    return frequencyOfBudgetRenewal;
}

public void setFrequencyOfBudgetRenewal(String frequencyOfBudgetRenewal) {
    this.frequencyOfBudgetRenewal = frequencyOfBudgetRenewal;
}

public Double getSustainableSpending() {
    return sustainableSpending;
}

public void setSustainableSpending(Double sustainableSpending) {
    this.sustainableSpending = sustainableSpending;
}

public Double getCurrentBudget() {
    return currentBudget;
}

public void setCurrentBudget(Double currentBudget) {
    this.currentBudget = currentBudget;
}

public LocalDate getLastRenewed() {
    return lastRenewed;
}

public void setLastRenewed(LocalDate lastRenewed) {
    this.lastRenewed = lastRenewed;
}
```

```
    public Consumer () {

    }

    public Consumer(String firstName, String lastName, Integer age, String dateOfBirth, String email, String
phoneNumber, String password) {
        this.firstName = firstName;
        this.lastName = lastName;
        this.age = age;
        this.dateOfBirth = dateOfBirth;
        this.email = email;
        this.phoneNumber = phoneNumber;
        this.password = password;
    }

    public Consumer(Integer consumerID, String firstName, String lastName, Integer age, String dateOfBirth, String
email, String phoneNumber, String password) {
        this.consumerID = consumerID;
        this.firstName = firstName;
        this.lastName = lastName;
        this.age = age;
        this.dateOfBirth = dateOfBirth;
        this.email = email;
        this.phoneNumber = phoneNumber;
        this.password = password;
    }

    public Consumer(Integer consumerID,  String firstName, String lastName, Integer age, String dateOfBirth,
            String email, String phoneNumber, String password, Double budgetSize, Double currentBudget,
            LocalDate lastRenewed, String frequencyOfBudgetRenewal, Double sustainableSpending, Double stockBudget) {
```

```
        this.consumerID = consumerID;
        this.firstName = firstName;
        this.lastName = lastName;
        this.age = age;
        this.dateOfBirth = dateOfBirth;
        this.email = email;
        this.phoneNumber = phoneNumber;
        this.password = password;
        this.budgetSize = budgetSize;
        this.currentBudget = currentBudget;
        this.lastRenewed = lastRenewed;
        this.frequencyOfBudgetRenewal = frequencyOfBudgetRenewal;
        this.sustainableSpending = sustainableSpending;
        this.stockBudget = stockBudget;
    }
}
```

Producer.java

```
package com.mycompany.Users;

public class Producer extends User {

    private Integer producerID;

    public Integer getProducerID() {
        return producerID;
    }

    public void setProducerID(Integer producerID) {
        this.producerID = producerID;
    }

    public Producer () {

    }

    public Producer(String firstName, String lastName, int age, String dateOfBirth, String email, String phoneNumber,
String password) {
        this.firstName = firstName;
        this.lastName = lastName;
        this.age = age;
        this.dateOfBirth = dateOfBirth;
        this.email = email;
        this.phoneNumber = phoneNumber;
        this.password = password;
    }
```

```
    public Producer(int producerID, String firstName, String lastName, int age, String dateOfBirth, String email,
String phoneNumber, String password) {
        this.producerID = producerID;
        this.firstName = firstName;
        this.lastName = lastName;
        this.age = age;
        this.dateOfBirth = dateOfBirth;
        this.email = email;
        this.phoneNumber = phoneNumber;
        this.password = password;
    }

    public Producer(int producerID, String firstName, String lastName, int age, String dateOfBirth, String email,
String phoneNumber, String password, Double stockBudget) {
        this.producerID = producerID;
        this.firstName = firstName;
        this.lastName = lastName;
        this.age = age;
        this.dateOfBirth = dateOfBirth;
        this.email = email;
        this.phoneNumber = phoneNumber;
        this.password = password;
        this.stockBudget = stockBudget;
    }
}
```

JSON Files

userOpinion1.json


```
[ {
  "question" : "Do you think that the economy is positively contributing to your personal wellbeing?",
  "answer" : "Yes, Somewhat",
  "weight" : 4
}, {
  "question" : "Compared to last month, do you think the economy is:",
  "answer" : "Much Better",
  "weight" : 5
}, {
  "question" : "Over the next 3 years, how do you feel the economy will change?",
  "answer" : "Stay the same",
  "weight" : 3
}, {
  "question" : "How would you describe the overall condition of the economy right now?",
  "answer" : "Stable",
  "weight" : 4
}, {
  "question" : "In the past 3 months, your cost of living has:",
  "answer" : "Increased Significantly",
  "weight" : 2
}, {
  "question" : "Inflation over the next year will:",
  "answer" : "Dropped slightly",
  "weight" : 4
}, {
  "question" : "How secure do you feel in your current job or income source?",
  "answer" : "Very Insecure",
```

```
  "weight" : 1
}, {
  "question" : "Do you expect job opportunities to increase, decrease, or stay the same in your area?",
  "answer" : "Increase Slightly",
  "weight" : 4
}, {
  "question" : "Approximately, what is the total value of your assets (property, investments, savings, etc.)?",
  "answer" : "£10,000 - £50,000",
  "weight" : 2
}, {
  "question" : "Approximately, what is your annual salary?",
  "answer" : "£10,000 - £25,000",
  "weight" : 2
}, {
  "question" : "Do you believe current economic policies are working in your favour?",
  "answer" : "Not Really",
  "weight" : 2
} ]
```

tharunnadeswaran2207@gmail.com.json (stores vernam key)


```
{
  "vernamKey" : "10101000111110010011111011010"
}
```

glossaryOfEconomicTerms.json


```
[
    {
    "word": "Alignment adjustment",
    "definition": "The adjustment applied to the expenditure and income measures of Gross Domestic Product that allows
ONS to balance these with the output measure of Gross Domestic Product."
    },

    {
    "word": "Appreciation",
    "definition": "An increase in the value of an asset or currency relative to another asset or currency."
    },

    {
    "word": "Asset",
    "definition": "An item or claim that usually provides a flow of benefits - either income or services - to its
owner. Real assets include land, buildings or machinery owned. Financial assets include cash and securities, and
credit extended to customers."
    },

    {
    "word": "Balance of payments",
    "definition": "The accounting record of UK transactions with the rest of the world, including international
institutions. Also known as the Pink Book."
    },

    {
    "word": "Balance sheet",
    "definition": "A statement, drawn up at a particular point in time, recording the assets held by and financial
claims (liabilities) of a business, organisation or country."
```

```
    },

    {
    "word": "Base rate",
    "definition": "Set by the Bank of England in the conduct of monetary policy, it is the rate of interest used by
commercial banks as the basis for their lending rates to the public."
    },

    {
    "word": "Basic prices",
    "definition": "Prices excluding taxes and subsidies on products."
    },

    {
    "word": "Blue Book",
    "definition": "The familiar name of the 'United Kingdom National Accounts' produced annually by ONS. It contains
estimates of Gross Domestic Product measured by the output, expenditure and income approaches."
    },.....
]
```

rateTheEconomyQuestions.json


```
[
  {
    "question": "Do you think that the economy is positively contributing to your personal wellbeing?",
    "answers": {
      "Yes, Significantly": 5,
      "Yes, Somewhat": 4,
      "Neutral": 3,
      "No, not really": 2,
      "No, not at all": 1
    }
  },
  {
    "question": "Compared to last month, do you think the economy is:",
    "answers": {
      "Much Better": 5,
      "Slightly Better": 4,
      "About the same": 3,
      "Slightly Worse": 2,
      "Much Worse": 1
    }
  },
  {
    "question": "Over the next 3 years, how do you feel the economy will change?",
    "answers": {
      "Improve greatly": 5,
      "Improve somewhat": 4,
      "Stay the same": 3,
      "Worsen somewhat": 2,
      "Worsen greatly": 1
```

```
    }
  },
  {
    "question": "How would you describe the overall condition of the economy right now?",
    "answers": {
      "Thriving": 5,
      "Stable": 4,
      "Mixed": 3,
      "Struggling": 2,
      "In Crisis": 1
    }
  },
  {
    "question": "In the past 3 months, your cost of living has:",
    "answers": {
      "Decreased": 5,
      "Stayed the Same": 4,
      "Increased Slightly": 3,
      "Increased Significantly": 2,
      "Become unmanageable": 1
    }
  },
  {
    "question": "Inflation over the next year will:",
    "answers": {
      "Dropped significantly": 5,
      "Dropped slightly": 4,
      "Stays stable": 3,
      "Increase slightly": 2,
      "Increase significantly": 1,
      "Not Sure": 0
```

```
    }
  },
  {
    "question": "How secure do you feel in your current job or income source?",
    "answers": {
      "Very Secure": 5,
      "Somewhat Secure": 4,
      "Neutral": 3,
      "Somewhat Insecure": 2,
      "Very Insecure": 1
    }
  },
  {
    "question": "Do you expect job opportunities to increase, decrease, or stay the same in your area?",
    "answers": {
      "Increase Significantly": 5,
      "Increase Slightly": 4,
      "Stay the Same": 3,
      "Decrease Slightly": 2,
      "Decrease Significantly": 1
    }
  },
  {
    "question": "Approximately, what is the total value of your assets (property, investments, savings, etc.)?",
    "answers": {
      "Over £500,000": 6,
      "£200,000 - £500,000": 5,
      "£100,000 - £200,000": 4,
      "£50,000 - £100,000": 3,
      "£10,000 - £50,000": 2,
      "Less than £10,000": 1,
```

```
      "Preferred Not To Say": 0
    }
  },
  {
    "question": "Approximately, what is your annual salary?",
    "answers": {
      "Over £150,000": 6,
      "£100,000 - £150,000": 5,
      "£50,000 - £100,000": 4,
      "£25,000 - £50,000": 3,
      "£10,000 - £25,000": 2,
      "Less than £10,000": 1,
      "Prefer Not To Say": 0
    }
  },
  {
    "question": "Do you believe current economic policies are working in your favour?",
    "answers": {
      "Yes, Strongly": 5,
      "Somewhat": 4,
      "Neutral": 3,
      "Not Really": 2,
      "Definitely Not": 1
    }
  }
]
```

tickerInfo.json


```
[
    {"companyName": "NVIDIA", "tickerSymbol": "NVDA"},
    {"companyName": "Microsoft", "tickerSymbol": "MSFT"},
    {"companyName": "Apple", "tickerSymbol": "AAPL"},
    {"companyName": "Amazon", "tickerSymbol": "AMZN"},
    {"companyName": "Google", "tickerSymbol": "GOOGL"},
    {"companyName": "Meta", "tickerSymbol": "META"},
    {"companyName": "Broadcom", "tickerSymbol": "AVGO"},
    {"companyName": "Tesla", "tickerSymbol": "TSLA"},
    {"companyName": "Berkshire Hathaway B", "tickerSymbol": "BRK-B"},
    {"companyName": "JPMorgan", "tickerSymbol": "JPM"},
    {"companyName": "Walmart", "tickerSymbol": "WMT"},
    {"companyName": "Oracle", "tickerSymbol": "ORCL"},
    {"companyName": "Eli Lilly", "tickerSymbol": "LLY"},
    {"companyName": "Visa A", "tickerSymbol": "V"},
    {"companyName": "Tencent ADR", "tickerSymbol": "TCEHY"},
    {"companyName": "Mastercard", "tickerSymbol": "MA"},
    {"companyName": "Netflix", "tickerSymbol": "NFLX"},
    {"companyName": "Exxon Mobil", "tickerSymbol": "XOM"},
    {"companyName": "Costco", "tickerSymbol": "COST"},
    {"companyName": "J&J", "tickerSymbol": "JNJ"},
    {"companyName": "Home Depot", "tickerSymbol": "HD"},
    {"companyName": "Industrial Commercial Bank of China", "tickerSymbol": "IDCBY"},
    {"companyName": "Palantir", "tickerSymbol": "PLTR"},
    {"companyName": "P&G", "tickerSymbol": "PG"},
    {"companyName": "Bank of America", "tickerSymbol": "BAC"},
    {"companyName": "Chevron", "tickerSymbol": "CVX"},
    {"companyName": "Coca-Cola", "tickerSymbol": "KO"},
    {"companyName": "AMD", "tickerSymbol": "AMD"},
```

```
    {"companyName": "IBM", "tickerSymbol": "IBM"}
]
```

Frontend Code

start.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<link rel="stylesheet" href="css\start.css">

<head>
    <title>Money Matters</title>
</head>

<body>
    <div id="centreDiv">

        <!-- Title -->
        <div id="createAnAccount">
            <h1>MoneyMatters</h1>
            <h4>By Tharun Nadeswaran</h4>
        </div>

        <!-- Buttons for Login and Sign Up -->
        <div id="registrationButtons">

            <!-- Login button -->
            <a th:href="@{/login}">
                <button id="login-button">Login</button>
            </a>

            <br>
```

```
            <!-- Sign Up button -->
            <a th:href="@{/signup}">
                <button id="signup-button">Sign Up</button>
            </a>

        </div>
    </div>
</body>
</html>
```

start.css

```css
body {
    margin: 0;
    height: 100vh;
    display: flex;
    flex-direction: column;
    justify-content: center;
    align-items: center;
    background-color: #d4e9cd;
    font-family: "Segoe UI", Tahoma, Geneva, Verdana, sans-serif;
}

h1 {
    margin-top: 0px;
    margin-bottom: 5px;
}

h4 {
    margin-top: 0px;
    margin-bottom: 5px;
}

#createAnAccount {
    margin-bottom: 10px;
    text-align: center;
}

#registrationButtons {
    margin: 10px;
}
```

```css
a {
    text-decoration: none;
}

#registrationButtons button {
    margin-bottom: 12px;
    width: 35vw;
    height: 5vh;
    background-color: #20e3448f;
    border: solid black 1px;
    color: #000000;
    font-size: 16px;
    font-family: "Segoe UI", Tahoma, Geneva, Verdana, sans-serif;
    border-radius: 5px;
    cursor: pointer;
}

#registrationButtons button:hover {
    background-color: #20e344c5;
}
```

signup.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<link rel="stylesheet" href="/css/signup.css" />

<head>
    <title>Money Matters</title>
</head>

<body>
    <h1>Register</h1>

    <div id="centreDiv">
        <!-- Sign-up form -->
        <form id="signup-inputs" action="/signup" method="post">

            <div class="inputWrapper">
                <label>First Name:</label>
                <input type="text" id="firstName" name="firstName" placeholder="John" required>
            </div>

            <div class="inputWrapper">
                <label>Last Name:</label>
                <input type="text" id="lastName" name="lastName" placeholder="Doe" required>
            </div>

            <div class="inputWrapper">
                <label>Age:</label>
                <input type="number" id="age" name="age" placeholder="18" required>
            </div>
```

```
<div class="inputWrapper">
    <label>Date Of Birth:</label>
    <input type="date" id="dateOfBirth" name="dateOfBirth" required>
</div>

<div class="inputWrapper">
    <label>Email:</label>
    <input type="text" id="email" name="email" placeholder="johndoe123@gmail.com" required>
</div>

<div class="inputWrapper">
    <label>Phone Number:</label>
    <input type="text" id="phoneNumber" name="phoneNumber" placeholder="+44" required>
</div>

<div class="inputWrapper">
    <label>Password:</label>
    <div class="passwordField">
        <input type="password" id="password" name="password" placeholder="********" required>
        <!-- Eye icon to toggle password visibility -->
        <img id="showPasswordImage" src="/images/view.png">
    </div>
</div>

<div class="inputWrapper">
    <label>Select Economic Agent:</label>
    <div class="radioOptions">
        <label class="typeOfEconomicAgentRadio">
            <input type="radio" name="typeOfEconomicAgent" value="Consumer" required> Consumer
        </label>
```

```
                <label class="typeOfEconomicAgentRadio">
                    <input type="radio" name="typeOfEconomicAgent" value="Producer"> Producer
                </label>
            </div>
        </div>

        <div id="signUpButtons">
            <button type="submit">Create Account</button>
            <a id="backToHomeButton" th:href="@{/start}">Back To Home</a>
        </div>

    </form>
</div>

<script th:inline="javascript">
    // Toggle the password visibility when eye icon is clicked
    function showPassword() {
        const passwordInput = document.getElementById("password");
        const buttonImage = document.getElementById("showPasswordImage");
        const isShow = passwordInput.type === "text";

        if (isShow) {
            // Hide password
            passwordInput.type = "password";
            buttonImage.src = "/images/view.png";
        }

        else {
            // Show password
            passwordInput.type = "text";
            buttonImage.src = "/images/hide.png";
```

```
            }
        }

        // Show alert
        function showWindowsAlert() {
            let windowsAlert = /*[[${windowsAlert}]]*/ false;
            let windowsAlertMessage = /*[[${windowsAlertMessage}]]*/ "";

            if (windowsAlert === true) {
                window.alert(windowsAlertMessage);
            }
        }

        document.getElementById("showPasswordImage").addEventListener("click", showPassword);
        window.onload = showWindowsAlert;
    </script>
</body>
</html>
```

signup.css

```css
body {
    margin: 0;
    display: flex;
    flex-direction: column;
    align-items: center;
    background-color: #d4e9cd;
    font-family: "Segoe UI", Tahoma, Geneva, Verdana, sans-serif;
}

h1 {
    margin-top: 30px;
    margin-bottom: 10px;
}

#centreDiv {
    display: flex;
    flex-direction: column;
    align-items: center;
    justify-content: center;
    height: 100%;
    width: 100%;
}

form {
    width: 100%;
    max-width: 600px;
    padding: 20px;
    box-sizing: border-box;
}
```

```css
.inputWrapper {
    display: flex;
    flex-direction: column;
    width: 100%;
    margin-bottom: 20px;
}

label {
    font-size: 18px;
    color: #000000;
    margin-bottom: 5px;
}

input[type="text"], input[type="password"], input[type="number"], input[type="date"] {
    padding: 10px;
    border: 1px solid #ffffff;
    border-radius: 5px;
    font-size: 14px;
    font-family: inherit;
    width: 100%;
    box-sizing: border-box;
}

input[type="text"]:focus, input[type="password"]:focus, input[type="number"]:focus, input[type="date"]:focus,
input[type="radio"]:focus {
    outline: none;
    border-color: #20e344;
    box-shadow: 0 0 3px #20e344;
}
```

```css
.passwordField {
    display: flex;
    align-items: center;
    position: relative;
}

.passwordField input {
    flex: 1;
}

.passwordField img {
    width: 25px;
    height: 25px;
    margin-left: 10px;
    cursor: pointer;
    background-color: transparent;
    border-radius: 5px;
}

.radioOptions {
    display: flex;
    gap: 20px;
    margin-top: 8px;
}

.typeOfEconomicAgentRadio {
    font-size: 15px;
    font-family: inherit;
    display: flex;
    align-items: center;
}
```

```css
#signUpButtons {
    display: flex;
    flex-direction: column;
    align-items: center;
    margin-top: 20px;
}

#signUpButtons button, #backToHomeButton {
    margin-bottom: 12px;
    width: 100%;
    height: 5vh;
    background-color: #20e3448f;
    border: solid black 1px;
    color: #000000;
    font-size: 16px;
    font-family: inherit;
    border-radius: 5px;
    cursor: pointer;
    text-align: center;
    text-decoration: none;
    display: flex;
    align-items: center;
    justify-content: center;
}

#signUpButtons button:hover, #backToHomeButton:hover {
    background-color: #20e344c5;
}
```

login.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<link rel="stylesheet" href="/css/login.css"/>

<head>
    <title>Money Matters</title>
</head>

<body>
    <h1>Login</h1>

    <form id="login-inputs" action="/login" method="post">

        <div class="inputWrapper">
            <label>Email:</label>
            <input type="text" id="email" name="email" placeholder="johndoe123@gmail.com" required>
        </div>

        <div class="inputWrapper" id="passwordInput">
            <label>Password:</label>
            <div class="passwordField">
                <input type="password" id="password" name="password" placeholder="********" required>
                <!-- Eye icon to toggle password visibility -->
                <img id="showPasswordImage" src="/images/view.png">
            </div>
        </div>

        <div id="loginButtons">
            <button type="submit">Login</button>
```

```
        <a id="backToHomeButton" th:href="@{/start}">Back To Home</a>
    </div>
</form>

<script th:inline="javascript">
    // Toggle the password visibility when the eye icon is clicked
    function showPassword() {
        const passwordInput = document.getElementById("password");
        const buttonImage = document.getElementById("showPasswordImage");
        const isShow = passwordInput.type === "text";

        if (isShow) {
            // Hide password
            passwordInput.type = "password";
            buttonImage.src = "/images/view.png";
        }

        else {
            // Show password
            passwordInput.type = "text";
            buttonImage.src = "/images/hide.png";
        }
    }

    // Show alert
    function showWindowsAlert() {
        let windowsAlert = /*[[${windowsAlert}]]*/ false;
        let windowsAlertMessage = /*[[${windowsAlertMessage}]]*/ "";

        if (windowsAlert === true) {
            window.alert(windowsAlertMessage);
```

```
            }
        }

        window.onload = showWindowsAlert;
        document.getElementById("showPasswordImage").addEventListener("click", showPassword);
    </script>
</body>
</html>
```

login.css

```
body {
    margin: 0;
    height: 100vh;
    display: flex;
    flex-direction: column;
    justify-content: center;
    align-items: center;
    background-color: #d4e9cd;
    font-family: "Segoe UI", Tahoma, Geneva, Verdana, sans-serif;
}

.inputWrapper {
    width: 35vw;
    margin-bottom: 20px;
}

label {
    display: block;
    margin-bottom: 5px;
    font-size: 18px;
    color: #000000;
}

input[type="text"], input[type="password"] {
    width: 100%;
    height: 6vh;
    padding: 10px;
    border: 1px solid #ffffff;
    border-radius: 5px;
```

```
    font-family: "Segoe UI", Tahoma, Geneva, Verdana, sans-serif;
    font-size: 14px;
    box-sizing: border-box;
}

.passwordField {
    display: flex;
    align-items: center;
    position: relative;
}

.passwordField input {
    flex: 1;
}

.passwordField img {
    width: 25px;
    height: 25px;
    margin-left: 10px;
    cursor: pointer;
    background-color: transparent;
    border-radius: 5px;
}

#loginButtons {
    display: flex;
    flex-direction: column;
    align-items: center;
    margin-top: 10px;
    width: 35vw;
}
```

```css
#loginButtons button {
    margin-bottom: 12px;
    width: 100%;
    height: 5vh;
    background-color: #20e3448f;
    border: solid black 1px;
    color: #000000;
    font-size: 16px;
    font-family: "Segoe UI", Tahoma, Geneva, Verdana, sans-serif;
    border-radius: 5px;
    cursor: pointer;
}

#loginButtons button:hover {
    background-color: #20e344c5;
}

#backToHomeButton {
    display: flex;
    justify-content: center;
    align-items: center;
    width: 35vw;
    height: 5vh;
    margin-bottom: 12px;
    background-color: #20e3448f;
    border: solid black 1px;
    color: #000000;
    font-size: 16px;
    font-family: "Segoe UI", Tahoma, Geneva, Verdana, sans-serif;
    border-radius: 5px;
```

```
    text-decoration: none;
    cursor: pointer;
}

#backToHomeButton:hover {
    background-color: #20e344c5;
}

input:focus {
    outline: none;
    border-color: #20e344;
    box-shadow: 0 0 3px #20e344;
}
```

user-dashboard.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">

<head>
    <link rel="stylesheet" href="/css/user-dashboard.css" />
    <title>Money Matters</title>
</head>

<body>
    <div id="all">
        <h1>User Dashboard</h1>

        <!-- Navigation bar -->
        <nav id="dashboardNavbar">
            <a th:href="@{/news}">Economic News</a>
            <a th:href="@{/figures-economic}">Economic Figures</a>
            <a th:href="@{/exchange-rate-economic}">Exchange Rates</a>
            <a th:href="@{/rate-the-economy}">Rate The Economy</a>
            <a th:href="@{/budget-tracker}">Budget Tracker</a>
            <a th:href="@{/investor}">Investor Centre</a>
            <a th:href="@{/cost-of-production}">Cost Of Production</a>
            <a th:href="@{/start}">Log Out</a>
        </nav>

        <!-- AI Bot Section -->
        <div id="aiFormContainer">
            <h2>Ask A Question</h2>
            <div id="aiReplies"></div>
            <form id="askAIBot" name="askAIBot">
```

```
            <input type="text" id="userPrompt" name="userPrompt" placeholder="Ask A Question">
            <button type="submit" onclick="generatingAnswer()">Ask</button>
        </form>
        <p id="generateAnswer"></p>
    </div>

    <!-- Glossary Section -->
    <div id="glossary">
        <h2>Glossary</h2>
        <form id="glossarySearchInput" name="glossarySearch" action="/user-dashboard" method="post">
            <input type="hidden" name="formName" value="glossarySearch">
            <input type="text" id="glossarySearchInput" name="glossarySearchInput">
            <input type="submit" value="Search">
            <span th:text="${userWantedWord}"></span>
        </form>
    </div>
</div>

<script th:inline="javascript">
    function generateGlossary() {
        const wordsDefinitionsPairs = /*[[${wordsDefinitionsPairs}]]*/ {};
        const glossaryDiv = document.getElementById("glossary");
        const words = Object.keys(wordsDefinitionsPairs);
        const definitions = Object.values(wordsDefinitionsPairs);

        for (let i = 0; i < words.length; i++) {
            const tempDiv = document.createElement("div");
            const wordSpan = document.createElement("span");
            const definitionSpan = document.createElement("span");

            wordSpan.innerText = words[i] + ": ";
```

```
            definitionSpan.innerText = definitions[i];
            tempDiv.append(wordSpan, definitionSpan);
            glossaryDiv.append(tempDiv);
        }
    }

    // Send user question to backend and display AI response
    function generatingAnswer() {
        const formName = "askAIBot";

        const userQuestion = document.createElement("p");
        userQuestion.classList.add("user-message");
        userQuestion.innerText = document.getElementById("userPrompt").value;

        const aiReply = document.createElement("p");
        aiReply.classList.add("ai-message");

        const aiReplies = document.getElementById("aiReplies");

        document.getElementById("generateAnswer").innerText = "Generating Response...";

        document.getElementById("askAIBot").addEventListener("submit", function(e) {
            e.preventDefault();
            const userPrompt = document.getElementById("userPrompt").value;

            // Fetch request to backend endpoint
            fetch("/user-dashboard", {
                method: "POST",
                headers: {
                    "Content-Type": "application/x-www-form-urlencoded"
                },
```

```
                body: new URLSearchParams({formName: formName, userPrompt: userPrompt})
            })
            .then(response => response.json())
            .then(data => {
                aiReply.textContent = JSON.parse(data.aiReply); // Parse AI reply
                aiReplies.append(userQuestion);
                aiReplies.append(aiReply);
                document.getElementById("generateAnswer").innerText = "";
            })
            .catch(error => console.error("Error:", error));
        });
    }

    function showWindowsAlert() {
        let windowsAlert = /*[[${windowsAlert}]]*/ false;
        let windowsAlertMessage = /*[[${windowsAlertMessage}]]*/ "";

        if (windowsAlert === true) {
            window.alert(windowsAlertMessage);
        }
    }

    window.onload = showWindowsAlert;
    generateGlossary();
</script>
</body>
</html>
```

user-dashboard.css

```css
body {
    margin: 1vw;
    display: flex;
    flex-direction: column;
    justify-content: flex-start;
    align-items: center;
    background-color: #d4e9cd;
    font-family: "Segoe UI", Tahoma, Geneva, Verdana, sans-serif;
    padding-top: 70px;
}

#all {
    padding: 20px;
    background-color: #ffffff9e;
    border-radius: 12px;
    box-shadow: 0 0 20px #0000001a;
    width: 1020px;
    max-width: 1020px;
}

h1 {
    margin-top: 0;
    margin-bottom: 5px;
    font-size: 2rem;
    text-align: center;
}

h2 {
    margin-top: 0;
```

```
    margin-bottom: 15px;
    font-size: 1.6rem;
    text-align: center;
}

button, input[type="submit"], button[type="submit"] {
    padding: 0.5em 1em;
    background-color: #20e3448f;
    border: 1px solid #000000;
    color: #000000;
    border-radius: 8px;
    cursor: pointer;
    font-size: 1rem;
    transition: background 0.2s ease, transform 0.1s ease;
}

button:hover, input[type="submit"]:hover, button[type="submit"]:hover {
    background-color: #20e344cc;
    transform: translateY(-1px);
}

#askAIBot {
    display: flex;
    align-items: center;
}

#askAIBot input[type="text"] {
    padding: 8px 12px;
    border-radius: 8px;
    border: 1px solid #00000024;
    font-size: 1rem;
```

```
    width: 300px;
    margin-bottom: 10px;
}

#generateAnswer {
    font-weight: 600;
    margin-top: 10px;
    color: #000;
}

#aiReplies p {
    margin: 6px 0;
    padding: 10px;
    border-radius: 8px;
    background-color: #000000;
}

#glossary {
    margin-top: 30px;
    padding: 20px;
    border-radius: 12px;
    background-color: #ffffff9e;
    box-shadow: 0 0 15px rgba(0, 0, 0, 0.08);
}

#glossary input[type="text"] {
    padding: 8px 12px;
    font-size: 1rem;
    border-radius: 8px;
    border: 1px solid #00000024;
    margin-bottom: 10px;
```

```
    width: 60%;
}

#glossary input[type="submit"] {
    width: 150px;
}

#glossary div {
    margin: 6px 0;
}

#glossary span:first-child {
    font-weight: 700;
    color: #20e344;
}

#all > div > a, #all > div > button {
    display: block;
    text-align: center;
}

#dashboardNavbar {
    position: fixed;
    top: 0;
    left: 0;
    width: 100%;
    background-color: #20e3448f;
    display: flex;
    justify-content: center;
    align-items: center;
    gap: 10px;
```

```
    padding: 10px 0;
    box-shadow: 0 2px 8px #00000033;
    z-index: 1000;
}

#dashboardNavbar a {
    text-decoration: none;
    color: #000000;
    padding: 8px 16px;
    border-radius: 8px;
    font-weight: 500;
    transition: background 0.2s ease, transform 0.1s ease;
}

#dashboardNavbar a:hover {
    background-color: #20e344cc;
}

#aiFormContainer {
    margin: 20px auto;
    padding: 30px;
    width: 950px;
    background: #ffffffee;
    border-radius: 18px;
    box-shadow: 0 8px 28px rgba(0,0,0,0.12);
    display: flex;
    flex-direction: column;
    backdrop-filter: blur(6px);
    transition: transform 0.2s ease;
}
```

```css
#askAIBot {
    display: flex;
    gap: 10px;
}

#askAIBot input[type="text"] {
    flex: 1;
    padding: 12px 16px;
    font-size: 1rem;
    border: 1px solid #ccc;
    border-radius: 12px;
    outline: none;
    transition: border-color 0.2s ease, box-shadow 0.2s ease;
}

#askAIBot input[type="text"]:focus {
    border-color: #20e344;
    box-shadow: 0 0 8px #20e34455;
}

#aiReplies {
    display: flex;
    flex-direction: column;
    gap: 12px;
    margin-top: 10px;
    margin-bottom: 10px;
    padding: 15px;
    border-radius: 14px;
    background: #f9faf9;
    min-height: 100px;
    font-size: 1rem;
```

```
    color: #333;
    line-height: 1.5;
    box-shadow: inset 0 1px 4px rgba(0,0,0,0.05);
    max-height: 400px;
    overflow-y: auto;
}

#aiReplies .user-message {
    align-self: flex-end;
    background: #e9ecef;
    color: #000;
    padding: 10px 14px;
    border-radius: 14px 14px 0 14px;
    max-width: 70%;
    box-shadow: 0 2px 6px rgba(0,0,0,0.08);
    animation: fadeIn 0.3s ease;
}

#aiReplies .ai-message {
    align-self: flex-start;
    background: #20e344;
    color: #fff;
    padding: 10px 14px;
    border-radius: 14px 14px 14px 0;
    max-width: 70%;
    box-shadow: 0 2px 6px rgba(0,0,0,0.08);
    animation: fadeIn 0.3s ease;
}

#generateAnswer {
    margin-top: 8px;
```

```
    font-size: 0.95rem;
    color: #666;
    font-style: italic;
    text-align: center;
}
```

news.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<link rel="stylesheet" href="/css/news.css" />

<head>
    <title>Money Matters</title>
</head>

<body>
    <!-- Navigation bar -->
    <nav id="dashboardNavbar">
        <a th:href="@{/news}">Economic News</a>
        <a th:href="@{/figures-economic}">Economic Figures</a>
        <a th:href="@{/exchange-rate-economic}">Exchange Rates</a>
        <a th:href="@{/rate-the-economy}">Rate The Economy</a>
        <a th:href="@{/budget-tracker}">Budget Tracker</a>
        <a th:href="@{/investor}">Investor Centre</a>
        <a th:href="@{/cost-of-production}">Cost Of Production</a>
        <a th:href="@{/user-dashboard}">DashBoard</a>
    </nav>

    <div id="all">
        <h1>Economic News</h1>
        <!-- Form to filter and sort news articles -->
        <form id="sort" action="/news" method="post">
            <label>Select News Outlets:</label>
            <br>
            <input type="hidden" name="formName" value="filterNews">
```

```
            <!-- Each checkbox represents a news source -->
            <label>BBC News - Centrist</label>
            <input type="checkbox" name="newsSource" th:checked="${selectedSources != null and
selectedSources.contains(0)}" value=0>
            <br>

            <label>The Telegraph - Right</label>
            <input type="checkbox" name="newsSource" th:checked="${selectedSources != null and
selectedSources.contains(1)}" value=1>
            <br>

            <label>The Economist - Centrist</label>
            <input type="checkbox" name="newsSource" th:checked="${selectedSources != null and
selectedSources.contains(2)}" value=2>
            <br>

            <label>The Independent - Left</label>
            <input type="checkbox" name="newsSource" th:checked="${selectedSources != null and
selectedSources.contains(3)}" value=3>
            <br>

            <label>CNBC - Centrist</label>
            <input type="checkbox" name="newsSource" th:checked="${selectedSources != null and
selectedSources.contains(4)}" value=4>
            <br>

            <label>Sort Articles By:</label>
            <select id="sort" name="sort">
                <option value="date (latest)" th:selected="${selectedSort == 'date (latest)'}">Date (Latest)</option>
                <option value="date (earliest)" th:selected="${selectedSort == 'date (earliest)'}">Date
(Earliest)</option>
```

```
            <option value="title (a-z)" th:selected="${selectedSort == 'title (a-z)'}">Title (A - Z)</option>
            <option value="title (z-a)" th:selected="${selectedSort == 'title (z-a)'}">Title (Z - A)</option>
        </select>
        <br>
        <input type="submit" value="Sort">
    </form>

    <div id="newsSearch">
        <label>Search News Article:</label>
        <input type="text" id="searchBar" placeholder="Type to search...">
    </div>

    <!-- Container for displaying news articles -->
    <div id="news"></div>
</div>

<script th:inline="javascript">
    const news = /*[[${economicNews}]]*/ [];
    const newsDiv = document.getElementById("news");

    // Function to display news articles
    function showNews(news) {
        for (let i = 0; i < news.length; i++) {
            const newsContainer = document.createElement("div");
            newsContainer.id = news[i].EconomicNewsID;
            newsContainer.className = "newsContainer";

            let title = document.createElement("span");
            title.innerText = news[i].title;
            title.id = news[i].EconomicNewsID + "title";
            title.className = "title";
```

```
let description = document.createElement("span");
description.innerText = news[i].description;
description.id = news[i].EconomicNewsID + "-description";
description.className = "description";

let source = document.createElement("span");
source.innerText = news[i].source;
source.id = news[i].EconomicNewsID + "-source";
source.className = "source";

let pubDate = document.createElement("span");
pubDate.innerText = news[i].pubDate;
pubDate.id = news[i].EconomicNewsID + "-pubDate";
pubDate.className = "pubDate";

let link = document.createElement("a");
link.href = news[i].link;
link.innerText = news[i].link;
link.id = news[i].EconomicNewsID + "-link";
link.className = "link";

let image = null;
if (news[i].image != null) {
    image = document.createElement("img");
    image.src = news[i].image;
    image.width = 240;
    image.height = 135;
    image.id = news[i].EconomicNewsID + "-image";
    image.className = "image";
}
```

```
let formName = document.createElement("input");
formName.type = "hidden";
formName.name = "formName";
formName.value = "summarise"

// Summarise button
let summaryButtonDiv = document.createElement("div");
summaryButtonDiv.id = "summaryButtonDiv";

let summaryButton = document.createElement("button");
summaryButton.innerText = "Summarise Article";
summaryButton.id = news[i].EconomicNewsID;
summaryButton.className = "summaryButton";
summaryButton.addEventListener("click", function() {
    event.preventDefault();
    generatingSummary(this);
});

summaryButtonDiv.appendChild(summaryButton);

// Append all elements to news container
newsContainer.append(formName);
newsContainer.append(title);
newsContainer.append(document.createElement("br"));
newsContainer.append(description);
newsContainer.append(document.createElement("br"));
newsContainer.append(source);
newsContainer.append(document.createElement("br"));
newsContainer.append(pubDate);
newsContainer.append(document.createElement("br"));
```

```
            newsContainer.append(link);
            newsContainer.append(document.createElement("br"));
            if (image != null) {
                newsContainer.append(image);
                newsContainer.append(document.createElement("br"));
            }
            newsContainer.append(summaryButtonDiv);
            newsContainer.append(document.createElement("br"));
            newsContainer.append(document.createElement("br"));
            newsDiv.append(newsContainer);
        }
    }

    function getDivId(button) {
        return button.closest(".newsContainer").id;
    }

    // Generate AI summary of an article
    function generatingSummary(button) {
        const currentDiv = document.getElementById(getDivId(button));
        const articleLink = currentDiv.querySelector("a").href;
        const aiSummary = document.createElement("p");
        button.innerText = "Generating Response...";

        fetch("/news", {
            method: "POST",
            headers: { "Content-Type": "application/x-www-form-urlencoded" },
            body: new URLSearchParams({formName: "summarise", link: articleLink})
        })
        .then(response => response.json())
        .then(data => {
```

```
                aiSummary.textContent = JSON.parse(data.aiReply);
                button.remove();
                currentDiv.append(aiSummary);
            })
            .catch(error => console.error("Error:", error));
        }

        function searchBar() {
            const searchBar = document.getElementById("searchBar");
            searchBar.addEventListener("input", function() {
                const query = this.value.toLowerCase();
                newsDiv.innerHTML = "";

                if (query) {
                    const matches = news.filter(oneNews => oneNews.title.toLowerCase().includes(query));
                    if (matches) showNews(matches);
                }

                if (this.value === "") {
                    showNews(news);
                }
            });
        }

        showNews(news);
        searchBar();
    </script>
</body>
</html>
```

news.css

```
body {
    margin: 1vw;
    display: flex;
    flex-direction: column;
    justify-content: flex-start;
    align-items: center;
    background-color: #d4e9cd;
    font-family: "Segoe UI", Tahoma, Geneva, Verdana, sans-serif;
    color: #000000;
    padding-top: 70px;
}

#all {
    padding: 20px;
    background-color: #ffffff9e;
    border-radius: 12px;
    box-shadow: 0 0 20px #0000001a;
    width: 1020px;
    max-width: 1020px;
}

h1 {
    margin-top: 0;
    margin-bottom: 15px;
    font-size: 2rem;
    text-align: center;
}

h2 {
```

```
    margin-top: 0;
    margin-bottom: 15px;
    font-size: 1.6rem;
    text-align: center;
}

#dashboardNavbar {
    position: fixed;
    top: 0;
    left: 0;
    width: 100%;
    background-color: #20e3448f;
    display: flex;
    justify-content: center;
    align-items: center;
    gap: 10px;
    padding: 10px 0;
    box-shadow: 0 2px 8px rgba(0,0,0,0.2);
    z-index: 1000;
}

#dashboardNavbar a {
    text-decoration: none;
    color: #000000;
    padding: 8px 16px;
    border-radius: 8px;
    font-weight: 500;
    transition: background 0.2s ease, transform 0.1s ease;
}

#dashboardNavbar a:hover {
```

```
    background-color: #20e344cc;
    transform: translateY(-1px);
}

button[type="submit"], input[type="submit"], button {
    padding: 0.5em 1em;
    background-color: #20e3448f;
    border: 1px solid #000000;
    color: #000000;
    border-radius: 8px;
    cursor: pointer;
    font-size: 1rem;
    transition: background 0.2s ease, transform 0.1s ease;
    width: 150px;
}

button[type="submit"]:hover, input[type="submit"]:hover, button:hover {
    background-color: #20e344cc;
    transform: translateY(-1px);
}

form#sort {
    width: 100%;
    max-width: 975px;
    background: #ffffff9e;
    padding: 20px;
    border-radius: 12px;
    box-shadow: 0 0 20px #00000014;
    margin-bottom: 15px;
}
```

```css
form#sort label {
    display: inline-block;
    margin: 6px 8px 6px 0;
    font-size: 1rem;
    font-weight: 600;
}

form#sort input[type="checkbox"] {
    margin-right: 12px;
    transform: translateY(1px);
}

select[name="sort"] {
    padding: 8px 12px;
    font-size: 1rem;
    border-radius: 8px;
    border: 1px solid #00000024;
    margin: 10px 0 10px 8px;
    background: #ffffff;
}

form#sort input[type="submit"] {
    margin-top: 8px;
    width: 180px;
}

#newsSearch {
    width: 100%;
    max-width: 975px;
    background: #ffffff9e;
    padding: 16px 20px;
```

```
    border-radius: 12px;
    box-shadow: 0 0 20px #00000014;
    margin: 10px 0 20px;
    display: flex;
    align-items: center;
    gap: 12px;
}

#newsSearch label {
    font-size: 1rem;
    font-weight: 500;
    white-space: nowrap;
}

#searchBar {
    flex: 1 1 auto;
    padding: 10px 12px;
    font-size: 1rem;
    border-radius: 8px;
    border: 1px solid #00000024;
    outline: none;
}

#searchBar:focus {
    box-shadow: 0 0 0 3px rgba(32, 227, 68, 0.25);
}

#news {
    width: 100%;
    max-width: 1020px;
    display: grid;
```

```
    grid-template-columns: repeat(3, minmax(0, 1fr));
    gap: 16px;
    margin-bottom: 40px;
}

img {
    width: 100%;
    height: 180px;
    object-fit: cover;
    border-radius: 10px;
    border: 1px solid #00000014;
}

.title {
    font-weight: 700;
    font-size: 1.05rem;
    line-height: 1.3;
}

.description {
    color: #1f1f1f;
    font-size: 0.98rem;
}

.source, .pubDate {
    font-size: 0.9rem;
    color: #2e2e2e;
    opacity: 0.9;
}

.link {
```

```
    word-break: break-word;
    text-decoration: none;
    border-bottom: none;
    display: inline-block;
    padding-bottom: 2px;
}

.newsContainer {
    background: #ffffff;
    border-radius: 12px;
    padding: 16px;
    box-shadow: 0 0 12px #0000000f;
    transition: transform 0.12s ease, box-shadow 0.2s ease;
}

#summaryButtonDiv {
    display: flex;
    flex-direction: column;
    align-items: center;
}

.summaryButton {
    margin-top: 10px;
}

.newsContainer:hover {
    transform: translateY(-2px);
    box-shadow: 0 6px 20px #0000001f;
}

.newsContainer p {
```

```
    margin: 8px;
    padding: 10px 12px;
    border-radius: 10px;
    background: #d0edc2;
    border: 1px solid #0000001c;
    font-size: 0.98rem;
}
```

figures-economic.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<script src="https://cdn.plot.ly/plotly-latest.min.js"></script>
<link rel="stylesheet" href="/css/figures-economic.css"/>

<head>
    <title>Money Matters</title>
</head>

<body>
    <div id="all">
        <h1>Economic Figures</h1>

        <!-- Navigation bar -->
        <nav id="dashboardNavbar">
            <a th:href="@{/news}">Economic News</a>
            <a th:href="@{/figures-economic}">Economic Figures</a>
            <a th:href="@{/exchange-rate-economic}">Exchange Rates</a>
            <a th:href="@{/rate-the-economy}">Rate The Economy</a>
            <a th:href="@{/budget-tracker}">Budget Tracker</a>
            <a th:href="@{/investor}">Investor Centre</a>
            <a th:href="@{/cost-of-production}">Cost Of Production</a>
            <a th:href="@{/user-dashboard}">DashBoard</a>
        </nav>

        <!-- Form to select a country and indicator -->
        <div>
            <form id="countrySelect" action="/figures-economic" method="post">
                <label>Select a Country:</label>
```

```
                <select id="countryCode" name="countryCode">
                    <option th:each="entry : ${countryCodes}"
                            th:value="${entry.value}"
                            th:text="${entry.key}"
                            th:selected="${selectedCountry} == ${entry.key}">
                    </option>
                </select>
                <br>

                <label>Select an Indicator:</label>
                <select id="indicatorCode" name="indicatorCode">
                    <option value="NY.GDP.MKTP.CD" th:selected="${selectedIndicator == 'GDP'}">GDP</option>
                    <option value="NY.GDP.PCAP.CD" th:selected="${selectedIndicator == 'GDP Per Capita'}">GDP per
Capita</option>
                    <option value="FP.CPI.TOTL.ZG" th:selected="${selectedIndicator == 'Inflation (CPI)'}">Inflation
(CPI)</option>
                    <option value="SL.UEM.TOTL.ZS" th:selected="${selectedIndicator == 'Real Interest Rate'}">Real
Interest Rate</option>
                    <option value="FR.INR.RINR" th:selected="${selectedIndicator == 'Total Unemployment'}">Total
Unemployment</option>
                    <option value="SI.POV.GINI" th:selected="${selectedIndicator == 'Gini Index'}">Gini Index</option>
                    <option value="GC.DOD.TOTL.GD.ZS" th:selected="${selectedIndicator == 'Total Central Government
Debt'}">Total Central Government Debt</option>
                    <option value="BN.GSR.GNFS.CD" th:selected="${selectedIndicator == 'Net Trade In Goods &
Services'}">Net Trade In Goods & Services</option>
                    <option value="NE.TRD.GNFS.ZS" th:selected="${selectedIndicator == 'Trade (% of GDP)'}">Trade (%
of GDP)</option>
                    <option value="GC.TAX.TOTL.CN" th:selected="${selectedIndicator == 'Tax revenue (current
LCU)'}">Tax revenue (current LCU)</option>
                    <option value="GC.TAX.TOTL.GD.ZS" th:selected="${selectedIndicator == 'Tax revenue (% of
GDP)'}">Tax revenue (% of GDP)</option>
```

```
                    <option value="FI.RES.TOTL.CD" th:selected="${selectedIndicator == 'Total reserves - includes
gold'}">Total reserves - includes gold</option>
                </select>
                <br>

                <button type="submit">Submit</button>
            </form>
        </div>

        <!-- Container for graph -->
        <div id="graphContainer"></div>
    </div>

    <script th:inline="javascript">

        function showGraph() {
            let yearDataPairs = /*[[${yearDataPairs}]]*/ {};
            if (yearDataPairs && Object.keys(yearDataPairs).length > 0) {
                let years = Object.keys(yearDataPairs);
                let data = Object.values(yearDataPairs);

                var line1 = {
                    x: years,
                    y: data,
                    type: 'scatter',
                    name: 'Line 1'
                };

                var plotting = [line1];

                const indicatorCode = /*[[${indicatorCode}]]*/ "";
```

```
const country = /*[[${country}]]*/ "";
let yaxisTitle = "";

switch (indicatorCode) {
    case "NY.GDP.MKTP.CD":
        yaxisTitle = "GDP (current USD)";
        break;

    case "NY.GDP.PCAP.CD":
        yaxisTitle = "GDP per Capita (current USD)";
        break;

    case "FP.CPI.TOTL.ZG":
        yaxisTitle = "CPI (annual %)";
        break;

    case "SL.UEM.TOTL.ZS":
        yaxisTitle = "Real interest rate (%)";
        break;

    case "FR.INR.RINR":
        yaxisTitle = "Unemployment (% of total labor force)";
        break;

    case "SI.POV.GINI":
        yaxisTitle = "Gini Index";
        break;

    case "GC.DOD.TOTL.GD.ZS":
        yaxisTitle = "Debt (% of GDP)";
        break;
```

```
        case "BN.GSR.GNFS.CD":
            yaxisTitle = "Net Trade (BoP, current US$)";
            break;

        case "NE.TRD.GNFS.ZS":
            yaxisTitle = "Trade (% of GDP)";
            break;

        case "GC.TAX.TOTL.CN":
            yaxisTitle = "Tax revenue (current LCU)";
            break;

        case "GC.TAX.TOTL.GD.ZS":
            yaxisTitle = "Tax revenue (% of GDP)";
            break;

        case "FI.RES.TOTL.CD":
            yaxisTitle = "Total reserves (current US$)";
            break;

        default:
            yaxisTitle = "";
            break;
    }

    var layout = {
        title: country,
        xaxis: {title: "Year"},
        yaxis: {title: yaxisTitle},
        dragmode: false,
```

```
                    zoom: false,
                    paper_bgcolor: "transparent",
                    plot_bgcolor: "transparent"
                };

                Plotly.newPlot("graphContainer", plotting, layout, {displayModeBar: false, responsive: true});
            }
        }

        showGraph();
    </script>
</body>
</html>
```

figures-economic.css

```css
body {
    margin: 1vw;
    display: flex;
    flex-direction: column;
    justify-content: flex-start;
    align-items: center;
    background-color: #d4e9cd;
    font-family: "Segoe UI", Tahoma, Geneva, Verdana, sans-serif;
    padding-top: 70px;
}

#all {
    padding: 20px;
    background-color: #ffffff9e;
    border-radius: 12px;
    box-shadow: 0 0 20px rgba(0, 0, 0, 0.1);
    width: 1020px;
    max-width: 1020px;
}

h1 {
    margin-top: 0;
    margin-bottom: 15px;
    font-size: 2rem;
    text-align: center;
}

h2 {
    margin-top: 0;
```

```
    margin-bottom: 15px;
    font-size: 1.6rem;
    text-align: center;
}

#dashboardNavbar {
    position: fixed;
    top: 0;
    left: 0;
    width: 100%;
    background-color: #20e3448f;
    display: flex;
    justify-content: center;
    align-items: center;
    gap: 10px;
    padding: 10px 0;
    box-shadow: 0 2px 8px rgba(0,0,0,0.2);
    z-index: 1000;
}

#dashboardNavbar a {
    text-decoration: none;
    color: #000000;
    padding: 8px 16px;
    border-radius: 8px;
    font-weight: 500;
    transition: background 0.2s ease, transform 0.1s ease;
}

#dashboardNavbar a:hover {
    background-color: #20e344cc;
```

```
    transform: translateY(-1px);
}

button[type="submit"], input[type="submit"] {
    padding: 0.5em 1em;
    background-color: #20e3448f;
    border: 1px solid #000000;
    color: #000000;
    border-radius: 8px;
    cursor: pointer;
    font-size: 1rem;
    transition: background 0.2s ease, transform 0.1s ease;
    width: 150px;
}

button[type="submit"]:hover, input[type="submit"]:hover {
    background-color: #20e344cc;
    transform: translateY(-1px);
}

span, label {
    font-size: 1rem;
    font-weight: 500;
    color: #000000;
    display: block;
}

#countrySelect {
    padding: 20px;
    display: flex;
    flex-direction: column;
```

```
    gap: 2px;
}

select {
    padding: 10px 40px 10px 12px;
    font-size: 1rem;
    font-family: inherit;
    border: 1px solid #00000024;
    border-radius: 8px;
    background: #fff;
    background-size: 16px;
    cursor: pointer;
    transition: border-color 0.2s ease, box-shadow 0.2s ease;
    box-shadow: 0 1px 2px rgba(0,0,0,0.05);
    width: 100%;
    max-width: 420px;
    box-sizing: border-box;
}

select:hover {
    border-color: #20e344;
}
select:focus {
    border-color: #20e344;
    box-shadow: 0 0 0 3px rgba(32, 227, 68, 0.3);
    outline: none;
}
```

exchange-rate-economic.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<script src="https://cdn.plot.ly/plotly-latest.min.js"></script>
<link rel="stylesheet" href="/css/exchange-rate-economic.css"/>

<head>
    <title>Money Matters</title>
</head>

<body>
    <div id="all">
        <h1>Exchange Rates</h1>

        <!-- Navigation bar -->
        <nav id="dashboardNavbar">
            <a th:href="@{/news}">Economic News</a>
            <a th:href="@{/figures-economic}">Economic Figures</a>
            <a th:href="@{/exchange-rate-economic}">Exchange Rates</a>
            <a th:href="@{/rate-the-economy}">Rate The Economy</a>
            <a th:href="@{/budget-tracker}">Budget Tracker</a>
            <a th:href="@{/investor}">Investor Centre</a>
            <a th:href="@{/cost-of-production}">Cost Of Production</a>
            <a th:href="@{/user-dashboard}">DashBoard</a>
        </nav>

        <!-- Form to select base/wanted currency and date -->
        <div>
            <form id="compare2Currencies" action="/exchange-rate-economic" method="post">
                <label>Select the Base Currency:</label>
```

```
            <select id="baseCurrency" name="baseCurrency">
                <option th:each="entry : ${avaliableCurrencies}"
                        th:value="${entry.value}"
                        th:text="${entry.key}"
                        th:selected="${selectedBaseCurrency} == ${entry.value}">
                </option>
            </select>
            <br>

            <label>Select the Wanted Currency:</label>
            <select id="wantedCurrency" name="wantedCurrency">
                <option th:each="entry : ${avaliableCurrencies}"
                        th:value="${entry.value}"
                        th:text="${entry.key}"
                        th:selected="${selectedWantedCurrency} == ${entry.value}">
                </option>
            </select>
            <br>

            <label>Select a Time Period:</label>
            <input type="date" id="date" name="date" required>
            <br>

            <button type="submit">Submit</button>
        </form>
    </div>

    <!-- Container for exchange rate graph -->
    <div id="graphContainer"></div>

    <!-- Container for currency converter -->
```

```
    <div id="currencyConverter"></div>
</div>

<script type="module" th:inline="javascript">
    import { format } from "https://cdn.skypack.dev/date-fns";

    // Set default date in the form to yesterday's date and restrict max date
    function setDefaultDate() {
        let dateInput = document.getElementById("date");
        let latestDate = new Date();
        latestDate.setDate(latestDate.getDate() - 1); // yesterday
        latestDate = latestDate.toISOString().split('T')[0];

        dateInput.value = latestDate;
        dateInput.max = latestDate;
    }

    function drawGraph() {
        let seriesOfCurrencies = /*[[${seriesOfCurrencies}]]*/ {};
        let baseCurrency = /*[[${baseCurrency}]]*/ "";
        let wantedCurrency = /*[[${wantedCurrency}]]*/ "";

        if (seriesOfCurrencies && Object.keys(seriesOfCurrencies).length > 0) {
            let years = Object.keys(seriesOfCurrencies);
            let data = Object.values(seriesOfCurrencies);

            var line1 = {
                x: years,
                y: data,
                type: 'scatter',
                name: baseCurrency + " to " + wantedCurrency
```

```
        };

        var plotting = [line1];

        var layout = {
            title: baseCurrency + " to " + wantedCurrency,
            xaxis: {title: "Year"},
            yaxis: {title: /*[[${wantedCurrency}]]*/ ""},
            dragmode: false,
            zoom: false,
            paper_bgcolor: "transparent",
            plot_bgcolor: "transparent"
        };

        Plotly.newPlot("graphContainer", plotting, layout, {displayModeBar: false, responsive: true});
    }
}

function showCurrencyConverter() {
    let date = /*[[${date}]]*/ "";
    let baseCurrency = /*[[${baseCurrency}]]*/ "";
    let wantedCurrency = /*[[${wantedCurrency}]]*/ "";
    let convertedCurrency = /*[[${convertedCurrency}]]*/ "";

    if (date !== null) {
        const currencyConverterDiv = document.getElementById("currencyConverter");

        const currencyConverterH2 = document.createElement("h2");
        currencyConverterH2.innerText = "Currency Converter";
        currencyConverterDiv.append(currencyConverterH2);
```

```
                date = new Date(date);
                date = format(date, "dd/MM/yyyy");

                const dateSpan = document.createElement("span");
                dateSpan.innerText = "Date: " + date;
                currencyConverterDiv.append(dateSpan);

                // Show conversion rate
                const baseToWantCurrencySpan = document.createElement("span");
                let convertedCurrency2dp = parseFloat(convertedCurrency).toFixed(2);
                baseToWantCurrencySpan.innerText = "1.00 " + baseCurrency + " = " + convertedCurrency2dp + " " +
wantedCurrency;
                currencyConverterDiv.append(baseToWantCurrencySpan);
                currencyConverterDiv.append(document.createElement("br"));

                // Input for custom amount to convert
                let convertCurrencySpan = document.createElement("span");
                convertCurrencySpan.innerText = "Enter " + baseCurrency + ":";
                currencyConverterDiv.append(convertCurrencySpan);

                let initialCurrency = document.createElement("input");
                initialCurrency.type = "number";
                initialCurrency.step = "0.01";
                initialCurrency.min = "0.01";
                initialCurrency.required = true;
                currencyConverterDiv.append(initialCurrency);

                let convertCurrencyButton = document.createElement("button");
                convertCurrencyButton.textContent = "Convert Currency";
                currencyConverterDiv.append(convertCurrencyButton);
```

```
                const baseToWantCurrencySpan2 = document.createElement("span");

                // Convert on button click
                convertCurrencyButton.addEventListener("click", function() {
                    if (initialCurrency.checkValidity()) {
                        baseToWantCurrencySpan2.innerText = parseFloat(initialCurrency.value).toFixed(2) + " " +
baseCurrency + " = " + calculateConvertedCurrency(convertedCurrency, initialCurrency) + " " + wantedCurrency;
                        currencyConverterDiv.append(baseToWantCurrencySpan2);
                    }
                });
            }
        }

        // Calculate converted currency based on rate
        function calculateConvertedCurrency(convertedCurrency, initialCurrency) {
            return (convertedCurrency * initialCurrency.value).toFixed(2);
        }

        // Reset CSS for empty converter container
        function resetCSS() {
            const currencyConverterDiv = document.getElementById("currencyConverter");

            if (currencyConverterDiv.innerHTML.trim() === "") {
                currencyConverterDiv.style.all = "unset";
            }
        }

        setDefaultDate();
        drawGraph();
        showCurrencyConverter();
        resetCSS();
```

```
    </script>
</body>
</html>
```

exchange-rate-economic.css

```css
body {
    margin: 1vw;
    display: flex;
    flex-direction: column;
    justify-content: flex-start;
    align-items: center;
    background-color: #d4e9cd;
    font-family: "Segoe UI", Tahoma, Geneva, Verdana, sans-serif;
    padding-top: 70px;
}

#all {
    padding: 20px;
    background-color: #ffffff9e;
    border-radius: 12px;
    box-shadow: 0 0 20px rgba(0, 0, 0, 0.1);
    width: 1020px;
    max-width: 1020px;
}

h1 {
    margin-top: 0;
    margin-bottom: 15px;
    font-size: 2rem;
    text-align: center;
}

h2 {
    margin-top: 0;
```

```
    margin-bottom: 15px;
    font-size: 1.6rem;
    text-align: center;
}

#dashboardNavbar {
    position: fixed;
    top: 0;
    left: 0;
    width: 100%;
    background-color: #20e3448f;
    display: flex;
    justify-content: center;
    align-items: center;
    gap: 10px;
    padding: 10px 0;
    box-shadow: 0 2px 8px rgba(0,0,0,0.2);
    z-index: 1000;
}

#dashboardNavbar a {
    text-decoration: none;
    color: #000000;
    padding: 8px 16px;
    border-radius: 8px;
    font-weight: 500;
    transition: background 0.2s ease, transform 0.1s ease;
}

#dashboardNavbar a:hover {
    background-color: #20e344cc;
```

```
    transform: translateY(-1px);
}

button[type="submit"], input[type="submit"] {
    padding: 0.5em 1em;
    background-color: #20e3448f;
    border: 1px solid #000000;
    color: #000000;
    border-radius: 8px;
    cursor: pointer;
    font-size: 1rem;
    transition: background 0.2s ease, transform 0.1s ease;
    width: 150px;
}

button[type="submit"]:hover, input[type="submit"]:hover {
    background-color: #20e344cc;
    transform: translateY(-1px);
}

#compare2Currencies {
    padding: 20px;
    display: flex;
    flex-direction: column;
    gap: 2px;
}

select {
    padding: 10px 40px 10px 12px;
    font-size: 1rem;
    border: 1px solid #00000024;
```

```
    border-radius: 8px;
    background: #ffffff;
    background-size: 16px;
    cursor: pointer;
    transition: border-color 0.2s ease, box-shadow 0.2s ease;
    box-shadow: 0 1px 2px rgba(0,0,0,0.05);
    width: 100%;
    max-width: 420px;
    box-sizing: border-box;
}

select:hover {
    border-color: #20e344;
}
select:focus {
    border-color: #20e344;
    box-shadow: 0 0 0 3px rgba(32, 227, 68, 0.3);
    outline: none;
}

input[type="date"] {
    padding: 8px 12px;
    border: 1px none #ffffff;
    border-radius: 6px;
    font-size: 16px;
    color: #333;
    background-color: #ffffff;
    width: 100%;
    max-width: 395px;
    box-shadow: 0 1px 2px rgba(0,0,0,0.05);
}
```

```css
input[type="date"]:focus {
    box-shadow: 0 0 0 3px rgba(32, 227, 68, 0.3);
    outline: none;
}

span, label {
    font-size: 1rem;
    font-weight: 500;
    color: #000000;
    display: block;
}

#currencyConverter {
    display: flex;
    flex-direction: column;
    align-items: center;
    gap: 12px;
    padding: 20px;
    background-color: #ffffff9e;
    border-radius: 12px;
    box-shadow: 0 0 20px rgba(0,0,0,0.1);
    max-width: 400px;
    margin: 20px auto;
}

#currencyConverter h2 {
    margin: 0;
    font-size: 1.6rem;
    text-align: center;
    color: #000000;
```

```
}

#currencyConverter span {
    font-size: 1rem;
    color: #000000;
    display: block;
    text-align: center;
}

#currencyConverter input[type="number"] {
    padding: 8px 12px;
    font-size: 1rem;
    border: 1px solid #00000024;
    border-radius: 8px;
    box-shadow: 0 1px 2px rgba(0,0,0,0.05);
    width: 100%;
    max-width: 200px;
    box-sizing: border-box;
    transition: border-color 0.2s ease, box-shadow 0.2s ease;
}

#currencyConverter input[type="number"]:focus {
    border-color: #20e344;
    box-shadow: 0 0 0 3px rgba(32, 227, 68, 0.3);
    outline: none;
}

#currencyConverter button {
    padding: 0.5em 1em;
    background-color: #20e3448f;
    border: 1px solid #000000;
```

```
    color: #000000;
    border-radius: 8px;
    cursor: pointer;
    font-size: 1rem;
    transition: background 0.2s ease, transform 0.1s ease;
    width: 250px;
}

#currencyConverter button:hover {
    background-color: #20e344cc;
    transform: translateY(-1px);
}
```

rate-the-economy.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<script src="https://cdn.plot.ly/plotly-latest.min.js"></script>
<link rel="stylesheet" href="/css/rate-the-economy.css"/>

<head>
    <title>Money Matters</title>
</head>

<body>
    <div id="all">
        <h1>Rate The Economy</h1>

        <!-- Navigation bar -->
        <nav id="dashboardNavbar">
            <a th:href="@{/news}">Economic News</a>
            <a th:href="@{/figures-economic}">Economic Figures</a>
            <a th:href="@{/exchange-rate-economic}">Exchange Rates</a>
            <a th:href="@{/rate-the-economy}">Rate The Economy</a>
            <a th:href="@{/budget-tracker}">Budget Tracker</a>
            <a th:href="@{/investor}">Investor Centre</a>
            <a th:href="@{/cost-of-production}">Cost Of Production</a>
            <a th:href="@{/user-dashboard}">DashBoard</a>
        </nav>

        <!-- Form for rating the economy -->
        <div id="rateTheEconomyDiv">
            <form id="rateTheEconomyForm" action="/rate-the-economy" method="post"></form>
        </div>
```

```
    <h2>Your Review</h2>
    <div id="latestConsumerOpinion"></div>

    <!-- Form to select past review dates -->
    <h2>See Other Reviews</h2>
    <div id="chooseDateDiv">
        <form id="chooseDateForm" action="/rate-the-economy" method="post">
            <input type="hidden" name="formName" value="chooseDateForm">
            <label>Choose a Date:</label>
            <select id="selectedDate" name="selectedDate">
                <option th:each="entry : ${avaliableDates}"
                        th:value="${entry.value}"
                        th:text="${entry.key}"></option>
            </select>
            <input type="submit">
        </form>
    </div>

    <!-- Container for all user opinions pie charts -->
    <div id="allUserOpinions"></div>
    <div id="averageConsumerOpinion"></div>
</div>

<script th:inline="javascript">
    function loadRateTheEconomyForm() {
        const currentForm = document.getElementById("rateTheEconomyForm");
        const allQuestionsAndAnswerPairs = /*[[${allQuestionsAndAnswerPairs}]]*/ [];

        const formName = document.createElement("input");
        formName.type = "hidden";
```

```
            formName.name = "formName";
            formName.value = "rateTheEconomyForm";
            currentForm.append(formName);

            // Loop through all questions
            for (let i = 0; i < allQuestionsAndAnswerPairs.length; i++) {
                const questionLabel = document.createElement("label");
                questionLabel.className = "question";
                questionLabel.innerText = "Question " + (i + 1) + ": "  + allQuestionsAndAnswerPairs[i].question + "
";
                currentForm.appendChild(questionLabel);

                const answers = allQuestionsAndAnswerPairs[i].answers;

                let questionNumber = 1;
                // Loop through all answer options for the question
                for (const [key, value] of Object.entries(answers)) {
                    const answerRadio = document.createElement("input");
                    answerRadio.type = "radio";
                    answerRadio.name = "answer" + (i+1);
                    answerRadio.value = key;
                    answerRadio.required = true;

                    const answerLabel = document.createElement("label");
                    answerLabel.className = "options";

                    const answerSpan = document.createElement("span");
                    answerSpan.innerText = (questionNumber) + ") "  + key;

                    answerLabel.appendChild(answerRadio);
                    answerLabel.appendChild(answerSpan);
```

```
            currentForm.appendChild(answerLabel);
            questionNumber++;
        }

        currentForm.appendChild(document.createElement("br"));
    }

    const submitButton = document.createElement("button");
    submitButton.type = "submit";
    submitButton.innerText = "Submit";
    currentForm.appendChild(submitButton);
}

// Generate pie charts for all user opinions
function loadPieCharts() {
    let allUserOpinions = /*[[${allUserOpinions}]]*/ {};
    let allUserOpinionsDiv = document.getElementById("allUserOpinions");

    for (let question in allUserOpinions) {
        let answersFrequencyPairs = allUserOpinions[question];
        let answers = Object.keys(answersFrequencyPairs);
        let frequencies = Object.values(answersFrequencyPairs);

        const data = [{
            values: frequencies,
            labels: answers,
            type: "pie"
        }];

        const layout = {
            title: {text: question, font: {size: 10}},
```

```
                    height: 400,
                    width: 500,
                    dragmode: false,
                    zoom: false,
                    paper_bgcolor: "transparent",
                    plot_bgcolor: "transparent",
                };

                let chartDiv = document.createElement("div");
                chartDiv.className = "pieChart";
                Plotly.newPlot(chartDiv, data, layout, {displayModeBar: false, responsive: true});
                allUserOpinionsDiv.appendChild(chartDiv);
            }

            // Display average consumer opinion
            const averageConsumerOpinion = /*[[${averageConsumerOpinion}]]*/ "";
            if (averageConsumerOpinion != null && averageConsumerOpinion !== "") {
                const averageConsumerOpinionDiv = document.getElementById("averageConsumerOpinion");
                const averageConsumerOpinionSpan = document.createElement("span");
                averageConsumerOpinionSpan.innerText = "Average Consumers' Happiness Of The Economy: " +
averageConsumerOpinion + "%";
                averageConsumerOpinionDiv.append(averageConsumerOpinionSpan);
                averageConsumerOpinionDiv.append(document.createElement("br"));
            }
        }

        // Show the user's latest review
        function loadLatestConsumerOpinion() {
            const latestConsumerOpinionDiv = document.getElementById("latestConsumerOpinion");

            const latestConsumerOpinionDateSpan = document.createElement("span");
```

```
            const latestConsumerOpinionDate = /*[[${consumerOpinionDate}]]*/ "";

            if (latestConsumerOpinionDate == null) {
                latestConsumerOpinionDateSpan.innerText = "Complete The Form To See Your Review";
            } else {
                latestConsumerOpinionDateSpan.innerText = "Latest Form Submission: " + latestConsumerOpinionDate;
            }
            latestConsumerOpinionDiv.append(latestConsumerOpinionDateSpan);
            latestConsumerOpinionDiv.append(document.createElement("br"));

            const latestConsumerOpinionSpan = document.createElement("span");
            const latestConsumerOpinion = /*[[${consumerOpinion}]]*/ "";
            latestConsumerOpinionSpan.innerText = "Your Happiness Of The Economy: " + latestConsumerOpinion + "%";

            if (latestConsumerOpinion != null) {
                latestConsumerOpinionDiv.append(latestConsumerOpinionSpan);
            }
            latestConsumerOpinionDiv.append(document.createElement("br"));
        }

        loadRateTheEconomyForm();
        loadLatestConsumerOpinion();
        loadPieCharts();
    </script>
</body>
</html>
```

rate-the-economy.css

```css
body {
    margin: 1vw;
    display: flex;
    flex-direction: column;
    justify-content: flex-start;
    align-items: center;
    background-color: #d4e9cd;
    font-family: "Segoe UI", Tahoma, Geneva, Verdana, sans-serif;
    padding-top: 70px;
}

#all {
    padding: 20px;
    background-color: #ffffff9e;
    border-radius: 12px;
    box-shadow: 0 0 20px rgba(0, 0, 0, 0.1);
    width: 1020px;
    max-width: 1020px;
}

h1 {
    margin-top: 0;
    margin-bottom: 15px;
    font-size: 2rem;
    text-align: center;
}

h2 {
    margin-top: 0;
```

```
    margin-bottom: 15px;
    font-size: 1.6rem;
    text-align: center;
}

#dashboardNavbar {
    position: fixed;
    top: 0;
    left: 0;
    width: 100%;
    background-color: #20e3448f;
    display: flex;
    justify-content: center;
    align-items: center;
    gap: 10px;
    padding: 10px 0;
    box-shadow: 0 2px 8px rgba(0,0,0,0.2);
    z-index: 1000;
}

#dashboardNavbar a {
    text-decoration: none;
    color: #000000;
    padding: 8px 16px;
    border-radius: 8px;
    font-weight: 500;
    transition: background 0.2s ease, transform 0.1s ease;
}

#dashboardNavbar a:hover {
    background-color: #20e344cc;
```

```
    transform: translateY(-1px);
}

button[type="submit"], input[type="submit"] {
    padding: 0.5em 1em;
    background-color: #20e3448f;
    border: 1px solid #000000;
    color: #000000;
    border-radius: 8px;
    cursor: pointer;
    font-size: 1rem;
    transition: background 0.2s ease, transform 0.1s ease;
    width: 150px;
}

button[type="submit"]:hover, input[type="submit"]:hover {
    background-color: #20e344cc;
    transform: translateY(-1px);
}

#rateTheEconomyForm {
    padding: 20px;
}

label.question {
    font-weight: 400;
    font-size: 1.2rem;
    margin-bottom: 10px;
    display: block;
    text-align: left;
}
```

```css
label.options {
    display: flex;
    align-items: center;
    padding: 12px 16px;
    margin-bottom: 10px;
    border: 2px solid #00000024;
    border-radius: 10px;
    box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
    cursor: pointer;
}

label.options input[type="radio"] {
    display: none;
}

label.options:hover {
    background-color: #d0edc2;
    transform: translateY(-1px);
}

label.options input[type="radio"]:checked + span {
    font-weight: 700;
    border-radius: 10px;
    padding: 12px 16px;
}

#latestConsumerOpinion {
    margin-top: -25px;
    padding: 20px;
    border-radius: 12px;
```

```
    text-align: left;
}

#latestConsumerOpinion span {
    margin: 5px 0;
    font-size: 1.2rem;
    color: #000000;
}

#averageConsumerOpinion {
    margin-top: 20px;
    padding: 20px;
    border-radius: 12px;
    text-align: left;
}

#averageConsumerOpinion span {
    margin: 5px 0;
    font-size: 1.2rem;
    color: #000000;
}

#allUserOpinions {
    display: flex;
    gap: 20px;
    justify-content: center;
    flex-wrap: wrap;
    margin-top: 20px;
}

#chooseDateDiv {
```

```
    margin-top: 20px;
    padding: 20px;
    border-radius: 12px;
    display: flex;
    flex-direction: column;
    align-items: center;
    width: 300px;
}

#chooseDateForm {
    display: flex;
    flex-direction: column;
    width: 100%;
}

#chooseDateForm label {
    font-size: 1.2rem;
    font-weight: 500;
    margin-bottom: 10px;
}

#selectedDate {
    padding: 8px 12px;
    font-size: 1rem;
    border-radius: 8px;
    border: 1px solid #00000024;
    margin-bottom: 15px;
    width: 100%;
    box-sizing: border-box;
}
```

```css
#chooseDateForm input[type="submit"] {
    padding: 0.5em 1em;
    background-color: #20e3448f;
    border: 1px solid #000000;
    color: #000000;
    border-radius: 8px;
    cursor: pointer;
    font-size: 1rem;
    transition: background 0.2s ease, transform 0.1s ease;
    width: 150px;
}

#chooseDateForm input[type="submit"]:hover {
    background-color: #20e344cc;
    transform: translateY(-1px);
}
```

budget-tracker.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<script src="https://cdn.plot.ly/plotly-latest.min.js"></script>
<link rel="stylesheet" href="/css/budget-tracker.css"/>

<head>
    <title>Money Matters</title>
</head>

<body>
    <div id="all">
        <h1>Budget Tracker</h1>

        <!-- Navigation bar -->
        <nav id="dashboardNavbar">
            <a th:href="@{/news}">Economic News</a>
            <a th:href="@{/figures-economic}">Economic Figures</a>
            <a th:href="@{/exchange-rate-economic}">Exchange Rates</a>
            <a th:href="@{/rate-the-economy}">Rate The Economy</a>
            <a th:href="@{/budget-tracker}">Budget Tracker</a>
            <a th:href="@{/investor}">Investor Centre</a>
            <a th:href="@{/cost-of-production}">Cost Of Production</a>
            <a th:href="@{/user-dashboard}">DashBoard</a>
        </nav>

        <div>
            <!Display current user budget data -->
            <div id="userBudget">
                <h2>Current Budget Data</h2>
```

```
    <div id="userCurrentData"></div>
</div>

<!-- Form to update user's budget -->
<div id="updateBudget">
    <h2>Update Budget Data</h2>
    <form id="budget-inputs" action="/budget-tracker" method="post">
        <input type="hidden" name="formName" value="userBudget">

        <label>Enter Current Budget (£):</label>
        <input type="number" id="budgetSize" name="budgetSize" placeholder="100" step="0.01" required>
        <br>

        <label>Frequency of Budget Renewal:</label>
        <select name="frequencyOfRenewal" required>
            <option value="Daily">Daily</option>
            <option value="Weekly">Weekly</option>
            <option value="Monthly">Monthly</option>
        </select>
        <br>

        <input type="submit">
    </form>
</div>

<!-- Form to add a purchased good -->
<div id="addGood">
    <h2>Add Good</h2>
    <form id="good-inputs" action="/budget-tracker" method="post">
        <input type="hidden" name="formName" value="userGoods">
```

```
                <label>Enter the name of the good you have purchased:</label>
                <input type="text" id="goodName" name="goodName" placeholder="Pepsi" required>
                <br>

                <label>Enter the price of the good you have purchased:</label>
                <input type="number" id="goodPrice" name="goodPrice" placeholder="1" step="0.01" required>
                <br>

                <label>Enter the quantity of the good you have purchased:</label>
                <input type="number" id="goodQuantity" name="goodQuantity" placeholder="1" step="1" required>
                <br>

                <input type="submit">
            </form>
        </div>

        <!-- Table to display latest goods purchased -->
        <div id="latestGoodsPurchased">
            <h2>Goods Purchased</h2>

            <!-- Form to filter purchases -->
            <form id="filterBudget" action="/budget-tracker" method="post">
                <input type="hidden" name="formName" value="filterBudget">
                <select id="sort" name="sort" required>
                    <option value="date (latest)" th:selected="${selectedSort == 'date (latest)'}">Date
(Latest)</option>
                    <option value="date (earliest)" th:selected="${selectedSort == 'date (earliest)'}">Date
(Earliest)</option>
                    <option value="most expensive" th:selected="${selectedSort == 'most expensive'}">Most To Least
Expensive</option>
                    <option value="least expensive" th:selected="${selectedSort == 'least expensive'}">Least To
```

```
Most Expensive</option>
                        <option value="most bought" th:selected="${selectedSort == 'most bought'}">Most To Least
Bought</option>
                        <option value="least bought" th:selected="${selectedSort == 'least bought'}">Least To Most
Bought</option>
                    </select>
                    <input type="submit">
                </form>

                <!-- Table showing purchases -->
                <table border="1">
                    <thead>
                        <tr>
                            <th>Date</th>
                            <th>Name</th>
                            <th>Price (£)</th>
                            <th>Quantity</th>
                            <th>Total Price (£)</th>
                        </tr>
                    </thead>
                    <tbody id="tableContents"></tbody>
                </table>
            </div>

            <!-- Risky purchases -->
            <div id="outliers">
                <h2>Risky Purchases</h2>
            </div>

            <div id="graphContainer1" style="width: 100%; height: 500px;"></div>
            <div id="graphContainer2" style="width: 100%; height: 500px;"></div>
```

```
        </div>
    </div>

    <script type="module" th:inline="javascript">
        import {format} from "https://cdn.skypack.dev/date-fns";

        let userCurrentDataDiv = document.getElementById("userCurrentData");

        function showBudgetSize() {
            let budgetSize = /*[[${budgetSize}]]*/ "";
            let budgetSizeSpan = document.createElement("span");
            budgetSizeSpan.innerText = "Budget Size: £" + budgetSize.toFixed(2);
            userCurrentDataDiv.append(budgetSizeSpan);
        }

        function showCurrentBudget() {
            let currentBudget = /*[[${currentBudget}]]*/ "";
            let currentBudgetSpan = document.createElement("span");
            currentBudgetSpan.innerText = "Current Budget: £" + currentBudget.toFixed(2);
            userCurrentDataDiv.append(currentBudgetSpan);
        }

        function showSustainableSpending() {
            let sustainableSpending = /*[[${sustainableSpending}]]*/ "";
            let sustainableSpendingSpan = document.createElement("span");

            if (!sustainableSpending || sustainableSpending === "" || sustainableSpending === "null") {
                sustainableSpendingSpan.innerText = "Sustainable Spending Per Day: £ N/A";
            }

            else {
```

```
                sustainableSpendingSpan.innerText = "Sustainable Spending Per Day: £" +
parseFloat(sustainableSpending).toFixed(2);
            }

            userCurrentDataDiv.append(sustainableSpendingSpan);
        }

        function showFrequencyOfBudgetRenewal() {
            let frequencyOfBudgetRenewal = /*[[${frequencyOfBudgetRenewal}]]*/ "";
            let frequencyOfBudgetRenewalSpan = document.createElement("span");
            frequencyOfBudgetRenewalSpan.innerText = "Frequency Of Budget Renewal: " + frequencyOfBudgetRenewal;
            userCurrentDataDiv.append(frequencyOfBudgetRenewalSpan);
        }

        function showTodaySpending() {
            let todaySpending = /*[[${todaySpending}]]*/ "";
            let todaySpendingSpan = document.createElement("span");
            todaySpendingSpan.innerText = "Today's Spending: £" + parseInt(todaySpending).toFixed(2);
            userCurrentDataDiv.append(todaySpendingSpan);
        }

        function showOutliers() {
            let outliers = /*[[${riskyPurchases}]]*/ [];
            let outliersDiv = document.getElementById("outliers");

            for (let i = 0; i < outliers.length; i++) {
                let outlierSpan = document.createElement("span");
                outlierSpan.innerText = outliers[i].goodName + " at £" + outliers[i].price.toFixed(2) + " is a risky
purchase.";
                outliersDiv.append(outlierSpan);
                outliersDiv.appendChild(document.createElement("br"));
```

```
    }
}

function showFutureBudget() {
    let futureBudget = /*[[${futureBudget}]]*/ [];
    futureBudget = JSON.parse(futureBudget);

    let months = /*[[${months}]]*/ [];

    let essentials = [];
    let leisure = [];
    let savings = [];

    for (let i = 0; i < futureBudget.length; i++) {
        let yearData = futureBudget[i];
        essentials.push(parseFloat(yearData[0]).toFixed(2));
        leisure.push(parseFloat(yearData[1]).toFixed(2));
        savings.push(parseFloat(yearData[2]).toFixed(2));
    }

    var line1 = {x: months, y: essentials, type: "scatter", name: "Essentials"};
    var line2 = {x: months, y: leisure, type: "scatter", name: "Leisure"};
    var line3 = {x: months, y: savings, type: "scatter", name: "Savings"};

    var plotting = [line1, line2, line3];

    var layout = {
        title: "Future Budget",
        xaxis: {title: "Months", type: "category"},
        yaxis: {title: "Budget Components (£)"},
        dragmode: false,
```

```
            zoom: false,
            paper_bgcolor: "transparent",
            plot_bgcolor: "transparent"
        };

        Plotly.newPlot("graphContainer2", plotting, layout, {displayModeBar: false, responsive: true});
    }

    function showLatestGoodsPurchased() {
        const consumerPurchases = /*[[${consumerPurchases}]]*/ [];
        const tableContents = document.getElementById("tableContents");

        for (let i = 0; i < consumerPurchases.length; i++) {
            const row = document.createElement("tr");
            const cell1 = document.createElement("td");
            const cell2 = document.createElement("td");
            const cell3 = document.createElement("td");
            const cell4 = document.createElement("td");
            const cell5 = document.createElement("td");

            let purchaseDate = format(new Date(consumerPurchases[i].purchaseDate), "dd/MM/yyyy");
            let goodName = consumerPurchases[i].goodName;
            let goodPrice = consumerPurchases[i].goodPrice.toFixed(2);
            let goodQuantity = consumerPurchases[i].goodQuantity;
            let totalPrice = consumerPurchases[i].totalPrice.toFixed(2);

            cell1.textContent = purchaseDate;
            cell2.textContent = goodName;
            cell3.textContent = goodPrice;
            cell4.textContent = goodQuantity;
            cell5.textContent = totalPrice;
```

```
            row.append(cell1, cell2, cell3, cell4, cell5);
            tableContents.append(row);
        }
    }

    function showWeeklySpending() {
        const dateTotalPricePairs = /*[[${dateTotalPricePairs}]]*/ {};
        const dates = Object.keys(dateTotalPricePairs);
        const totalPrice = Object.values(dateTotalPricePairs);

        var bars = {x: dates, y: totalPrice, type: "bar"};
        var plotting = [bars];

        var layout = {
            title: "This Week's Spendings",
            xaxis: {title: "Days"},
            yaxis: {title: "Amount Spent (£)"},
            dragmode: false,
            zoom: false,
            paper_bgcolor: "transparent",
            plot_bgcolor: "transparent"
        };

        Plotly.newPlot("graphContainer1", plotting, layout, {displayModeBar: false, responsive: true});
    }

    function showWindowsAlert() {
        let windowsAlert = /*[[${windowsAlert}]]*/ false;
        let windowsAlertMessage = /*[[${windowsAlertMessage}]]*/ "";
```

```
            if (windowsAlert === true) {
                window.alert(windowsAlertMessage);
            }
        }

        // Call functions to display all dynamic data
        showWindowsAlert();
        showBudgetSize();
        showCurrentBudget();
        showFrequencyOfBudgetRenewal();
        showSustainableSpending();
        showOutliers();
        showTodaySpending();
        showFutureBudget();
        showWeeklySpending();
        showLatestGoodsPurchased();
    </script>
</body>
</html>
```

budget-tracker.css

```css
body {
    margin: 1vw;
    display: flex;
    flex-direction: column;
    justify-content: flex-start;
    align-items: center;
    background-color: #d4e9cd;
    font-family: "Segoe UI", Tahoma, Geneva, Verdana, sans-serif;
    padding-top: 70px;
}

#all {
    padding: 20px;
    background-color: #ffffff9e;
    border-radius: 12px;
    box-shadow: 0 0 20px rgba(0, 0, 0, 0.1);
    width: 1020px;
    max-width: 1020px;
}

h1 {
    margin-top: 0;
    margin-bottom: 15px;
    font-size: 2rem;
    text-align: center;
}

h2 {
    margin-top: 0;
```

```
    margin-bottom: 15px;
    font-size: 1.6rem;
    text-align: center;
}

#dashboardNavbar {
    position: fixed;
    top: 0;
    left: 0;
    width: 100%;
    background-color: #20e3448f;
    display: flex;
    justify-content: center;
    align-items: center;
    gap: 10px;
    padding: 10px 0;
    box-shadow: 0 2px 8px rgba(0,0,0,0.2);
    z-index: 1000;
}

#dashboardNavbar a {
    text-decoration: none;
    color: #000000;
    padding: 8px 16px;
    border-radius: 8px;
    font-weight: 500;
    transition: background 0.2s ease, transform 0.1s ease;
}

#dashboardNavbar a:hover {
    background-color: #20e344cc;
```

```
    transform: translateY(-1px);
}

button[type="submit"], input[type="submit"] {
    padding: 0.5em 1em;
    background-color: #20e3448f;
    border: 1px solid #000000;
    color: #000000;
    border-radius: 8px;
    cursor: pointer;
    font-size: 1rem;
    transition: background 0.2s ease, transform 0.1s ease;
    width: 150px;
}

button[type="submit"]:hover, input[type="submit"]:hover {
    background-color: #20e344cc;
    transform: translateY(-1px);
}

#userBudget, #updateBudget, #addGood, #latestGoodsPurchased, #outliers, #graphContainer1, #graphContainer2 {
    margin-top: 24px;
    padding: 20px;
    border-radius: 14px;
    background: #ffffff;
}

#userCurrentData {
    display: grid;
    grid-template-columns: repeat(2, minmax(0,1fr));
    gap: 12px 18px;
```

```
    margin-top: 14px;
}

form {
    display: flex;
    flex-direction: column;
    gap: 12px;
    padding: 20px;
    border-radius: 12px;
    border: 1px solid var(--border);
}

label {
    font-weight: 500;
    font-size: 1.05rem;
}

input[type="text"], input[type="number"], select {
    padding: 10px 12px;
    font-size: 1rem;
    border-radius: 8px;
    border: 1px solid #000000;
    background: #ffffff;
    outline: none;
    width: 300px;
}

input[type="text"]:focus, input[type="number"]:focus, select:focus {
    box-shadow: 0 0 0 3px #20e34440;
    border-color: #20e34466;
}
```

```css
#latestGoodsPurchased form#filterBudget {
    flex-direction: row;
    align-items: center;
    gap: 10px;
    padding: 10px 14px;
    border-radius: 10px;
}

table {
    width: 100%;
    border-collapse: collapse;
    margin-top: 16px;
    border: 1px solid var(--border);
    border-radius: 12px;
    overflow: hidden;
    font-size: 0.95rem;
}

thead th {
    text-align: left;
    padding: 12px 14px;
    background: #d6f0ca;
    font-weight: 600;
    position: sticky;
    top: 0;
    z-index: 1;
}

tbody tr:nth-child(odd) {
    background: #fdfdfd;
```

```
}

tbody tr:nth-child(even) {
    background: #f7fff4;
}

tbody tr:hover {
    background: #eaffea;
}

td {
    padding: 12px 14px;
    border-bottom: 1px solid #00000010;
}

#outliers span {
    display: block;
    margin: 6px 0;
    padding: 10px 12px;
    background: #fff9f4;
    border-radius: 8px;
    font-size: 1rem;
}

#graphContainer1, #graphContainer2 {
    width: 100%;
    height: 500px;
    margin-top: 24px;
    border: 1px solid var(--border);
    border-radius: 14px;
    box-shadow: 0 4px 14px rgba(0,0,0,.06);
```

```
    padding: 10px;
}
```

investor.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<script src="https://cdn.plot.ly/plotly-latest.min.js"></script>
<link rel="stylesheet" href="/css/investor.css"/>

<head>
    <title>Money Matters</title>
</head>

<body>
    <div id="all">
        <h1>Investor Centre</h1>

        <!-- Navigation bar -->
        <nav id="dashboardNavbar">
            <a th:href="@{/news}">Economic News</a>
            <a th:href="@{/figures-economic}">Economic Figures</a>
            <a th:href="@{/exchange-rate-economic}">Exchange Rates</a>
            <a th:href="@{/rate-the-economy}">Rate The Economy</a>
            <a th:href="@{/budget-tracker}">Budget Tracker</a>
            <a th:href="@{/investor}">Investor Centre</a>
            <a th:href="@{/cost-of-production}">Cost Of Production</a>
            <a th:href="@{/user-dashboard}">DashBoard</a>
        </nav>

        <!-- User budget display -->
        <div id="userData"></div>
```

```
<!-- Form to select stock and time period -->
<div id="selectCompanyDiv">
    <form id="selectCompany" action="/investor" method="post">
        <input type="hidden" name="formName" value="selectCompany">
        <label>Choose a Stock: </label>
        <select id="tickerSymbol" name="tickerSymbol">
            <option th:each="entry : ${allTickerInfo}"
                    th:value="${entry.value}"
                    th:text="${entry.key}"
                    th:selected="${selectedTickerSymbol == entry.value}">
            </option>
        </select>

        <label>Choose a Time Period: </label>
        <select id="timeSpan" name="timeSpan">
            <option th:each="entry : ${timePeriods}"
                    th:value="${entry.value}"
                    th:text="${entry.key}"
                    th:selected="${selectedTimeSpan == entry.value}">
            </option>
        </select>

        <input type="submit">
    </form>
</div>

<!-- Graph container for stock price and volume -->
<div id="graphContainer" th:class="${graphContainerDiv == 'graphContainerDiv' ? 'show' : 'hide'}"></div>
```

```
        <!-- Company details display -->
        <div id="companyDetails" th:class="${companyDetailsDiv == 'companyDetailsDiv' ? 'show' : 'hide'}"></div>

        <!-- Purchase stock form -->
        <div id="purchaseStockDiv" th:class="${purchaseStockDiv == 'purchaseStockDiv' ? 'show' : 'hide'}">
            <h2 id=purchaseStockHeading></h2>
            <form id="purchaseStock" action="/investor" method="post">
                <input type="hidden" name="formName" value="purchaseStock">
            </form>
        </div>

        <!-- Sell stock form -->
        <div id="sellStockDiv" th:class="${sellStockDiv == 'sellStockDiv' ? 'show' : 'hide'}">
            <h2 id=sellStockHeading></h2>
            <form id="sellStock" action="/investor" method="post">
                <input type="hidden" name="formName" value="sellStock">
            </form>
        </div>

        <!-- Display of owned stocks -->
        <div id="ownedStocksDiv">
            <h2>Owned Stocks</h2>
        </div>
    </div>

    <script th:inline="javascript">
        function showUserData() {
            const stockBudget = /*[[${stockBudget}]]*/ 0;
            const userDataDiv = document.getElementById("userData");
```

```
    const labelSpan = document.createElement("span");
    labelSpan.innerText = "Budget Available For Stocks: £" + stockBudget.toFixed(2);
    userDataDiv.append(labelSpan);
}

// Display user's owned stocks
function showOwnedStock() {
    const ownedStocksDiv = document.getElementById("ownedStocksDiv");

    const allStockNames = /*[[${allStockNames}]]*/ [];
    const allNetQuantities = /*[[${allNetQuantities}]]*/ [];
    const allTotalAskPrices = /*[[${allTotalAskPrices}]]*/ [];
    const allCurrentAskPrices = /*[[${allCurrentAskPrices}]]*/ [];
    const allAverageAskPrices = /*[[${allAverageAskPrices}]]*/ [];
    const allPotentialProfit = /*[[${allPotentialProfit}]]*/ [];

    for (let i = 0; i < allStockNames.length; i++) {
        const currentStockDiv = document.createElement("div");
        currentStockDiv.className = "currentStock";

        const stockNameSpan = document.createElement("span");
        stockNameSpan.innerText = "Stock: " + allStockNames[i];

        const quantitySpan = document.createElement("span");
        quantitySpan.innerText = "Quantity: " + allNetQuantities[i];

        const totalAskPricesSpan = document.createElement("span");
        totalAskPricesSpan.innerText = "Bought Stock Price: £" + allTotalAskPrices[i].toFixed(2);
```

```
                const averageAskPricesSpan = document.createElement("span");
                averageAskPricesSpan.innerText = "Average Bought Stock Price: £" +
allAverageAskPrices[i].toFixed(2);

                const currentAskPricesSpan = document.createElement("span");
                currentAskPricesSpan.innerText = "Current Stock Price: £" + allCurrentAskPrices[i].toFixed(2);

                const potentialProfitFor1StockSpan = document.createElement("span");
                const potentialTotalProfitSpan = document.createElement("span");

                if (allPotentialProfit[i] < 0) {
                    potentialProfitFor1StockSpan.innerText = "Potential Loss If One Stock Sold: £" + (-1 *
allPotentialProfit[i]).toFixed(2);
                    potentialTotalProfitSpan.innerText = "Potential Total Loss: £" + (-1 * allPotentialProfit[i]
* allNetQuantities[i]).toFixed(2);
                }

                else {
                    potentialProfitFor1StockSpan.innerText = "Potential Profit If One Stock Sold: £" +
allPotentialProfit[i].toFixed(2);
                    potentialTotalProfitSpan.innerText = "Potential Total Profit: £" + (allPotentialProfit[i] *
allNetQuantities[i]).toFixed(2);
                }

                currentStockDiv.append(stockNameSpan);
                currentStockDiv.append(quantitySpan);
                currentStockDiv.append(totalAskPricesSpan);
                currentStockDiv.append(averageAskPricesSpan);
```

```
            currentStockDiv.append(currentAskPricesSpan);
            currentStockDiv.append(potentialProfitFor1StockSpan);
            currentStockDiv.append(potentialTotalProfitSpan);
            ownedStocksDiv.append(currentStockDiv);
        }
    }

    // Display selected company details
    function showCompanyDetails() {
        let companyRecord = /*[[${companyInformation}]]*/ {};
        let askPrice = /*[[${askPrice}]]*/ "";

        let companyDetailsDiv = document.getElementById("companyDetails");

        let shortName = companyRecord["shortName"];
        let sector = companyRecord["sector"];
        let industry = companyRecord["industry"];
        let fullTimeEmployees = companyRecord["fullTimeEmployees"];
        let country = companyRecord["country"];
        let website = companyRecord["website"];

        let shortNameSpan = document.createElement("span");
        shortNameSpan.innerText = "Company: " + shortName;

        let sectorSpan = document.createElement("span");
        sectorSpan.innerText = "Sector: " + sector;

        let industrySpan = document.createElement("span");
        industrySpan.innerText = "Industry: " + industry;
```

```
let fullTimeEmployeesSpan = document.createElement("span");
fullTimeEmployeesSpan.innerText = "Full Time Employees: " + fullTimeEmployees;

let countrySpan = document.createElement("span");
countrySpan.innerText = "Country: " + country;

let websiteSpan = document.createElement("span");
let websiteLink = document.createElement("a");
websiteLink.href = website;
websiteLink.innerText = website;
websiteSpan.innerText = "Website: ";

let askPriceSpan = document.createElement("span");
askPriceSpan.innerText = "Ask Price: £" + askPrice.toFixed(2);

let SMA10DayTrend = /*[[${SMA10DayTrend}]]*/ "";
let SMA10DayGradient = /*[[${SMA10DayGradient}]]*/ "";

let SMA50DayTrend = /*[[${SMA50DayTrend}]]*/ "";
let SMA50DayGradient = /*[[${SMA50DayGradient}]]*/ "";

let SMA100DayTrend = /*[[${SMA100DayTrend}]]*/ "";
let SMA100DayGradient = /*[[${SMA100DayGradient}]]*/ "";

let SMA10DayTrendSpan = document.createElement("span");
SMA10DayTrendSpan.innerText = "10-Day SMA Trend: " + SMA10DayTrend;

let SMA10DayGradientSpan = document.createElement("span");
```

```
SMA10DayGradientSpan.innerText = "10-Day SMA Gradient: " + SMA10DayGradient.toFixed(2);

let SMA50DayTrendSpan = document.createElement("span");
SMA50DayTrendSpan.innerText = "50-Day SMA Trend: " + SMA50DayTrend;

let SMA50DayGradientSpan = document.createElement("span");
SMA50DayGradientSpan.innerText = "50-Day SMA Gradient: " + SMA50DayGradient.toFixed(2);

let SMA100DayTrendSpan = document.createElement("span");
SMA100DayTrendSpan.innerText = "100-Day SMA Trend: " + SMA100DayTrend;

let SMA100DayGradientSpan = document.createElement("span");
SMA100DayGradientSpan.innerText = "100-Day SMA Gradient: " + SMA100DayGradient.toFixed(2);

companyDetailsDiv.append(shortNameSpan);
companyDetailsDiv.append(sectorSpan);
companyDetailsDiv.append(industrySpan);
companyDetailsDiv.append(fullTimeEmployeesSpan);
companyDetailsDiv.append(countrySpan);
websiteSpan.appendChild(websiteLink);
companyDetailsDiv.append(websiteSpan);
companyDetailsDiv.append(askPriceSpan);

companyDetailsDiv.append(document.createElement("br"));

companyDetailsDiv.append(SMA10DayTrendSpan);
companyDetailsDiv.append(SMA10DayGradientSpan);

companyDetailsDiv.append(SMA50DayTrendSpan);
```

```
    companyDetailsDiv.append(SMA50DayGradientSpan);

    companyDetailsDiv.append(SMA100DayTrendSpan);
    companyDetailsDiv.append(SMA100DayGradientSpan);
}

// Show purchase stock form
function showPurchaseStock() {
    const stockBudget = /*[[${stockBudget}]]*/ 0;

    const askPrice = /*[[${askPrice}]]*/ "";
    const companyName = /*[[${companyName}]]*/ "";

    let maxBuy = Math.floor(stockBudget / askPrice);

    const purchaseStockForm = document.getElementById("purchaseStock");
    const purchaseStocksHeading = document.getElementById("purchaseStockHeading");
    purchaseStocksHeading.innerText = "Purchase Stocks";

    const askPriceLabel = document.createElement("label");
    askPriceLabel.innerText = "Buy Stock From " + companyName + ": £" + askPrice.toFixed(2);
    purchaseStockForm.append(askPriceLabel);

    const quantityLabel = document.createElement("label");
    quantityLabel.innerText = "Quantity: ";
    purchaseStockForm.append(quantityLabel);

    const quantityBuy = document.createElement("input");
    quantityBuy.id = "quantityBuy";
```

```
    quantityBuy.name = "quantityBuy";
    quantityBuy.type = "number";
    quantityBuy.step = "1";
    quantityBuy.max = maxBuy;
    quantityBuy.min = 1;
    purchaseStockForm.append(quantityBuy);

    let submitInput = document.createElement("input");
    submitInput.type = "submit";
    purchaseStockForm.append(submitInput);
}

// Show sell stock form
function showSellStock() {
    const askPrice = /*[[${askPrice}]]*/ "";
    const companyName = /*[[${companyName}]]*/ "";

    const allStockNames = /*[[${allStockNames}]]*/ [];
    const allNetQuantities = /*[[${allNetQuantities}]]*/ [];
    let maxSell = 0;

    for (let i = 0; i < allStockNames.length; i++) {
        if (allStockNames[i] === companyName) {
            maxSell = allNetQuantities[i];
            break;
        }
    }

    const sellStockForm = document.getElementById("sellStock");
```

```
    const sellStocksHeading = document.getElementById("sellStockHeading");
    sellStocksHeading.innerText = "Sell Stocks";

    const askPriceLabel = document.createElement("label");
    askPriceLabel.innerText = "Sell Stock From " + companyName + ": £" + askPrice.toFixed(2);
    sellStockForm.append(askPriceLabel);

    const quantityLabel = document.createElement("label");
    quantityLabel.innerText = "Quantity: ";
    sellStockForm.append(quantityLabel);

    const quantitySell = document.createElement("input");
    quantitySell.id = "quantitySell";
    quantitySell.name = "quantitySell";
    quantitySell.type = "number";
    quantitySell.step = "1";
    quantitySell.max = maxSell;
    quantitySell.min = 1;
    sellStockForm.append(quantitySell);

    let submitInput = document.createElement("input");
    submitInput.type = "submit";
    sellStockForm.append(submitInput);
}

// Display stock price and volume chart
function showStockPriceAndVolumeGraph() {
    const stockPriceComponents = /*[[${stockPriceComponents}]]*/ [[]];
    const dates = stockPriceComponents[0];
```

```
const open = stockPriceComponents[1];
const high = stockPriceComponents[2];
const low = stockPriceComponents[3];
const close = stockPriceComponents[4];
const volume = stockPriceComponents[5];

const companyName = /*[[${companyName}]]*/ "";

const Line1 = {
    x: dates,
    open: open,
    high: high,
    low: low,
    close: close,
    type: "candlestick",
    name: "Price",
    yaxis: "y1"
};

const Bar1 = {
    x: dates,
    y: volume,
    type: "bar",
    name: "Volume",
    yaxis: "y2",
    marker: { color: "rgba(100, 100, 255, 0.5)" }
};

const plotting = [Line1, Bar1];
```

```
            const layout = {
                title: { text: companyName + "'s Stock Price and Volume" },
                xaxis: { title: "Date", showgrid: false, tickangle: -45 },
                yaxis: { title: "Volume", side: "right", overlaying: "y2", showgrid: false },
                yaxis2: { title: "Price", side: "left", showgrid: true },
                dragmode: false,
                zoom: false,
                paper_bgcolor: "transparent",
                plot_bgcolor: "transparent"
            };

            Plotly.newPlot("graphContainer", plotting, layout, {displayModeBar: false, responsive: true});
        }

        showUserData();
        showOwnedStock();
        showStockPriceAndVolumeGraph();
        showCompanyDetails();
        showPurchaseStock();
        showSellStock();
    </script>
</body>
</html>
```

investor.css

```css
body {
    margin: 1vw;
    display: flex;
    flex-direction: column;
    justify-content: flex-start;
    align-items: center;
    background-color: #d4e9cd;
    font-family: "Segoe UI", Tahoma, Geneva, Verdana, sans-serif;
    padding-top: 70px;
}

#all {
    padding: 20px;
    background-color: #ffffff9e;
    border-radius: 12px;
    box-shadow: 0 0 20px rgba(0, 0, 0, 0.1);
    width: 1020px;
    max-width: 1020px;
}

h1 {
    margin-top: 0;
    margin-bottom: 15px;
    font-size: 2rem;
    text-align: center;
}

h2 {
    margin-top: 0;
```

```
    margin-bottom: 15px;
    font-size: 1.6rem;
    text-align: center;
}

#dashboardNavbar {
    position: fixed;
    top: 0;
    left: 0;
    width: 100%;
    background-color: #20e3448f;
    display: flex;
    justify-content: center;
    align-items: center;
    gap: 10px;
    padding: 10px 0;
    box-shadow: 0 2px 8px rgba(0,0,0,0.2);
    z-index: 1000;
}

#dashboardNavbar a {
    text-decoration: none;
    color: #000000;
    padding: 8px 16px;
    border-radius: 8px;
    font-weight: 500;
    transition: background 0.2s ease, transform 0.1s ease;
}

#dashboardNavbar a:hover {
    background-color: #20e344cc;
```

```
    transform: translateY(-1px);
}

button[type="submit"], input[type="submit"] {
    padding: 0.5em 1em;
    background-color: #20e3448f;
    border: 1px solid #000000;
    color: #000000;
    border-radius: 8px;
    cursor: pointer;
    font-size: 1rem;
    transition: background 0.2s ease, transform 0.1s ease;
    width: 150px;
}

button[type="submit"]:hover, input[type="submit"]:hover {
    background-color: #20e344cc;
    transform: translateY(-1px);
}

#graphContainer,
#companyDetails,
#purchaseStockDiv,
#sellStockDiv {
    display: none;
}

#userData,
#selectCompanyDiv,
#graphContainer.show,
#companyDetails.show,
```

```css
#purchaseStockDiv.show,
#sellStockDiv.show,
#ownedStocksDiv {
    display: block;
    padding: 20px;
    border-radius: 12px;
    box-shadow: 0 0 20px rgba(0, 0, 0, 0.1);
    margin-bottom: 20px;
    width: 100%;
    box-sizing: border-box;
}

#graphContainer {
    width: 100%;
    height: 500px;
}

form {
    display: flex;
    flex-direction: column;
    gap: 12px;
    max-width: 420px;
    width: 100%;
    box-sizing: border-box;
}

input[type="number"],
input[type="text"],
input[type="date"],
select {
    padding: 10px 12px;
```

```
    font-size: 1rem;
    border: 1px solid #00000024;
    border-radius: 8px;
    background: #ffffff;
    box-shadow: 0 1px 2px rgba(0,0,0,0.05);
    width: 100%;
    transition: border-color 0.2s ease, box-shadow 0.2s ease;
    box-sizing: border-box;
}

input[type="date"] { padding: 8px 12px; border-radius: 6px; max-width: 395px; }

input[type="number"]:hover,
input[type="text"]:hover,
input[type="date"]:hover,
select:hover {
    border-color: #20e344;
}

input[type="number"]:focus,
input[type="text"]:focus,
input[type="date"]:focus,
select:focus {
    border-color: #20e344;
    box-shadow: 0 0 0 3px rgba(32, 227, 68, 0.3);
    outline: none;
}

span, label {
    font-size: 1rem;
    font-weight: 500;
```

```
    color: #000000;
    display: block;
}

#ownedStocksDiv {
    display: grid;
    flex-direction: column;
    grid-template-columns: repeat(3, minmax(0, 1fr));
    gap: 12px;
}

.currentStock {
    gap: 8px;
    padding: 8px;
    border: 1px none #ddd;
    border-radius: 6px;
}

#companyDetails a {
    color: #20a844;
    text-decoration: none;
}

#companyDetails a:hover {
    text-decoration: underline;
}
```

cost-of-production.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<script src="https://cdn.plot.ly/plotly-latest.min.js"></script>
<link rel="stylesheet" href="/css/cost-of-production.css"/>

<head>
    <title>Money Matters</title>
</head>

<body>
    <div id="all">
        <h1>Cost Of Production</h1>

        <!-- Navigation bar  -->
        <nav id="dashboardNavbar">
            <a th:href="@{/news}">Economic News</a>
            <a th:href="@{/figures-economic}">Economic Figures</a>
            <a th:href="@{/exchange-rate-economic}">Exchange Rates</a>
            <a th:href="@{/rate-the-economy}">Rate The Economy</a>
            <a th:href="@{/budget-tracker}">Budget Tracker</a>
            <a th:href="@{/investor}">Investor Centre</a>
            <a th:href="@{/cost-of-production}">Cost Of Production</a>
            <a th:href="@{/user-dashboard}">DashBoard</a>
        </nav>

        <!-- Table displaying company factor input details -->
        <div id="allFactorInputsDiv">
            <h2>Company Details</h2>
            <table border="1">
```

```
            <thead>
                <tr>
                    <th>Type</th>
                    <th>Name</th>
                    <th>Description</th>
                    <th>Marginal Cost (£)</th>
                    <th>Marginal Revenue (£)</th>
                    <th>Marginal Profit (£)</th>
                </tr>
            </thead>
            <tbody id="tableContents"></tbody>
        </table>
    </div>

    <!-- Form to insert new factor input -->
    <div id="factorInputInsertDiv">
        <h2>Insert Factor Input</h2>
        <form id="factorInputInsert" action="/cost-of-production" method="post">
            <input type="hidden" name="formName" value="factorInputInsert"/>

            <label>Enter Type of Factor Input</label>
            <select id="factorInputType" name="factorInputType">
                <option value="Capital">Capital</option>
                <option value="Labour">Labour</option>
            </select>
            </br>

            <label>Name of Factor Input:</label>
            <input type="text" id="factorInputName" name="factorInputName"/>
            </br>
```

```
            <label>Provide a Description of the Factor:</label>
            <input type="text" id="factorInputDescription" name="factorInputDescription"/>
            </br>

            <label>What is Marginal Cost of the factor input:</label>
            <input type="number" id="marginalCost" name="marginalCost" step="0.01" placeholder="7.30"/>
            </br>

            <label>What is Marginal Revenue of the factor input:</label>
            <input type="number" id="marginalRevenue" name="marginalRevenue" step="0.01" placeholder="10.40"/>
            </br>

            <button type="submit">Submit</button>
            </br>
        </form>
    </div>

    <!-- Display overall cost/revenue/profit -->
    <div id="costOfProductionInformation">
        <h2>Current Data</h2>
    </div>

    <!-- Form to calculate optimal factor input based on maximum cost -->
    <div id="optimalFactorInput">
        <h2>Optimal Workers</h2>
        <form id="factorInputInsert" action="/cost-of-production" method="post">
            <input type="hidden" name="formName" value="optimalFactorInput"/>
            <label>Enter The Maximum Cost (this will be used to find the optimal workers):</label>
            <input type="number" id="maximumCost" name="maximumCost" step="0.01"/>
            </br>
            <button type="submit">Submit</button>
```

```
        </br>
    </form>
</div>

<!-- Form to update existing factor input -->
<div id="factorInputUpdateDiv">
    <h2>Update Factor Input</h2>
    <form id="factorInputUpdate" action="/cost-of-production" method="post">
        <input type="hidden" name="formName" value="factorInputUpdate"/>
        <label>Select Factor Input</label>
        <select id="factorInputName" name="factorInputName">
            <option th:each="entry : ${allFactorInputs}"
                    th:value="${entry.value.factorInputName}"
                    th:text="${entry.key}">
            </option>
        </select>
        </br>

        <label>Change Marginal Cost of the factor input:</label>
        <input type="number" id="marginalCost" name="marginalCost" step="0.01" placeholder="7.30"/>
        </br>

        <label>Change Marginal Revenue of the factor input:</label>
        <input type="number" id="marginalRevenue" name="marginalRevenue" step="0.01" placeholder="10.40"/>
        </br>

        <button type="submit">Submit</button>
        </br>
    </form>
</div>
```

```
        <!-- Form to delete factor input -->
        <div id="factorInputDeleteDiv">
            <h2>Fire Factor Input</h2>
            <form id="factorInputDelete" action="/cost-of-production" method="post">
                <input type="hidden" name="formName" value="factorInputDelete"/>
                <label>Fire Factor Input</label>
                <select id="factorInputDelete" name="factorInputDelete">
                    <option th:each="entry : ${allFactorInputs}"
                            th:value="${entry.value.factorInputName}"
                            th:text="${entry.key}">
                    </option>
                </select>
                </br>

                <button type="submit">Submit</button>
                </br>
            </form>
        </div>

        <!-- Undo last operation -->
        <form id="undo" action="/cost-of-production" method="post">
            <input type="hidden" name="formName" value="undo"/>
            <input type="submit" value="Undo"/>
            </br>
        </form>

        <!-- Container for graph of costs & revenue -->
        <div id="graphContainer"></div>

    </div>
```

```
<script th:inline="javascript">
    function showWindowsAlert() {
        let windowsAlert = /*[[${windowsAlert}]]*/ false;
        let windowsAlertMessage = /*[[${windowsAlertMessage}]]*/ "";

        if (windowsAlert === true) {
            window.alert(windowsAlertMessage);
        }
    }

    // Function to display cost of production summary information
    function showCostOfProductionInformation() {
        const currentDiv = document.getElementById("costOfProductionInformation");

        const numberOfLabour = /*[[${numberOfLabour}]]*/ "";
        const numberOfLabourSpan = document.createElement("span");
        numberOfLabourSpan.innerText = "Total Labour: " + numberOfLabour;
        currentDiv.append(numberOfLabourSpan);

        const numberOfCapital = /*[[${numberOfCapital}]]*/ "";
        const numberOfCapitalSpan = document.createElement("span");
        numberOfCapitalSpan.innerText = "Total Capital: " + numberOfCapital;
        currentDiv.append(numberOfCapitalSpan);

        const totalCost = /*[[${totalCost}]]*/ "";
        const totalCostSpan = document.createElement("span");
        totalCostSpan.innerText = "Total Cost: £" + totalCost;
        currentDiv.append(totalCostSpan);

        const averageCost = /*[[${averageCost}]]*/ "";
        const averageCostSpan = document.createElement("span");
```

```
    averageCostSpan.innerText = "Average Cost: £" + averageCost;
    currentDiv.append(averageCostSpan);

    const totalRevenue = /*[[${totalRevenue}]]*/ "";
    const totalRevenueSpan = document.createElement("span");
    totalRevenueSpan.innerText = "Total Revenue: £" + totalRevenue;
    currentDiv.append(totalRevenueSpan);

    const averageRevenue = /*[[${averageRevenue}]]*/ "";
    const averageRevenueSpan = document.createElement("span");
    averageRevenueSpan.innerText = "Average Revenue: £" + averageRevenue;
    currentDiv.append(averageRevenueSpan);

    const totalProfit = /*[[${totalProfit}]]*/ "";
    const totalProfitSpan = document.createElement("span");
    totalProfitSpan.innerText = "Total Profit: £" + totalProfit;
    currentDiv.append(totalProfitSpan);

    const averageProfit = /*[[${averageProfit}]]*/ "";
    const averageProfitSpan = document.createElement("span");
    averageProfitSpan.innerText = "Average Profit: £" + averageProfit;
    currentDiv.append(averageProfitSpan);

    const cobbDouglasFunction = /*[[${cobbDouglasFunction}]]*/ "";
    const cobbDouglasFunctionSpan = document.createElement("span");
    cobbDouglasFunctionSpan.innerText = "Addition to GDP: " + cobbDouglasFunction;
    currentDiv.append(cobbDouglasFunctionSpan);
}

// Function to display optimal factor input results
function showOptimalFactorInput() {
```

```
            const currentDiv = document.getElementById("optimalFactorInput");

            const bestCombination = /*[[${bestCombination}]]*/ {};
            for (let i = 0; i < bestCombination.length; i++) {
                const bestCombinationSpan = document.createElement("span");
                bestCombinationSpan.innerText = bestCombination[i].factorInputName + " Profit: £" +
bestCombination[i].marginalProfit.toFixed(2);
                currentDiv.append(bestCombinationSpan);
            }
        }

        // Function to fill factor input table
        function showTable() {
            const tableBody = document.getElementById("tableContents");
            let allFactorInputs = /*[[${allFactorInputs}]]*/ {};
            allFactorInputs = Object.values(allFactorInputs);

            for (let i = 0; i < allFactorInputs.length; i++) {
                const row = document.createElement("tr");
                const cell1 = document.createElement("td");
                const cell2 = document.createElement("td");
                const cell3 = document.createElement("td");
                const cell4 = document.createElement("td");
                const cell5 = document.createElement("td");
                const cell6 = document.createElement("td");

                cell1.textContent = allFactorInputs[i].factorInputType;
                cell2.textContent = allFactorInputs[i].factorInputName;
                cell3.textContent = allFactorInputs[i].factorInputDescription;
                cell4.textContent = allFactorInputs[i].marginalCost.toFixed(2);
                cell5.textContent = allFactorInputs[i].marginalRevenue.toFixed(2);
```

```
            cell6.textContent = allFactorInputs[i].marginalProfit.toFixed(2);

            row.append(cell1, cell2, cell3, cell4, cell5, cell6);
            tableBody.append(row);
        }
    }

    // Function to display costs & revenue graph
    function showGraph() {
        let cumulativeMarginalCosts = /*[[${cumulativeMarginalCosts}]]*/ {};
        let cumulativeMarginalRevenue = /*[[${cumulativeMarginalRevenue}]]*/ {};

        if (Object.keys(cumulativeMarginalCosts).length > 0 && Object.keys(cumulativeMarginalRevenue).length > 0)
{
            let numberOfInputs = Object.keys(cumulativeMarginalCosts);
            let costs = Object.values(cumulativeMarginalCosts);
            let revenue = Object.values(cumulativeMarginalRevenue);

            var line1 = { x: numberOfInputs, y: costs, type: 'scatter', name: 'Costs' };
            var line2 = { x: numberOfInputs, y: revenue, type: 'scatter', name: 'Revenue' };
            var plotting = [line1, line2];

            var layout = {
                title: "Costs & Revenue",
                xaxis: {title: "Inputs"},
                yaxis: {title: "Cost/Revenue"},
                dragmode: false,
                zoom: false,
                paper_bgcolor: "transparent",
                plot_bgcolor: "transparent",
                autosize: true
```

```
                };

                Plotly.newPlot("graphContainer", plotting, layout, {displayModeBar: false, responsive: true});
            }
        }

        window.onload = showWindowsAlert;
        showCostOfProductionInformation();
        showTable();
        showGraph();
        showOptimalFactorInput();
    </script>
</body>
</html>
```

cost-of-production.css

```css
body {
    margin: 1vw;
    display: flex;
    flex-direction: column;
    justify-content: flex-start;
    align-items: center;
    background-color: #d4e9cd;
    font-family: "Segoe UI", Tahoma, Geneva, Verdana, sans-serif;
    padding-top: 70px;
}

#all {
    padding: 20px;
    background-color: #ffffff9e;
    border-radius: 12px;
    box-shadow: 0 0 20px rgba(0, 0, 0, 0.1);
    width: 1020px;
    max-width: 1020px;
}

h1 {
    margin-top: 0;
    margin-bottom: 15px;
    font-size: 2rem;
    text-align: center;
}

h2 {
    margin-top: 0;
```

```
    margin-bottom: 15px;
    font-size: 1.6rem;
    text-align: center;
}

#dashboardNavbar {
    position: fixed;
    top: 0;
    left: 0;
    width: 100%;
    background-color: #20e3448f;
    display: flex;
    justify-content: center;
    align-items: center;
    gap: 10px;
    padding: 10px 0;
    box-shadow: 0 2px 8px rgba(0,0,0,0.2);
    z-index: 1000;
}

#dashboardNavbar a {
    text-decoration: none;
    color: #000000;
    padding: 8px 16px;
    border-radius: 8px;
    font-weight: 500;
    transition: background 0.2s ease, transform 0.1s ease;
}

#dashboardNavbar a:hover {
    background-color: #20e344cc;
```

```
    transform: translateY(-1px);
}

button[type="submit"], input[type="submit"], button {
    padding: 0.6em 1.2em;
    background-color: #20e3448f;
    border: 1px solid #000;
    color: #000;
    border-radius: 8px;
    cursor: pointer;
    font-size: 1rem;
    transition: background 0.2s ease, transform 0.1s ease;
}

button[type="submit"]:hover, input[type="submit"]:hover, button:hover {
    background-color: #20e344cc;
    transform: translateY(-2px);
}

#undo {
    position: fixed;
    top: 90px;
    right: 15px;
    z-index: 9998;
}

table {
    width: 100%;
    border-collapse: collapse;
    margin-top: 16px;
    border: 1px solid var(--border);
```

```
    border-radius: 12px;
    overflow: hidden;
    font-size: 0.95rem;
}

thead th {
    text-align: left;
    padding: 12px 14px;
    background: #d6f0ca;
    font-weight: 600;
    position: sticky;
    top: 0;
    z-index: 1;
}

tbody tr:nth-child(odd) {
    background: #fdfdfd;
}

tbody tr:nth-child(even) {
    background: #f7fff4;
}

tbody tr:hover {
    background: #eaffea;
}

td {
    padding: 12px 14px;
    border-bottom: 1px solid #00000010;
}
```

```
#costOfProductionInformation, #allFactorInputsDiv, #factorInputInsertDiv, #factorInputUpdateDiv,
#factorInputDeleteDiv, #optimalFactorInput {
    margin-top: 25px;
    padding: 20px;
    background: #fafafa;
    border: 1px solid #ddd;
    border-radius: 12px;
    box-shadow: 0 2px 8px rgba(0,0,0,0.03);
}

#graphContainer {
    margin-top: 25px;
    background: #fafafa;
    border: 1px solid #ddd;
    border-radius: 12px;
    box-shadow: 0 2px 8px rgba(0,0,0,0.03);
}

label {
    display: block;
    margin: 12px 0 6px;
    font-weight: 500;
}

span {
    display: block;
    margin: 12px 0 6px;
    font-weight: 500;
}
```

```css
input[type="text"], input[type="number"], select {
    width: 100%;
    max-width: 350px;
    padding: 10px;
    margin-bottom: 15px;
    border-radius: 8px;
    border: 1px solid #ccc;
    font-size: 1rem;
    box-sizing: border-box;
}

input:focus, select:focus {
    outline: none;
    border-color: #20e344;
    box-shadow: 0 0 8px rgba(32, 227, 68, 0.3);
}
```

404.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">

<head>
    <title>ERROR 404</title>
</head>

<body>
    <h1>ERROR 404</h1>
    <h2>Page not found</h2>
</body>
</html>
```

Testing

System Tests

To ensure MoneyMatters works properly, I have let multiple people test the system, through port forwarding the project through VSCode inbuilt feature. These tests ensured that everything was working as expected and my objectives were being fulfilled.

The table below shows what features I have tested, and the expected and actual outputs.

These tests will focus on Typical, Erroneous and Boundary data. There will be videos going through each test and how the system works as well.

The testing videos can be accessed via the link to the youtube videos, which are already timestamped, or through the google drive link or the youtube playlist

Google Drive Link:
https://drive.google.com/drive/folders/1ADFD0KtMQKTdViscW3L3vQtZ3axkIWCl?usp=drive_link

Youtube Testing Playlist:
youtube.com/playlist?list=PLtb2r8nMpfzv8dzVdkdJ_DsaMebzlMkpz&si=6Bso9Kyz3NTQD6C3

Test №	Description	Type	Expected	Pass or Fail
1a	Register for an account following the password constraints	T	An account will be created, where user data will be stored either in the Consumer or Producer Table. Random Vernam Key will be generated and stored in a JSON file	PASSED https://youtu.be/zgme3YQhJso?si=XozS9IuxOXnZqOSt&t=106 1:46
1b	Register for an account when the email has already been used	E	An account won't be made. Prompts the user to use another email.	PASSED https://youtu.be/zgme3YQhJso?si=4H9-RNAAZHUhw98M&t=219

				3:39
1c	Register for an account that doesn't follow the password constraints	E	An account won't be made. Prompts the user to follow the password constraints	PASSED https://youtu.be/zgme3YQhJso?si=he5vxzVzcjmSdZQY&t=30 0:30
2a	Login into account with correct credentials	T	Users have access to the system. New random Vernam Key will be generated and overwrite previous Key in a JSON file	PASSED https://youtu.be/zgme3YQhJso?si=7PDDaSbUskIBv_83&t=296 4:56
2b	Login into account with incorrect credentials	E	Users won't have access to the system. Prompts the user to retry the login process.	PASSED https://youtu.be/zgme3YQhJso?si=efW2GA4KOoVFixJ5&t=381 6:21
3a	Producers attempting to access Consumer only features	E	Tells producers that they are unable to access consumer only features, and prompts them to switch to a producer account.	PASSED https://youtu.be/dXuc-Ir1gYk?si=RYCGjHmK3jT2tZqW&t=81 1:21
3b	Consumer attempting to access Producer only features	E	Tells consumers that they are unable to access producer only features, and prompts them to switch to a consumer account.	PASSED https://youtu.be/dXuc-Ir1gYk?si=8pG4tOW5fsMNIxyq&t=28 0:28
4a	User searches a valid word in the	T	See their desired word and definition.	PASSED

	glossary			https://youtu.be/dXuc-Ir1gYk?si=ZE-n3kcE20rC_dBb&t=133 2:13
4b	User searches for a valid word, but in a different format, such as a mixture of upper and lower case, and white spaces.	B	See their desired word and definition.	PASSED https://youtu.be/dXuc-Ir1gYk?si=O-sXo8G4tafnVa7c&t=211 3:11
4c	User searches an invalid word in the glossary.	E	Display “word not found”	PASSED https://youtu.be/dXuc-Ir1gYk?si=7Tw9vpXbED6w0ADU&t=251 4:11
5a	Users ask A.I. a question about the economy.	T	Displays the relevant answer to the question.	PASSED https://youtu.be/dXuc-Ir1gYk?si=rQsEqiLaM6uk_E2G&t=275 4:35
6a	Users can dynamically search news by its title	T	Only displays sorted news	PASSED https://youtu.be/UKAcp1wmeKo?si=v97zcd8V4ZrXezdS&t=25 0:25
6b	Users sort news by new outlets, date or title	T	Only displays sorted news	PASSED https://youtu.be/UKAcp1wmeKo?si=j5EsvablZfi_xBHk&t=217

				3:37
6c	Users summarise news article	T	Displays summarised news	PASSED https://youtu.be/UKAcp1wmeKo?si=2ONRTHMU9YFak-wg&t=319 5:19
7a	User selects a valid country and macro-economic indicator	T	Displays a graph showing changes of the macro-economic indicator	PASSED https://youtu.be/Mn8vOEu2JRY?si=U7FphuHqNLKtfmav&t=7 0:07
7b	User selects a combination of a country and macro-economic indicator, but there is no data available from the World Bank API to access it.	E	No graph is displayed. User has to select a different combination of country and macro-economic indicator	PASSED https://youtu.be/Mn8vOEu2JRY?si=U7FphuHqNLKtfmav&t=90 1:30
8a	User selects a base currency, a wanted currency different to the base currency, and a date that is not in the future.	T	Displays a graph showing the relationship between the 2 currencies, over the selected date and current date. Users are then able to use a currency converter.	PASSED https://youtu.be/Mn8vOEu2JRY?si=U7FphuHqNLKtfmav&t=180 3:00
8b	User selects a base currency, a wanted currency that is the same as the base currency, and a date that is not in the future.	B	Doesn't display a graph, but the currency converter prompt will show.	PASSED https://youtu.be/Mn8vOEu2JRY?si=Dp_0-ZzqQ6-d9tcM&t=272 4:32
8c	User selects a base currency, a wanted	E	User is prompted to select a date in the past.	PASSED

	currency, and a date that is in the future.			https://youtu.be/Mn8vOEu2JRY?si=zhQKIeQEbB5THRXY&t=321 5:21
9a	Questions and multiple choice answers from a JSON file are loaded to the screen.	T	User is able to see the questions and answer the question by selecting a choice	PASSED https://youtu.be/5E2q5lv4NHI?si=-glkqQ_lD2Ye-QUx&t=1 0:01
9b	Users' answers are saved to a JSON file and their response is stored in a database.	T	The question, answer and the weight of each question is stored in a JSON file.	PASSED https://youtu.be/5E2q5lv4NHI?si=nZIt4s0O4pFoZB3Y&t=107 1:47
9c	User wants to redo the survey on the same day.	T	A new file and entry on the database is not created. Instead, the old user's response is overwritten with the new one.	PASSED https://youtu.be/5E2q5lv4NHI?si=kC124Wmgm-ZF16He&t=171 2:51
9d	User wants to view the reviews of society at different dates	T	Pie charts with data from multiple users are shown on the page	PASSED https://youtu.be/5E2q5lv4NHI?si=l9EC6IM9xuE2sRnZ&t=265 4:25
10a	User wants to update their budget	T	Database receives a relevant update, changing budget information, changing the information on the website	PASSED https://youtu.be/YD0nc3x3bbU?si=RXQariTnOiB-Gz1v&t=46

				0:46
10b	User wants to purchase a good, which is under their remaining budget	T	Database receives a relevant update. Information on the website changes	PASSED https://youtu.be/YD0nc3x3bbU?si=ji3ScyZpeqqI4pHe&t=120 2:00
10c	User wants to purchase a good which is equal to their remaining budget	B	Database receives a relevant update. Information on the website changes	PASSED https://youtu.be/YD0nc3x3bbU?si=GXPnbWoeF20aakAs&t=233 3:53
10d	User wants to purchase a good which is above to their remaining budget	E	User is unable to make the purchase.	PASSED https://youtu.be/YD0nc3x3bbU?si=jtDkybxQNoWFNLeY&t=310 5:10
10e	User sorts their purchases	T	Data is sorted correctly	PASSED https://youtu.be/YD0nc3x3bbU?si=o1QrE39e49w7tsuL&t=372 6:12
10f	User updates their frequency of renewable to “Monthly”	T	Sustainable spending is shown of the website	PASSED https://youtu.be/YD0nc3x3bbU?si=mGPMbAg2dx3wHDDh&t=425 7:05
10g	User purchases	T	Risky purchases should	PASSED

	goods that cost too high from the normal prices		be shown on the website	https://youtu.be/YD0nc3x3bbU?si=XMM6S0haFgq8ZHBA&t=517 8:37
10h	Users want to view how they spend everyday in a week and their future budget projections	T	Weekly Spendings and Future Budget Projections are displayed on the website	PASSED https://youtu.be/YD0nc3x3bbU?si=cd0Pe2PgLMITNBqL&t=640 10:42
11a	User chooses a stock and a time period	T	Displays an appropriate graph with appropriate company details and SMA trends	PASSED https://youtu.be/OGlk2mFMwCQ?si=W5nDPcbWQWbnW5qr&t=10 0:10
11b	User buys stock, when they have a valid budget	T	User's stock budget will decrease appropriately, but they are able to permanently own stock, until they want to sell it	PASSED https://youtu.be/OGlk2mFMwCQ?si=W5nDPcbWQWbnW5qr&t=249 4:09
11c	User buys too much stock, as they have a low stock budget.	E	User is unable to buy stocks. Prompts user to buy a lower number of stocks	PASSED https://youtu.be/OGlk2mFMwCQ?si=W5nDPcbWQWbnW5qr&t=340 5:40
11d	User sells existing stock	T	User's stock budget will change appropriately, losing their stock, unless they buy it again	PASSED https://youtu.be/OGlk2mFMwCQ?si=W5nDPcbWQWbn

				W5qr&t=390 6:30
11e	User attempts to sell stock they don't own	E	User is unable to sell stocks. Prompts user to sell lower number of stocks	PASSED https://youtu.be/OGlk2mFMwCQ?si=W5nDPcbWQWbnW5qr&t=515 8:35
12a	Producer inserts a factor input	T	Displays new factor input to the table	PASSED https://youtu.be/XAF4mDyM8ZY?si=uk37y86UHXg9BVbt&t=30 0:30
12b	Producer updates a factor input	T	Displays updated factor input to the table	PASSED https://youtu.be/XAF4mDyM8ZY?si=uk37y86UHXg9BVbt&t=85 1:25
12c	Producer deletes a factor input	T	Removes factor input from the table	PASSED https://youtu.be/XAF4mDyM8ZY?si=uk37y86UHXg9BVbt&t=120 2:00
12d	Producer inputs the maximum cost to select the most profitable workers based on that cost.	T	Displays most profitable workers based on the maximum cost the producer is willing to spend	PASSED https://youtu.be/XAF4mDyM8ZY?si=uk37y86UHXg9BVbt&t=160 2:40

12e	Producer inputs the maximum cost, but their maximum cost is too low to select the most profitable workers.	E	No worker is displayed	PASSED https://youtu.be/XAF4mDyM8ZY?si=VtcUSp5lRSecf1b5&t=235 3:55
12f	Producer attempts to undo any changes made to factor inputs	T	Undoes previous change to factor input	PASSED https://youtu.be/XAF4mDyM8ZY?si=VtcUSp5lRSecf1b5&t=265 4:25
12g	Producer attempts to undo any changes made to factor inputs, but they have made no prior changes beforehand	B	Nothing happens	PASSED https://youtu.be/XAF4mDyM8ZY?si=VtcUSp5lRSecf1b5&t=310 5:10

Unit Tests

Parsing Data from Java to Python to Java (Flask API)

This JUnit test can check that a Python Flask API is working correctly. It sends different sets of sample data to several Flask endpoints, each representing a specific feature such as income calculation, AI-generated answers, summarising news articles, and stock analysis. The test sends random requests to the Flask API and confirms that each one responds successfully with a 200 status code, serving as a way to verify that all parts of the Python Flask API are running properly and can handle valid input.

```
package com.mycompany.MoneyMatters;

import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.Random;
import static org.junit.jupiter.api.Assertions.assertEquals;
import org.junit.jupiter.api.Test;
import com.fasterxml.jackson.databind.ObjectMapper;

public class PythonFlaskAPITest
    private ArrayList<LinkedHashMap<String, ?>> getPayloads() {
        final Random randomNumberGenerator = new Random();
        ArrayList<LinkedHashMap<String, ?>> payloads = new ArrayList<>();

        LinkedHashMap<String, Double> payload1 = new LinkedHashMap<>();
```

```
        payload1.put("income", randomNumberGenerator.nextDouble(10, 1000000));

        LinkedHashMap<String, Double> payload2 = new LinkedHashMap<>();
        payload2.put("totalRevenue", randomNumberGenerator.nextDouble(10, 1000000));
        payload2.put("totalCost", randomNumberGenerator.nextDouble(10, 1000000));
        payload2.put("numberOfLabour", randomNumberGenerator.nextDouble(10, 1000000));
        payload2.put("numberOfCapital", randomNumberGenerator.nextDouble(10, 1000000));

        LinkedHashMap<String, String> payload3 = new LinkedHashMap<>();
        payload3.put("userPrompt", "Define GDP");

        LinkedHashMap<String, String> payload4 = new LinkedHashMap<>();
        payload4.put("link",
"https://www.independent.co.uk/news/business/business-rates-warning-supermarkets-price-b2852194.html");

        LinkedHashMap<String, Object> payload5 = new LinkedHashMap<>();
        LinkedHashMap<String, Double> incomeComponents = new LinkedHashMap<>();
        incomeComponents.put("Essentials", randomNumberGenerator.nextDouble(10, 1000000));
        incomeComponents.put("Leisure", randomNumberGenerator.nextDouble(10, 1000000));
        incomeComponents.put("Savings", randomNumberGenerator.nextDouble(10, 1000000));
        payload5.put("incomeComponents", incomeComponents);
        payload5.put("growthRate", randomNumberGenerator.nextDouble(0, 10));

        LinkedHashMap<String, Double> payload6 = new LinkedHashMap<>();
        payload6.put("essentials", randomNumberGenerator.nextDouble(10, 1000000));

        LinkedHashMap<String, String> payload7 = new LinkedHashMap<>();
        payload7.put("tickerSymbol", "AAPL");

        LinkedHashMap<String, String> payload8 = new LinkedHashMap<>();
        payload8.put("tickerSymbol", "AAPL");
```

```
        payload8.put("startDate", LocalDate.now().toString());

        LinkedHashMap<String, String> payload9 = new LinkedHashMap<>();
        payload9.put("tickerSymbol", "AAPL");

        LinkedHashMap<String, Object> payload10 = new LinkedHashMap<>();
        payload10.put("prices", new ArrayList<Double>());
        payload10.put("quantities", new ArrayList<Integer>());

        LinkedHashMap<String, Object> payload11 = new LinkedHashMap<>();
        payload11.put("tickerSymbol", "AAPL");
        payload11.put("window", randomNumberGenerator.nextInt(10, 250));

        payloads.add(payload1);
        payloads.add(payload2);
        payloads.add(payload3);
        payloads.add(payload4);
        payloads.add(payload5);
        payloads.add(payload6);
        payloads.add(payload7);
        payloads.add(payload8);
        payloads.add(payload9);
        payloads.add(payload10);
        payloads.add(payload11);
        return payloads;
    }

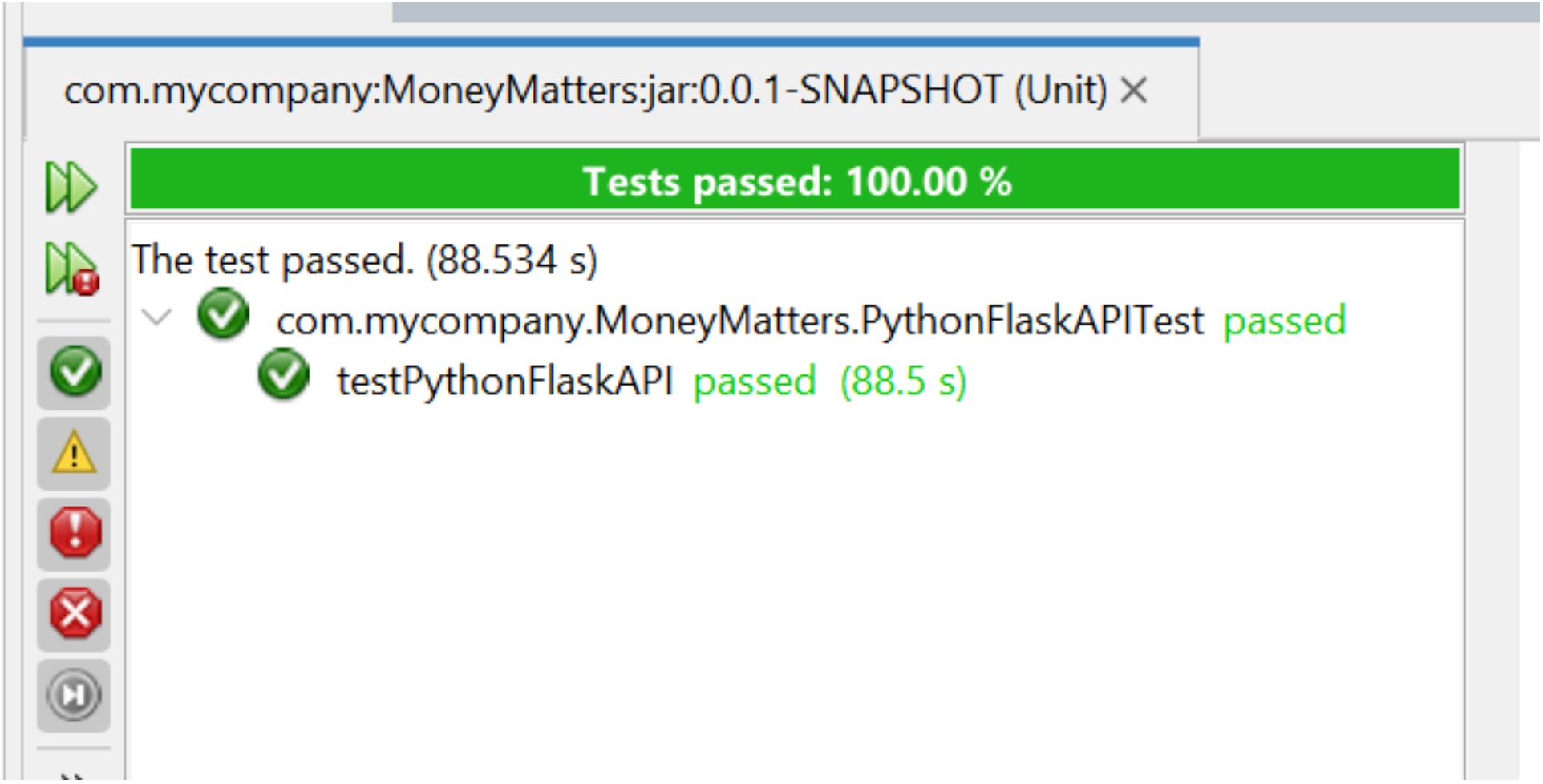
    @Test
    public void testPythonFlaskAPI() throws Exception {
        final HttpClient client = HttpClient.newHttpClient();
        final ObjectMapper mapper = new ObjectMapper();
```

```
        final String baseURL = "http://localhost:5000/";
        final String[] endpoints = {"calculate-Income-Components", "perform-cobb-douglas-function", "ai-answer",
"summarise-news-article", "future-budget", "sustainable-spending", "company-information", "stock-data",
"stock-ask-price", "get-last-week-purchases"};
        final ArrayList<LinkedHashMap<String, ?>> payloads = getPayloads();

        for (int i = 0; i < endpoints.length; i++) {
            HttpRequest request = HttpRequest.newBuilder()
                .uri(URI.create(baseURL + endpoints[i]))
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(mapper.writeValueAsString(payloads.get(i))))
                .build();

            HttpResponse<String> response = client.send(request, HttpResponse.BodyHandlers.ofString());
            assertEquals(200, response.statusCode());

            System.out.println("Endpoint: " + endpoints[i]);
            System.out.println("Sample Data: " + payloads.get(i).toString());
            System.out.println("Response: " + mapper.readValue(response.body(), Object.class));
            System.out.println("");
        }
    }
}
```


com.mycompany:MoneyMatters:jar:0.0.1-SNAPSHOT (Unit)
Tests passed: 100.00 %
The test passed. (88.534 s)
com.mycompany.MoneyMatters.PythonFlaskAPITest passed
testPythonFlaskAPI passed (88.5 s)

Vernam Cipher Test

This JUnit test verifies that the VernamCipher class works correctly for encryption. It first ensures a clean testing environment by deleting any existing test files before and after each test. The test cases then check three main functions: converting plain text to binary (convertPlainTextToBinary), correctly performing the XOR encryption operation (XORfunction), and saving and loading Vernam cipher keys as JSON files (storeVernamKeysInJSON and loadVernamKey). The test confirms that the VernamCipher class can properly encrypt data, handle binary conversion, and manage encryption keys in the filesystem.

```
package com.mycompany.MoneyMatters;

import static org.junit.jupiter.api.Assertions.*;
import java.io.File;
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import com.mycompany.Algorithms.VernamCipher;

public class VernamCipherTest {

    private final String folderName = "VernamKeys";
    private final String email = "test@gmail.com";
    private final File folder = new File(folderName);
    private final File file = new File(folderName + File.separator + email + ".json");

    @BeforeEach
    void cleanBefore() throws Exception {
        deleteIfExists(file);
        if (folder.exists()) {
```

```
            folder.delete();
        }
    }

    @AfterEach
    void cleanAfter() throws Exception {
        deleteIfExists(file);
        if (folder.exists()) {
            folder.delete();
        }
    }

    private void deleteIfExists(File f) throws Exception {
        if (f.exists()) {
            f.delete();
        }
    }

    @Test
    void testConvertPlainTextToBinary() {
        String text = "A";
        String plainText = "1000001";
        String actual = VernamCipher.convertPlainTextToBinary(text);
        assertEquals(plainText, actual);
    }

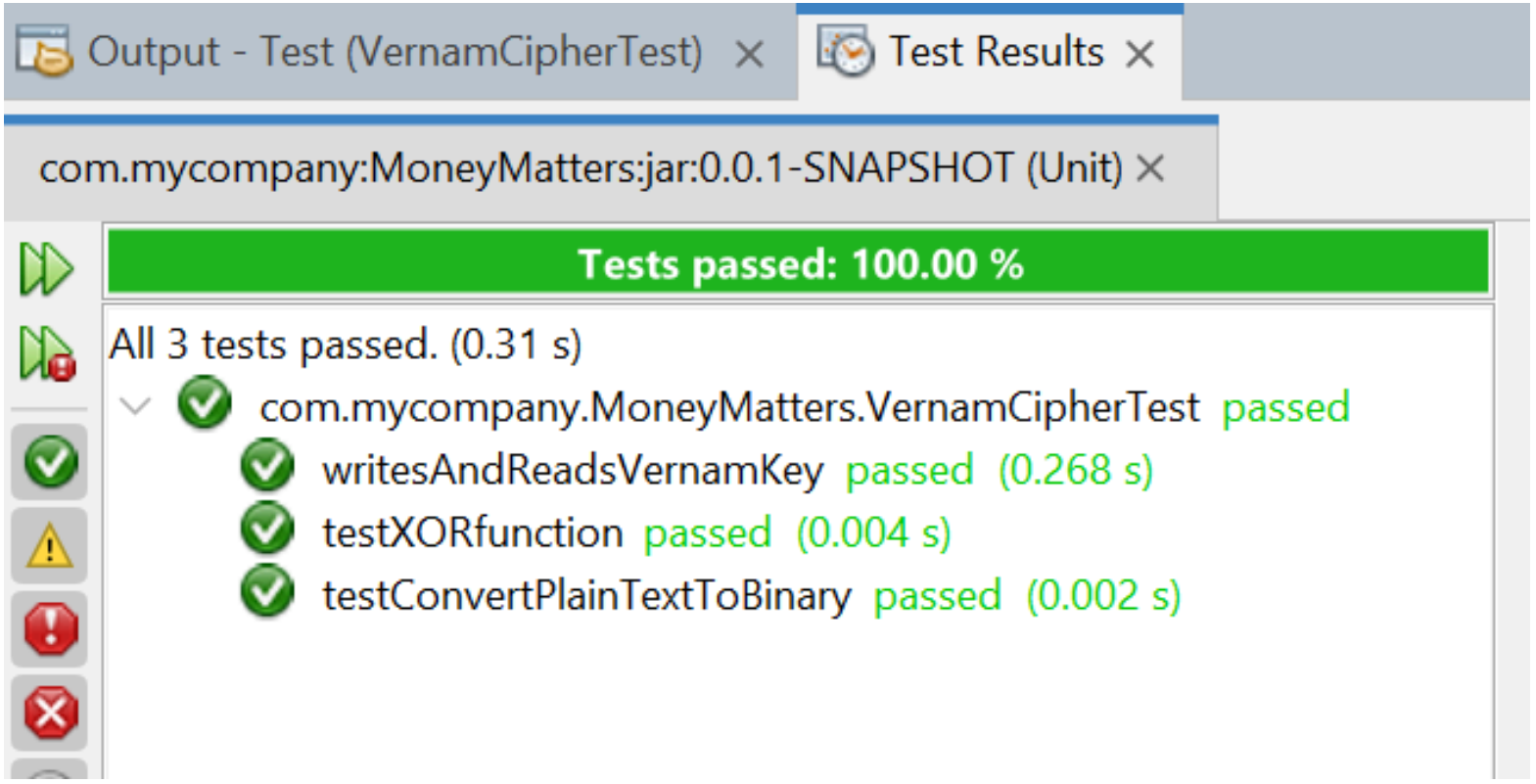
    @Test
    void testXORfunction() {
        String plainText = "1010101";
        String vernamKey = "1100110";
        String expectedCipherText = "0110011";
```

```
        String actualCipherText = VernamCipher.XORfunction(plainText, vernamKey);
        assertEquals(expectedCipherText, actualCipherText);
    }

    @Test
    void writesAndReadsVernamKey() {
        final String vernamKey = "1010101";
        VernamCipher.storeVernamKeysInJSON(email, vernamKey);

        assertTrue(folder.exists());
        assertTrue(file.exists());

        String loadedVernamKey = VernamCipher.loadVernamKey(email);
        assertEquals(vernamKey, loadedVernamKey);
    }
}
```


Output - Test (VernamCipherTest)
Test Results
com.mycompany:MoneyMatters:jar:0.0.1-SNAPSHOT (Unit)
Tests passed: 100.00 %
All 3 tests passed. (0.31 s)
com.mycompany.MoneyMatters.VernamCipherTest passed
writesAndReadsVernamKey passed (0.268 s)
testXORfunction passed (0.004 s)
testConvertPlainTextToBinary passed (0.002 s)

DDL Scripts

This test class confirms that each table in my database is generated correctly. I used Mockito so that the tests could run without connecting to the real database. This test confirms that the database is able to correctly create tables.

```
package com.mycompany.MoneyMatters;

import java.lang.reflect.Method;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import static org.mockito.Mockito.*;
import com.mycompany.Repository.SQLQueriesDDL;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.DriverManager;
import org.mockito.MockedStatic;

class DatabaseTest {

    private Connection mockConnection;
    private PreparedStatement mockPreparedStatement;

    @BeforeEach
    void setup() throws Exception {
        mockConnection = mock(Connection.class);
        mockPreparedStatement = mock(PreparedStatement.class);
        when(mockConnection.prepareStatement(anyString())).thenReturn(mockPreparedStatement);
    }

    @Test
```

```
    void testCreateConsumerTable() throws Exception {
        try (MockedStatic<DriverManager> mockedDriverManager = mockStatic(DriverManager.class)) {
            mockedDriverManager.when(() -> DriverManager.getConnection(anyString(), anyString(),
anyString())).thenReturn(mockConnection);

            Method method = SQLQueriesDDL.class.getDeclaredMethod("createConsumerTable");
            method.setAccessible(true);
            method.invoke(null);

            verify(mockPreparedStatement, times(1)).executeUpdate();
            verify(mockPreparedStatement, times(1)).close();
            verify(mockConnection, times(1)).close();
        }
    }

    @Test
    void testCreateProducerTable() throws Exception {
        try (MockedStatic<DriverManager> mockedDriverManager = mockStatic(DriverManager.class)) {
            mockedDriverManager.when(() -> DriverManager.getConnection(anyString(), anyString(),
anyString())).thenReturn(mockConnection);

            Method method = SQLQueriesDDL.class.getDeclaredMethod("createProducerTable");
            method.setAccessible(true);
            method.invoke(null);

            verify(mockPreparedStatement, times(1)).executeUpdate();
            verify(mockPreparedStatement, times(1)).close();
            verify(mockConnection, times(1)).close();
        }
    }
```

```
    @Test
    void testCreateEconomyRaterTable() throws Exception {
        try (MockedStatic<DriverManager> mockedDriverManager = mockStatic(DriverManager.class)) {
            mockedDriverManager.when(() -> DriverManager.getConnection(anyString(), anyString(),
anyString())).thenReturn(mockConnection);

            Method method = SQLQueriesDDL.class.getDeclaredMethod("createEconomyRaterTable");
            method.setAccessible(true);
            method.invoke(null);

            verify(mockPreparedStatement, times(1)).executeUpdate();
            verify(mockPreparedStatement, times(1)).close();
            verify(mockConnection, times(1)).close();
        }
    }

    @Test
    void testCreateGoodsTable() throws Exception {
        try (MockedStatic<DriverManager> mockedDriverManager = mockStatic(DriverManager.class)) {
            mockedDriverManager.when(() -> DriverManager.getConnection(anyString(), anyString(),
anyString())).thenReturn(mockConnection);

            Method method = SQLQueriesDDL.class.getDeclaredMethod("createGoodsTable");
            method.setAccessible(true);
            method.invoke(null);

            verify(mockPreparedStatement, times(1)).executeUpdate();
            verify(mockPreparedStatement, times(1)).close();
            verify(mockConnection, times(1)).close();
        }
    }
```

```
    @Test
    void testCreatePurchaseTable() throws Exception {
        try (MockedStatic<DriverManager> mockedDriverManager = mockStatic(DriverManager.class)) {
            mockedDriverManager.when(() -> DriverManager.getConnection(anyString(), anyString(),
anyString())).thenReturn(mockConnection);

            Method method = SQLQueriesDDL.class.getDeclaredMethod("createPurchaseTable");
            method.setAccessible(true);
            method.invoke(null);

            verify(mockPreparedStatement, times(1)).executeUpdate();
            verify(mockPreparedStatement, times(1)).close();
            verify(mockConnection, times(1)).close();
        }
    }

    @Test
    void testCreateFactorInputTable() throws Exception {
        try (MockedStatic<DriverManager> mockedDriverManager = mockStatic(DriverManager.class)) {
            mockedDriverManager.when(() -> DriverManager.getConnection(anyString(), anyString(),
anyString())).thenReturn(mockConnection);

            Method method = SQLQueriesDDL.class.getDeclaredMethod("createFactorInputTable");
            method.setAccessible(true);
            method.invoke(null);

            verify(mockPreparedStatement, times(1)).executeUpdate();
            verify(mockPreparedStatement, times(1)).close();
            verify(mockConnection, times(1)).close();
        }
```

```
    }

    @Test
    void testCreateStocksTable() throws Exception {
        try (MockedStatic<DriverManager> mockedDriverManager = mockStatic(DriverManager.class)) {
            mockedDriverManager.when(() -> DriverManager.getConnection(anyString(), anyString(),
anyString())).thenReturn(mockConnection);

            Method method = SQLQueriesDDL.class.getDeclaredMethod("createStocksTable");
            method.setAccessible(true);
            method.invoke(null);

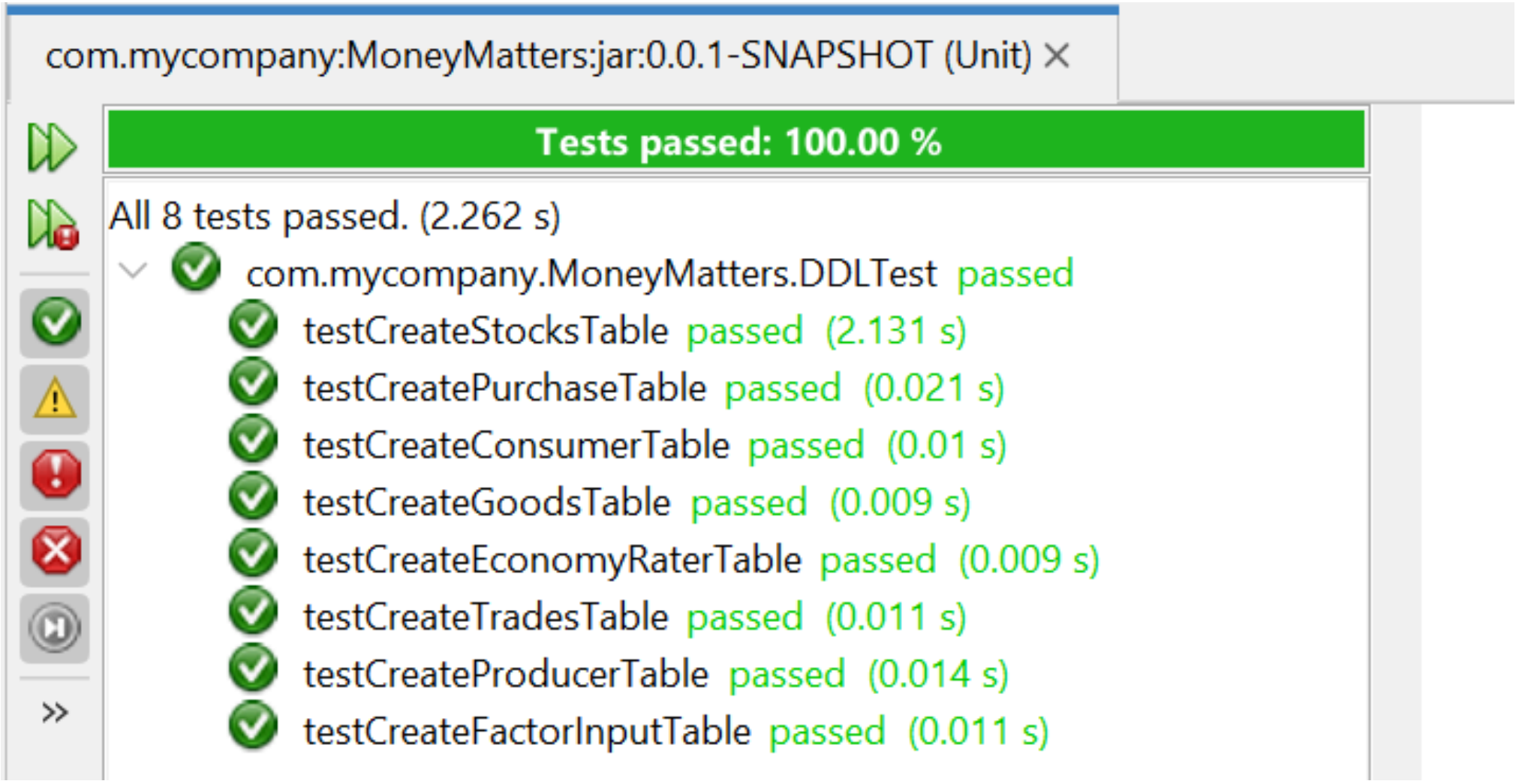
            verify(mockPreparedStatement, times(1)).executeUpdate();
            verify(mockPreparedStatement, times(1)).close();
            verify(mockConnection, times(1)).close();
        }
    }

    @Test
    void testCreateTradesTable() throws Exception {
        try (MockedStatic<DriverManager> mockedDriverManager = mockStatic(DriverManager.class)) {
            mockedDriverManager.when(() -> DriverManager.getConnection(anyString(), anyString(),
anyString())).thenReturn(mockConnection);

            Method method = SQLQueriesDDL.class.getDeclaredMethod("createTradesTable");
            method.setAccessible(true);
            method.invoke(null);

            verify(mockPreparedStatement, times(1)).executeUpdate();
            verify(mockPreparedStatement, times(1)).close();
            verify(mockConnection, times(1)).close();
```

```
        }
    }
}
```


com.mycompany:MoneyMatters:jar:0.0.1-SNAPSHOT (Unit)
Tests passed: 100.00 %
All 8 tests passed. (2.262 s)
com.mycompany.MoneyMatters.DDLTest passed
testCreateStocksTable passed (2.131 s)
testCreatePurchaseTable passed (0.021 s)
testCreateConsumerTable passed (0.01 s)
testCreateGoodsTable passed (0.009 s)
testCreateEconomyRaterTable passed (0.009 s)
testCreateTradesTable passed (0.011 s)
testCreateProducerTable passed (0.014 s)
testCreateFactorInputTable passed (0.011 s)

DQL and DML Script

I built this test class to make sure that the Consumer Table can interact with the database when inserting and retrieving consumer data. I used Mockito to mock the database components so the tests could run safely without connecting to the actual database. In the insert test, I checked that all the correct parameters were set in the SQL statement and that the database update was executed. In the get data test, I returned a record from the database and verified that a Consumer object was created with the expected details. This confirmed that my methods can handle both writing to and reading from the database. Whilst this test is only for the Consumer Table, it is reflective of all tables in the database.

```
package com.mycompany.MoneyMatters;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import com.mycompany.Repository.SQLQueriesConsumers;
import com.mycompany.Users.Consumer;
import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.ArgumentMatchers.anyString;
import static org.mockito.Mockito.*;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import org.mockito.MockedStatic;

public class ConsumerTableTest {
    private SQLQueriesConsumers sqlConsumers;
    private Connection mockConnection;
    private PreparedStatement mockPreparedStatement;
    private ResultSet mockResultSet;
```

```
    @BeforeEach
    void setup() throws Exception {
        sqlConsumers = new SQLQueriesConsumers();
        mockConnection = mock(Connection.class);
        mockPreparedStatement = mock(PreparedStatement.class);
        mockResultSet = mock(ResultSet.class);
        when(mockConnection.prepareStatement(anyString())).thenReturn(mockPreparedStatement);
        when(mockPreparedStatement.executeQuery()).thenReturn(mockResultSet);
    }

    @Test
    void testInsertUserDataIntoTable() throws Exception {
        try (MockedStatic<DriverManager> mockedDriverManager = mockStatic(DriverManager.class)) {
            mockedDriverManager.when(() -> DriverManager.getConnection(anyString(), anyString(),
anyString())).thenReturn(mockConnection);

            Consumer consumer = new Consumer(2207, "Tharun", "Nadeswaran", 17, "2008-01-30",
                                             "tharunnadeswaran2207@gmail.com", "+44 9535 134533", "password",
                                             5000.00, 2000.00, null, "Monthly", 0.00, 10000.00);

            sqlConsumers.insertUserDataIntoTable(consumer);

            verify(mockPreparedStatement, times(1)).setString(1, "Tharun");
            verify(mockPreparedStatement, times(1)).setString(2, "Nadeswaran");
            verify(mockPreparedStatement, times(1)).setInt(3, 17);
            verify(mockPreparedStatement, times(1)).executeUpdate();
            verify(mockPreparedStatement).close();
            verify(mockConnection).close();
        }
    }
```

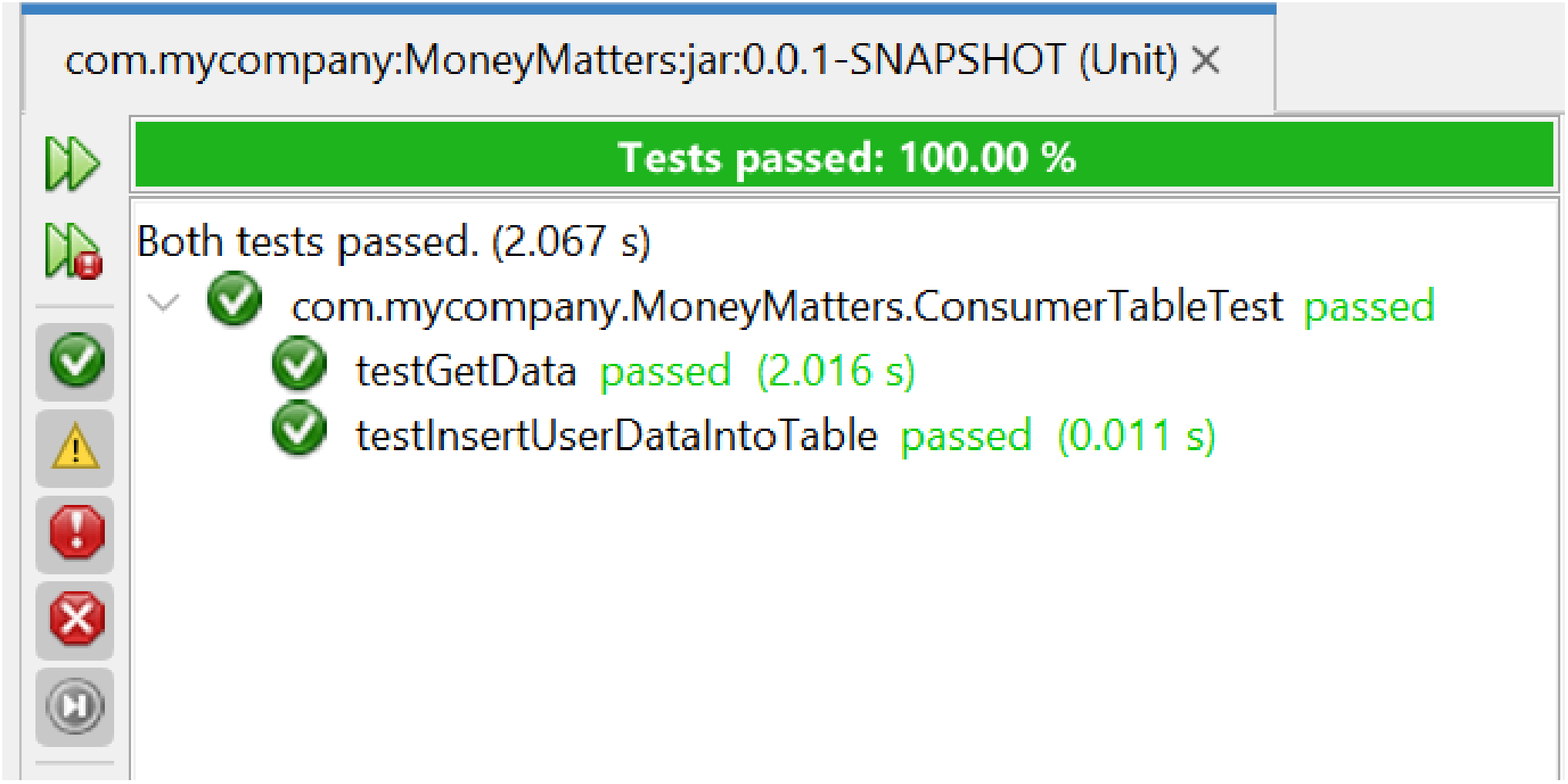
```
    @Test
    void testGetData() throws Exception {
        try (MockedStatic<DriverManager> mockedDriverManager = mockStatic(DriverManager.class)) {
            mockedDriverManager.when(() -> DriverManager.getConnection(anyString(), anyString(),
anyString())).thenReturn(mockConnection);

            when(mockConnection.prepareStatement(anyString())).thenReturn(mockPreparedStatement);
            when(mockPreparedStatement.executeQuery()).thenReturn(mockResultSet);
            when(mockResultSet.next()).thenReturn(true);
            when(mockResultSet.getInt("ConsumerID")).thenReturn(2207);
            when(mockResultSet.getString("FirstName")).thenReturn("Tharun");
            when(mockResultSet.getString("LastName")).thenReturn("Nadeswaran");
            when(mockResultSet.getInt("Age")).thenReturn(17);
            when(mockResultSet.getString("DateOfBirth")).thenReturn("2008-01-30");
            when(mockResultSet.getString("Email")).thenReturn("tharunnadeswaran2207.com");
            when(mockResultSet.getString("PhoneNumber")).thenReturn("+44 9535 134533");
            when(mockResultSet.getString("Password")).thenReturn("password");
            when(mockResultSet.getDouble("BudgetSize")).thenReturn(5000.00);
            when(mockResultSet.getDouble("CurrentBudget")).thenReturn(2000.00);
            when(mockResultSet.getDate("LastRenewed")).thenReturn(java.sql.Date.valueOf("2025-01-01"));
            when(mockResultSet.getString("FrequencyOfRenewal")).thenReturn("Monthly");
            when(mockResultSet.getDouble("SustainableSpending")).thenReturn(0.00);
            when(mockResultSet.getDouble("StockBudget")).thenReturn(10000.00);

            Consumer consumer = sqlConsumers.getData(2207);

            assertNotNull(consumer);
            assertEquals("Tharun", consumer.getFirstName());
            assertEquals(2207, consumer.getConsumerID());
```

```
            verify(mockPreparedStatement).setInt(1, 2207);
            verify(mockPreparedStatement).close();
            verify(mockConnection).close();
        }
    }
}
```


com.mycompany:MoneyMatters:jar:0.0.1-SNAPSHOT (Unit)
Tests passed: 100.00 %
Both tests passed. (2.067 s)
com.mycompany.MoneyMatters.ConsumerTableTest passed
testGetData passed (2.016 s)
testInsertUserDataIntoTable passed (0.011 s)

Cost Restraint Test

This test checks whether the program can correctly identify the most profitable combination of factor inputs given a fixed production cost limit. Each test focuses on a different case: algorithm selects the optimal inputs within a set budget, returns an empty list when no inputs are affordable, only one item fits the cost constraint, and handles empty input. By using small data sets, I confirmed that the method can maximise profit while respecting cost boundaries.

```
package com.mycompany.MoneyMatters;

import static org.junit.jupiter.api.Assertions.*;

import java.util.ArrayList;
import java.util.Set;
import java.util.stream.Collectors;
import org.junit.jupiter.api.Test;
import com.mycompany.Logic.CostOfProductionLogic;
import com.mycompany.Records.FactorInput;

class CostRestraintTest {

    @Test
    void maximiseProfitWithFixedCost() {
        ArrayList<FactorInput> factorInputs = new ArrayList<>();
        factorInputs.add(new FactorInput(1, "Labour", "Worker 1", "Construction", 100, 300));
        factorInputs.add(new FactorInput(2, "Labour", "Worker 2", "Construction", 150, 200));
        factorInputs.add(new FactorInput(3, "Labour", "Worker 3", "Construction", 200, 500));
        factorInputs.add(new FactorInput(4, "Capital", "Machine 1", "Construction", 120, 180));

        int maxCost = 300;
```

```
        ArrayList<FactorInput> bestCombination = CostOfProductionLogic.showBestCombination((ArrayList<FactorInput>)
factorInputs, maxCost);
        Set<Integer> actualIDs =
bestCombination.stream().map(FactorInput::getFactorInputID).collect(Collectors.toSet());

        Set<Integer> expectedIDs = Set.of(1, 3);
        assertEquals(expectedIDs, actualIDs);
    }

    @Test
    void testNoAffordableFactorInputs() {
        ArrayList<FactorInput> factorInputs = new ArrayList<>();

        factorInputs.add(new FactorInput(1, "Labour", "Worker 1", "Construction", 500, 1000));
        factorInputs.add(new FactorInput(2, "Capital", "Machine 1", "Construction", 600, 900));

        int maxCost = 100;

        ArrayList<FactorInput> bestCombination = CostOfProductionLogic.showBestCombination((ArrayList<FactorInput>)
factorInputs, maxCost);
        assertTrue(bestCombination.isEmpty());
    }

    @Test
    void testSingleSelection() {
        ArrayList<FactorInput> factorInputs = new ArrayList<>();

        factorInputs.add(new FactorInput(1, "Labour", "Worker 1", "Construction", 400, 800));
        factorInputs.add(new FactorInput(2, "Capital", "Machine 1", "Construction", 100, 200));
```

```
        int maxCost = 400;

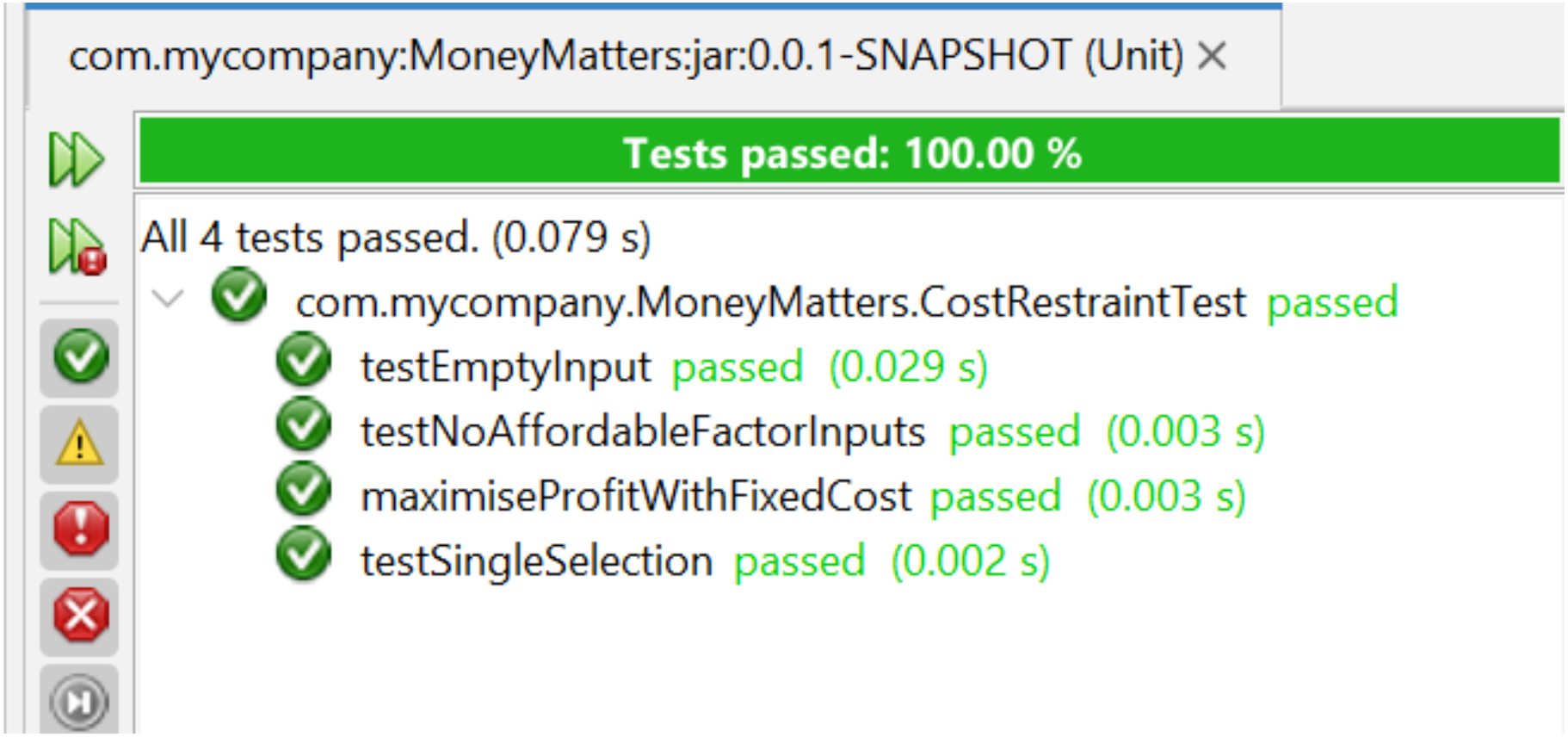
        ArrayList<FactorInput> bestCombination = CostOfProductionLogic.showBestCombination((ArrayList<FactorInput>)
factorInputs, maxCost);
        Set<Integer> actualIDs = bestCombination.stream()
                .map(FactorInput::getFactorInputID)
                .collect(Collectors.toSet());

        Set<Integer> expectedIDs = Set.of(1);
        assertEquals(expectedIDs, actualIDs);
    }

    @Test
    void testEmptyInput() {
        ArrayList<FactorInput> factorInputs = new ArrayList<>();
        int maxCost = 500;

        ArrayList<FactorInput> bestCombination = CostOfProductionLogic.showBestCombination((ArrayList<FactorInput>)
factorInputs, maxCost);

        assertTrue(bestCombination.isEmpty());
    }
}
```


com.mycompany:MoneyMatters:jar:0.0.1-SNAPSHOT (Unit)
Tests passed: 100.00 %
All 4 tests passed. (0.079 s)
com.mycompany.MoneyMatters.CostRestraintTest passed
testEmptyInput passed (0.029 s)
testNoAffordableFactorInputs passed (0.003 s)
maximiseProfitWithFixedCost passed (0.003 s)
testSingleSelection passed (0.002 s)

Cost Restraint Test With Random Factor Inputs

This test checks whether the program can choose the best combination of workers or resources while staying under a certain budget. It does this by creating random examples of workers (like Labour or Capital) with different costs and profits, then setting a random cost restraint. The test runs with different random data and prints out all the workers, the cost restraint, and the best list of Factor Inputs. This makes sure the program can work correctly in different situations.

```
package com.mycompany.MoneyMatters;

import com.mycompany.Logic.CostOfProductionLogic;
import com.mycompany.Records.FactorInput;
import java.util.ArrayList;
import java.util.Random;
import org.junit.jupiter.api.RepeatedTest;

public class CostRestraintTestRandom {

    final private Random randomNumberGenerator = new Random();

    private String randomStringGenerator() {
        final String uppercase = "qwertyuiopasdfghjklzxcvbnm".toUpperCase();
        final String lowercase = "qwertyuiopasdfghjklzxcvbnm".toLowerCase();
        final String numbers = "1234567890";
        final String availableCharacters = uppercase + lowercase + numbers;
        String randomString = "";

        for (int i = 0; i < randomNumberGenerator.nextInt(1, 30); i++) {
            randomString += availableCharacters.charAt(randomNumberGenerator.nextInt(0,
availableCharacters.length()));
```

```
        }

        return randomString;
    }

    private ArrayList<FactorInput> randomFactorInputGenerator(int size) {
        ArrayList<FactorInput> factorInputs = new ArrayList<>();
        final String[] factorInputTypes = {"Capital", "Labour"};

        for (int i = 0; i < size; i++) {
            int randomID = randomNumberGenerator.nextInt(0, 1000000);
            String randomFactorInputName = randomStringGenerator();
            String randomFactorInputDescription = randomStringGenerator();
            double randomMarginalCost = randomNumberGenerator.nextDouble(0, 1000);
            double randomMarginalRevenue = randomNumberGenerator.nextDouble(0, 1000);

            factorInputs.add(new FactorInput(randomID, factorInputTypes[randomNumberGenerator.nextInt(0, 2)],
randomFactorInputName, randomFactorInputDescription, randomMarginalCost, randomMarginalRevenue));
        }

        return factorInputs;
    }

    public void maximiseProfitWithFixedCostRandom() {
        final ArrayList<FactorInput> factorInputs = randomFactorInputGenerator(5);
        final int maxCost = randomNumberGenerator.nextInt(0, 2500);
        ArrayList<FactorInput> bestFactorInputs = CostOfProductionLogic.showBestCombination(factorInputs, maxCost);

        System.out.println("All Workers:");
        for (int i = 0; i < factorInputs.size(); i++) {
            System.out.println(factorInputs.get(i).toString());
```

```
        }

        System.out.println("\nMax Cost Restraint: " + maxCost + "\n");

        System.out.println("Best Workers:");
        for (int i = 0; i < bestFactorInputs.size(); i++) {
            System.out.println(bestFactorInputs.get(i).toString());
        }
    }
}
```

By comparing the actual data (the list of all workers) with the expected data (the “optimal” workers), I can confirm that the logic behind the cost and profit calculation works as intended. This comparison helps ensure that the method works with the cost constraint and maximises total profit, demonstrating that the selection process produces accurate results under different random conditions.

All Workers

ID	Type	Name	Description	Marginal Cost	Marginal Revenue	Marginal Profit
632805	Labour	vLUGB	shIgH4Hl	352.53	611.30	258.77
980179	Capital	vYbg3v	RvSnpFJrRe	289.13	185.41	-103.72
153250	Capital	P	ZGsN5	16.79	848.93	832.14
306811	Labour	WqYD	quvk	26.25	812.63	786.38
22009	Labour	Fd4hGImpOs9OQ	9oJvl9x2b	815.38	390.44	-424.94

Max Cost Restraint: 66

Expected Optimal Workers

ID	Type	Name	Description	Marginal Cost	Marginal Revenue	Marginal Profit
153250	Capital	P	ZGsN5	16.79	848.93	832.14
306811	Labour	WqYD	quvk	26.25	812.63	786.38

Actual Optimal Workers

ID	Type	Name	Description	Marginal Cost	Marginal Revenue	Marginal Profit
153250	Capital	P	ZGsN5	16.79	848.93	832.14
306811	Labour	WqYD	quvk	26.25	812.63	786.38

Evaluation

Objective Analysis

Objective 1 - News Tab

I believe I have fulfilled this objective, as the system is able to gather news from multiple news sources and is able to summarise each article through the use of Ollama AI. I am very pleased with how I was able to gather data from public .XML files, and how I can manipulate the data to allow users to view the important information of each article, such as the headline, description, and link to the full article. I successfully implemented a feature where users can sort articles by date or headline and can search for specific articles by headline.

However, due to the lack of time and knowledge to train A.I. models to process current economic news and understand possible changes to the economy, the system is unable to provide predictions about the future economy.

Objective 2 - Economic Figures & Diagrams

I believe that I accomplished this objective, as the user can view real-time economic indicators from multiple APIs, such as the World Bank API and FrankFurter API. The website displays statistics such as Real GDP, Inflation Rates and Exchange Rates. I am happy that this process happens dynamically, which allows diagrams to show changes in the data over time, and a currency converter allows users to compare values between currencies.

However, due to the lack of time and knowledge to train A.I. models to handle changes of macro economic indicators, the system is unable to provide predictions about the future economy.

Objective 3 - Economy Rater

I am pleased that I was able to create a system that gathers data from a JSON file I made, where the user's rating will be stored in a database and all their answers will be stored in a separate JSON file. This allows me to create a feature that asks users with fixed questions associated with the current economic climate, and through Javascript users are able to see feedback from the rest of society at specific dates through pie charts.

Objective 4 - Investment Simulator

I am happy that I was able to learn OOP in Python, learning how to use static methods and composition, and my ability to use complex mathematical algorithms, such as Linear Regression and calculate Simple Moving Averages to highlight the trend in stock prices over a period of time. I am also happy that I was able to successfully implement multithreading, which speeds the process of parsing data from the yfinance library to my website. Using a combination of these technologies, I was able to create a system that can gather real-time stock market values, and display diagrams indicating historical trends and the volume of stocks. I am also happy to be able to use more complex SQL

queries, such as being able to gather all stocks data held by a specific user or using aggregate functions.

Objective 5 - Budget Tracker

I believe that this objective has been met as I was to create my own algorithm that calculates whether a user purchase is deemed to be risky, if the standard deviation of the good is 1.5x greater than the typical price of the good, allows users to stay informed of their purchases. I am happy that I was able to include multithreading, which uses queue operations, so that I am able to parse past spending simultaneously, loading the pages faster. I am glad that I was able to incorporate matrix and vector calculation, to calculate future budgets based on the current inflation rates and being able to split income based on essentials, leisure, and saving. I used recursive binary search and merge sort, to allow users to search their purchases. This objective can successfully support budgeting decisions.

Objective 6 - Cost Of Production

This objective allows producers to manage factor input data. I am very pleased that I was able to implement a Combinatorial Subset Search to find the group of most profitable factor inputs under a cost restraint, which allows firms to know their most productive workers. The system is able to take data about the factor inputs, and store it in the database, which allows me to generate graphs showing the increasing trend in marginal cost and marginal trend, showing whether firms are making an overall profit or loss. I am glad that I have used a stack to create an undo button which allows producers to undo any changes they made to their factor inputs, which leads to the appropriate change made to the database.

Objective 7 - Login System

The system collects details about the user, where the account made must have a unique email address. Passwords must meet complexity requirements and are stored using Vernam Cipher encryption. I am happy that I was able to store user's passwords as ciphertext in the database and the vernam key in a JSON file, and I am glad I was able to create a randomly unique Vernam Key, every time the user login into the system, which overwrites the JSON file and the database entry.

Objective 8 - Friendly to use UI Design

The system has a clean layout with structured headings, and differentiated sections. Navigation menus enable intuitive access to features. Buttons, text fields, tables, and graphs are used to present data and support user input. Error handling has been used to prevent mistakes through the use of error messages and undo functionality.

Objective 9 - User Dashboard

I am glad that I was able to create a system that provides a searchable glossary of economic terms, by parsing a list of JSON objects to Java. I am happy that I was able to use Python Flask API, to send JSON payloads from Python to Java and vice versa so that users can ask questions to Ollama AI, and receive answers to questions about finance or the economy.

User Feedback

I gave my end user access to the system along with a feedback form so I could see how well I met each of my objectives. They could rate each one out of 5, with 5 being the highest. The end user can provide a description of how well I met each objective and what could be improved.

Objective	Score & Description
News Tab	**5** It is very pleasant to see which news outlet supports which political view, such as the left or right. Additionally, the AI summariser is a nice feature, being able to process the information in a quick way is very nice in a world where time is limited. But something that would be a nice add-on is to see which country or region the news is referring to, potentially being a stretch goal. Other than that this feature has been perfected.
Economic Figures and Diagrams	**4** I am very pleased to see how you were able to display information about the economy in simple graphs, showing the trends on macroeconomic indicators.
Economy Rater	**4** I like how users are able to assess the economy based on their own opinions, by answering questions. Perhaps you could reduce the total number of questions asked to a user to 5 and instead of giving the option to do it anytime, maybe make the questions be random and spread out to a couple of days or weeks, so that it doesn't feel too repetitive. Other than that the fact you can see other peoples options is a really nice addition.
Investment Simulator	**5** I am really impressed how you were able to replicate the stock market, using real life data. I like how users are able to view graphs and the trend of SMAs, which can warn users whether they should buy or sell stocks. I like how this feature is able to show the public that risk-taking is necessary for the economy as it leads to higher rewards.
Budget Tracker	**5** I really like this feature. I like how users are able to input their

	purchases to the system, leaving a record of how users are able to view their spending. The graphs showing weekly spending and future income projections is a nice touch, as users begin to understand the consequences of the cost of living crisis. Instead of automatically splitting a user's budget into essentials, leisure and saving, users should be able to manually select what proportion of their budget should be essentials, leisure or savings.
Cost Of Production	**4** I like how firms are able to see how profitable their workers are. Firms being able to assess the most profitable workers under a cost restraint is an excellent feature, as it informs which workers should be laid off and which workers should be promoted. The ability to undo changes to factor inputs is nice, as it allows firms to revert accidental changes made.
Login System	**4** It is simple and straight to the point. I like the added feature, where the user's passwords are encrypted, which is important to keep the user's finances and incomes hidden. However, it would be nice if there was a reset password system, which sends an email confirmation to change your password.
Friendly to use UI Design	**4** The UI is very friendly and intuitive. Users easily know how to navigate through the website. It would be nice if there were (i) buttons, so for someone who has no knowledge of the economy, they can hover above the information button, which summarises each feature or terminology.
User Dashboard	**5** I like how users are able to use the AI system to answer any concerns they may have over the current economy, or anything to do with finance. The glossary is a very nice feature. It is a good place for beginners to learn economic terms.

Possible Extensions

Overall, I am very pleased how my system has turned out. I was able to create a website that allows users to easily gather knowledge of the current economy, and allows consumers and producers to use a variety of features, to teach them how to effectively handle money, how to take risks to gain higher rewards, and how they can improve their productivity and profitability. However, after discussing with my end user, we realised that the system could have additional features to improve user experience and functionality. If I had more time, I would also implement the following features:

1. While the system gathers news articles from several sources and processes them using Ollama AI to generate summaries, I could include user-defined sources, allowing individuals to customise where the system retrieves news from, and enabling a broader, more personalised news feed.

2. I would have liked to add a feature that predicts future economic conditions. Right now, the system can show real-time data and let users see how things change over time through diagrams, but it can't actually forecast what might happen next. Adding prediction tools, such as regression models or machine-learning-based forecasts, but it wasn't realistic within the time-frame, and I didn't have enough machine-learning knowledge to include it in the system.

3. The system provides a set of fixed questions about the economy, allowing users to compare their views with the public. In the future, these questions could be made random and changed based on the current economic environment, which would improve its relevance and make the system more engaging.

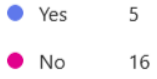
4. Although the system gathers real-time stock data, displays trends and moving averages, and lets users buy and sell stocks, the system could be improved by letting users compare stocks with other types of investments, such as bonds, real estate, or savings accounts, helping them understand how different assets perform over time.

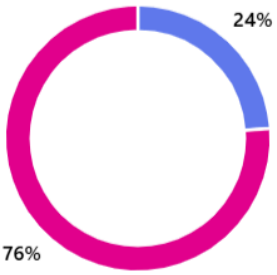
5. I would have liked to implement a simulation feature that would have allowed the system to analyse different scenarios for a firm, showing how changes in market price, labour productivity, or random demand and supply shocks could impact costs, revenue, and profitability over time. With more time, this could be expanded to give producers a clearer idea of potential outcomes under various conditions.

Appendix

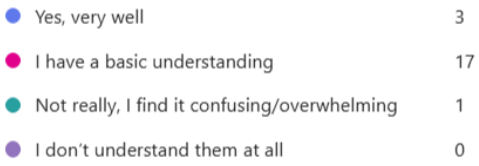
Appendix A - Initial Survey Results

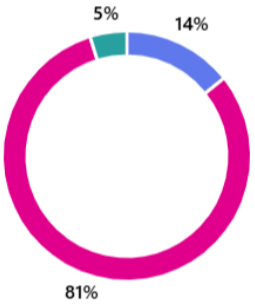
1. What is your name?
2. Do you currently study A-Level Economics?





3. Do you feel you understand economic terms (like inflation, GDP, or unemployment rates) well enough to make informed decisions about your finances?



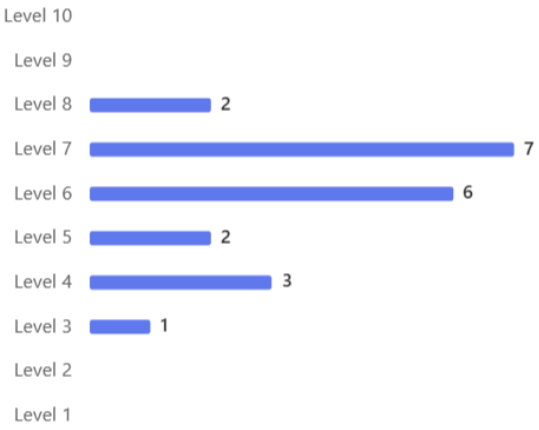


4. If you don't feel confident with economic terms well enough to make informed decisions about your finances, Why?
    - “I don't really follow on with the news and current events”
    - “Was never taught properly, how I was taught was always a one time thing and in not enough detail that would usually not get followed up on, so I would forget over time”
    - “I don't do enough research about the topic/ unreliable sources of information”
    - “Seems complex from someone outside financial spaces”
    - “Just don’t know enough and it’s boring”

5. How would you describe the state of the UK economy?
    - “From what I know, it's been losing its value over the years meaning the economy is failing (i think)”
    - “Inflation is quite bad currently, cost of living crisis, housing crisis”
    - “Really bad”
    - “Not great, but I honestly couldn't tell you. I know inflation is up because prices have clearly gone up.”
    - “I don’t know” - from 8 respondents

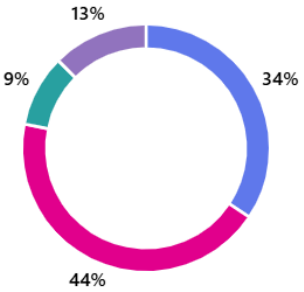
6. From a scale of 1 - 10 how do you rate your financial literacy (1 being financially illiterate - 10 being financially literate)





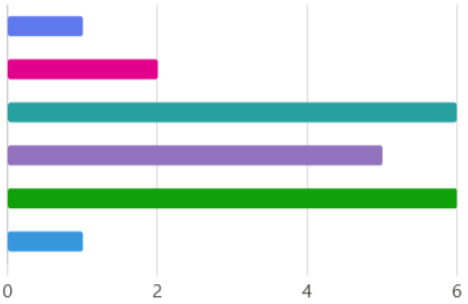
7. How do you mainly research the current economic climate?

Following the news, such as BBC News or The Guardian	11
Social Media, such as using Tiktok, Twitter, or listening to podcasts	14
Government & Institutional Reports, such as The Federal Reserve or UK Office for National Statistics	3
I don't research on the current state of the economy	4
Other	0



8. How often do you read or view news related to the economy each month?

Daily	1
Few times a week	2
Weekly	6
Few times a month	5
Rarely	6
Never	1



Tharun Nadeswaran - Money Matters
Student Code -
Centre Number -
580

9. What are the benefits when you research the current economic climate?
    - “Getting a better understanding of what is happening around the world and of things that may affect my future”
    - “You can make informed decisions”
    - “Helps with budgeting, understanding what exactly you are spending on and why.”
    - “Helps me understand economic better and what to invest in the future”
    - “I am more knowledgeable of the current state of the economy and I can make more financial decisions.”
    - “I am able to have conversations with people about the state of the economy”

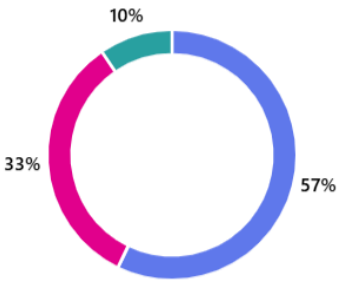
10. What problems do you face when researching the current economic climate?
    - “A lot of right wing content, it would be amazing if you could sort by different types of viewpoints on websites”
    - “Gathering accurate or trustworthy figures and information across the different articles and sites”
    - “Hard to navigate where to research as there is always a level above of knowledge I don’t know”
    - “Misinformation and not knowing where to search.”
    - “Just don't. It's not interesting to me. Too confusing and I don't have the time nor will.”

11. Do you think that society over-exaggerates the current economic climate (such as a rise in A.I. leads to unemployment, or the cost of living crisis)? Why/Why not?
    - “No, as these examples aren’t spoken, to me at least, enough”
    - “I don't think so, because I pay for things, have 2 part time jobs and it's very evident that prices are very high.”
    - “No because it’s a serious thing with trump throwing tariffs around chaotically”
    - “Yes, because in the case of most innovations in the past more jobs have been created than those lost.”
    - “I think that even though the overall jobs may stay the same, there are more high paying jobs that require more education (e.g. software engineering). However, the number of lower paying (or repetitive) jobs (e.g. cashier, accountant) will reduce, meaning that people will become unemployed if they work in certain sectors.”
    - “Yes, many suggested economic articles I see does exaggerated these, probably to get more people believing one thing is that much of an impact”

12. What do you think about the government's approach to managing inflation and prices?
    - "Stop printing money that cannot be sustained, investing in long term growth opportunities"
    - "Borrowing more and spending less on public services will solve these issues in the long-term."
    - "They have just increased interest rates and increased prices which is not good, especially for loans for borrowing."
    - "Bad, it keeps going up"
    - "I don't know" - 17 respondents

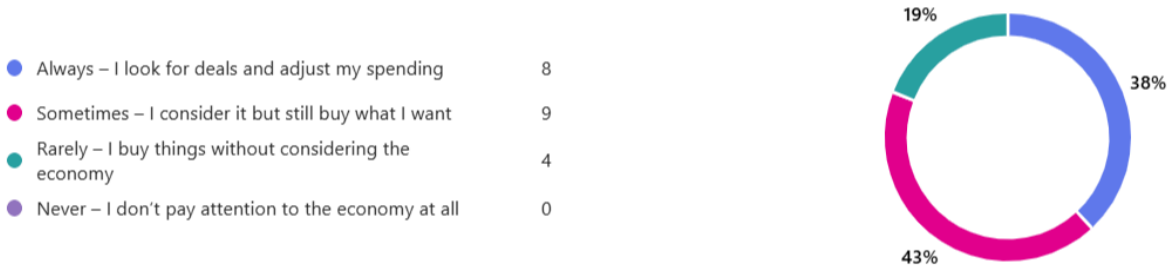
13. How much do you think inflation impacts your personal spending?

A lot – I notice prices going up	12
A little – but I don't think about it much	7
Not at all – I don't really pay attention	2
I don't know how inflation affects my spending	0

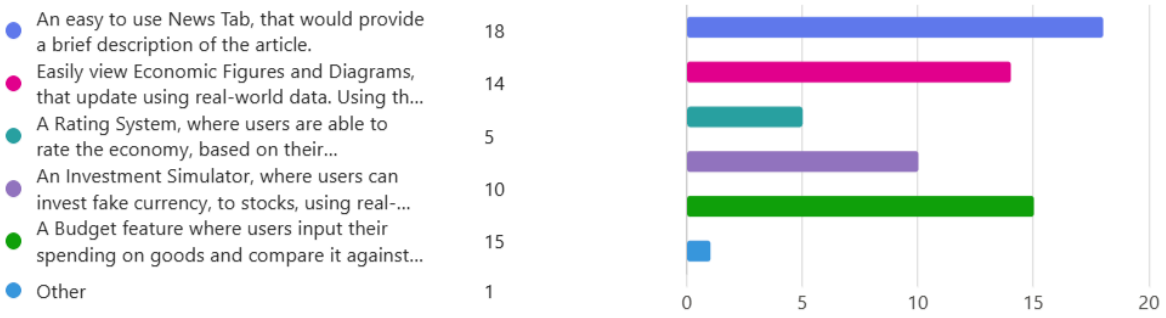


14. Would you find it beneficial to track your spending? Why/Why not?
    - "Yes, I spend too much on things I don't need and then realise too late when I have barely any money left"
    - "I find it beneficial because I would like to save enough for future spending like university costs"
    - "Yes, easier to track how much you have left and to reach a certain goal if you want to buy something"
    - "I do find it useful as I like to have a general idea of how much money I'm spending and taking in every month'"
    - "Yes, it would allow me to create a steady savings amount each month."
    - None of the respondents argued against the idea of tracking spending.
    - "Yes, tracking your spending can be very beneficial. It helps you understand where your money is going, spot bad habits, and make better budgeting decisions. By keeping an eye on your expenses, you're more likely to stay within your limits, save more, and avoid unnecessary debt. It also gives you a clearer picture of your financial health, which can reduce stress and help you reach your goals faster."

15. How often do you make financial decisions based on sales, discounts, price markups?

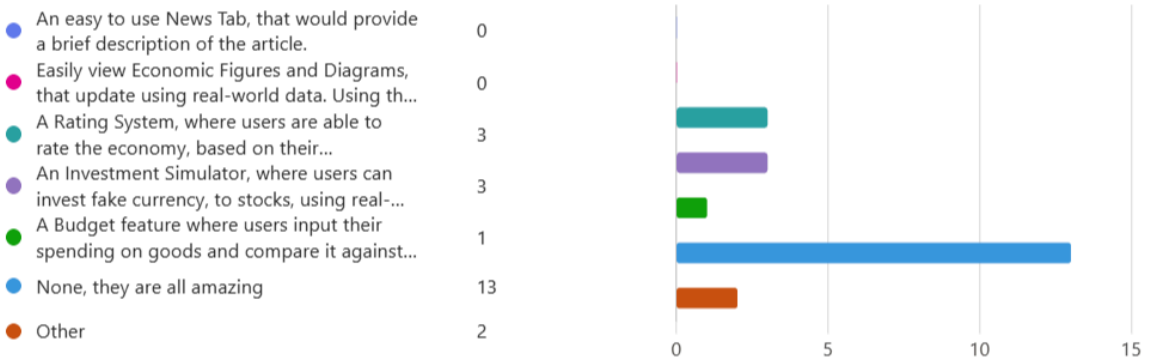


16. What features would you find beneficial to understand shifts in the economy?



- Other - "Don't really get the question or understand the options"

17. Are there any features you dislike about the system?



- Other - “Same as previous question”
- Other - “I don't understand the rating system”

18. If you disliked any features in Question 17, Why?
    - “Subjective opinions may differ based on social class and other factors” - referring to Rate the Economy
    - “It's not that i hate the idea, it's more like it won't be used by the normal individual but it will definitely be used by someone who's invested into the stock market but not someone who wishes to just learn about the economy” - referring to the Investment Simulator

- “Some people may be uncomfortable sharing personal information on a database.” - referring to Budget Tracker
- “Even if you have an investment simulator, it still doesn’t fundamentally teach you anything, it just allows you to try your luck if you don’t have the knowledge” - referring to the Investment Simulator
- “I don’t dislike the aim of the feature, but it must be clear to help users to minimise the risks of their investments.” - referring to the Investment Simulator

19. Would you feel confident to make effective financial decisions, if there was a system that has all the features you have selected in Question 16?



Appendix B - The Prototype Code

Controllers

MoneyMatters.java

```
package com.mycompany.MoneyMatters;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class MoneyMattersApplication {
    public static void main(String[] args) {
        SpringApplication.run(MoneyMattersApplication.class, args);
    }
}
```

LoginController.java

```
package com.mycompany.MoneyMatters;

import org.springframework.web.bind.annotation.RestController;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.servlet.ModelAndView;

import jakarta.servlet.http.HttpSession;

@RestController
public class LoginController {

    @GetMapping("/")
    public ModelAndView redirectToStartScreen() {
        ModelAndView mv = new ModelAndView("redirect:start");
        return mv;
    }

    @GetMapping("start")
    public ModelAndView showStartScreen(HttpSession session) {
        session.invalidate();
        ModelAndView mv = new ModelAndView("login/start");
        return mv;
    }

    @GetMapping("signup")
    public ModelAndView showSignupScreen() {
        ModelAndView mv = new ModelAndView("login/signup");
        return mv;
    }

    @PostMapping("signup")
    public ModelAndView storeSignupData(Model modeL, String firstName,
String lastName, int age, String dateOfBirth, String email, String
phoneNumber, String password, String typeOfEconomicAgent) {

        ModelAndView mv = new ModelAndView();
        if (typeOfEconomicAgent.equals("Consumer")) {
            Consumer consumer = new Consumer(firstName, lastName, age,
dateOfBirth, email, phoneNumber, password);
            SQLQueriesConsumers.insertDataIntoConsumerTable(consumer);
        }

        else if (typeOfEconomicAgent.equals("Producer")) {
            Producer producer = new Producer(firstName, lastName, age,
```

```
dateOfBirth, email, phoneNumber, password);
            SQLQueriesProducers.insertDataIntoProducerTable(producer);
        }

        else {
            mv.setViewName("login/wrong-details");
            return mv;
        }

        mv.setViewName("login/login");
        return mv;
    }

    @GetMapping("/login")
    public ModelAndView showLoginScreen() {
        ModelAndView mv = new ModelAndView("login/login");
        return mv;
    }

    @PostMapping("/login")
    public ModelAndView authenticateLogin(String email, String password,
HttpSession session) {
        int consumerID = SQLQueriesConsumers.authenticateLogin(email,
password);
        int producerID = SQLQueriesProducers.authenticateLogin(email,
password);

        ModelAndView mv = new ModelAndView();
        mv.setViewName("redirect:user-dashboard");

        if (consumerID > 0) {
            Consumer consumer =
SQLQueriesConsumers.getConsumerData(consumerID);
            session.setAttribute("user", consumer);
        }

        else if (producerID > 0) {
            Producer producer =
SQLQueriesProducers.getProducerData(producerID);
            session.setAttribute("user", producer);
        }

        else {
            mv.setViewName("login/wrong-details");
        }

        return mv;
    }
```

```
    @GetMapping("/wrong-details")
    public ModelAndView showWrongDetailsScreen() {
        ModelAndView mv = new ModelAndView("login/wrong-details");
        return mv;
    }
}
```

UserDashboardController.java

```
package com.mycompany.MoneyMatters;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.servlet.ModelAndView;

import jakarta.servlet.http.HttpSession;

@RestController
public class UserDashboardController {

    @GetMapping("/user-dashboard")
    public ModelAndView showUserDashboard(HttpSession session) {
        ModelAndView mv = new ModelAndView("userdashboard/user-dashboard");

        if (session.getAttribute("user") instanceof Consumer) {
            System.out.println("Consumer is logged in");
            Consumer consumer = (Consumer) session.getAttribute("user");

            consumer.getBudgetSize();

            try {
                Double budgetSize =
SQLQueriesConsumers.getBudgetSize(consumer);
                consumer.setBudgetSize(budgetSize);
                mv.addObject("budgetSize", budgetSize);

                String frequencyOfBudgetRenewal =
SQLQueriesConsumers.getFrequencyOBudgetRenewal(consumer);
                mv.addObject("frequencyOfBudgetRenewal",
frequencyOfBudgetRenewal);

consumer.setFrequencyOfBudgetRenewal(frequencyOfBudgetRenewal);
            }

            catch (Exception e) {
                System.out.println("Error retrieving budget data: " +
e.getMessage());
                mv.addObject("error", "Could not retrieve budget data.
Please try again later.");
            }
        }

        return mv;
    }
}
```

Tharun Nadeswaran - Money Matters
Student Code -
Centre Number -
590

EconomicNewsController.java

```
package com.mycompany.MoneyMatters;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.servlet.ModelAndView;

@RestController
public class EconomicNewsController {

    @GetMapping("news-BBCNews")
    public ModelAndView showBBCNews() {
        String URL =
"https://feeds.bbci.co.uk/news/business/economy/rss.xml";
        String newsContent = WebScraperLogic.scrapeNews(URL);

        ModelAndView mv = new ModelAndView("economicnews/news-BBCNews");
        mv.addObject("newsContent", newsContent);
        return mv;
    }

    @GetMapping("news-TheTelegraph")
    public ModelAndView showTheTelegraph() {
        String URL = "https://www.telegraph.co.uk/business/rss.xml";
        String newsContent = WebScraperLogic.scrapeNews(URL);

        ModelAndView mv = new ModelAndView();
        mv.setViewName("economicnews/news-TheTelegraph");
        mv.addObject("newsContent", newsContent);
        return mv;
    }
}
```

BudgetTrackerController.Java

```
package com.mycompany.MoneyMatters;

import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.servlet.ModelAndView;

import jakarta.servlet.http.HttpSession;

@RestController
public class BudgetTrackerController {

    @GetMapping("budget-tracker")
    public ModelAndView showBudgetTracker(HttpSession session) {
        ModelAndView mv = new ModelAndView("budgettracker/budget-tracker");
        Consumer consumer = (Consumer) session.getAttribute("user");

        if (consumer == null) {
            mv.setViewName("redirect:/login");
            return mv;
        }

        return mv;
    }

    @PostMapping("budget-tracker")
    public ModelAndView storeBudgetData(Model model, String budgetSize,
String frequencyOfRenewal, HttpSession session) {
        ModelAndView mv = new ModelAndView("redirect:user-dashboard");

        if (budgetSize == null || budgetSize.isEmpty() ||
frequencyOfRenewal.equals("N/A")) {
            mv.setViewName("redirect:budget-tracker");
            return mv;
        }

        Consumer consumer = (Consumer) session.getAttribute("user");

        SQLQueriesConsumers.updateBudgetSize(Double.parseDouble(budgetSize),
consumer);

SQLQueriesConsumers.updateFrequencyOfBudgetRenewal(frequencyOfRenewal,
consumer);
        return mv;
    }
```

```
    @GetMapping("good-purchased")
    public ModelAndView showGoodPurchased(HttpSession session) {
        ModelAndView mv = new ModelAndView("budgettracker/good-purchased");
        Consumer consumer = (Consumer) session.getAttribute("user");
        mv.addObject("consumerPurchases",
SQLQueriesConsumers.getConsumerPurchase(consumer));

        if (consumer == null) {
            mv.setViewName("redirect:/login");
            return mv;
        }

        return mv;
    }

    @PostMapping("good-purchased")
    public ModelAndView storeGoodPurchased(Model model, String goodName,
String goodPrice, String goodQuantity, HttpSession session) {
        ModelAndView mv = new ModelAndView("redirect:user-dashboard");
        Consumer consumer = (Consumer) session.getAttribute("user");

        double budgetSize = SQLQueriesConsumers.getBudgetSize(consumer);
        if (budgetSize < Double.parseDouble(goodPrice)) {
            mv.setViewName("redirect:budget-tracker");
            return mv;
        }

        if (SQLQueriesGoods.doesGoodIDExist(goodName,
Double.parseDouble(goodPrice))) {
            SQLQueriesGoods.insertGood(goodName,
Double.parseDouble(goodPrice));
        }

        GoodRecord good = new GoodRecord(SQLQueriesGoods.getGoodID(goodName,
Double.parseDouble(goodPrice)), goodName, Double.parseDouble(goodPrice));
        SQLQueriesPurchase.insertPurchase(consumer, good, goodQuantity);

        if (goodQuantity == null || goodQuantity.isEmpty()) {
            mv.setViewName("redirect:good-purchased");
            return mv;
        }

        budgetSize -= Integer.parseInt(goodQuantity) *
Double.parseDouble(goodPrice);
        SQLQueriesConsumers.updateBudgetSize(budgetSize, consumer);
        return mv;
    }
```

```
}
```

User Classes

UserInterface.java

```
package com.mycompany.MoneyMatters;

public interface UserInterface {

    public String getFirstName();
    public void setFirstName(String firstName);

    public String getLastName();
    public void setLastName(String lastName);

    public int getAge();
    public void setAge(int age);

    public String getDateOfBirth();
    public void setDateOfBirth(String dateOfBirth);

    public String getEmail();
    public void setEmail(String email);

    public String getPassword();
    public void setPassword(String password);

    public String getPhoneNumber();
    public void setPhoneNumber(String phoneNumber);
}
```

User.java

```java
package com.mycompany.MoneyMatters;

import org.springframework.context.annotation.Scope;
import org.springframework.stereotype.Component;

@Component
@Scope(value = "prototype")
public abstract class User implements UserInterface {
    protected String firstName;
    protected String lastName;
    protected int age;
    protected String dateOfBirth;
    protected String email;
    protected String phoneNumber;
    protected String password;

    @Override
    public String getFirstName() {
        return firstName;
    }

    @Override
    public void setFirstName(String firstName) {
        this.firstName = firstName;
    }

    @Override
    public String getLastName() {
        return lastName;
    }

    @Override
    public void setLastName(String lastName) {
        this.lastName = lastName;
    }

    @Override
    public int getAge() {
        return age;
    }

    @Override
    public void setAge(int age) {
        this.age = age;
    }
```

```
    @Override
    public String getDateOfBirth() {
        return dateOfBirth;
    }

    @Override
    public void setDateOfBirth(String dateOfBirth) {
        this.dateOfBirth = dateOfBirth;
    }

    @Override
    public String getEmail() {
        return email;
    }

    @Override
    public void setEmail(String email) {
        this.email = email;
    }

    @Override
    public String getPhoneNumber() {
        return phoneNumber;
    }

    @Override
    public void setPhoneNumber(String phoneNumber) {
        this.phoneNumber = phoneNumber;
    }

    @Override
    public String getPassword() {
        return password;
    }

    @Override
    public void setPassword(String password) {
        this.password = password;
    }
}
```

Consumer.java

```
package com.mycompany.MoneyMatters;

import org.springframework.context.annotation.Scope;
import org.springframework.stereotype.Component;

@Component
@Scope(value = "prototype")
public class Consumer extends User {

    private int consumerID;
    private double budgetSize;
    private double spendingAmount;
    private String frequencyOfBudgetRenewal;
    private double sustainableSpending;

    public int getConsumerID() {
        return consumerID;
    }

    public void setConsumerID(int consumerID) {
        this.consumerID = consumerID;
    }

    public double getBudgetSize() {
        return budgetSize;
    }

    public void setBudgetSize(double budgetSize) {
        this.budgetSize = budgetSize;
    }

    public double getSpendingAmount() {
        return spendingAmount;
    }

    public void setSpendingAmount(double spendingAmount) {
        this.spendingAmount = spendingAmount;
    }

    public String getFrequencyOfBudgetRenewal() {
        return frequencyOfBudgetRenewal;
    }

    public void setFrequencyOfBudgetRenewal(String frequencyOfBudgetRenewal)
{
        this.frequencyOfBudgetRenewal = frequencyOfBudgetRenewal;
    }
```

```
    public double getSustainableSpending() {
        return sustainableSpending;
    }

    public void setSustainableSpending(double sustainableSpending) {
        this.sustainableSpending = sustainableSpending;
    }

    public Consumer () {

    }

    public Consumer(String firstName, String lastName, int age, String
dateOfBirth, String email, String phoneNumber, String password) {
        this.firstName = firstName;
        this.lastName = lastName;
        this.age = age;
        this.dateOfBirth = dateOfBirth;
        this.email = email;
        this.phoneNumber = phoneNumber;
        this.password = password;
    }

    public Consumer(int consumerID, String firstName, String lastName, int
age, String dateOfBirth, String email, String phoneNumber, String password)
{
        this.consumerID = consumerID;
        this.firstName = firstName;
        this.lastName = lastName;
        this.age = age;
        this.dateOfBirth = dateOfBirth;
        this.email = email;
        this.phoneNumber = phoneNumber;
        this.password = password;
    }
}
```

Producer.java

```
package com.mycompany.MoneyMatters;

import org.springframework.context.annotation.Scope;
import org.springframework.stereotype.Component;

@Component
@Scope(value = "prototype")
public class Producer extends User {

    private int producerID;

    public int getProducerID() {
        return producerID;
    }

    public void setProducerID(int producerID) {
        this.producerID = producerID;
    }

    public Producer () {

    }

    public Producer(String firstName, String lastName, int age, String
dateOfBirth, String email, String phoneNumber, String password) {
        this.firstName = firstName;
        this.lastName = lastName;
        this.age = age;
        this.dateOfBirth = dateOfBirth;
        this.email = email;
        this.phoneNumber = phoneNumber;
        this.password = password;
    }

    public Producer(int producerID, String firstName, String lastName, int
age, String dateOfBirth, String email, String phoneNumber, String password)
{
        this.producerID = producerID;
        this.firstName = firstName;
        this.lastName = lastName;
        this.age = age;
        this.dateOfBirth = dateOfBirth;
        this.email = email;
        this.phoneNumber = phoneNumber;
        this.password = password;
    }
}
```

Tharun Nadeswaran - Money Matters
Student Code -
Centre Number -
600

SQLQueries

SQLQueries.java

```
package com.mycompany.MoneyMatters;

public abstract class SQLQueries {

    protected static final String url =
"jdbc:sqlserver://localhost\\SQLEXPRESS:1433;databaseName=MONEYMATTERSDB;enc
rypt=true;trustServerCertificate=true;";
    protected static final String databaseUser = "bot1";
    protected static final String databasePassword = "password";

    public String getUrl() {
        return url;
    }

    public String getDatabaseUser() {
        return databaseUser;
    }

    public String getDatabasePassword() {
        return databasePassword;
    }
}
```

SQLQueriesConsumers.java

```
package com.mycompany.MoneyMatters;

import java.sql.Connection;
import java.sql.Date;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;

import org.springframework.stereotype.Repository;

@Repository
public class SQLQueriesConsumers extends SQLQueries {

    public static void insertDataIntoConsumerTable(Consumer consumer) {
        String query = "INSERT INTO Consumers (FirstName, LastName, Age,
DateOfBirth, Email, PhoneNumber, Password) VALUES (?, ?, ?, ?, ?, ?, ?)";

        try (Connection conn = DriverManager.getConnection(url,
databaseUser, databasePassword);
            PreparedStatement pstmt = conn.prepareStatement(query)) {

            pstmt.setString(1, consumer.getFirstName());
            pstmt.setString(2, consumer.getLastName());
            pstmt.setInt(3, consumer.getAge());
            pstmt.setDate(4, Date.valueOf(consumer.getDateOfBirth()));
            pstmt.setString(5, consumer.getEmail());
            pstmt.setString(6, consumer.getPhoneNumber());
            pstmt.setString(7, consumer.getPassword());

            pstmt.executeUpdate();
        }

        catch (SQLException e) {
            System.out.println("Connection failed: " + e.getMessage());
        }
    }

    public static Consumer getConsumerData(int consumerID) {
        String query = "SELECT * FROM Consumers WHERE ConsumerID = ?";

        try (Connection conn = DriverManager.getConnection(url,
databaseUser, databasePassword);
            PreparedStatement pstmt = conn.prepareStatement(query)) {
```

```
            pstmt.setInt(1, consumerID);
            ResultSet rs = pstmt.executeQuery();

            if (rs.next()) {
                Consumer consumer = new Consumer();
                consumer.setConsumerID(rs.getInt("ConsumerID"));
                consumer.setFirstName(rs.getString("FirstName"));
                consumer.setLastName(rs.getString("LastName"));
                consumer.setAge(rs.getInt("Age"));
                consumer.setDateOfBirth(rs.getString("DateOfBirth"));
                consumer.setEmail(rs.getString("Email"));
                consumer.setPhoneNumber(rs.getString("PhoneNumber"));
                consumer.setPassword(rs.getString("Password"));

                return consumer;
            }

            else {
                System.out.println("No user found with ID: " + consumerID);
                return null;
            }

        }

        catch (SQLException e) {
            System.out.println("Error retrieving user: " + e.getMessage());
            return null;
        }
    }

    public static int authenticateLogin(String email, String password) {
        String query = "SELECT ConsumerID FROM Consumers WHERE Email = ? AND
Password = ?";

        try (Connection conn = DriverManager.getConnection(url,
databaseUser, databasePassword);
            PreparedStatement pstmt = conn.prepareStatement(query)) {

            pstmt.setString(1, email);
            pstmt.setString(2, password);

            ResultSet rs = pstmt.executeQuery();

            if (rs.next()) {
                int consumerID = rs.getInt("ConsumerID");
                return consumerID;
            }
```

```
            else {
                System.out.println("Invalid email or password.");
                return -1;
            }
        }

        catch (SQLException e) {
            System.out.println("Login failed: " + e.getMessage());
            return -1;
        }
    }

    public static void updateBudgetSize(Double budgetSize, Consumer
consumer) {
        String query = "UPDATE Consumers SET BudgetSize = ? WHERE ConsumerID
= ?";

        try (Connection conn = DriverManager.getConnection(url,
databaseUser, databasePassword);
            PreparedStatement pstmt = conn.prepareStatement(query)) {

            pstmt.setDouble(1, budgetSize);
            pstmt.setInt(2, consumer.getConsumerID());

            int rows = pstmt.executeUpdate();
            if (rows > 0) {
                System.out.println("Budget data updated");
            }
        }

        catch (SQLException e) {
            System.out.println("Connection failed: " + e.getMessage());
        }
    }

    public static void updateFrequencyOfBudgetRenewal(String
frequencyOfRenewal, Consumer consumer) {
        String query = "UPDATE Consumers SET FrequencyOfRenewal = ? WHERE
ConsumerID = ?";

        try (Connection conn = DriverManager.getConnection(url,
databaseUser, databasePassword);
            PreparedStatement pstmt = conn.prepareStatement(query)) {

            pstmt.setString(1, frequencyOfRenewal);
            pstmt.setInt(2, consumer.getConsumerID());
```

```
            int rows = pstmt.executeUpdate();
            if (rows > 0) {
                System.out.println("FrequencyOfRenewal data updated");
            }
        }

        catch (SQLException e) {
            System.out.println("Connection failed: " + e.getMessage());
        }
    }

    public static Double getBudgetSize(Consumer consumer) {
        String query = "SELECT BudgetSize FROM Consumers WHERE ConsumerID =
?";

        try (Connection conn = DriverManager.getConnection(url,
databaseUser, databasePassword);
            PreparedStatement pstmt = conn.prepareStatement(query)) {

            pstmt.setInt(1, consumer.getConsumerID());
            ResultSet rs = pstmt.executeQuery();

            if (rs.next()) {
                return rs.getDouble("BudgetSize");
            }

            else {
                System.out.println("No budget found with ID: " +
consumer.getConsumerID());
                return null;
            }
        }

        catch (SQLException e) {
            System.out.println("Error retrieving user: " + e.getMessage());
            return null;
        }
    }

    public static String getFrequencyOBudgetRenewal(Consumer consumer) {
        String query = "SELECT FrequencyOfRenewal FROM Consumers WHERE
ConsumerID = ?";

        try (Connection conn = DriverManager.getConnection(url,
databaseUser, databasePassword);
            PreparedStatement pstmt = conn.prepareStatement(query)) {
```

```
            pstmt.setInt(1, consumer.getConsumerID());
            ResultSet rs = pstmt.executeQuery();

            if (rs.next()) {
                return rs.getString("FrequencyOfRenewal");
            }

            else {
                System.out.println("No budget found with ID: " +
consumer.getConsumerID());
                return null;
            }
        }

        catch (SQLException e) {
            System.out.println("Error retrieving user: " + e.getMessage());
            return null;
        }
    }

    public static ArrayList<ConsumerPurchasesRecord>
getConsumerPurchase(Consumer consumer) {
        String query = "SELECT Goods.GoodName, Goods.GoodPrice,
Purchase.QuantityBought, Purchase.PurchaseDate FROM Consumers, Goods,
Purchase WHERE Consumers.ConsumerID = Purchase.ConsumerID AND Goods.GoodID =
Purchase.GoodID AND Consumers.ConsumerID = ? ORDER BY Purchase.PurchaseDate
DESC";

        ArrayList<ConsumerPurchasesRecord> consumerPurchases = new
ArrayList<>();

        try (Connection conn = DriverManager.getConnection(url,
databaseUser, databasePassword);
            PreparedStatement pstmt = conn.prepareStatement(query)) {

            pstmt.setInt(1, consumer.getConsumerID());

            ResultSet rs = pstmt.executeQuery();
            while (rs.next()) {
                consumerPurchases.add(new ConsumerPurchasesRecord(
                    rs.getString("GoodName"),
                    rs.getDouble("GoodPrice"),
                    rs.getInt("QuantityBought"),
                    rs.getDate("PurchaseDate").toString()
                ));
            }
```

```
        }

        catch (SQLException e) {
            System.out.println("Connection failed: " + e.getMessage());
        }

        return consumerPurchases;
    }
}
```

SQLQueriesGoods.java

```
package com.mycompany.MoneyMatters;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;

import org.springframework.stereotype.Repository;

@Repository
public class SQLQueriesGoods extends SQLQueries {

    public static void insertGood(String goodName, Double goodPrice) {
        String query = "INSERT INTO Goods (GoodName, GoodPrice) VALUES (?,
?)";

        try (Connection conn = DriverManager.getConnection(url,
databaseUser, databasePassword);
            PreparedStatement pstmt = conn.prepareStatement(query)) {

            System.out.println("Connection Established");

            pstmt.setString(1, goodName);
            pstmt.setDouble(2, goodPrice);

            pstmt.executeUpdate();
        }

        catch (SQLException e) {
            System.out.println("Connection failed: " + e.getMessage());
        }
    }

    public static boolean doesGoodIDExist(String goodName, Double goodPrice)
{
```

```
        String query = "SELECT COUNT(*) AS count FROM Goods WHERE GoodName =
? AND GoodPrice = ?";

        try (Connection conn = DriverManager.getConnection(url,
databaseUser, databasePassword);
             PreparedStatement pstmt = conn.prepareStatement(query)) {

            System.out.println("Connection Established");

            pstmt.setString(1, goodName);
            pstmt.setDouble(2, goodPrice);

            ResultSet rs = pstmt.executeQuery();
            if (rs.next()) {
                int count = rs.getInt("count");

                if (count >= 1) {
                    return false;
                }

                else {
                    return true;
                }
            }
        }

        catch (SQLException e) {
            System.out.println("Connection failed: " + e.getMessage());
        }

        return false;
    }

    public static int getGoodID(String goodName, Double goodPrice) {
        String query = "SELECT GoodID From Goods WHERE GoodName = ? AND
GoodPrice = ?";

        try (Connection conn = DriverManager.getConnection(url,
databaseUser, databasePassword);
             PreparedStatement pstmt = conn.prepareStatement(query)) {

            System.out.println("Connection Established");

            pstmt.setString(1, goodName);
            pstmt.setDouble(2, goodPrice);

            ResultSet rs = pstmt.executeQuery();
```

```
            if (rs.next()) {
                int goodID = rs.getInt("GoodID");
                return goodID;
            }
        }

        catch (SQLException e) {
            System.out.println("Connection failed: " + e.getMessage());
        }

        return -1;
    }
}
```

SQLQueriesPurchase.java

```
package com.mycompany.MoneyMatters;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.SQLException;

import org.springframework.stereotype.Repository;

@Repository
public class SQLQueriesPurchase extends SQLQueries {

    public static void insertPurchase(Consumer consumer, GoodRecord good,
String quantity) {
        String query = "INSERT INTO Purchase (ConsumerID, GoodID,
QuantityBought) VALUES (?, ?, ?)";

        try (Connection conn = DriverManager.getConnection(url,
databaseUser, databasePassword);
            PreparedStatement pstmt = conn.prepareStatement(query)) {

            pstmt.setInt(1, consumer.getConsumerID());
            pstmt.setInt(2, good.goodID());
            pstmt.setInt(3, Integer.parseInt(quantity));

            pstmt.executeUpdate();
        }

        catch (SQLException e) {
            System.out.println("Connection failed: " + e.getMessage());
        }
    }
}
```

SQLQueriesProducers.java

```
package com.mycompany.MoneyMatters;

import java.sql.Connection;
import java.sql.Date;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;

import org.springframework.stereotype.Repository;

@Repository
```

```
public class SQLQueriesProducers extends SQLQueries {

    public static void insertDataIntoProducerTable(Producer producer) {
        String query = "INSERT INTO Producers (FirstName, LastName, Age,
DateOfBirth, Email, PhoneNumber, Password) VALUES (?, ?, ?, ?, ?, ?, ?)";

        try (Connection conn = DriverManager.getConnection(url,
databaseUser, databasePassword);
             PreparedStatement pstmt = conn.prepareStatement(query)) {

            pstmt.setString(1, producer.getFirstName());
            pstmt.setString(2, producer.getLastName());
            pstmt.setInt(3, producer.getAge());
            pstmt.setDate(4, Date.valueOf(producer.getDateOfBirth()));
            pstmt.setString(5, producer.getEmail());
            pstmt.setString(6, producer.getPhoneNumber());
            pstmt.setString(7, producer.getPassword());

            pstmt.executeUpdate();
        }

        catch (SQLException e) {
            System.out.println("Connection failed: " + e.getMessage());
        }
    }

    public static Producer getProducerData(int producerID) {
        String query = "SELECT * FROM Producers WHERE ProducerID = ?";

        try (Connection conn = DriverManager.getConnection(url,
databaseUser, databasePassword);
             PreparedStatement pstmt = conn.prepareStatement(query)) {

            pstmt.setInt(1, producerID);
            ResultSet rs = pstmt.executeQuery();

            if (rs.next()) {
                Producer producer = new Producer();
                producer.setProducerID(rs.getInt("ProducerID"));
                producer.setFirstName(rs.getString("FirstName"));
                producer.setLastName(rs.getString("LastName"));
                producer.setAge(rs.getInt("Age"));
                producer.setDateOfBirth(rs.getString("DateOfBirth"));
                producer.setEmail(rs.getString("Email"));
                producer.setPhoneNumber(rs.getString("PhoneNumber"));
                producer.setPassword(rs.getString("Password"));
                return producer;
            }
```

```
            else {
                System.out.println("No user found with ID: " + producerID);
                return null;
            }
        }

        catch (SQLException e) {
            System.out.println("Error retrieving user: " + e.getMessage());
            return null;
        }
    }

    public static int authenticateLogin(String email, String password) {
        String query = "SELECT * FROM Producers WHERE Email = ? AND Password
= ?";

        try (Connection conn = DriverManager.getConnection(url,
databaseUser, databasePassword);
            PreparedStatement pstmt = conn.prepareStatement(query)) {

            pstmt.setString(1, email);
            pstmt.setString(2, password);

            ResultSet rs = pstmt.executeQuery();

            if (rs.next()) {
                int producerID = rs.getInt("ProducerID");
                return producerID;
            }

            else {
                System.out.println("Invalid email or password.");
                return -1;
            }

        }

        catch (SQLException e) {
            System.out.println("Login failed: " + e.getMessage());
            return -1;
        }
    }
}
```

WebScraperLogic.java

```
package com.mycompany.MoneyMatters;

import java.io.*;
import java.net.URL;
import org.jsoup.Jsoup;
import org.jsoup.nodes.Document;
import org.jsoup.nodes.Element;
import org.jsoup.parser.Parser;
import org.jsoup.select.Elements;
import org.springframework.context.annotation.Scope;
import org.springframework.stereotype.Component;

@Component
@Scope(value = "prototype")
public class WebScraperLogic {

    public static String scrapeNews (String URL) {

        StringBuilder sb = new StringBuilder();

        try {
            InputStream inputLink = new URL(URL).openStream();
            Document doc = Jsoup.parse(inputLink, "UTF-8", "",
Parser.xmlParser());
            Elements headlines = doc.select("item");

            int EconomicNewsID = 1;

            for (Element headline : headlines) {
                String title = headline.select("title").text();
                String description = headline.select("description").text();
                String pubDate = headline.select("pubDate").text();
                String link = headline.select("link").text();

                sb.append(new EconomicNewsRecord(EconomicNewsID, title,
description, pubDate, link).toString());
                EconomicNewsID++;
            }
        }

        catch (IOException e) {
            System.out.println("An error occurred in scraping the news");
        }

        return sb.toString();
    }
}
```

Tharun Nadeswaran - Money Matters
Student Code -
Centre Number -
614

Records

ConsumerPurchasesRecord.java

```
public record ConsumerPurchasesRecord(String goodName, Double goodPrice, int
goodQuantity, String purchaseDate) {

    @Override
    public String toString() {
        return "Good Name: " + goodName + ", Good Price: " + goodPrice + ",
Good Quantity: " + goodQuantity + ", Purchase Date: " + purchaseDate + ",
<br>";
    }
}
```

EconomicNewsRecord.java

```
public record EconomicNewsRecord(int EconomicNewsID, String title, String
description, String pubDate, String link) {

    @Override
    public String toString() {
        return  EconomicNewsID + ") " + title + "<br>" +
                EconomicNewsID + ") " + description + "<br>" +
                EconomicNewsID + ") " + pubDate + "<br>" +
                EconomicNewsID + ") " + link + "<br><br>";
    }
}
```

GoodRecord.java

```
package com.mycompany.MoneyMatters;
public record GoodRecord (int goodID, String goodName, double goodPrice) {}
```

Frontend Code

start.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">

<head>
    <title>Money Matters</title>
</head>

<body>
    <h1>Money Matters Prototype - Start Screen</h1>

    <a th:href = "@{/login}">
        <button>Login</button>
    </a>

    <br>

    <a th:href = "@{/signup}">
        <button>Sign Up</button>
    </a>
</body>
</html>
```

signup.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">

<head>
    <title>Money Matters</title>
</head>

<body>
    <h1>Sign Up</h1>

    <form action = "/signup" method = "post">

        <div>
            <label for = "firstName">First Name:</label>
            <input type = "text" id = "firstName" name = "firstName"
placeholder = "John">

            <br>

            <label for = "lastName">Last Name:</label>
            <input type = "text" id = "lastName" name = "lastName"
placeholder = "Doe">

            <br>

            <label for = "age">Age:</label>
            <input type = "number" id = "age" name = "age" placeholder = 18>

            <br>

            <label for = "dateOfBirth">Date Of Birth:</label>
            <input type = "date" id = "dateOfBirth" name = "dateOfBirth">

            <br>

            <label for = "email">Email:</label>
            <input type = "text" id = "email" name = "email" placeholder =
"johndoe123@gmail.com">

            <br>

            <label for = "phoneNumber">Phone Number:</label>
            <input type = "text" id = "phoneNumber" name = "phoneNumber"
placeholder = "+44">

            <br>
```

```
            <label for = "password">Password:</label>
            <input type = "password" id = "password" name = "password"
placeholder = "••••••••">

            <br>

            <label for = "typeOfEconomicAgent">Select Economic
Agent:</label><br>
            <select id = "typeOfEconomicAgent" name = "typeOfEconomicAgent">
                <option value = "Consumer">Consumer</option>
                <option value = "Producer">Producer</option>
            </select>
        </div>

        <div>
            <input type = "submit">
            <input type = "reset">
        </div>

    </form>

    <div>
        <a th:href = "@{/start}">
            <button>Back</button>
        </a>
    </div>

</body>
</html>
```

login.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">

<head>
    <title>Money Matters</title>
</head>

<body>
    <h1>Login</h1>

    <form action = "/login" method = "post">

        <div>
            <label for = "email">Email:</label>
            <input type = "text" id = "email" name = "email" placeholder =
"johndoe123@gmail.com">

            <br>

            <label for = "password">Password:</label>
            <input type = "password" id = "password" name = "password"
placeholder = "********">

            <br>
        </div>

        <div>
            <input type = "submit">
            <input type = "reset">
        </div>
    </form>

    <div>
        <a th:href = "@{/start}">
            <button>Back</button>
        </a>
    </div>
</body>
</html>
```

wrong-details.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">

<head>
    <title>Money Matters</title>
</head>

<body>
    <h1>You have entered incorrect details / account doesn't exist</h1>
    <p>Go to the start screen</p>

    <a th:href = "@{/start}">
        <button>Start</button>
    </a>
</body>
</html>
```

user-dashboard.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">

<head>
    <title>Money Matters</title>
</head>

<body>
    <h1>Welcome to User Dashboard</h1>

    <a th:href = "@{/start}">
        <button>Log Out</button>
    </a>

    <div>
        <p><b>Current Budget (£):</b> <span th:text =
"${budgetSize}"></span></p>
        <p><b>Frequency of Budget Renewal:</b> <span th:text =
"${frequencyOfBudgetRenewal}"></span></p>

        <p><b>Budget Tracker</b></p>
        <a th:href = "@{/budget-tracker}">
            <button>Budget Tracker</button>
        </a>
    </div>

    <div>
        <p>Display <b>BBC News</b></p>

        <a th:href = "@{/news-BBCNews}">
            <button>BBC News</button>
        </a>
    </div>

    <br>

    <div>
        <p>Display <b>The Telegraph</b></p>

        <a th:href = "@{/news-TheTelegraph}">
            <button>The Telegraph</button>
        </a>
    </div>

    <br>
</body>
</html>
```

news-BBCNews.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">

<head>
    <title>Money Matters</title>
</head>

<body>
    <h1>BBC News</h1>

    <p th:utext = "${newsContent}"></p>

    <a th:href = "@{user-dashboard}">
        <button>Back to Dashboard</button>
    </a>

</body>
</html>
```

news-TheTelegraph.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">

<head>
    <title>Money Matters</title>
</head>

<body>
    <h1>The Telegraph</h1>

    <p th:utext = "${newsContent}"></p>

    <a th:href = "@{/user-dashboard}">
        <button>Back to Dashboard</button>
    </a>

</body>
</html>
```

budget-tracker.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <title>Money Matters</title>
</head>

<body>
    <h1>Budget Tracker</h1>

    <form action = "/budget-tracker" method = "post">
        <div>
            <label for = "budgetSize">Enter Current Budget (£):</label>
            <input type = "text" id = "budgetSize" name = "budgetSize"
placeholder = "100">

            <br>

            <label>Frequency of Budget Renewal:</label>
            <select name="frequencyOfRenewal">
                <option value="N/A">N/A</option>
                <option value="Daily">Daily</option>
                <option value="Weekly">Weekly</option>
                <option value="Monthly">Monthly</option>
            </select>

            <br>
        </div>

        <div>
            <input type = "submit">
            <input type = "reset">
        </div>
    </form>

    <br>

    <a th:href = "@{/good-purchased}">
        <button>Purchase Good</button>
    </a>

    <br>

    <a th:href = "@{/user-dashboard}">
        <button>Back to Dashboard</button>
    </a>
</body>
</html>
```

good-purchased.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">

<head>
    <title>Money Matters</title>
</head>

<body>
    <h1>Adding Good</h1>

    <form action = "/good-purchased" method = "post">
        <div>
            <label for = "goodName">Enter the name of the good you have
purchased:</label>
            <input type = "text" id = "goodName" name = "goodName"
placeholder = "Pepsi">

            <br>

            <label for = "goodPrice">Enter the price of the good you have
purchased:</label>
            <input type = "text" id = "goodPrice" name = "goodPrice"
placeholder = "1">

            <br>

            <label for = "goodQuantity">Enter the quantity of the good you
have purchased:</label>
            <input type = "text" id = "goodQuantity" name = "goodQuantity"
placeholder = "1">

            <br>
        </div>

        <div>
            <input type = "submit">
            <input type = "reset">
        </div>
    </form>

    <div>
        <h2>Latest Goods Purchased</h2>
        <p th:utext = "${consumerPurchases}"></p>
    </div>

    <a th:href = "@{/user-dashboard}">
```

```
        <button>Back to Dashboard</button>
    </a>
</body>
</html>
```

404.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">

<head>
    <meta charset="UTF-8" />
    <title>ERROR 404</title>
</head>
<body>
    <h1>ERROR 404</h1>
    <h2>Page not found</h2>
</body>
</html>
```

References

Figures

Figure 1: Foresters Friendly Society screenshot showing statistics of UK Financial Literacy. (*Foresters Friendly Society*, 2023).
Source: https://www.forestersfriendlysociety.co.uk/how-good-is-the-uks-financial-literacy/ (Accessed: 25 March 2025).
Page - 9

Extract 1a: Foresters Friendly Society Statistics of UK Financial Literacy. (*Foresters Friendly Society*, 2023).
Source: https://www.forestersfriendlysociety.co.uk/how-good-is-the-uks-financial-literacy/ (Accessed: 25 March 2025).
Page - 9

Extract 1b: Foresters Friendly Society - Sally Waterfield, Head of Marketing comments. (*Foresters Friendly Society*, 2023).
Source: https://www.forestersfriendlysociety.co.uk/how-good-is-the-uks-financial-literacy/ (Accessed: 25 March 2025).
Page - 10

Figure 2a: The Telegraph webpage screenshot showing the "Money" tab highlighted. (*The Telegraph*, 2025).
Source: https://www.telegraph.co.uk/ (Accessed: 09 April 2025).
Page - 14

Figure 2b: The Telegraph webpage screenshot showing the "Money" section. (*The Telegraph*, 2025).
Source: https://www.telegraph.co.uk/money/ (Accessed: 09 April 2025).
Page - 15

Figure 2c: The Telegraph webpage screenshot showing a pop-up to force the user to make an account to read the news. (*The Telegraph*, 2025).
Source: https://www.telegraph.co.uk/money/tax/trump-handed-britain-brexit-dividend-vat-starmer/ (Accessed: 09 April 2025).
Page - 15

Figure 2d: The Telegraph webpage screenshot showing a pop-up to convince the user to pay £25 for 1 year subscription. (*The Telegraph*, 2025).
Source: https://www.telegraph.co.uk/money/tax/trump-handed-britain-brexit-dividend-vat-starmer/ (Accessed: 09 April 2025).
Page - 16

Figure 2e: The Telegraph webpage screenshot that gives the user the option to pay for a subscription. (*The Telegraph*, 2025).
Source:
https://www.telegraph.co.uk/customer/subscribe/01sps250/?ICID=subscribecta_flash-overlay-ab-test_asset&redirectTo=https%3A%2F%2Fwww.telegraph.co.uk%2Fmoney%2Ftax%2Ftrump-handed-britain-brexit-dividend-vat-starmer%2F (Accessed: 09 April 2025).
Page - 16

Figure 3: FINANCIAL TIMES webpage screenshot showing a paywall. (FINANCIAL TIMES, 2025).
Source:
https://www.ft.com/content/92b98c54-56de-46f1-a762-7a4a49d6c446
(Accessed: 25 March 2025).
Page - 19

Guides

Guide 1: Youtube tutorial from Telusko teaching the basics of SpringBoot in Java
https://youtu.be/35EQXmHKZYs?si=2-Um8SGl-GLHFlCt

Guide 2: Youtube tutorial from CodeSpy to show how to retrieve user input from the front end and how it can be processed via the backend
https://youtu.be/4aNNedLQHGY?si=n9E-KLOd-5taR6g7

Guide 3: Guide showing how to parse JSON payloads using Jackson in Java
https://mkyong.com/java/jackson-how-to-parse-json/

Guide 4: Guide demonstrating how to create and use Java Threads, for multithreading and concurrency.
https://jenkov.com/tutorials/java-concurrency/creating-and-starting-threads.html

Guide 5: YouTube video on Multithreading and Concurrency in Java Threads
https://www.youtube.com/watch?v=mTGdtC9f4EU&list=PLL8woMHwr36EDxjUoCzboZjedsnhLP1j4&index=1

Guide 6: GitHub repository showing how to use Ollama (Python Library)
https://github.com/ollama/ollama-python

Guide 7: GitHub repository showing how to use Yahoo Finance library to get stock data
https://github.com/yahoo-finance/yahoo-finance

Guide 8: JavaScript Graphing Library (Plotly)
https://plotly.com/javascript/

Guide 9: Guide showing how to perform a subset combinatorial search
https://medium.com/@CalvinChankf/a-general-approach-for-subsets-combinations-and-permutations-5c8fe3aff0ae

Guide 10: Guide showing how to have a JDBC (Java Database Connectivity)
https://www.baeldung.com/java-jdbc

In addition with the use of these guides, I have used AI tools, such as ChatGPT to either learn how systems work, or being able to debug elements of my code, when the error messages felt too confusing. For example, I used AI tools to help my understanding with Multithreading, SpringBoot and Linear Regression, understand how to use API calls with JSON payloads, and stylise some web pages using CSS.